No-Code : L'art de bâtir et d'automatiser sans écrire une ligne

Applications, workflows et processus métier à la portée de tous grâce aux outils visuels

Oussama Ghorbel

Bibliothèque Oussama Ghorbel

© 2026 Oussama Ghorbel. Tous droits réservés.

Aucune partie de cet ouvrage ne peut être reproduite, stockée ou transmise, sous quelque forme ou par quelque moyen que ce soit, sans l’autorisation écrite préalable de l’auteur, sauf courtes citations dans le cadre d’une critique ou d’une recension, conformément aux lois sur la propriété intellectuelle.

Ouvrage conçu et dirigé par Oussama Ghorbel, rédigé avec l’assistance d’outils d’intelligence artificielle.

Bibliothèque Oussama Ghorbel — Première édition, 2026.

Avertissement

Le présent ouvrage est fourni à titre strictement éducatif et informatif. Il ne constitue en aucun cas un conseil financier personnalisé, une recommandation d'investissement, ni une sollicitation à acheter ou vendre un instrument financier.

Les informations contenues dans ce livre sont générales et ne tiennent pas compte de votre situation financière personnelle, de vos objectifs ni de votre tolérance au risque. Avant toute décision d'investissement, consultez un conseiller financier agréé et indépendant.

L'auteur et l'éditeur déclinent toute responsabilité quant aux pertes éventuelles résultant de l'utilisation des informations présentées dans cet ouvrage.

Table des matières

Introduction 8

Partie 1 — Fondations : penser no-code avant d'outiller 23

Chapitre 1 – La promesse no-code : mythes, réalités et promesses tenues 24

Chapitre 2 – Penser comme un produit : du besoin à la spec visuelle . . 37

Chapitre 3 – Cartographier ses processus métier avant de les automatiser 48

Chapitre 4 – Choisir sa stack no-code : la méthode en 7 critères 62

Partie 2 — Les données : le socle invisible de toute application . . 73

Chapitre 5 – Modéliser ses données sans SQL : la pensée relationnelle 74

Chapitre 6 – Airtable en profondeur : le tableur qui a appris le relationnel 86

Chapitre 7 – Notion, Coda, SmartSuite : quand le document devient base de données 98

Chapitre 8 – Bases de données externes : Xano, Supabase, Google Sheets comme backend 114

Chapitre 9 – Qualité, gouvernance et vie des données 130

Partie 3 — Construire des applications visuelles qui tiennent la route 146

Chapitre 10 – Bubble : le couteau suisse du SaaS no-code 147

Chapitre 11 – Glide, Softr, WeWeb, Toddle : apps à partir de données existantes 161

Chapitre 12 – Power Apps, AppSheet, Retool : l'entreprise passe au low-code 177

Chapitre 13 – UX et design system en contexte no-code 189

Chapitre 14 – Sécurité, authentification et gestion des accès 203

Chapitre 15 – Tests, déploiement et maintenance continue 215

Partie 4 — Automatiser : connecter, orchestrer, fluidifier 226

Chapitre 16 – Zapier, Make, n8n, Power Automate : la guerre des orchestrateurs 227

Chapitre 17 – Concevoir des workflows robustes : patterns et anti-patterns 239

Chapitre 18 – Automatisations métier types : le catalogue des 50 recettes 251

Chapitre 19 – RPA et automatisation desktop : quand l'API n'existe pas 264

Chapitre 20 – Webhooks, APIs et code light : l'extension inévitable . . 276

Partie 5 — Industrialiser : IA, équipe, gouvernance et passage à l'échelle 286

Chapitre 21 – IA générative intégrée : GPT, Claude, agents dans vos apps 287

Chapitre 22 – Travailler en équipe sur du no-code 299

Chapitre 23 – Gouvernance, shadow IT et relation avec la DSI 311

© 2026 Oussama Ghorbel

Chapitre 24 – Monétiser, transmettre, évoluer : la vie après le lancement 326

Conclusion 339

Bibliographie commentée 348

© 2026 Oussama Ghorbel

Introduction

Introduction : L'aube des bâtisseurs visuels

Depuis les débuts de l'informatique, l'histoire de la création logicielle est celle d'une longue et implacable marche vers l'abstraction. Dans les années 1950, programmer signifiait câbler physiquement des machines ou perforer des cartes. Puis est venu le langage assembleur, suivi des premiers langages compilés comme Fortran et C. À chaque saut technologique, une frange de puristes s'est offusquée, arguant que s'éloigner du langage machine revenait à sacrifier la performance et la maîtrise. Pourtant, l'abstraction a toujours fini par s'imposer, car elle est la seule réponse viable à la complexité croissante de nos ambitions numériques.

Aujourd'hui, nous vivons le chapitre le plus récent de cette évolution : le No-Code.

Pendant des décennies, le code textuel a été le gardien absolu du temple numérique. Si vous aviez une idée, un problème à résoudre ou un processus à optimiser, vous deviez soit apprendre à parler le langage des machines, soit engager quelqu'un pour le faire à votre place. Cette barrière à l'entrée a créé un goulot d'étranglement mondial. D'un côté, des millions de professionnels, d'entrepreneurs et de créatifs comprenaient parfaitement les problèmes de leur industrie ; de l'autre, une pénurie chronique de développeurs capables de traduire ces compréhensions en logiciels fonctionnels. Le résultat ? Un cimetière d'idées brillantes jamais concrétisées et des entreprises entières paralysées par des tableurs Excel tentaculaires et fragiles.

Le No-Code n'est pas une mode passagère, ni un simple outil de prototypage pour amateurs. C'est un changement de paradigme fondamental. En remplaçant la syntaxe textuelle par des interfaces visuelles, des bases de données relationnelles intuitives et des orchestrateurs de flux de travail, nous avons transféré le pouvoir de création de ceux qui savent *comment* écrire le code à ceux qui savent *quoi* construire et *pourquoi*.

© 2026 Oussama Ghorbel

Ce livre, « No-Code : L'art de bâtir et d'automatiser sans écrire une ligne », est né d'un constat simple mais vertigineux : les outils sont désormais là, ils sont extraordinairement puissants, mais la méthode, la rigueur et la pensée systémique font encore cruellement défaut. Trop de projets no-code échouent non pas à cause des limites de la technologie, mais à cause d'une mauvaise modélisation des données, d'une incompréhension des processus métier ou d'une architecture visuelle bancale. L'objectif de cet ouvrage est de vous fournir bien plus qu'un tutoriel technique ; il vise à vous forger un esprit d'architecte logiciel.

Le grand malentendu : la fin du code ?

Il est crucial, dès les premières pages de ce livre, de dissiper un mythe tenace. Le No-Code ne signe pas l'arrêt de mort du développement traditionnel. Le code n'a pas disparu ; il a simplement été encapsulé, optimisé et masqué là où il apporte le plus de valeur : dans l'infrastructure.

Les plateformes no-code que nous explorerons sont, en réalité, d'immenses moteurs de code écrits par des ingénieurs brillants pour vous permettre de manipuler des concepts de haut niveau. Lorsque vous glissez-déposez un bouton et lui assignez une action, vous manipulez des abstraits de programmation orientée objet et des architectures événementielles. Le No-Code et le Low-Code (qui permet d'injecter des snippets de code personnalisés lorsque les limites visuelles sont atteintes) ne remplacent pas les ingénieurs en logiciel ; ils les libèrent des tâches répétitives de création de formulaires et de tableaux de bord internes, pour qu'ils puissent se concentrer sur les algorithmes complexes, l'infrastructure à grande échelle et l'innovation profonde.

Pour le reste du monde – les 99 % de professionnels qui ne sont pas ingénieurs en logiciel – le No-Code est une émancipation. C'est la capacité de reprendre le contrôle de ses propres outils de travail.

À qui s'adresse ce livre ?

J'ai conçu cet ouvrage comme un compagnon de route pour plusieurs profils, unis par une même soif d'autonomie et d'efficacité :

Les entrepreneurs et les makers. Vous avez une idée de SaaS, de marketplace ou de service numérique. Vous ne voulez pas (ou ne pouvez

© 2026 Oussama Ghorbel

pas) lever des fonds pour engager une équipe de développement avant même d'avoir validé votre marché. Vous apprendrez ici à construire votre produit minimum viable (MVP), à itérer en temps réel avec vos utilisateurs, et à poser les fondations d'une entreprise technologique sans dépendre d'un associé technique.

Les responsables opérationnels et les intrapreneurs. Vous travaillez dans une PME ou un grand groupe et vous étouffez sous les processus manuels, les ressaisies de données et les validations par email. Vous êtes ce qu'on appelle un « Citizen Developer ». Ce livre vous apprendra à cartographier vos processus, à concevoir des portails clients ou employés, et à automatiser les flux de travail pour libérer des centaines d'heures de valeur ajoutée chaque année.

Les freelances, consultants et agences. Vous accompagnez vos clients dans leur transformation numérique. Maîtriser la stack no-code vous permet de passer du statut de conseiller qui livre des slides à celui de bâtisseur qui livre des systèmes fonctionnels, déployés et maintenables.

Les développeurs curieux. Vous codez au quotidien, mais vous êtes fatigué de réinventer la roue pour chaque back-office ou application interne. Vous découvrirez comment le no-code peut devenir votre arme secrète pour livrer des interfaces et des automatisations en un temps record, tout en gardant la main sur les architectures complexes.

La philosophie de l'ouvrage : penser avant de cliquer

La plus grande erreur des débutants en no-code est de choisir un outil avant même d'avoir défini le problème. C'est l'équivalent d'acheter une perceuse industrielle avant de savoir si l'on doit construire une cabane à oiseaux ou un gratte-ciel. L'outil ne fait pas l'architecte.

La promesse de ce livre est de vous enseigner la *pensée produit* et la *pensée système*. Avant d'ouvrir Bubble, Glide, Make ou Airtable, vous apprendrez à déconstruire un besoin métier, à modéliser une base de données relationnelle saine, et à dessiner des workflows résilients. Les plateformes no-code évoluent à une vitesse fulgurante ; celles qui dominent le marché aujourd'hui pourraient être obsolètes dans cinq ans. En revanche, la logique relationnelle, la modélisation des processus

© 2026 Oussama Ghorbel

métier et les principes de l'expérience utilisateur (UX) sont éternels. C'est cette couche de connaissance inaltérable que nous allons bâtir ensemble.

Le voyage qui vous attend : annonce du plan

Pour vous guider de l'idée brute jusqu'à l'industrialisation de vos solutions, cet ouvrage est structuré en cinq grandes parties, conçues pour être lues séquentiellement lors de votre apprentissage, puis utilisées comme manuel de référence au fil de vos projets.

Partie 1 — Fondations : penser no-code avant d'outiller

Tout édifice durable repose sur des fondations solides. Cette première partie est dédiée à la préparation intellectuelle et stratégique de vos projets.

Nous commencerons par décortiquer la promesse du no-code, en séparant les faits des effets d'annonce, pour ancrer vos attentes dans la réalité. Ensuite, nous apprendrons à « penser comme un produit ». Vous découvrirez comment transformer une plainte utilisateur ou une intuition en une spécification visuelle claire. Le chapitre 3 vous initiera à la cartographie des processus métier : on n'automatise jamais un processus que l'on n'a pas d'abord compris et optimisé sur le papier. Enfin, nous établirons une méthode rigoureuse en 7 critères pour choisir votre « stack » (votre combinaison d'outils) no-code, vous évitant ainsi les erreurs coûteuses liées au choix d'une plateforme inadaptée à votre cas d'usage.

Partie 2 — Les données : le socle invisible de toute application

C'est le secret le mieux gardé du développement, qu'il soit traditionnel ou visuel : l'interface n'est qu'une fenêtre sur vos données. Si votre base de données est mal conçue, votre application sera lente, buggée et impossible à faire évoluer.

Nous débuterons par une introduction à la pensée relationnelle, vous permettant de modéliser des données complexes sans jamais écrire une ligne de SQL. Nous plongerons ensuite en profondeur dans Airtable, le tableur qui a révolutionné la donnée relationnelle accessible, avant d'explorer comment des outils documentaires comme Notion, Coda et

© 2026 Oussama Ghorbel

SmartSuite ont transformé nos pages blanches en bases de données structurées. Pour les projets nécessitant une puissance et une scalabilité supérieures, nous explorerons les backends externes comme Xano, Supabase, et même l'usage (et les limites) de Google Sheets. Cette partie se clôturera sur les enjeux cruciaux de la qualité, de la gouvernance et du cycle de vie de vos données.

Partie 3 — Construire des applications visuelles qui tiennent la route

Une fois vos données structurées, il est temps de construire l'interface qui permettra à vos utilisateurs d'interagir avec elles. Le marché des frontends no-code est vaste et segmenté.

Nous analyserons d'abord Bubble, le couteau suisse incontournable pour bâtir des SaaS complexes de A à Z. Nous comparerons ensuite les outils de nouvelle génération comme Glide, Softr, WeWeb et Toddle, qui excellent dans la création d'applications élégantes à partir de données existantes. L'univers de l'entreprise n'est pas en reste : nous décortiquerons l'approche low-code de Power Apps, AppSheet et Retool pour les besoins internes robustes. Mais construire ne suffit pas ; il faut bien construire. Les chapitres 13 à 15 vous armeront des principes de l'UX et du design system en contexte visuel, des protocoles de sécurité et de gestion des accès, ainsi que des méthodologies de tests, de déploiement et de maintenance continue.

Partie 4 — Automatiser : connecter, orchestrer, fluidifier

Une application isolée est une île déserte. La vraie magie du no-code réside dans sa capacité à faire dialoguer les systèmes entre eux. C'est le système nerveux de votre entreprise numérique.

Nous dresserons un état des lieux de la guerre des orchestrateurs en comparant Zapier, Make, n8n et Power Automate, pour vous aider à choisir le bon moteur selon votre logique et votre budget. Vous apprendrez ensuite à concevoir des workflows robustes, en identifiant les patterns de succès et les anti-patterns qui mènent à des boucles infinies ou des pertes de données. Vous y découvrirez un catalogue de 50 recettes d'automatisation métier types, applicables immédiatement. Nous aborderons également le RPA (Robotic Process Automation) pour les cas désespérés où aucune API n'existe, avant de franchir la frontière du « code light » : l'utilisation des webhooks et des scripts JavaScript

© 2026 Oussama Ghorbel

légers pour repousser les limites inévitables des outils visuels.

Partie 5 — Industrialiser : IA, équipe, gouvernance et passage à l'échelle

La dernière partie de ce livre est dédiée à la maturité. Ce qui commence comme un projet personnel ou un outil d'équipe doit souvent évoluer pour supporter la croissance de l'entreprise.

Nous explorerons l'intégration de l'IA générative (GPT, Claude, agents autonomes) directement au cœur de vos applications et workflows, transformant vos outils en assistants intelligents. Travailler seul est une chose, mais collaborer en équipe sur des environnements no-code exige des règles strictes que nous détaillerons. Nous aborderons ensuite le sujet épineux de la gouvernance, du « Shadow IT » et de la relation, souvent complexe, que vous devrez nouer avec la Direction des Systèmes d'Information (DSI) de votre entreprise. Enfin, nous parlerons de l'après-lancement : comment monétiser vos créations, les transmettre, et les faire évoluer (notez que ces réflexions sur la monétisation et le ROI sont partagées à titre indicatif et ne remplacent pas l'avis d'un expert financier).

Les limites et les responsabilités du créateur

Avec le grand pouvoir de créer des logiciels en quelques clics vient une grande responsabilité. Le no-code abaisse les barrières à l'entrée, mais il abaisse aussi les barrières à la création de mauvais logiciels, de failles de sécurité et de processus défaillants.

Lorsque vous construisez une application qui gère les données personnelles de vos clients, qui déclenche des paiements ou qui envoie des communications critiques, vous endossez le rôle d'éditeur de logiciel. Vous devez comprendre les enjeux du RGPD, de la sécurité des accès et de la redondance des données. Ce livre ne fera pas de vous un expert en cybersécurité ou un juriste, mais il éveillera votre conscience sur ces sujets et vous donnera les cadres de travail pour collaborer efficacement avec les experts de votre organisation. Le no-code n'est pas une zone de non-droit ; c'est un nouveau territoire qui exige la même rigueur éthique et technique que le développement traditionnel.

© 2026 Oussama Ghorbel

Mode d'emploi : comment lire cet ouvrage

Vous tenez entre vos mains (ou sur votre écran) un manuel conçu pour être à la fois un récit d'apprentissage et une boîte à outils.

Si vous débutez totalement dans l'univers du no-code et de la tech, je vous recommande une lecture linéaire, du premier au dernier chapitre. L'enchaînement des concepts est pensé pour construire votre compréhension couche par couche, de la modélisation abstraite jusqu'au déploiement concret. Prenez le temps de faire les exercices mentaux, de dessiner vos propres schémas sur papier, et de résister à l'envie d'ouvrir votre navigateur pour créer un compte sur la première plateforme venue.

Si vous êtes déjà un pratiquant du no-code, un « maker » qui a déjà bricolé quelques automatisations sur Zapier ou monté un CRM sur Airtable, vous pouvez utiliser ce livre comme un guide de perfectionnement. Plongez directement dans la Partie 2 pour consolider vos bases de données, ou dans la Partie 4 pour transformer vos automatisations fragiles en workflows de production résilients. Les chapitres sur la gouvernance (Partie 5) seront particulièrement utiles si vous cherchez à faire adopter vos solutions à l'échelle de votre département ou de votre entreprise.

Tout au long de votre lecture, vous remarquera que je m'efforce de rester « agnostique » vis-à-vis des outils. Bien que nous explorions en détail des plateformes spécifiques (Bubble, Make, Airtable, etc.) pour ancrer les concepts dans la réalité, l'objectif n'est pas de faire de vous un évangéliste d'une marque. Les outils naissent, pivote, changent leurs tarifs, et parfois disparaissent. Votre véritable actif, c'est votre capacité à analyser un problème, à le découper en entités de données, à concevoir une logique de traitement et à imaginer une interface humaine. Cette compétence, aucune plateforme ne pourra jamais vous la facturer ou vous la retirer.

Le mot de la fin avant de commencer

Nous vivons une époque extraordinaire pour quiconque souhaite avoir un impact sur le monde qui l'entoure. Le fossé entre l'intention et la création n'a jamais été aussi étroit. Là où il fallait hier des mois, des dizaines de milliers d'euros et une équipe d'ingénieurs pour tester une

© 2026 Oussama Ghorbel

hypothèse de marché ou optimiser une chaîne logistique, il suffit aujourd'hui d'un week-end, d'une connexion internet et d'une pensée structurée.

Le no-code n'est pas une baguette magique qui transforme une mauvaise idée en un succès planétaire. C'est un amplificateur de compétence. Si vous connaissez votre métier, si vous comprenez vos clients et si vous maîtrisez les rouages de vos processus, le no-code décuplera votre valeur. Il fera de vous un profil hybride, rare et profondément recherché : celui qui comprend le besoin métier et qui sait le traduire, seul et rapidement, en une solution technologique vivante.

Oubliez la syntaxe. Oubliez les points-virgules et les erreurs de compilation. Préparez-vous à dessiner, à relier, à orchestrer et à bâtir. Tournez la page, et entrons ensemble dans l'ère de la construction visuelle.

Chapitre 1 – La promesse no-code : mythes, réalités et promesses tenues

L'anatomie d'une révolution silencieuse

Dans l'industrie technologique, les effets de mode se succèdent à un rythme effréné. Chaque année apporte son lot de buzzwords, de paradigmes censés « tout changer », et de conférences où l'on prédit la fin du monde tel que nous le connaissons. Pourtant, au milieu de ce bruit assourdissant, une révolution s'opère en silence. Elle ne fait pas la une des journaux grand public, elle ne génère pas de spéculations boursières irrationnelles, mais elle modifie en profondeur la structure même de notre économie numérique. Cette révolution, c'est le no-code.

Pour comprendre la portée de ce mouvement, il faut cesser de le regarder comme une simple catégorie de logiciels. Le no-code n'est pas un outil, c'est une commodification de la création logicielle. Pendant cinquante ans, le code a été une matière première rare, coûteuse et difficile à extraire. Il nécessitait des années d'apprentissage, une syntaxe stricte et une tolérance zéro à l'erreur de frappe. Aujourd'hui, le code devient une matière première abondante, malléable et accessible. Nous passons d'une ère où le logiciel était un *produit fini* livré par des experts,

© 2026 Oussama Ghorbel

à une ère où le logiciel est un *matériau brut* que chaque professionnel peut façonner pour résoudre ses propres problèmes.

Cette transition n'est pas sans heurts. Comme toute disruption, elle s'accompagne d'incompréhensions, de résistances corporatives et d'attentes irréalistes. Avant de plonger dans la mécanique des outils et l'architecture des données, il est impératif de nettoyer notre esprit des idées reçues qui polluent le discours ambiant. Démystifier le no-code, c'est se donner les moyens de l'utiliser avec lucidité.

Mythe n°1 : "Le no-code va remplacer les développeurs"

C'est la crainte la plus répandue, souvent alimentée par des titres de presse racoleurs et des éditeurs de logiciels en quête de financement. L'idée que le citizen developer (le développeur citoyen, c'est-à-dire un utilisateur métier créant ses propres applications) allait rendre l'ingénieur en logiciel obsolète est non seulement fausse, mais elle témoigne d'une méconnaissance profonde de ce qu'est le développement logiciel.

L'histoire de l'informatique est une longue suite d'outils qui devaient « remplacer les programmeurs ». Les langages de haut niveau comme COBOL ou C devaient vider les salles de serveurs. Les environnements de développement rapide (RAD) des années 90 devaient sonner le glas du codeur. Plus récemment, les frameworks modernes et les générateurs de code assistés par IA ont suscité les mêmes angoisses. À chaque fois, le résultat a été inverse : en abaissant le coût de production des tâches basiques, on a explosé la demande globale en logiciels, créant ainsi un besoin encore plus criant d'architectes, d'experts en sécurité et d'ingénieurs capables de résoudre des problèmes complexes.

Le no-code ne remplace pas le développeur ; il le libère. Aujourd'hui, une part colossale du temps des équipes techniques est absorbée par ce qu'on appelle le « CRUD » (Create, Read, Update, Delete) : la création de formulaires, de tableaux de bord internes, de portails clients basiques et de scripts de synchronisation entre deux API. Ce travail est nécessaire, mais il est répétitif et souvent peu épanouissant pour un ingénieur.

En confiant la construction de ces outils internes et de ces MVP (Produits Minimum Viables) aux experts métier via des plateformes visuelles, on élève le développeur traditionnel au rang qui devrait

© 2026 Oussama Ghorbel

toujours être le sien : celui d'architecte système. Le développeur de demain ne passera plus ses journées à centrer des divs en CSS ou à écrire des endpoints REST basiques. Il concevra les infrastructures scalables, optimisera les algorithmes de recommandation, sécurisera les pipelines de données et créera les briques sur-mesure que les plateformes no-code intégreront ensuite. Le no-code ne tue pas le développeur ; il tue le « codeur à la chaîne ».

Mythe n°2 : "C'est juste pour des prototypes ou des jouets"

L'autre grand malentendu concerne la robustesse. Il est fréquent d'entendre des DSI (Directeurs des Systèmes d'Information) balayer le no-code d'un revers de la main en affirmant que c'était « bien pour un hackathon ou un prototype, mais impensable pour la production ». Cette vision était peut-être défendable en 2015. Elle est aujourd'hui dangereusement obsolète.

Les plateformes no-code modernes ne sont pas des générateurs de sites web statiques. Ce sont des environnements d'exécution complexes, hébergés sur des infrastructures cloud de niveau entreprise (AWS, Google Cloud), capables de gérer des millions de requêtes, de traiter des paiements sécurisés conformes aux normes PCI-DSS, et de respecter les exigences strictes du RGPD.

Prenons l'exemple du secteur de la logistique. Des entreprises utilisent aujourd'hui des applications mobiles construites en no-code pour que leurs chauffeurs sur le terrain scannent des colis, capturent des signatures électroniques et mettent à jour des bases de données relationnelles en temps réel, le tout synchronisé avec des ERP (Progiciels de Gestion Intégrés) comme SAP ou Oracle. Dans le secteur de la santé, des cliniques déploient des portails patients no-code pour gérer la prise de rendez-vous, le suivi des dossiers médicaux et la facturation, avec des niveaux de chiffrement et de contrôle d'accès qui rivalisent avec des solutions codées sur mesure.

La confusion vient du fait que le no-code excelle dans la résolution de problèmes métier complexes, mais pas nécessairement dans la résolution de problèmes *informatiques* complexes. Si vous devez créer une marketplace reliant des milliers d'acheteurs et de vendeurs avec un système d'enchères en temps réel, le no-code est parfaitement adapté. Si

© 2026 Oussama Ghorbel

vous devez développer un moteur de rendu 3D pour un jeu vidéo, ou un algorithme de compression vidéo propriétaire, le no-code sera inutile. Le no-code est un outil de production pour le logiciel d'entreprise et le SaaS (Software as a Service) B2B/B2C, pas pour la recherche fondamentale en informatique.

Mythe n°3 : "Zéro compétence technique requise"

C'est le mythe le plus pernicieux, celui qui conduit à l'échec de milliers de projets. Le terme « no-code » est un triomphe du marketing sur la sémantique. Il laisse entendre que la création de logiciel s'apparente à l'assemblage de blocs de construction pour enfants, où la logique émerge magiquement de l'empilement physique.

La réalité est que le no-code exige une compétence technique profonde, mais d'une nature différente. Vous n'avez pas besoin de mémoriser la syntaxe d'une boucle `for` en JavaScript ou les spécificités d'un framework React. En revanche, vous devez absolument maîtriser la *logique computationnelle* et l'*architecture des systèmes*.

Construire une application no-code robuste exige de comprendre :

• **La pensée relationnelle :** Comment les entités de données s'articulent entre elles (relations un-à-un, un-à-plusieurs, plusieurs-à-plusieurs).

• **La gestion des états (State Management) :** Comment l'interface utilisateur réagit et se met à jour lorsque les données sous-jacentes changent.

• **La logique booléenne et conditionnelle :** Comment structurer des arbres de décision complexes sans créer des workflows illisibles.

• **Les concepts d'API et de Webhooks :** Comment les systèmes s'appellent les uns les autres, gèrent l'authentification (OAuth, clés API) et traitent les réponses asynchrones (JSON, payloads).

Le no-code remplace la *syntaxe* par la *sémantique*. L'erreur de compilation due à un point-virgule manquant disparaît, mais elle est remplacée par l'erreur de logique : un workflow qui se déclenche en boucle infinie parce que vous n'avez pas compris le concept de récursivité, ou une base de données qui s'effondre sous la charge parce que vous n'avez pas indexé vos champs de recherche. Le no-code ne

© 2026 Oussama Ghorbel

dispense pas de penser comme un ingénieur ; il vous oblige à penser comme un *architecte*.

La réalité du terrain : ce que le no-code change vraiment

Une fois les mythes écartés, que reste-t-il ? La véritable promesse du no-code, celle qui justifie l'engouement mondial, repose sur trois piliers fondamentaux : la compression du temps de mise sur le marché, l'alignement entre le besoin et la solution, et la modification de la structure des coûts.

La compression de la boucle de rétroaction

Dans le développement traditionnel, le cycle est long : le métier rédige un cahier des charges, le produit le traduit en tickets, l'ingénierie développe, la QA (Qualité Assurance) teste, et l'ops déploie. Entre l'idée initiale et son test face à l'utilisateur final, il s'écoule des mois. Le no-code réduit ce cycle à des heures, voire des minutes. L'expert métier peut modifier un formulaire, ajuster une règle de calcul ou ajouter un champ en base de données et voir le résultat instantanément. Cette agilité permet une itération basée sur la réalité du terrain plutôt que sur des hypothèses figées dans un document Word de cent pages.

L'alignement parfait entre le problème et la solution

Le plus grand fossé dans la tech est celui qui sépare celui qui a le problème de celui qui code la solution. Ce fossé est rempli de malentendus, de traductions approximatives et de frustrations. Avec le no-code, la personne qui connaît le mieux le problème (le responsable logistique, le directeur financier, le responsable RH) devient celle qui construit la solution. On élimine l'effet « téléphone arabe ». Le logiciel devient l'extension directe de la pensée de l'expert métier.

La dynamique économique et le ROI

Le no-code transforme radicalement l'économie des projets numériques, bien que ces observations économiques restent informatives et ne constituent pas un conseil en investissement personnalisé. Il fait passer la création logicielle d'une logique de CAPEX (dépenses d'investissement lourdes, avec des équipes de développement mobilisées sur de longs cycles) à une logique d'OPEX (dépenses d'exploitation, via

© 2026 Oussama Ghorbel

des abonnements SaaS flexibles et des heures de configuration plutôt que de développement).

Cela étant dit, sur le plan purement opérationnel, le no-code permet à des entreprises de valider des marchés avec un budget de quelques centaines d'euros par mois en outils, là où il fallait auparavant lever des centaines de milliers d'euros pour financer une V1 technique. C'est une démocratisation du droit à l'erreur et à l'expérimentation.

Les limites inhérentes : quand le no-code montre ses coutures

Il serait malhonnête de dresser le portrait du no-code sans aborder ses ombres. En tant que bâtisseur visuel, vous allez inévitablement vous heurter à des murs. Connaître ces limites à l'avance est la marque d'un professionnel mature.

Le verrouillage propriétaire (Vendor Lock-in)

C'est le risque majeur. Lorsque vous codez une application en Python ou en Node.js, le code vous appartient. Vous pouvez l'héberger où vous voulez, le modifier, le fork. Lorsque vous construisez sur Bubble, Airtable ou Make, vous louez un terrain sur lequel vous bâtissez votre maison. Si la plateforme double ses tarifs, change ses conditions d'utilisation, ou fait faillite, votre application est en danger. Vous êtes captif de l'écosystème.

La parade : Nous verrons dans la Partie 2 comment mitiger ce risque en découplant autant que possible votre couche de données (votre base de données externe) de votre couche de présentation (votre frontend no-code), préservant ainsi votre actif le plus précieux : votre donnée.

Le plafond de verre de la performance et de l'UX

Les plateformes no-code génèrent du code en arrière-plan. Ce code est optimisé pour la flexibilité et la généralité, pas pour la performance extrême. Si votre application nécessite des animations UI de 60 images par seconde ultra-fluides, ou si elle doit traiter et afficher des dizaines de milliers de lignes de données en une fraction de seconde sur le navigateur du client, vous atteindrez les limites du moteur de rendu de la plateforme. Le no-code est excellent pour les interfaces fonctionnelles,

© 2026 Oussama Ghorbel

moins pour les expériences immersives ou les applications grand public où le design sur-mesure est l'argument de vente principal.

Le problème du "Dernier Kilomètre" (The Last Mile Problem)

C'est le piège classique dans lequel tombent les débutants. Vous construisez 90 % de votre application en un temps record grâce aux briques natives de la plateforme. Mais il vous reste 10 % de fonctionnalités très spécifiques, propres à votre métier, qui ne sont pas prévues par l'outil. Vous allez alors passer 90 % de votre temps total de projet à essayer de "hacker" la plateforme pour forcer ces 10 % restants, créant au passage une dette technique visuelle monstrueuse et des workflows instables.

La règle d'or du bâtisseur no-code : Si l'outil résiste trop, c'est que vous vous battez contre sa nature. Il faut soit adapter le processus métier à l'outil (ce qui est souvent une bonne pratique de standardisation), soit faire appel à du "code light" (API externe, micro-service sur mesure) pour gérer cette exception.

La dette technique visuelle

On pense souvent que la dette technique est réservée au code textuel. C'est faux. Un workflow Make ou Zapier comportant 150 étapes, avec des filtres imbriqués, des variables d'environnement codées en dur et aucune gestion d'erreur, est l'équivalent visuel du "code spaghetti". Lorsque ce workflow cassera (et il cassera, car les API externes changent, les tokens expirent), le déboguer sera un cauchemar. La rigueur, la documentation et les tests sont encore plus vitaux en no-code qu'en code traditionnel, car les interfaces visuelles masquent souvent la complexité sous-jacente.

Le nouveau contrat social de la tech

Ce premier chapitre pose le cadre de notre voyage. Le no-code n'est ni une baguette magique qui dispense de réfléchir, ni un jouet réservé aux amateurs. C'est un levier de puissance inouï, qui redéfinit le contrat social au sein des entreprises et sur le marché.

D'un côté, les entreprises s'engagent à fournir des outils sécurisés, gouvernés et intégrés, acceptant de lâcher un peu de contrôle centralisé

© 2026 Oussama Ghorbel

au profit de l'agilité périphérique. De l'autre, les experts métier et les makers s'engagent à endosser la responsabilité de ce qu'ils créent. Ils ne sont plus de simples utilisateurs qui soumettent un ticket au support IT en se plaignant d'un bug ; ils deviennent les éditeurs de leurs propres solutions, garants de la qualité de leurs données, de la sécurité de leurs accès et de la maintenance de leurs automatisations.

Vous n'êtes plus un passager du numérique. Vous prenez les commandes. Mais pour piloter, il ne suffit pas de savoir où se trouvent les pédales et le manche. Il faut comprendre l'aérodynamisme, la météo, et la destination.

Dans le chapitre suivant, nous allons laisser de côté la technologie pure pour nous concentrer sur l'essentiel : la pensée produit. Car avant de savoir *comment* construire avec des blocs visuels, il faut impérativement savoir *quoi* construire, pour *qui*, et *pourquoi*. L'outil le plus puissant du monde ne sauvera jamais une mauvaise idée ou un processus métier fondamentalement bancal. Préparez-vous à changer de casquette : nous entrons maintenant dans l'esprit de l'architecte.

© 2026 Oussama Ghorbel

Partie 1 — Fondations : penser no-code avant d'outiller

© 2026 Oussama Ghorbel

Chapitre 1 – La promesse no-code : mythes, réalités et promesses tenues

Il existe une fascination presque magique autour du no-code. L'idée de pouvoir materialiser une idée, de la transformer en une application fonctionnelle, en un système automatisé, sans avoir à taper la moindre ligne de code dans un éditeur texte obscur, ressemble à l'accomplissement d'une vieille prophétie technologique. Pourtant, comme toute révolution, le no-code charrie avec son lot de promesses mirobolantes une quantité égale de malentendus.

Mon objectif dans ce chapitre, et plus largement dans cet ouvrage, n'est pas de vous vendre du rêve. Le no-code n'est pas une baguette magique qui transforme une idée floue en un SaaS coté en bourse du jour au lendemain. C'est un paradigme de construction, un langage visuel qui exige de la rigueur, de la pensée systémique et une compréhension profonde des besoins métier. Avant d'apprendre à cliquer sur des boutons et à relier des modules, il est impératif de déconstruire les mythes, de comprendre l'histoire de ces outils et de poser les fondations d'une pratique saine et durable.

1.1. Du tableur à Bubble : une brève histoire des outils visuels

Pour comprendre vers où nous allons en 2026, il faut regarder d'où nous venons. L'histoire du no-code ne commence pas avec les startups de la Silicon Valley des années 2010. Elle commence bien plus tôt, avec l'invention du tableur électronique.

En 1979, VisiCalc (suivi de près par Lotus 1-2-3 et Microsoft Excel) a introduit un concept révolutionnaire : la séparation entre la donnée brute et la logique qui la manipule, le tout sur une interface visuelle en grille. Un comptable ou un gestionnaire pouvait soudainement créer des modèles financiers complexes, simuler des scénarios et automatiser des calculs sans jamais apprendre le FORTRAN ou le BASIC. Le tableur est, par essence, le premier outil no-code de l'histoire de l'informatique moderne. Il a démocratisé la puissance de calcul en la rendant accessible aux « métiers ».

Dans les années 1990 et 2000, le web a connu sa propre vague d'outils visuels. Dreamweaver a tenté de rendre le HTML visuel, avant que les CMS comme WordPress, Wix ou Squarespace ne prennent le relais. Mais ces outils étaient limités à la présentation : ils permettaient de créer des vitrines, des blogs, des boutiques e-commerce standardisées. La logique sous-jacente restait figée ou nécessitait l'intervention d'un développeur pour être modifiée.

Le véritable tournant, celui qui a donné naissance au mouvement no-code tel que nous le connaissons aujourd'hui, s'est produit avec l'avènement de l'économie des API (Application Programming Interfaces) au début des années 2010. Les logiciels ont commencé à savoir se parler. C'est dans ce contexte qu'est né Zapier en 2012, suivi par des outils comme IFTTT. Soudain, il devenait possible de connecter une adresse email à un tableur, puis à un outil de facturation, sans écrire de script d'intégration.

Puis vint la vague des bases de données relationnelles visuelles (Airtable, 2015) et enfin, les moteurs d'applications web complexes (Bubble, 2012 mais popularisé vers 2018). La promesse avait muté : on ne parlait plus seulement d'automatiser des tâches ou d'afficher du contenu, mais de *construire des logiciels sur mesure*, avec leurs propres bases de données, leurs règles métier, leurs interfaces utilisateurs et leurs systèmes d'authentification. Nous sommes passés de l'automatisation de surface à l'ingénierie logicielle visuelle.

1.2. No-code, low-code, pro-code : où se situe la frontière ?

Il est courant d'entendre ces termes utilisés de manière interchangeable, ce qui est une erreur fondamentale. Pour bien choisir ses outils et calibrer ses attentes, il faut comprendre le spectre du développement logiciel.

Le Pro-Code (développement traditionnel)

C'est l'écriture de code source dans des langages comme Python, JavaScript, C++ ou Ruby. Le développeur a un contrôle absolu sur chaque octet, chaque requête réseau, chaque pixel. La flexibilité est infinie, mais la courbe d'apprentissage est abrupte et le temps de développement est long. C'est le domaine de l'ingénierie logicielle pure.

© 2026 Oussama Ghorbel

Le Low-Code (faible code)

Le low-code s'adresse principalement aux développeurs qui souhaitent accélérer leur travail, ou aux profils très techniques. Des plateformes comme OutSystems, Mendix, ou dans une moindre mesure Retool, fournissent des interfaces visuelles pour assembler des composants, mais exigent une connaissance approfondie des concepts de programmation. Pour créer une interaction complexe ou une requête spécifique, vous devrez inévitablement écrire du JavaScript, du SQL ou comprendre la structure d'un payload JSON. Le low-code abstractise l'infrastructure et l'interface, mais pas la logique complexe.

Le No-Code (sans code)

Le no-code pousse l'abstraction à son paroxysme. L'outil traduit vos intentions visuelles (glisser-déposer, configuration de menus, formules à la Excel) en code machine en arrière-plan. Des outils comme Glide, Softr, Make ou Bubble (dans son usage standard) permettent de bâtir des systèmes entiers sans jamais voir une ligne de code.

Cependant, la frontière est poreuse. C'est pourquoi j'aime utiliser le terme de **« code light »** ou de **« no-code augmenté »**. Dans la pratique, un maker no-code expérimenté finira toujours par injecter de minuscules fragments de code (une expression régulière, un snippet de JavaScript pour manipuler un tableau, une formule SQL dans un backend comme Xano) pour contourner une limitation de l'outil. Le no-code n'est pas l'absence de code, c'est la minimisation du code au profit de la logique visuelle.

1.3. Les trois promesses (vitesse, coût, autonomie) et leurs limites

L'industrie du no-code s'est construite sur trois piliers marketing. Il est crucial de les analyser avec lucidité pour éviter les désillusions.

1. La promesse de la vitesse (Time-to-Market)

Le mythe : « Construisez votre MVP (Minimum Viable Product) en un week-end. »

La réalité : Il est tout à fait vrai que le no-code permet de réduire le temps de développement initial de plusieurs mois à quelques semaines,

© 2026 Oussama Ghorbel

voire quelques jours. Prototyper une idée, la confronter au marché et itérer est infiniment plus rapide. Cependant, cette vitesse initiale crée souvent une illusion. Passer de la version 1.0 (le prototype) à la version 2.0 (le produit robuste, sécurisé, capable de gérer des milliers d'utilisateurs et des cas limites) prendra du temps. La vitesse du no-code réside dans l'itération, pas dans la magie de la finition.

2. La promesse du coût réduit

Le mythe : « Le no-code coûte une fraction du prix d'une équipe de développeurs. »

La réalité : Au démarrage, c'est indéniable. Payer 50 € par mois pour une stack no-code est bien moins cher que de salarier un ingénieur. Mais le no-code repose sur un modèle d'abonnement SaaS. À mesure que votre application grossit, que votre base de données s'alourdit et que vos automatisations s'exécutent des milliers de fois par jour, les coûts des plateformes (Bubble, Make, Airtable) augmentent de manière exponentielle. Ce qu'on appelle la « SaaS Tax » (la taxe des abonnements) peut rendre une application no-code très coûteuse à l'échelle. Le no-code est un levier de trésorerie au démarrage, pas nécessairement une économie à long terme sur les marges unitaires.

3. La promesse de l'autonomie

Le mythe : « Libérez-vous de la DSI (Direction des Systèmes d'Information) et des développeurs. »

La réalité : Le no-code transfère le pouvoir de création aux équipes métier, ce qui est une excellente chose. Mais il ne vous dispense pas de penser comme un créateur de produit. Vous n'avez plus besoin de savoir *comment* coder une base de données, mais vous devez absolument savoir *pourquoi* et *comment* structurer vos données relationnellement. L'autonomie technique ne doit pas être confondue avec l'absence de besoin en compétences produit.

1.4. Le citizen developer : profil, compétences requises, pièges

Le terme « citizen developer » (développeur citoyen) a été popularisé par le cabinet Gartner pour décrire un employé qui crée des

© 2026 Oussama Ghorbel

applications pour son usage propre ou pour son équipe, sans que le développement logiciel ne soit son métier principal. Ce peut être un directeur marketing, un responsable des opérations, un analyste financier ou un chef de projet.

Le profil et les compétences réelles

Contrairement à ce que l'on pourrait croire, le citizen developer n'a pas besoin d'être un « geek » ou un passionné de mathématiques. Les compétences qui prédisent le mieux le succès en no-code sont :

• **La pensée systémique :** La capacité à décomposer un processus complexe en une série d'étapes logiques et conditionnelles (Si X arrive, alors faire Y, sinon faire Z).

• **L'empathie utilisateur (UX) :** Comprendre comment l'utilisateur final va interagir avec l'outil, anticiper ses erreurs, simplifier son parcours.

• **La modélisation de données :** Savoir organiser l'information (qu'est-ce qu'un client ? qu'est-ce qu'une commande ? comment sont-ils liés ?).

• **La persévérance face à l'abstraction :** Accepter de ne pas comprendre ce qui se passe sous le capot, mais savoir isoler un bug par déduction logique.

Les pièges du citizen developer

Le plus grand danger pour le maker débutant est le biais de l'expert. Ayant conçu l'outil, il en comprend intuitivement chaque recoin. Il a alors tendance à créer des interfaces déroutantes pour les autres, à omettre les messages d'erreur clairs et à négliger les cas limites (que se passe-t-il si l'utilisateur clique deux fois très vite sur le bouton de paiement ?).

Un autre piège classique est le syndrome de l'objet brillant : multiplier les outils, ajouter des fonctionnalités « parce que c'est possible » et perdre de vue le problème métier initial que l'application était censée résoudre.

1.5. Quand le no-code devient une dette technique déguisée

© 2026 Oussama Ghorbel

En ingénierie logicielle, la « dette technique » désigne le coût implicite des raccourcis pris lors du développement. En no-code, cette dette existe bel et bien, mais elle prend des formes insidieuses.

Parce qu'il est très facile de créer une automatisation sur Make ou Zapier en cinq minutes, on a tendance à le faire sans documentation, sans convention de nommage, et sans architecture globale. Six mois plus tard, vous vous retrouvez avec ce que j'appelle le **« plat de spaghettis visuel »** : des centaines de scénarios d'automatisation qui s'entrecroisent, des variables nommées `Data_1` et `Test_Final_V2`, et des bases de données Airtable remplies de colonnes orphelines.

Cette dette technique no-code se manifeste par :

1. **Le Shadow IT (informatique fantôme) :** Des processus critiques de l'entreprise reposent sur le compte personnel d'un employé qui a créé l'automatisation. Si cet employé quitte l'entreprise, le processus s'effondre.
2. **L'absence de tests :** En pro-code, on écrit des tests unitaires. En no-code, on teste souvent « à la main » en cliquant sur les boutons. Quand l'application grandit, chaque nouvelle fonctionnalité risque de casser une ancienne fonctionnalité sans que personne ne s'en aperçoive.
3. **Le verrouillage propriétaire (Vendor Lock-in) :** En construisant toute votre logique métier à l'intérieur des murs d'un seul éditeur (par exemple, en utilisant exclusivement les bases de données et les workflows internes de Bubble), vous devenez otage de leurs augmentations de tarifs et de leurs pannes de serveur.

Pour éviter cela, le maker no-code doit s'imposer la même discipline qu'un ingénieur logiciel : documenter ses architectures, utiliser des environnements de test, et séparer autant que possible les données (le backend) de la logique (les workflows) et de l'interface (le frontend).

1.6. Panorama 2026 des catégories d'outils (bases, apps, automations, IA)

L'écosystème no-code a mûri. En 2026, nous ne parlons plus seulement d'outils isolés, mais de plateformes composables qui s'intègrent à l'Intelligence Artificielle générative de manière native.

© 2026 Oussama Ghorbel

Voici comment se structure le paysage actuel.

Les Bases de Données et Backends

C'est le socle de toute application.

• *Les tableurs relationnels :* Airtable reste le standard pour les processus internes et le CRM léger, tandis que SmartSuite s'est imposé pour la gestion de projets complexes.

• *Les backends dédiés :* Xano et Supabase offrent des bases de données relationnelles robustes, scalables, avec des API générées automatiquement. Ils permettent de s'affranchir des limites de lignes imposées par les outils visuels tout en restant accessibles via des interfaces visuelles.

Les Constructeurs d'Applications (Frontends)

• *Les moteurs complexes :* Bubble demeure le leader incontesté pour bâtir des SaaS B2B ou B2C sur mesure, avec une logique de workflow extrêmement poussée.

• *Les outils orientés données :* Glide, Softr et Stacker excellent pour transformer instantanément une base Airtable, Google Sheets ou SQL en portail client ou application interne.

• *La nouvelle garde :* WeWeb et Toddle permettent de créer des frontends sur mesure qui se connectent à n'importe quel backend (Xano, Supabase), offrant une liberté de design totale et une scalabilité accrue.

Les Orchestrateurs et Automations

• *Make (ex-Integromat)* et *n8n* dominent le marché des makers exigeants grâce à leur gestion visuelle des branches conditionnelles, des boucles et des erreurs. n8n, avec son approche open-source et auto-hébergeable, est devenu le choix de prédilection pour les entreprises soucieuses de la souveraineté de leurs données.

• *Zapier* reste le roi de la simplicité et du nombre d'intégrations natives, idéal pour les connexions rapides.

L'Intelligence Artificielle et les Agents

© 2026 Oussama Ghorbel

En 2026, l'IA n'est plus un simple module que l'on ajoute à la fin d'un workflow. Les LLM (Large Language Models) sont intégrés nativement dans les interfaces. On ne construit plus seulement des formulaires, on construit des *agents* capables de lire des emails non structurés, d'en extraire les données, de prendre des décisions basées sur des bases de connaissances internes (RAG), et de déclencher des actions dans le CRM. Le no-code est devenu le système nerveux qui permet aux cerveaux artificiels d'interagir avec les outils de l'entreprise.

Exercices de fin de chapitre

La théorie ne vaut rien sans la pratique. Avant de plonger dans la modélisation de vos données ou la création de vos premiers workflows, je vous invite à réaliser ces trois exercices. Ils vous permettront de calibrer votre approche et de préparer le terrain pour la suite de ce livre.

Exercice 1 : Auto-évaluation — Quel type de maker êtes-vous ?

Ce quiz de 20 questions n'est pas un examen, mais un miroir. Il vise à identifier vos forces naturelles et les domaines où vous devrez être plus vigilant.

Pour chaque affirmation, notez-vous de **1 (Pas du tout d'accord / Pas du tout moi)** à **5 (Tout à fait d'accord / Tout à fait moi)**.

Bloc A : Logique et Systèmes

1. J'aime démonter les objets (ou les processus) pour comprendre comment ils fonctionnent à l'intérieur.

2. Quand je joue à un jeu de société, je suis celui qui lit les règles et repère les failles dans la mécanique.

3. Je suis à l'aise avec les formules conditionnelles dans Excel (ex: SI, RECHERCHEV).

4. J'ai tendance à organiser mes dossiers et fichiers personnels selon une taxonomie stricte et logique.

5. Face à un problème complexe, mon premier réflexe est de le découper en sous-problèmes plus petits.

© 2026 Oussama Ghorbel

Bloc B : Design et Expérience Utilisateur (UX)

6. Je suis souvent frustré par les applications ou les sites web que j'utilise au quotidien quand ils ne sont pas intuitifs.

7. J'accorde de l'importance à l'esthétique et à l'harmonie visuelle, même dans mes documents de travail internes.

8. Je me mets facilement à la place de quelqu'un qui ne connaît pas du tout le sujet que je suis en train d'expliquer.

9. J'aime dessiner des schémas, des croquis ou des diagrammes pour faire comprendre une idée.

10. Je pense que la forme (le design) est tout aussi importante que le fond (la fonction).

Bloc C : Orientation Métier et Projet

11. Je préfère qu'un projet soit « fait et utile » plutôt que « parfait mais en retard ».

12. Je comprends facilement comment l'entreprise gagne de l'argent et où sont les goulots d'étranglement.

13. J'aime parler aux clients, aux utilisateurs ou aux autres départements pour comprendre leurs frustrations.

14. Je suis capable de dire « non » à une fonctionnalité intéressante si elle ne sert pas l'objectif principal du projet.

15. Je mesure le succès d'un outil à son taux d'adoption par les utilisateurs, pas à sa complexité technique.

Bloc D : Tolérance à la frustration et Résolution de problèmes

16. Quand un outil informatique plante, je cherche la solution sur des forums ou dans la documentation avant d'appeler le support.

17. Je ne me décourage pas facilement face à une erreur incompréhensible ; je la vois comme un puzzle à résoudre.

18. J'accepte que ma première version d'un projet soit médiocre, tant que je peux l'améliorer ensuite.

19. Je suis capable de remettre en question ma propre approche si je réalise que je fais fausse route.

© 2026 Oussama Ghorbel

20. J'aime apprendre de nouvelles interfaces logicielles, même si cela me demande un effort cognitif au début.

Interprétation de vos résultats :

• **Score total entre 80 et 100 :** Vous avez le profil complet du *Product Builder*. Vous avez une affinité naturelle pour la pensée produit. Le no-code sera pour vous un terrain de jeu extraordinaire.

• **Score élevé dans le Bloc A et D (Logique et Résolution) :** Vous êtes un *Architecte Système*. Vous excellerez dans les bases de données (Partie 2) et les automatisations complexes (Partie 4). Attention à ne pas négliger l'interface utilisateur.

• **Score élevé dans le Bloc B et C (Design et Métier) :** Vous êtes un *Product Designer / Ops Manager*. Vous saurez créer des outils que les gens aimeront utiliser et qui résoudront de vrais problèmes. Appuyez-vous sur des templates ou des collaborateurs pour la logique pure des bases de données.

• **Score inférieur à 50 :** Le no-code va vous demander un effort d'adaptation. Commencez par des outils très guidés (comme Glide ou Softr) avant de vous attaquer à des toiles blanches comme Bubble.

Exercice 2 : Cartographier son écosystème numérique actuel en 30 minutes

Avant de vouloir automatiser ou remplacer des outils, il faut comprendre l'existant. Cet exercice de cartographie est indispensable pour tout maker.

Matériel : Une grande feuille de papier (ou un tableau blanc), des post-its de trois couleurs différentes, et un stylo.

Étape 1 : Identifier les silos de données (Post-its jaunes - 10 min)

Listez tous les endroits où votre entreprise (ou votre projet) stocke de l'information.

Exemples : Le CRM (HubSpot, Salesforce), le tableur de suivi financier, le drive partagé, l'outil de facturation, Notion, les emails.

© 2026 Oussama Ghorbel

Écrivez un post-it jaune pour chacun de ces outils et placez-les sur la feuille.

Étape 2 : Identifier les acteurs et les déclencheurs (Post-its bleus - 10 min)

Qui interagit avec ces données ?

Exemples : Le commercial qui saisit un lead, le client qui remplit un formulaire sur le site web, le comptable qui exporte un CSV à la fin du mois.

Placez les post-its bleus autour de votre cartographie.

tape 3 : Tracer les flux et les frictions (Dessins - 10 min)

Reliez les acteurs aux outils, et les outils entre eux, par des flèches.

• Utilisez une **flèche verte pleine** pour les connexions automatisées existantes (ex: une API native).

• Utilisez une **flèche rouge en pointillés** pour les transferts manuels (ex: le comptable qui copie-colle des données d'un PDF vers Excel).

• Entourez d'un **cercle rouge** les zones de friction (les endroits où les données sont dupliquées, où les erreurs de saisie sont fréquentes, ou où l'information met du temps à circuler).

Résultat attendu : Vous avez maintenant sous les yeux la carte de votre « dette d'automatisation ». Les cercles rouges sont les cibles prioritaires pour les chapitres sur les workflows et les automatisations métier (Partie 4).

Exercice 3 : Identifier 3 croyances limitantes sur la programmation et les reformuler

Le plus grand frein à l'apprentissage du no-code n'est pas technique, il est psychologique. Des années de culture populaire nous ont convaincus que la création de logiciels était réservée à une élite de génies des mathématiques tapant du code vert sur des écrans noirs.

Identifiez trois croyances qui vous ont, par le passé, empêché de tenter de créer vos propres outils, et reformulez-les à la lumière de ce premier chapitre.

© 2026 Oussama Ghorbel

Exemple 1 :

• *Croyance limitante :* « Je n'ai pas l'esprit logique, je suis plutôt créatif/littéraire, donc je ne saurai pas structurer une base de données. »

• *Reformulation no-code :* « La modélisation de données n'est pas des mathématiques abstraites, c'est de l'organisation sémantique. Si je suis capable de ranger une bibliothèque physique par genres, auteurs et dates, je suis capable de comprendre les relations entre des tables de données. »

Exemple 2 :

• *Croyance limitante :* « Si je construis quelque chose sans être développeur, ça va être bricolé, peu sécurisé, et ça va casser dès que l'entreprise grandira. »

• *Reformulation no-code :* « Le no-code professionnel repose sur des infrastructures cloud de niveau entreprise (AWS, Google Cloud) gérées par les plateformes. La sécurité et la scalabilité dépendent de ma rigueur à appliquer les bonnes pratiques de gouvernance et de design, pas de ma capacité à écrire du code en Python. »

Exemple 3 :

• *Croyance limitante :* « Ça va me prendre plus de temps d'apprendre à utiliser Bubble ou Make que de payer un freelance pour le faire à ma place. »

• *Reformulation no-code :* « Déléguer la création d'un outil métier dont je suis le seul expert revient à jouer au téléphone arabe. En apprenant le no-code, j'investis dans ma capacité à itérer, corriger et faire évoluer mon produit en temps réel, ce qui est le véritable avantage concurrentiel. »

À vous de jouer : Prenez un carnet, notez vos trois blocages personnels, et forcez-vous à écrire leur reformulation. Gardez cette page précieusement. Vous aurez besoin de la relire au chapitre 17, quand vous serez confronté à votre premier workflow d'automatisation qui tourne en boucle infinie et que le syndrome de l'imposteur viendra frapper à votre porte.

© 2026 Oussama Ghorbel

La promesse du no-code est réelle, mais elle se mérite par la rigueur. Tournez la page, nous allons maintenant apprendre à penser comme un produit.

© 2026 Oussama Ghorbel

Chapitre 2 – Penser comme un produit : du besoin à la spec visuelle

L'erreur la plus coûteuse en no-code ne se situe jamais dans l'outil. Elle se situe en amont, au moment où l'on décide de ce que l'on va construire. On voit souvent des équipes entières passer trois semaines à assembler une application Bubble magnifique, visuellement impeccable, techniquement solide… qui ne répondait à aucun besoin réel. L'interface était léchée, les workflows tournaient, la base de données était propre. Personne ne l'a utilisée. Pas parce que l'outil avait failli, mais parce que la pensée produit avait été escamotée.

Le no-code a un effet secondaire pervers : il rend la construction tellement accessible qu'il donne l'illusion que la réflexion préalable est optionnelle. On ouvre Bubble, on glisse un bouton, on ajoute une base de données, et on se retrouve à « bricoler une app ». C'est exactement l'inverse de ce qu'il faut faire. Plus l'outil est facile, plus la discipline amont doit être rigoureuse. Sinon, vous construisez vite — mais vous construisez à côté.

Ce chapitre est probablement le plus important du livre. Il ne vous apprendra pas à cliquer dans un logiciel. Il vous apprendra à penser avant de cliquer. Nous allons transformer une demande métier floue — « on aurait besoin d'un outil pour suivre nos projets » — en une spécification claire, exploitable, testable, que n'importe quel maker no-code (vous, un collègue, un freelance) pourrait implémenter sans ambiguïté.

La promesse est simple : quand vous aurez terminé ce chapitre, vous saurez passer d'une intuition à un plan. Et un bon plan en no-code, c'est souvent 80 % du travail déjà fait.

2.1. Cadrer : qui, quoi, pourquoi, quand, où, combien (QQOQCCP)

Avant d'ouvrir un tableur, un wireframe ou une base de données, posez-vous la question la plus ancienne du management, celle que les consultants en qualité utilisaient déjà dans les années 1960 : QQOQCCP.

© 2026 Oussama Ghorbel

Qui ? Quoi ? Où ? Quand ? Comment ? Combien ? Pourquoi ?

C'est bête comme une checklist. C'est redoutablement efficace.

Qui utilise l'outil ? Pas « l'entreprise », pas « les équipes ». Précisez. Est-ce le responsable commercial, le comptable, l'assistant de direction, les techniciens terrain ? Plus vous êtes précis, plus vous éviterez de construire une interface qui essaie de plaire à tout le monde et ne satisfait personne.

Quoi doit faire l'outil, concrètement ? Décrivez les actions, pas les concepts. « Gérer les clients » est un concept. « Saisir une nouvelle fiche client, lui attribuer un commercial référent, planifier un premier appel » sont des actions. La différence est immense.

Où l'outil sera-t-il utilisé ? Au bureau sur un écran 27 pouces ? Sur un chantier avec un téléphone et des gants ? Dans un entrepôt avec une tablette et une connexion 3G capricieuse ? Le contexte physique dicte des choix de design radicalement différents.

Quand intervient-il dans le quotidien ? Est-ce utilisé dix fois par jour en rafale, une fois par semaine pour un reporting, une fois par mois pour une clôture ? La fréquence d'usage change la tolérance à la friction : un outil quotidien doit être ultra-rapide, un outil mensuel peut se permettre une interface plus guidée.

Comment s'intègre-t-il à l'existant ? Avec quels autres outils doit-il échanger ? Quelles données doit-il consommer ou produire ? Un outil isolé est un outil mort.

Combien de personnes, de transactions, de lignes de données ? Dix utilisateurs ou cinq cents ? Cent fiches ou un million ? Ces ordres de grandeur dictent le choix de la stack technique bien plus que les préférences esthétiques.

Pourquoi, enfin — et c'est souvent la question qu'on oublie. Quel problème douloureux résout-on ? Quelle opportunité saisit-on ? Si vous ne pouvez pas formuler le « pourquoi » en une phrase simple, vous n'êtes pas prêt à construire.

Je vous propose un exercice mental. Prenez la prochaine demande d'outil que vous recevrez dans votre équipe. Avant de répondre,

© 2026 Oussama Ghorbel

forcez-vous à remplir une grille QQOQCCP en trente minutes, sans ouvrir aucun logiciel. Vous serez surpris de voir combien de zones d'ombre persistent, même pour des demandes qui semblaient limpides.

Un dernier point, crucial : le QQOQCCP n'est pas un formulaire administratif à remplir une fois puis à ranger dans un tiroir. C'est un document vivant. Il doit être relu, challengé, partagé avec les futurs utilisateurs. Affichez-le. Imprimez-le. Collez-le au mur. Il deviendra votre boussole chaque fois que vous serez tenté d'ajouter « juste une petite fonctionnalité de plus ».

2.2. Personas et jobs-to-be-done pour apps internes

Les personas, vous connaissez. Cette fiche fictive qui représente un utilisateur type, avec son prénom, son âge, ses frustrations. En marketing, c'est un outil classique. En no-code interne, c'est un outil sous-estimé.

Pour une application interne, vous n'avez pas besoin de personas sophistiqués. Deux ou trois suffisent. Mais ils doivent être honnêtes. Pas le persona idéalisé (« Julie, 34 ans, passionnée par l'optimisation des processus »), mais le persona réel (« Julie, 34 ans, gère 47 dossiers clients en parallèle, utilise Excel parce que c'est ce qu'on lui a montré il y a six ans, n'a jamais le temps de lire les documentations, déteste les formulaires à rallonge »).

Le vrai persona se construit en allant observer les gens travailler. Pas en les interrogeant dans une salle de réunion — ils vous diront ce qu'ils pensent que vous voulez entendre. En regardant leur écran, en comptant leurs clics, en notant les post-it collés sur leur moniteur, les fichiers Excel ouverts en permanence, les raccourcis clavier qu'ils ont inventés.

Mais les personas seuls ne suffisent pas. Ils décrivent qui est l'utilisateur, pas ce qu'il essaie d'accomplir. C'est là qu'intervient le cadre des **jobs-to-be-done** (JTBD), popularisé par Clayton Christensen. L'idée est simple : on n'achète pas un produit, on l'« embauche » pour faire un travail précis.

Appliqué aux apps internes, cela donne des formulations du type : « Quand je prépare la réunion commerciale du vendredi, je veux consolider l'avancement de tous les deals en moins de dix minutes, afin

© 2026 Oussama Ghorbel

d'arriver en réunion avec une vue claire sans y passer ma matinée. »

Décomposez la formule :

• **Quand** (le déclencheur, le contexte)

• **Je veux** (l'action)

• **Afin de** (le résultat attendu, la motivation profonde)

Cette structure a trois vertus. D'abord, elle ancre le besoin dans un moment réel, pas dans une abstraction. Ensuite, elle sépare l'action du résultat, ce qui ouvre la porte à plusieurs solutions possibles pour un même job. Enfin, elle explicite la motivation, ce qui permet de prioriser : un « afin de » qui fait gagner deux heures par semaine pèse plus lourd qu'un « afin de » cosmétique.

Pour une application interne moyenne, identifiez entre cinq et quinze jobs-to-be-done. Classez-les par fréquence et par douleur. Les jobs fréquents et douloureux sont vos priorités absolues. Les jobs rares et peu douloureux peuvent attendre — ou être traités par un simple tableur bien fait.

Un piège à éviter : confondre le job et la solution. « Je veux un tableau de bord avec des graphiques en camembert » n'est pas un job. C'est une solution pré-mâchée par l'utilisateur. Creusez : pourquoi un tableau de bord ? Pourquoi des camemberts ? Vous remonterez souvent à un job bien plus fondamental, du type « je veux détecter en un coup d'œil les deals qui dérapent ». Et là, les camemberts ne sont peut-être pas la bonne réponse.

2.3. User stories et critères d'acceptation en langage naturel

Une fois les personas et les jobs identifiés, il faut descendre d'un cran dans le détail. C'est le rôle des **user stories**.

Le format classique est connu : « En tant que [persona], je veux [action] afin de [bénéfice]. » C'est la même structure que le JTBD, mais appliquée à une fonctionnalité précise plutôt qu'à un objectif global.

Quelques exemples pour une application de gestion d'interventions techniques :

© 2026 Oussama Ghorbel

• En tant que technicien terrain, je veux consulter la fiche d'un équipement en scannant son QR code, afin de ne pas perdre de temps à chercher sa référence dans un catalogue.

• En tant que responsable maintenance, je veux recevoir une alerte quand un équipement dépasse son seuil d'heures de fonctionnement, afin de planifier une maintenance préventive avant la panne.

• En tant que comptable, je veux exporter les interventions du mois en CSV, afin de les intégrer à notre outil de facturation.

Une user story bien écrite tient en une phrase. Si vous avez besoin de trois lignes avec des « et aussi » et des « mais sauf que », c'est qu'elle contient en réalité plusieurs stories. Découpez.

Mais la user story seule ne suffit pas. Elle dit quoi, pas comment on sait que c'est fini. C'est là qu'interviennent les **critères d'acceptation**. Ce sont les conditions qui, une fois remplies, permettent de dire : « Cette story est terminée, on peut passer à la suivante. »

Je vous recommande de les rédiger en langage naturel, pas en Gherkin (le format « Given/When/Then » populaire dans le développement logiciel). Le Gherkin est excellent pour des équipes techniques aguerries, mais il crée une barrière inutile dans un contexte no-code où les métiers doivent pouvoir lire et valider les specs.

Pour la première story ci-dessus, les critères pourraient être :

• Le scan du QR code ouvre directement la fiche de l'équipement concerné, sans étape intermédiaire.

• La fiche affiche au minimum : référence, date d'installation, historique des trois dernières interventions, manuel technique en PDF.

• Si le QR code est illisible, un champ de recherche manuel par référence est disponible en secours.

• Le tout fonctionne en mode déconnecté : le technicien peut consulter la fiche même sans réseau, et les données se synchronisent au retour de la connexion.

Quatre critères. Précis. Testables. N'importe qui dans l'équipe peut lire cette liste et dire si la fonctionnalité est « finie » ou non. Pas de

© 2026 Oussama Ghorbel

débat, pas d'ambiguïté.

Une règle d'or : chaque user story doit avoir entre deux et six critères d'acceptation. En dessous, la story est probablement trop vague. Au-dessus, elle est probablement trop grosse et mérite d'être découpée.

Un dernier conseil, qui vaut de l'or : faites relire vos stories et vos critères par les futurs utilisateurs avant de commencer à construire. Pas par politesse. Par nécessité. Vous découvrirez immanquablement des malentendus, des implicites, des oublis. Mieux vaut les corriger sur du texte que sur du Bubble.

2.4. Wireframes papier avant wireframes numériques

Je vais vous faire une confidence : le meilleur outil de wireframing du monde, c'est une feuille A4 et un stylo. Pas Figma. Pas Balsamiq. Pas Sketch. Une feuille et un stylo.

La raison est simple. Le papier impose des contraintes qui sont en réalité des libérations. Vous ne pouvez pas passer quarante minutes à choisir la bonne teinte de bleu. Vous ne pouvez pas vous perdre dans les détails d'une ombre portée. Vous êtes forcé d'aller à l'essentiel : quelle information apparaît, dans quel ordre, quelle action est possible.

Le wireframe papier a aussi une vertu sociale : il est intimidant pour personne. Quand vous présentez une maquette Figma léchée à un utilisateur, il n'ose pas critiquer. Il sent que vous y avez passé du temps. Il hoche la tête. Quand vous lui montrez un croquis au feutre, il se sent autorisé à dire « attends, ce bouton-là, je ne le comprends pas » ou « en fait, la première chose que je cherche, c'est le montant total, et il est tout en bas de ton dessin ».

Voici comment procéder, et je vous invite à essayer :

1. **Prenez une feuille A4 en mode paysage.** Tracez un cadre qui représente l'écran (téléphone, tablette ou desktop, selon votre contexte).
2. **Listez les informations à afficher.** Pas leur design, juste leur existence. « Nom du client, montant du deal, date de clôture prévue, commercial en charge, étape du pipeline. »

© 2026 Oussama Ghorbel

3. **Hiérarchisez.** Numérotez de 1 à N dans l'ordre où l'œil doit les rencontrer. Ce qui est vu en premier est ce qui compte le plus.

4. **Placez les actions.** Chaque bouton, chaque lien, chaque champ de saisie. Nommez-les avec des verbes : « Modifier », « Archiver », « Ajouter une note ». Évitez les noms flous comme « Détails » ou « Options ».

5. **Dessinez les états.** Que voit-on quand la liste est vide ? Quand une recherche ne donne rien ? Quand une action est en cours ? Quand une erreur survient ? Ce sont souvent ces états secondaires qui font la qualité réelle d'une interface.

Faites trois à cinq écrans papier par user story principale. Photocopiez-les. Donnez-les aux utilisateurs. Demandez-leur de « jouer » l'interface avec leur doigt : « Tu arrives sur cet écran, tu veux faire X, tu cliques où ? » Vous verrez immédiatement ce qui fonctionne et ce qui coince.

Un exercice que j'aime particulièrement : le **wireframe à l'aveugle**. Décrivez votre écran à voix haute à quelqu'un qui ne le voit pas, et demandez-lui de le dessiner pendant que vous parlez. Si le résultat ressemble à ce que vous aviez en tête, votre description est claire. Sinon, votre mental n'est pas encore assez structuré pour passer au numérique.

Le passage au wireframe numérique (Figma, Whimsical, Balsamiq) ne doit intervenir que lorsque les écrans papier ont été validés par les utilisateurs. À ce stade, le numérique sert à affiner, pas à inventer. C'est une distinction cruciale : le numérique est un outil de précision, le papier est un outil de pensée.

2.5. Le MVP no-code : la règle du 80/20 appliquée aux apps

MVP. Minimum Viable Product. Le concept est rabâché dans toutes les formations produit du monde. Et pourtant, il est régulièrement mal appliqué, surtout en no-code.

La tentation est forte, quand on dispose d'outils puissants et qu'on a pris goût à leur rapidité, de construire « tant qu'à faire » un peu plus que le minimum. On ajoute un export PDF. Puis une notification email. Puis un tableau de bord. Puis un système de rôles avec trois niveaux d'accès. Trois semaines plus tard, on a construit un paquebot alors qu'on avait

© 2026 Oussama Ghorbel

besoin d'un canoë.

Le MVP no-code, c'est l'application qui résout le job-to-be-done principal, pour le persona principal, dans le contexte principal. Point.

Appliquez la règle du 80/20 avec brutalité. Vingt pour cent des fonctionnalités délivreront quatre-vingts pour cent de la valeur. Identifiez ces vingt pour cent et construisez-les. Le reste attendra.

Une méthode simple pour trancher. Prenez la liste de toutes les user stories que vous avez écrites. Pour chacune, posez deux questions :

1. **Si je ne livre pas cette story, l'outil est-il inutilisable ?**
2. **Si je ne livre pas cette story, les utilisateurs reviendront-ils à leur ancienne méthode (Excel, email, papier) ?**

Si la réponse aux deux questions est « non », la story n'est pas dans le MVP. Peu importe à quel point elle semble sympathique, élégante ou « facile à faire de toute façon ». Le MVP n'est pas une question de facilité, c'est une question de nécessité.

Voici un exemple concret. Une équipe commerciale demande un CRM interne. La liste des envies contient quarante-sept stories. Le MVP en contient sept :

- Créer une fiche client
- Créer une fiche deal rattachée à un client
- Passer un deal d'une étape à une autre dans le pipeline
- Voir la liste de ses deals en cours
- Ajouter une note horodatée sur un deal
- Rechercher un client par nom
- Modifier une fiche

Tout le reste — export PDF, dashboard, notifications, intégration email, prévisions, scoring — est hors MVP. Pas abandonné. Différé. On verra à l'usage ce qui manque vraiment, pas ce qu'on imaginait manquer.

Le MVP a aussi une vertu psychologique sous-estimée : il crée de la confiance. Quand vous livrez une première version en deux semaines, qui fonctionne, qui résout un vrai problème, vous gagnez un crédit

© 2026 Oussama Ghorbel

énorme auprès des utilisateurs et de la direction. Ce crédit vous servira pour itérer ensuite. À l'inverse, annoncer un projet de six mois qui finit par accoucher d'une souris, c'est tuer l'envie de no-code dans toute l'organisation pour des années.

Une règle stricte à se fixer : un MVP no-code doit pouvoir être livré en moins de quatre semaines de travail à temps partiel. Au-delà, c'est que le périmètre est trop large. Découpez encore.

2.6. Rédiger une 'product brief' d'une page

Tout ce que nous avons vu dans ce chapitre — QQOQCCP, personas, jobs-to-be-done, user stories, wireframes, MVP — doit converger vers un document unique, synthétique, partageable : la **product brief**.

Une page. Pas deux, pas dix. Une page.

Pourquoi une page ? Parce qu'une page se lit. Dix pages se survolent. Une page se partage par email, s'imprime, s'affiche au mur, se relit avant chaque session de construction. Dix pages finissent dans un dossier Google Drive que personne ne rouvre.

Voici une structure éprouvée. Elle tient en six blocs.

Bloc 1 — Le problème (3 lignes max). Quelle douleur, quel manque, quelle opportunité motive ce projet ? Formulez comme un humain, pas comme un consultant. « Les commerciaux passent 3h par semaine à consolider leur pipeline sur Excel, et les chiffres sont toujours obsolètes le vendredi matin. »

Bloc 2 — L'utilisateur principal (2 lignes). Un persona, pas cinq. Celui pour qui on construit en priorité. Les autres viendront après.

Bloc 3 — Le job-to-be-done principal (1 phrase). Le JTBD numéro un, celui qui justifie tout le projet.

Bloc 4 — Le périmètre MVP (liste à puces, 5 à 8 items). Les user stories incluses dans la V1. Rien d'autre.

Bloc 5 — Ce qui est explicitement exclu (liste à puces, 3 à 5 items). Tout aussi important que le bloc précédent. Dire « on ne fait pas le dashboard, ni l'export PDF, ni l'intégration email en V1 » évite les

© 2026 Oussama Ghorbel

malentendus et les dérives.

Bloc 6 — La mesure de succès (1 à 2 indicateurs). Comment saura-t-on, dans quatre semaines, que l'outil est un succès ? Pas un indicateur de vanité (« 100 % des commerciaux l'utilisent »), mais un indicateur de valeur (« le temps de consolidation hebdomadaire passe de 3h à 20 minutes »).

Une fois la brief rédigée, faites-la valider par trois personnes : le sponsor (celui qui paie ou qui décide), un futur utilisateur (celui qui va vivre avec), et un maker (celui qui va construire, peut-être vous). Si les trois signent, vous pouvez ouvrir Bubble, Glide, Make ou n'importe quel autre outil. Pas avant.

La brief a aussi une fonction secondaire, rarement soulignée : elle sert de contrat moral avec vous-même. Dans trois semaines, quand vous serez tenté d'ajouter « juste une petite feature de plus », vous rouvrirez la brief. Vous verrez le bloc « explicitement exclu ». Vous reposerez la souris. Vous finirez d'abord ce qui était prévu.

Un dernier mot. La brief n'est pas gravée dans le marbre. Elle peut évoluer. Mais toute évolution doit être décidée consciemment, documentée, validée à nouveau. Une brief qui change toutes les semaines n'est pas une brief agile, c'est une brief absente.

Pour aller plus loin : quelques lectures utiles

Le cadre des jobs-to-be-done vient des travaux de Clayton Christensen, notamment dans *Competing Against Luck*. Pour les user stories et l'approche agile appliquée au produit, *User Stories Applied* de Mike Cohn reste une référence accessible. Sur le wireframing et le design d'interface, *Don't Make Me Think* de Steve Krug est un classique court et percutant, applicable bien au-delà du web.

Ces lectures ne sont pas obligatoires. Tout ce qu'il vous faut pour bien démarrer tient dans ce chapitre. Mais si vous voulez approfondir, ce sont des portes d'entrée solides.

À retenir

Le no-code récompense la vitesse de construction, mais il punit impitoyablement la précipitation de la pensée. Prenez le temps de cadrer,

© 2026 Oussama Ghorbel

de comprendre les vrais jobs, d'écrire des stories testables, de dessiner sur papier, de réduire le périmètre au MVP, de synthétiser le tout en une brief d'une page. Ces heures de réflexion amont vous en économiseront des dizaines en construction, en corrections, en frustrations.

Quand vous ouvrez un outil no-code, vous devez déjà savoir exactement ce que vous allez construire, pour qui, et pourquoi. L'outil n'est que l'exécutant. Le produit naît dans votre tête, sur une feuille, dans une brief. Pas dans Bubble.

Exercices pratiques

Exercice 1 — Rédiger 5 user stories pour un processus de votre équipe. Choisissez un processus que vous connaissez bien (validation de congés, gestion des notes de frais, onboarding d'un nouveau collaborateur, suivi de commandes, etc.). Identifiez les deux personas principaux. Écrivez cinq user stories avec leur format « En tant que… je veux… afin de… » et, pour chacune, trois à cinq critères d'acceptation en langage naturel. Faites relire par un collègue qui connaît le même processus et demandez-lui s'il est d'accord avec vos critères.

Exercice 2 — Dessiner 3 écrans papier d'une app de gestion de congés. Prenez trois feuilles A4. Dessinez : (1) l'écran d'accueil d'un employé qui voit son solde et ses congés passés/futurs, (2) le formulaire de demande de congés, (3) l'écran du manager qui doit valider ou refuser les demandes en attente. Pour chaque écran, hiérarchisez l'information et nommez chaque action avec un verbe. Montrez vos croquis à deux collègues et demandez-leur de « jouer » l'interface avec leur doigt en commentant à voix haute.

Exercice 3 — Écrire la brief produit d'un outil interne en 1 page maximum. Reprenez un projet d'outil que vous avez en tête ou qu'on vous a demandé. Remplissez les six blocs : problème, utilisateur principal, JTBD principal, périmètre MVP, exclus explicites, mesure de succès. Tenez-vous à une page A4, police 11, marges normales. Si vous dépassez, reformulez et condensez — c'est l'exercice. Partagez la brief à trois personnes (sponsor, utilisateur, maker) et notez leurs retours.

© 2026 Oussama Ghorbel

Chapitre 3 – Cartographier ses processus métier avant de les automatiser

Il existe une tentation presque irrésistible, lorsque l'on découvre le no-code, de se jeter immédiatement sur les outils. On vient de signer un abonnement à Make, Zapier ou Bubble, et l'on ressent cette urgence de « connecter des choses », de créer des scénarios, de voir des données voyager d'une application à l'autre. C'est l'enthousiasme du maker. Pourtant, dans la pratique, on voit d'innombrables projets s'effondrer sous leur propre poids, non pas à cause d'une limitation technique de l'outil, mais à cause d'une faille originelle : l'automatisation d'un processus qui n'avait aucun sens dès le départ.

Le no-code est un amplificateur. Il amplifie votre capacité à exécuter, à déployer et à itérer. Mais il amplifie tout autant vos erreurs de conception. Avant de toucher à la moindre API, avant de configurer le premier webhook, il existe une étape non négociable, souvent négligée par les débutants mais vénérée par les architectes de systèmes expérimentés : la cartographie des processus métier.

Ce chapitre est dédié à l'art de dessiner la réalité de votre entreprise avant de chercher à la modifier. Nous allons apprendre à modéliser visuellement les flux de travail, à identifier les aberrations logiques, et à sélectionner avec une précision chirurgicale ce qui mérite réellement d'être outillé. Car automatiser le chaos ne produit que du chaos automatisé.

3.1. Pourquoi automatiser un mauvais processus accélère juste les problèmes

Bill Gates a un jour formulé une règle d or de la technologie en entreprise qui résonne parfaitement avec la philosophie no-code : « La première règle de toute technologie appliquée à une entreprise est que l'automatisation appliquée à une opération efficace en multipliera l'efficacité. La seconde est que l'automatisation appliquée à une opération inefficace en multipliera l'inefficacité. »

© 2026 Oussama Ghorbel

Cette citation devrait être gravée au-dessus de l'écran de tout créateur no-code. Imaginez une entreprise dont le processus d'onboarding d'un nouveau client implique les étapes suivantes : le commercial remplit un formulaire Excel, l'envoie par email à l'assistant de direction qui le copie-colle dans le CRM, puis l'assistant de direction envoie un email au directeur financier pour qu'il valide la création de la facture, qui à son tour notifie le chef de projet via Slack pour lancer la mission.

Si vous décidez d'automatiser ce processus tel quel avec Make ou Zapier, vous allez relier le formulaire au CRM, déclencher un email automatique au financier, et envoyer un message Slack automatique au chef de projet. Sur le papier, c'est une victoire : vous avez supprimé le copier-coller manuel. Mais dans les faits, vous venez de cristalliser une aberration. Pourquoi le directeur financier doit-il valider manuellement chaque facture de 500 euros ? Pourquoi le CRM n'est-il pas directement connecté au formulaire de saisie du commercial ? En automatisant ce processus sans le questionner, vous venez de rendre ce goulot d'étranglement structurel, invisible et difficile à détricoter. Vous avez accéléré la transmission de l'information, mais vous n'avez pas supprimé les frictions de validation.

L'erreur classique consiste à confondre la *vitesse d'exécution* avec la *pertinence de la logique*. Le no-code permet d'exécuter une logique à la vitesse de la lumière. Mais si la logique exige cinq validations humaines successives pour une tâche à faible risque, votre scénario no-code ne fera qu'envoyer des notifications de validation à la vitesse de la lumière, créant une fatigue décisionnelle chez vos managers.

De plus, automatiser un mauvais processus crée une dette technique redoutable. Dans le monde du code traditionnel, refactoriser une base de code mal pensée prend du temps. Dans le no-code, modifier un scénario Make de 80 modules qui a été construit sur des fondations métier bancales relève souvent du cauchemar. Les erreurs s'accumulent, les exceptions non gérées (les fameux *edge cases*) font planter le scénario en plein milieu de la nuit, et vous finissez par passer plus de temps à maintenir une automatisation toxique qu'à faire le travail manuellement.

La cartographie intervient ici comme un filtre de survie. Elle vous oblige à ralentir pour mieux accélérer ensuite. Elle vous force à poser la question qui tue : « Pourquoi faisons-nous cette étape ? ». Si la réponse

© 2026 Oussama Ghorbel

est « parce qu'on a toujours fait comme ça » ou « parce que l'ancien logiciel l'exigeait », vous tenez une opportunité d'élimination pure et simple. Le but n'est pas d'automatiser l'existant, mais de concevoir l'optimal, puis de l'automatiser.

3.2. BPMN simplifié : événements, tâches, gateways, pools

Pour cartographier, il faut un langage commun. Si vous demandez à trois personnes de votre équipe de vous décrire le processus de facturation, vous obtiendrez trois versions différentes, souvent contradictoires. Pour aligner tout le monde, le monde de l'entreprise utilise depuis des décennies le BPMN (Business Process Model and Notation).

Le BPMN est un standard international extrêmement riche, mais sa version complète est une usine à gaz réservée aux analystes métier certifiés. Pour le maker no-code, il existe une approche que l'on peut appeler le « BPMN Light ». Il s'agit de conserver les 20 % des symboles qui couvrent 99 % de nos besoins en modélisation de workflows automatisables.

Voici le vocabulaire visuel de base que vous devez maîtriser :

1. Les Événements (Les cercles)

Les événements sont ce qui déclenche, interrompt ou termine un processus.

• **Événement de début (Cercle fin) :** Le point de départ. Exemple : « Un client remplit le formulaire de contact » ou « Une nouvelle ligne est ajoutée dans Airtable ».

• **Événement intermédiaire (Double cercle fin) :** Souvent utilisé pour représenter une attente ou un déclencheur temporel. Exemple : « Attendre 3 jours » ou « Recevoir un webhook de Stripe ».

• **Événement de fin (Cercle épais) :** Le point d'arrivée du processus. Exemple : « Facture payée et archivée » ou « Client notifié de l'échec du paiement ».

2. Les Tâches (Les rectangles)

© 2026 Oussama Ghorbel

Les tâches représentent le travail accompli. C'est ici que vous identifierez ce qui sera fait par un humain et ce qui sera délégué à vos outils no-code.

• **Tâche utilisateur (Icône de personne) :** Une action nécessitant une intervention humaine. Exemple : « Le manager valide la demande de congé » ou « Le designer upload la maquette ».

• **Tâche de service / système (Icône d'engrenage) :** C'est le cœur de votre future automatisation. Une action réalisée par un logiciel. Exemple : « Générer le PDF via PDFMonkey » ou « Mettre à jour le statut dans Notion ».

• **Tâche manuelle (Icône de main) :** Une action physique ou hors système. Exemple : « Préparer le colis » ou « Signer le contrat papier ».

3. Les Passerelles ou Gateways (Les losanges)

Les passerelles gèrent la divergence et la convergence du flux. Ce sont vos futures conditions (IF/ELSE) dans Bubble ou vos routeurs (Routers) dans Make.

• **Passerelle exclusive (X dans le losange) :** Le flux prend *un seul* chemin parmi plusieurs options. Exemple : « Le montant de la commande est-il supérieur à 1000 € ? » (Si oui -> chemin A, Si non -> chemin B).

• **Passerelle parallèle (+ dans le losange) :** Le flux se divise en plusieurs chemins exécutés simultanément, ou fusionne pour attendre que tous les chemins soient terminés. Exemple : « Lors de la commande, envoyer un email de confirmation ET notifier l'entrepôt pour la préparation ».

• **Passerelle inclusive (O dans le losange) :** Un ou plusieurs chemins peuvent être empruntés selon les conditions. (À utiliser avec parcimonie car souvent source de confusion, préférez des passerelles exclusives en cascade).

4. Les Pools et Swimlanes (Les couloirs)

Un processus n'évolue pas dans le vide ; il traverse différents départements ou systèmes. Les couloirs (swimlanes) permettent d'attribuer la responsabilité de chaque tâche.

© 2026 Oussama Ghorbel

• **Pool :** Représente l'entité globale (ex: L'entreprise, ou le Client).

• **Swimlanes (Couloirs) :** Les subdivisions. Dans une optique no-code, je vous conseille vivement d'utiliser des couloirs basés sur les *acteurs systémiques* plutôt que seulement sur les départements humains. Par exemple : « Client », « Équipe Commerciale », « Système CRM (HubSpot) », « Orchestrateur (Make/Zapier) ». Cela vous permet de visualiser exactement où l'outil no-code va s'insérer pour remplacer ou assister l'humain.

Maîtriser ce BPMN Light vous donne le pouvoir de traduire n'importe quelle réunion métier en un schéma clair, qui deviendra directement le plan de construction (le blueprint) de votre application ou de votre scénario d'automatisation.

3.3. Cartographier avec Miro, Whimsical ou Lucidchart

Une fois la syntaxe comprise, il faut choisir le bon canevas. L'outil de cartographie doit être rapide, collaboratif et ne pas opposer de résistance à votre réflexion. Voici comment on peut classer les trois grands acteurs du marché pour un maker no-code :

Whimsical : La vitesse et la clarté

C'est un outil de prédilection pour la phase de réflexion solitaire ou en petit comité. Whimsical brille par son interface épurée et ses raccourcis clavier qui permettent de dessiner un flowchart à la vitesse de la pensée. Vous tapez une touche pour créer un rectangle, une autre pour le lier, et l'outil aligne tout automatiquement. Il possède une bibliothèque BPMN light intégrée qui vous évite de chercher les symboles. Whimsical est idéal pour passer de l'idée brute au schéma structuré en moins de trente minutes.

Miro : L'atelier collaboratif

Si vous devez animer un atelier avec cinq personnes de l'entreprise (le commercial, le comptable, le support client, etc.), Miro est imbattable. Son canevas infini permet de commencer par des post-its en vrac pour capturer la réalité du terrain, avant de les regrouper et de les transformer en flux BPMN. Miro est parfait pour le « chaos créatif » initial. L'astuce consiste à utiliser le mode présentation pour guider les parties prenantes étape par étape à travers le processus, en leur

© 2026 Oussama Ghorbel

demandant de valider chaque couloir.

Lucidchart : La rigueur institutionnelle

Lucidchart est plus dense, plus strict, et s'intègre souvent mieux dans les environnements d'entreprises très structurées ou lorsque vous devez collaborer avec la DSI (Direction des Systèmes d'Information). Si votre projet no-code doit s'interfacer avec des systèmes legacy complexes et que vous avez besoin de respecter la norme BPMN 2.0 stricte pour obtenir des validations de la direction, Lucidchart sera votre allié. Cependant, pour 90 % des projets de PME ou de startups, il peut s'avérer trop lourd et ralentir votre phase d'idéation.

La règle d'or de l'atelier de cartographie :

Ne cartographiez jamais seul dans votre coin. Le plus grand piège du maker est de modéliser le processus *tel qu'il devrait être* selon le manuel de l'entreprise, et non *tel qu'il est réellement*. Pour capturer la réalité, vous devez interroger les opérationnels. Vous découvrirez alors ce que l'on appelle les « sentiers de désir » (desire paths) : ces contournements que les employés ont inventés pour survivre à un mauvais logiciel interne. Vous apprendrez que « oui, officiellement on valide sur l'ERP, mais en vrai, Marc envoie un message WhatsApp à Julie le mardi pour qu'elle débloque la commande, sinon on perd le client ». C'est exactement ce workflow WhatsApp informel que votre système no-code devra absorber et structurer.

Dessinez toujours le « Happy Path » (le chemin où tout se passe parfaitement) en premier, puis ajoutez les chemins d'exceptions (les retours, les erreurs, les annulations). En no-code, les exceptions représentent souvent 80 % du temps de développement et de la complexité des scénarios Make. Les voir sur le papier avant de coder vous sauvera des semaines de frustration.

3.4. Identifier goulots, doublons et tâches à faible valeur ajoutée

Une fois votre BPMN Light étalé sur l'écran, le véritable travail d'architecte commence. Vous avez sous les yeux la radiographie de l'entreprise. Il faut maintenant identifier les pathologies. Pour cela, on peut s'inspirer des principes du Lean Management, et plus

© 2026 Oussama Ghorbel

particulièrement de la chasse au « Muda » (le gaspillage), en l'adaptant aux flux digitaux.

Voici les quatre anomalies majeures que vous devez traquer sur votre schéma :

1. Les goulots d'étranglement (Bottlenecks)

Un goulot d'étranglement est une étape où le flux s'accumule, créant une file d'attente. Sur votre schéma, repérez les tâches utilisateur qui sont les seules à pouvoir débloquer les étapes suivantes.

Exemple : Une passerelle exclusive qui attend la validation manuelle du PDG pour toute dépense marketing, même pour des achats de 50 €.

La solution no-code : Introduire des règles de gestion automatisées. Le système (votre application Bubble ou votre scénario Make) peut évaluer le montant. Si < 500 €, la tâche de validation est sautée (chemin direct). Si > 500 €, la notification part sur Slack avec un bouton d'approbation à un clic. Vous venez de fluidifier le goulot.

2. Les doublons et la ressaisie de données

Regardez les tâches de service et manuelles. Voyez-vous la même information transiter d'un système à un autre par l'intermédiaire d'un humain ?

Exemple : Le commercial saisit le nom du client dans le CRM (Tâche 1), puis ouvre l'outil de facturation pour recréer la fiche client (Tâche 2), puis copie l'adresse pour générer le contrat Word (Tâche 3).

La solution no-code : C'est le terrain de jeu favori des orchestrateurs comme Make. Le CRM devient la source unique de vérité (Single Source of Truth). Dès qu'une opportunité passe au statut "Gagnée", un webhook déclenche la création automatique de la fiche dans l'outil de facturation et la génération du contrat via une API. L'humain ne saisit la donnée qu'une seule fois.

3. Les tâches à faible valeur ajoutée (Low-Value Tasks)

Ce sont les tâches qui ne requièrent ni empathie, ni jugement complexe, ni créativité humaine, mais qui consomment du temps. Le copier-coller, le formatage de fichiers, l'envoi d'emails de relance

© 2026 Oussama Ghorbel

standards, le tri de pièces jointes.

La solution no-code : Ce sont vos "Quick Wins". Elles sont faciles à automatiser et offrent un retour sur investissement psychologique immédiat pour les équipes, qui voient leur charge mentale diminuer.

4. Les boucles de rétroaction inutiles (Loops)

Parfois, le processus fait des allers-retours entre deux services pour des clarifications mineures. Sur le BPMN, cela ressemble à une boucle infinie entre le couloir "Support" et le couloir "Technique".

La solution no-code : Standardiser la demande d'entrée. Créer un formulaire dynamique (avec des champs conditionnels) qui oblige le demandeur à fournir toutes les informations requises (captures d'écran, logs, ID client) *avant* que la tâche ne soit créée dans le système de ticketing. On coupe ainsi la boucle à la racine.

En appliquant ce filtre analytique à votre cartographie, vous passez d'une simple observation à un plan d'attaque. Vous ne cherchez plus à "tout automatiser", mais à "éliminer, simplifier, puis automatiser".

3.5. Matrice effort/valeur pour prioriser les automatisations

Vous avez maintenant une liste de 15 ou 20 optimisations potentielles identifiées sur votre cartographie. Le problème ? Vous n'avez ni le temps, ni le budget, ni la bande passante cognitive pour tout construire en même temps. Il faut prioriser. C'est ici que la matrice Effort/Valeur devient votre boussole stratégique.

Dessinez un graphique à deux axes :

- **Axe vertical (Y) : La Valeur Métier (Le gain).**
- **Axe horizontal (X) : L'Effort de réalisation (La complexité technique).**

Pour évaluer la **Valeur**, ne vous fiez pas uniquement à votre intuition. Calculez.

Formule simplifiée : (Temps moyen par tâche × Fréquence mensuelle × Coût horaire de la personne) + Coût des erreurs générées + Impact sur la satisfaction client.

© 2026 Oussama Ghorbel

Une tâche qui prend 10 minutes, faite 200 fois par mois par un employé, représente plus de 33 heures de travail pur par mois. La valeur de son automatisation est immense. À l'inverse, automatiser une tâche complexe qui n'arrive qu'une fois par an a une valeur très faible.

Pour évaluer l'**Effort** en contexte no-code, posez-vous ces questions techniques :

1. *Les API existent-elles ?* (Si oui, effort faible. Si non, il faudra faire du RPA ou du scraping, effort élevé).
2. *La donnée est-elle propre et structurée ?* (Si la donnée source est un email en texte libre, l'effort pour la parser avec de l'IA sera plus élevé que si elle vient d'un formulaire structuré).
3. *Y a-t-il beaucoup d'exceptions (edge cases) ?* (Le Happy Path est facile à coder ; gérer les 15 cas d'erreur possibles multiplie l'effort par trois).

En croisant ces deux axes, vous obtenez quatre quadrants qui dicteront votre feuille de route :

Quadrant 1 : Quick Wins (Haute Valeur / Faible Effort)

• *Description :* Des connexions API directes, des formulaires simples, des notifications conditionnelles.

• *Action :* À faire immédiatement. C'est par là qu'il faut commencer pour générer de l'adhésion en interne et prouver la valeur du no-code à votre direction ou à vos clients.

Quadrant 2 : Projets Stratégiques (Haute Valeur / Effort Élevé)

• *Description :* Refonte complète d'un CRM sur mesure, migration de bases de données complexes, création d'un portail client avec authentification et permissions fines.

• *Action :* À planifier. Ces projets nécessitent une modélisation de données rigoureuse (que nous verrons dans la Partie 2) et un découpage en phases (MVP, V2, V3). Ne vous lancez pas dedans sans avoir d'abord sécurisé les Quick Wins.

Quadrant 3 : Tâches de remplissage (Faible Valeur / Faible Effort)

© 2026 Oussama Ghorbel

• *Description :* Des petits scripts, des rangements de fichiers automatiques dans Google Drive, des alertes Slack sympathiques mais non critiques.

• *Action :* À faire quand vous avez un creux, ou à déléguer à un maker junior pour qu'il se fasse la main.

Quadrant 4 : Puits sans fond (Faible Valeur / Effort Élevé)

• *Description :* Automatiser un processus obsolète qui nécessite de contourner l'API d'un vieux logiciel legacy par du scraping web, pour un gain de temps marginal.

• *Action :* À abandonner ou à repenser radicalement. Parfois, la meilleure automatisation consiste à tuer le processus lui-même, ou à changer de logiciel source.

Cette matrice est un outil de communication redoutable. Quand un manager vous demande "pourquoi tu n'as pas encore automatisé mon rapport Excel du vendredi soir ?", vous pouvez lui montrer la matrice et lui expliquer objectivement que la complexité de son fichier non structuré (Effort élevé) couplée à son faible impact stratégique (Valeur faible) le place en bas de la liste, derrière l'automatisation du traitement des factures fournisseurs.

3.6. Documenter pour transmettre (pas pour archiver)

Le mot "documentation" évoque souvent des dossiers PDF poussiéreux, perdus dans les méandres d'un SharePoint que personne ne consulte. Dans l'univers du no-code, où les applications et les scénarios évoluent à une vitesse fulgurante, la documentation statique est pire qu'inutile : elle est dangereuse, car elle donne une fausse impression de vérité.

L'objectif de la cartographie n'est pas d'archiver le passé, mais de créer une **documentation vivante**, qui sert de colonne vertébrale à la transmission et à la maintenance. J'appelle cela le concept de "Process as Source of Truth" (Le processus comme source de vérité).

Voici comment transformer votre cartographie en un actif vivant pour votre équipe :

1. L'ancrage contextuel (Embed)

© 2026 Oussama Ghorbel

Ne laissez pas votre schéma BPMN dormir dans Whimsical ou Miro. Intégrez-le (via *embed*) directement à l'endroit où le travail se fait. Si vous utilisez Notion pour vos procédures opérationnelles standard (SOP), intégrez le diagramme du processus juste au-dessus des instructions textuelles. Si vous construisez une application interne pour le support client, affichez le schéma du processus de résolution de ticket dans un module d'aide contextuel. Le maker doit voir la logique globale pendant qu'il opère au niveau local.

2. Le couplage Vidéo + Schéma (L'approche Loom)

Un schéma est parfois aride pour un nouvel arrivant. Prenez l'habitude d'enregistrer une vidéo de 5 minutes avec un outil comme Loom, où vous commentez le schéma BPMN tout en montrant l'outil no-code en action.

Exemple de script : « Vous voyez cette passerelle exclusive sur le schéma ? C'est exactement ce qui se passe ici dans notre scénario Make. Si le client coche "Entreprise", le routeur envoie les données vers le module B pour vérifier le numéro de TVA intracommunautaire via l'API de l'État. »

Ce couplage visuel (la logique) et vidéo (l'implémentation) réduit drastiquement le temps d'onboarding des nouveaux makers ou des opérationnels.

3. Le versionning des processus

Dans le code traditionnel, on utilise Git pour versionner. En no-code, le versionning natif des outils est souvent limité (historique des modifications dans Bubble, logs dans Make). Votre cartographie BPMN doit servir de journal des modifications métier.

Une règle essentielle à imposer dans les équipes est la suivante : **on ne modifie jamais une application ou un scénario complexe sans avoir mis à jour le schéma BPMN au préalable.** Le schéma est le contrat. Si le métier change (ex: on ajoute une nouvelle étape de validation légale), le schéma est modifié, validé par les parties prenantes, et *ensuite* le maker implémente la modification dans l'outil. Cela évite les dérives où l'outil ne reflète plus la réalité de l'entreprise, créant ce qu'on appelle le "Shadow IT" (où les employés contournent l'outil

© 2026 Oussama Ghorbel

officiel car il ne correspond plus à leurs besoins réels).

4. Préparer l'audit et la scalabilité

À mesure que votre stack no-code grandit, vous allez connecter des dizaines d'outils, brasser des données sensibles, et impacter le chiffre d'affaires. Lorsque votre DSI (Direction des Systèmes d'Information) ou un auditeur externe viendra vous demander comment sont traitées les données personnelles de vos clients (RGPD), ou comment sont gérés les droits d'accès, sortir un schéma BPMN clair avec les couloirs de responsabilités et les tâches de service vous conférera une crédibilité immédiate. Vous ne serez plus perçu comme un "bricoleur du shadow IT", mais comme un véritable architecte de processus d'entreprise.

La cartographie n'est donc pas une perte de temps avant le "vrai" travail. C'est le "vrai" travail. L'outil no-code n'est que le maçon ; le schéma BPMN est le plan de l'architecte. Et on ne construit pas un bâtiment solide sans plan.

Exercices de fin de chapitre

Pour ancrer ces concepts, je vous invite à réaliser ces trois exercices pratiques. Ils ne nécessitent aucun outil no-code, seulement votre cerveau, un outil de diagramme (papier/crayon, Whimsical ou Miro), et un tableur.

Exercice 1 : Cartographier un processus récurrent en BPMN Light

Objectif : Maîtriser la syntaxe visuelle et identifier les acteurs.

Consigne :

1. Choisissez un processus récurrent dans votre environnement professionnel ou associatif (ex: l'onboarding d'un nouveau client, le traitement d'une demande de remboursement, ou la publication d'un article de blog).

2. Ouvrez votre outil de diagramme et créez trois *swimlanes* (couloirs) : « Client / Demandeur », « Équipe Opérationnelle », « Système / Outils ».

© 2026 Oussama Ghorbel

3. Dessinez le processus en utilisant exclusivement les symboles du BPMN Light (Événements de début/fin, Tâches utilisateur/système, Passerelles exclusives).

4. Identifiez au moins deux passerelles exclusives (ex: "Le dossier est-il complet ?").

5. Mettez en évidence (avec une couleur différente) les tâches qui sont actuellement faites manuellement mais qui pourraient devenir des "Tâches de service" (automatisées).

Exercice 2 : Calculer le temps gaspillé (La matrice de la douleur)

Objectif : Objectiver la valeur d'une automatisation par des données chiffrées.

Consigne :

1. Listez 5 tâches répétitives que vous ou votre équipe effectuez chaque semaine (ex: copier les leads de LinkedIn vers le CRM, générer les rapports hebdomadaires, trier les emails de support, etc.).

2. Créez un tableau avec les colonnes suivantes :

• *Nom de la tâche*

• *Temps moyen par exécution (en minutes)*

• *Fréquence mensuelle*

• *Temps total mensuel (en heures)*

• *Taux d'erreur estimé (%)*

3. Calculez le temps total perdu par mois pour ces 5 tâches.

4. Multipliez ce temps par un taux horaire moyen (ex: 30€/h ou 50€/h selon le profil). Vous venez de calculer le "coût de la non-automatisation". C'est l'argument financier que vous utiliserez pour justifier le temps passé à développer la solution no-code.

Exercice 3 : Construire sa matrice effort/valeur

Objectif : Apprendre à prioriser un backlog d'automatisation.

Consigne :

© 2026 Oussama Ghorbel

1. Reprenez les 5 tâches de l'exercice précédent, et ajoutez-en 5 autres (des projets plus ambitieux, comme "créer un portail client sur mesure").

2. Dessinez votre matrice Effort (X) / Valeur (Y).

3. Pour chaque tâche, attribuez une note de 1 à 5 pour la Valeur (1 = très faible impact, 5 = impact stratégique majeur ou gain de temps massif).

4. Attribuez une note de 1 à 5 pour l'Effort (1 = API native et données propres, 5 = besoin de scraping, données sales, multiples exceptions).

5. Placez vos 10 tâches sur le graphique.

6. **Analyse :** Sélectionnez les deux tâches situées dans le quadrant "Quick Wins" (Haute Valeur, Faible Effort). Ce sont vos devoirs pour la suite de ce livre. Lorsque nous aborderons les orchestrateurs dans la Partie 4, ce sont ces deux scénarios précis que vous devrez construire.

© 2026 Oussama Ghorbel

Chapitre 4 – Choisir sa stack no-code : la méthode en 7 critères

Vous avez clarifié votre besoin, modélisé vos données, cartographié vos processus. Il reste une question, et elle est décisive : quels outils allez-vous assembler pour donner vie à tout cela ? C'est ici que beaucoup de projets no-code trébuchent. Non pas parce qu'ils manquent de bonnes intentions, mais parce qu'ils choisissent leurs outils comme on choisit un téléphone : en regardant la fiche technique, les avis en ligne, ou le dernier tutoriel à la mode.

Une stack no-code n'est pas une collection d'outils. C'est un système. Et comme tout système, sa performance dépend moins de la qualité individuelle de ses composants que de la cohérence de leur assemblage. Un excellent outil mal adapté à votre contexte produira un mauvais résultat. Un outil modeste, mais parfaitement aligné avec vos contraintes, vous emmènera plus loin que vous ne l'imaginez.

Dans ce chapitre, je vous propose une méthode reproductible. Sept critères, un panorama honnête des grandes familles d'outils, et une façon de penser votre stack en couches. L'objectif n'est pas de vous dire quel outil est « le meilleur » — cette question n'a pas de sens hors contexte — mais de vous donner les moyens de trancher vous-même, en conscience.

4.1 Les 7 critères : données, UX, logique, intégrations, sécurité, coût, pérennité

De nombreux projets échouent pour la même raison : une décision prise sur un seul critère, souvent le prix ou la popularité. Une stack se juge sur sept dimensions, et aucune ne peut être négligée.

1. Les données. C'est le critère fondateur. Combien d'enregistrements allez-vous gérer ? À quelle vitesse croissent-ils ? Avez-vous besoin de relations complexes entre tables, d'historisation, de recherches full-text, de jointures multiples ? Une base qui tient sur 5 000 lignes ne se gère pas comme une base qui en compte 500 000. Certains outils s'effondrent au-delà de quelques dizaines de milliers d'enregistrements ; d'autres, au contraire, sont conçus pour scaler.

© 2026 Oussama Ghorbel

2. L'expérience utilisateur (UX). Qui utilisera l'application, et dans quel contexte ? Un commercial sur mobile, en déplacement, n'a pas les mêmes exigences qu'un analyste sur desktop avec deux écrans. Demandez-vous si vous avez besoin d'une interface responsive, d'une application native, d'un portail client, ou simplement d'un outil interne sobre et efficace. L'UX détermine aussi la vitesse d'adoption : une belle interface mal pensée sera rejetée ; une interface rustique mais parfaitement adaptée sera adoptée.

3. La logique métier. Votre application est-elle principalement un affichage de données avec quelques règles simples ? Ou contient-elle des calculs complexes, des workflows conditionnels à plusieurs branches, des boucles, des traitements asynchrones ? La profondeur de la logique permise varie énormément d'un outil à l'autre, et c'est souvent là que se joue la frontière entre no-code et low-code.

4. Les intégrations. Aucun outil ne vit seul. Vous devrez connecter votre application à votre CRM, à votre comptabilité, à votre messagerie, à des API tierces. Regardez le catalogue d'intégrations natives, la qualité de la documentation, la possibilité d'appeler des webhooks, de consommer des API REST, et — point crucial — la manière dont l'outil gère l'authentification OAuth auprès des services externes.

5. La sécurité et la conformité. Où sont hébergées les données ? L'outil est-il conforme au RGPD ? Propose-t-il l'authentification à deux facteurs, la gestion fine des rôles, un journal d'audit, une certification SOC 2 ou ISO 27001 ? Pour une application interne, ces questions peuvent sembler secondaires. Pour une application exposant des données clients, elles sont éliminatoires.

6. Le coût total de possession (TCO). Ne regardez jamais uniquement le prix mensuel affiché. Intégrez le coût des utilisateurs additionnels, du volume de données, des exécutions d'automatisations, des appels API, et surtout le coût humain : temps de formation, de maintenance, d'évolution. Un outil gratuit mais chronophage peut coûter dix fois plus cher qu'un outil payant bien documenté.

7. La pérennité. L'éditeur est-il solide financièrement ? A-t-il une trajectoire de développement claire ? La communauté est-elle vivante ? Que se passe-t-il si l'outil ferme dans deux ans ? Ce critère est le plus

© 2026 Oussama Ghorbel

difficile à évaluer, mais c'est aussi le plus stratégique. Construire sur une plateforme fragile, c'est hypothéquer l'avenir de votre produit.

Une règle simple : notez chaque critère de 1 à 5 pour chaque outil candidat, en fonction de votre contexte précis. La somme des notes vous donnera une première indication, mais ce sont les critères éliminatoires (souvent sécurité et pérennité) qui trancheront vraiment.

4.2 Panorama comparatif : Bubble vs Glide vs Softr vs Power Apps vs AppSheet

Plutôt que de vous présenter chaque outil isolément, comparons-les sur les axes qui comptent vraiment. Ces cinq plateformes couvrent l'essentiel des besoins en applications métier, mais elles n'adressent pas les mêmes publics.

Bubble est la plus puissante et la plus flexible. Vous pouvez y construire presque n'importe quelle application web, du simple annuaire au SaaS complet avec abonnements Stripe, tableaux de bord multi-locataires et workflows complexes. C'est aussi la plus longue à maîtriser : comptez plusieurs semaines avant d'être vraiment à l'aise. Bubble est adapté si vous construisez un produit destiné à évoluer, si vous avez besoin d'une liberté totale sur l'interface et la logique, et si vous acceptez une courbe d'apprentissage significative. Attention toutefois : la plateforme reste un écosystème fermé, et la montée en charge sur de très gros volumes de données impose une architecture réfléchie.

Glide part d'une idée simple : transformer une feuille de données (Google Sheets, Airtable, ou sa propre base) en application mobile élégante en quelques minutes. L'interface est contrainte mais soignée, l'expérience utilisateur excellente, et la prise en main quasi immédiate. Glide convient parfaitement aux applications internes légères, aux catalogues, aux annuaires, aux portails simples. En revanche, dès que la logique métier se complexifie ou que le volume de données dépasse quelques dizaines de milliers de lignes, vous atteindrez rapidement les limites.

Softr joue dans une catégorie proche, mais avec une orientation web plus marquée. Il se connecte nativement à Airtable et SmartSuite, et

© 2026 Oussama Ghorbel

permet de bâtir rapidement des portails clients, des annuaires, des sites membres, des places de marché simples. Softr brille quand vous avez déjà une base Airtable bien structurée et que vous voulez l'exposer à des utilisateurs externes avec une gestion fine des permissions. Moins flexible que Bubble, il compense par une rapidité de mise en œuvre remarquable.

Power Apps est la réponse de Microsoft à la demande d'applications métier en entreprise. Son avantage décisif : l'intégration native avec l'écosystème Microsoft 365, SharePoint, Dataverse, Teams. Si votre entreprise vit déjà sous Microsoft, Power Apps devient un choix presque évident pour des outils internes. La plateforme gère bien les gros volumes via Dataverse, propose une gouvernance solide, et s'inscrit dans les politiques de sécurité de l'entreprise. Son défaut majeur : une expérience utilisateur parfois rugueuse, et une complexité de licensing qui peut rendre le TCO difficile à prévoir.

AppSheet, racheté par Google, est le pendant de Power Apps dans l'univers Google Workspace. Il excelle pour les applications de terrain : inventaires, inspections, relevés, formulaires offline. La logique est déclarative (on décrit ce que l'application doit faire, pas comment), ce qui le rend accessible aux profils métier. Ses limites apparaissent dès que l'on cherche une interface sophistiquée ou des workflows complexes.

Pour trancher entre ces cinq plateformes, posez-vous trois questions : quelle est la complexité de la logique métier ? Quel est l'écosystème dominant dans votre entreprise (Microsoft, Google, indépendant) ? Et surtout, qui sont les utilisateurs finaux et sur quel terminal ?

4.3 Airtable vs Notion vs Coda vs Smartsheet : quand choisir quoi

Ces quatre outils forment une famille particulière : celle des bases de données habillées en documents, ou des documents qui ont appris le relationnel. Leur force commune est de rendre la donnée vivante, collaborative, et immédiatement exploitable par des non-techniciens. Leurs différences sont pourtant profondes.

Airtable reste la référence pour quiconque a besoin d'une vraie base de données relationnelle sans écrire de SQL. Ses vues (grille, Kanban,

© 2026 Oussama Ghorbel

calendrier, galerie, Gantt), ses formules, ses automatisations natives et son API robuste en font le socle de données idéal pour une stack no-code. Airtable est particulièrement adapté quand vos données sont structurées, quand vous avez besoin de relations claires entre tables, et quand vous souhaitez exposer ces données à d'autres outils (Softr, Glide, Make, etc.). Sa limite historique — un plafond de 50 000 à 100 000 enregistrements par base selon le plan — s'est progressivement desserrée, mais elle reste à considérer pour les très gros volumes.

Notion est avant tout un espace de travail collaboratif : documents, wikis, bases de données, tâches. Ses bases de données sont élégantes et intuitives, mais moins puissantes qu'Airtable sur le plan relationnel. Notion brille quand la documentation, la connaissance et les données doivent cohabiter dans un même espace. C'est l'outil privilégié des petites équipes, des startups, des agences, et de tous les contextes où la flexibilité prime sur la rigueur structurelle. Pour une application métier pure, il montrera vite ses limites ; pour un système d'information d'équipe combinant connaissances et données opérationnelles, il est difficile à battre.

Coda se situe entre les deux. Plus puissant qu'Airtable sur certains aspects (formules avancées, boutons d'action, packs d'intégration), il offre une expérience proche du document programmable. Coda est particulièrement adapté quand vous voulez construire des outils internes très personnalisés, où la logique métier est imbriquée dans le document lui-même. Sa philosophie « le document est l'application » séduit les équipes produit et opérationnelles. L'écosystème est plus petit que celui d'Airtable, ce qui peut peser dans la balance.

Smartsheet vise un public différent : les gestionnaires de projet, les PMO, les équipes opérationnelles habituées à Excel. Son interface en feuilles de calcul, ses diagrammes de Gantt, ses formulaires et ses automatisations orientées workflow en font un choix naturel pour le pilotage de projets, le suivi de portefeuilles, les processus d'approbation. Moins adapté pour construire des applications au sens strict, Smartsheet excelle dès qu'il s'agit d'orchestrer des processus transverses avec de nombreux intervenants.

Mon conseil : ne choisissez pas entre ces outils en regardant leurs fonctionnalités, mais en observant comment votre équipe travaille déjà.

© 2026 Oussama Ghorbel

Si elle vit dans des documents, Notion. Si elle raisonne en tables et en relations, Airtable. Si elle a besoin de documents qui calculent et agissent, Coda. Si elle pilote des projets complexes avec de multiples parties prenantes, Smartsheet.

4.4 Zapier vs Make vs n8n vs Power Automate : la guerre des orchestrateurs

Les orchestrateurs sont le système nerveux de votre stack. Ils relient vos applications entre eux, déclenchent des actions en cascade, gèrent les erreurs, assurent la fluidité de vos processus. Quatre acteurs dominent aujourd'hui le marché, avec des philosophies très différentes.

Zapier est le pionnier et le plus connu. Son catalogue d'intégrations est de loin le plus vaste (plus de 6 000 applications), son interface est limpide, et sa prise en main immédiate. Zapier est idéal pour les automatisations simples, linéaires, et pour les équipes qui veulent démarrer sans friction. Son défaut : le coût grimpe vite dès que le volume de tâches augmente, et les workflows complexes (branchements multiples, boucles, gestion fine des erreurs) deviennent rapidement difficiles à maintenir.

Make (anciennement Integromat) propose une approche visuelle beaucoup plus riche. Les scénarios se construisent comme des schémas, avec des routeurs, des itérateurs, des agrégateurs, une gestion avancée des erreurs. Make est l'outil de prédilection des makers qui veulent concevoir des automatisations robustes, avec une logique complexe, sans pour autant écrire de code. Son modèle de tarification, basé sur les opérations, est généralement plus avantageux que Zapier à volume équivalent. La courbe d'apprentissage est un peu plus marquée, mais l'investissement est rapidement rentabilisé.

n8n occupe une place à part : c'est un orchestrateur open source, auto-hébergeable, qui combine interface visuelle et possibilité d'injecter du code JavaScript quand nécessaire. n8n séduit les équipes techniques, les startups soucieuses de leur souveraineté, et tous ceux qui veulent garder le contrôle total sur leurs données et leurs exécutions. L'auto-hébergement demande des compétences DevOps, mais la version cloud propose une expérience clé en main. Pour les projets où la confidentialité des données est critique, n8n est souvent le seul choix

© 2026 Oussama Ghorbel

acceptable.

Power Automate est l'orchestrateur naturel de l'écosystème Microsoft. Si votre entreprise utilise déjà Microsoft 365, SharePoint, Dynamics, Power Automate s'impose pour tout ce qui touche aux processus internes. Il propose également une composante RPA (automatisation de l'interface graphique) particulièrement utile pour interagir avec des applications legacy dépourvues d'API. Son interface a beaucoup évolué, mais reste moins intuitive que Make pour les scénarios complexes hors écosystème Microsoft.

Pour choisir, regardez trois éléments : la complexité de vos workflows (Zapier pour le simple, Make pour le complexe, n8n pour le sur-mesure), l'écosystème dominant de votre entreprise (Power Automate si Microsoft), et votre exigence de souveraineté (n8n si l'auto-hébergement est requis).

4.5 Éviter le vendor lock-in : clauses, exports, API

Le vendor lock-in — cette situation où vous êtes prisonnier d'un outil parce que le coût de sortie est prohibitif — est le risque silencieux du no-code. Parce que vous construisez vite, vous construisez beaucoup, et vous vous attachez profondément à la plateforme qui vous a permis de le faire. Le jour où cette plateforme change ses prix, dégrade son service, ou ferme, vous découvrez que migrer reviendrait presque à tout rebâtir.

Ce risque ne s'élimine pas, il se gère. Et il se gère dès le départ, par quelques réflexes simples.

Exigez l'exportabilité des données. Avant de vous engager, vérifiez que vous pouvez exporter l'intégralité de vos données dans un format standard (CSV, JSON, SQL). Testez cet export sur un jeu de données réel, pas seulement sur la documentation. Certains outils prétendent offrir l'export mais livrent des fichiers inutilisables ou incomplets.

Privilégiez les API ouvertes. Une plateforme dotée d'une API REST documentée et complète vous laisse une porte de sortie : vous pourrez, le jour venu, reconstruire une interface ou une logique ailleurs tout en conservant vos données. À l'inverse, une plateforme sans API, ou avec une API limitée, vous enferme.

© 2026 Oussama Ghorbel

Séparez les couches. C'est le principe que nous développerons dans la section suivante : si vos données vivent dans une base indépendante (Xano, Supabase, Airtable) et que votre interface n'est qu'une couche de présentation, vous pouvez remplacer l'interface sans perdre vos données. Si vos données et votre logique sont enkystées dans la même plateforme, la migration devient un cauchemar.

Lisez les clauses contractuelles. Durée d'engagement, conditions de résiliation, propriété des données, conditions de modification des prix, sort des données en cas de fermeture du service : ces points doivent être examinés avant la signature, pas après. Pour les entreprises, impliquer le service juridique en amont évite bien des déconvenues.

Documentez votre stack. Tenez à jour un registre de vos outils, de leurs versions, de leurs configurations, de leurs intégrations. Le jour où il faudra migrer, cette documentation vaudra de l'or.

Le vendor lock-in zéro n'existe pas. Mais entre un lock-in maîtrisé, dont vous connaissez le coût de sortie, et un lock-in subi, la différence est immense.

4.6 Construire sa stack en couches (data / logic / UI / automation / IA)

Une stack no-code bien pensée s'organise en couches distinctes, chacune ayant un rôle précis et pouvant, idéalement, être remplacée sans affecter les autres. Cette architecture en couches est le meilleur antidote au vendor lock-in, et elle facilite considérablement la maintenance et l'évolution.

La couche data est le socle. Elle héberge vos données, leurs relations, leur historique. Selon vos besoins, ce sera Airtable pour la simplicité, Xano ou Supabase pour la puissance et la scalabilité, SmartSuite pour le relationnel orienté métier, Notion pour la flexibilité. Cette couche doit être choisie avec le plus grand soin : c'est elle qui portera votre système d'information sur la durée.

La couche logique contient les règles métier, les calculs, les validations. Elle peut vivre dans la couche data (formules Airtable, scripts Xano), dans l'orchestrateur (scénarios Make), ou dans l'interface (logique Bubble). L'important est de savoir où chaque règle réside, et

© 2026 Oussama Ghorbel

d'éviter la dispersion.

La couche UI est l'interface utilisateur. Elle peut être portée par l'outil de data lui-même (interfaces Airtable, pages Notion), par un builder dédié (Bubble, WeWeb, Toddle, Softr, Glide), ou par une solution entreprise (Power Apps, AppSheet). C'est la couche la plus visible, mais paradoxalement la plus facile à remplacer si les autres couches sont bien séparées.

La couche automation orchestre les flux entre applications. Make, Zapier, n8n, Power Automate en sont les acteurs principaux. Cette couche est le système nerveux de votre stack : elle doit être robuste, observable, et documentée.

La couche IA est devenue incontournable. Elle peut être intégrée (fonctions IA natives d'Airtable, de Notion, de Bubble), orchestrée (appels à GPT, Claude, Mistral via Make ou n8n), ou hébergée (agents autonomes sur Relevance AI, Flowise, Voiceflow). Pensez-la comme une capacité transversale, pas comme un outil isolé.

Un exemple concret de stack en couches pour une PME industrielle : Supabase pour la data, WeWeb pour l'UI client, Make pour l'automation, Claude via API pour l'assistance intelligente, et Airtable pour le back-office opérationnel. Chaque couche peut évoluer indépendamment. Si WeWeb ne convient plus, on peut migrer l'UI vers Bubble sans toucher aux données. Si Make devient trop cher, on peut basculer vers n8n sans réécrire l'interface.

Cette discipline architecturale demande un effort initial. Elle le rend au centuple sur la durée de vie du projet.

4.7 Budget réaliste d'une stack no-code selon la taille d'entreprise

Parlons argent, sans détour. Le no-code n'est pas gratuit. Il est moins cher que le développement traditionnel, il est plus rapide, il est plus agile — mais il a un coût, et ce coût doit être anticipé.

Pour un indépendant ou une micro-entreprise (1 à 5 personnes), une stack cohérente coûte typiquement entre 50 et 200 euros par mois. Vous combinerez par exemple Airtable (plan gratuit ou Team), Make

© 2026 Oussama Ghorbel

(plan Core), Softr ou Glide (plan de base), et quelques outils satellites. Le vrai coût est ailleurs : votre temps. Comptez plusieurs dizaines d'heures pour construire, tester, maintenir. C'est un investissement, pas une dépense. Notez toutefois que ces estimations budgétaires sont fournies à titre indicatif et ne constituent en aucun cas un conseil en investissement.

Pour une PME (10 à 100 personnes), la fourchette s'élargit considérablement : 300 à 2 000 euros par mois, selon le nombre d'utilisateurs et la complexité des outils. Les plans entreprise d'Airtable, de Bubble, de Make pèsent rapidement. S'y ajoutent souvent des outils spécialisés (signature électronique, emailing, analytics) qui alourdissent la note. C'est aussi à cette taille que la question de la maintenance devient critique : qui, en interne, s'occupe de la stack ? Faut-il former une personne, recruter, externaliser ?

Pour une ETI ou un grand groupe, les budgets se comptent en milliers, voire dizaines de milliers d'euros par mois. Les plateformes entreprise (Power Apps, AppSheet, Power Automate) ont des modèles de licensing complexes, souvent par utilisateur et par application. Les projets no-code s'inscrivent alors dans une stratégie plus large, avec gouvernance, DSI, shadow IT, centres d'excellence. Le coût direct des outils devient secondaire face aux coûts d'organisation, de formation, et de conformité.

Quelques règles pour ne pas vous tromper dans vos prévisions :

• **Projetez sur 24 mois**, pas sur 3. Les premières factures sont toujours légères ; c'est à mesure que l'usage grandit que les coûts explosent.

• **Intégrez le coût des exécutions.** Make, Zapier, Power Automate facturent à la tâche ou à l'opération. Une automation mal conçue peut consommer des milliers d'opérations par jour.

• **Prévoyez une marge de 20 à 30 %** pour les outils non anticipés : un connecteur spécifique, un outil d'analytics, un service d'envoi d'emails transactionnels.

• **Comparez toujours le TCO avec l'alternative.** Une stack no-code à 1 500 euros par mois peut être infiniment moins chère qu'un développement sur mesure à 80 000 euros, même en intégrant

© 2026 Oussama Ghorbel

trois ans de licence.

Le piège classique est de commencer avec des plans gratuits, de construire vite, puis de se retrouver coincé quand l'usage impose de passer aux plans payants. Anticipez cette transition dès le départ : elle est inévitable.

Exercices pratiques

Exercice 1 — Remplir la grille de sélection sur un cas réel. Prenez un projet concret de votre entreprise, actuel ou à venir. Pour chaque outil candidat, remplissez un tableau à sept lignes (données, UX, logique, intégrations, sécurité, coût, pérennité) et notez de 1 à 5. Identifiez les critères éliminatoires. Justifiez votre choix final en trois phrases maximum.

Exercice 2 — Comparer deux plateformes sur un mini-prototype. Choisissez deux outils concurrents pour un même besoin (par exemple Glide et Softr pour un annuaire, Make et Zapier pour une automation). Consacrez trente minutes à chacun pour construire un prototype minimal : trois écrans, cinq enregistrements, un workflow. Notez votre ressenti sur la prise en main, la clarté de la documentation, la fluidité. Vous apprendrez plus en une heure de pratique qu'en dix heures de lectures comparatives.

Exercice 3 — Calculer le TCO sur 24 mois. Pour votre stack candidate, listez tous les outils et leurs plans. Projetez l'évolution du nombre d'utilisateurs, du volume de données, du nombre d'exécutions d'automatisations. Calculez le coût mensuel à 6, 12, 18 et 24 mois. Ajoutez une estimation du temps humain de maintenance (en heures, valorisées au taux horaire moyen de votre équipe). Vous obtiendrez le véritable coût de votre stack — et souvent quelques surprises.

Choisir sa stack n'est pas un acte ponctuel. C'est une décision structurante, qui engage votre projet pour des mois, parfois des années. Prenez le temps de la penser. Les outils passent, les architectures bien conçues restent.

© 2026 Oussama Ghorbel

Partie 2 — Les données : le socle invisible de toute application

© 2026 Oussama Ghorbel

Chapitre 5 – Modéliser ses données sans SQL : la pensée relationnelle

Bienvenue dans la Partie 2 de cet ouvrage. Si la première partie vous a appris à penser comme un produit et à cartographier vos processus, nous allons maintenant descendre dans la salle des machines. Avant de choisir un outil, avant de dessiner la moindre interface utilisateur, avant même d'imaginer le premier workflow d'automatisation, il y a une étape que la grande majorité des créateurs no-code négligent, souvent par peur d'un jargon hérité de l'informatique traditionnelle : la modélisation des données.

Dans le monde du développement classique, cette étape est souvent associée au langage SQL et à la création de schémas de bases de données complexes. Le no-code a eu le grand mérite de masquer cette complexité technique. Cependant, il n'a pas supprimé le besoin de *logique*. Comme l'écrivait Fred Brooks dans son ouvrage fondateur *Le Mythe du Mois-Homme* : « Montrez-moi vos diagrammes de flux et cachez-moi vos tables, et je resterai perplexe. Montrez-moi vos tables, et je n'aurai généralement pas besoin de vos diagrammes de flux. » Les structures de données dictent la réalité de votre application. Si votre modèle de données est bancal, votre application no-code le sera aussi, peu importe la beauté de votre interface ou la puissance de vos automatisations Make ou Zapier.

Ce chapitre a pour objectif de vous transmettre les fondamentaux de la pensée relationnelle. Nous allons apprendre à structurer l'information de manière rigoureuse, sans écrire une seule ligne de SQL, en utilisant un langage clair et orienté métier. Car au fond, modéliser des données, ce n'est pas faire de l'informatique : c'est traduire la réalité de votre entreprise en un langage que la machine peut comprendre et orchestrer.

5.1. Entités, attributs, relations : le trio fondamental

Pour modéliser des données, il faut d'abord apprendre à observer le monde qui vous entoure avec les yeux d'un architecte de l'information. Toute activité humaine, tout processus métier, peut être décomposé en un trio fondamental : des entités, des attributs et des relations.

L'Entité : le nom

Une entité représente un concept, un objet ou un acteur majeur de votre domaine métier. C'est le « nom » autour duquel l'information s'agrège. Dans une application de gestion commerciale, vos entités seront probablement *Client*, *Produit*, *Commande*, *Facture*. Dans une application de ressources humaines, vous trouverez *Employé*, *Département*, *Congé*, *Équipement*.

Une bonne règle empirique pour identifier une entité est de chercher les noms communs autour desquels votre entreprise crée, lit, met à jour ou supprime de l'information (le fameux CRUD : *Create, Read, Update, Delete*). Si vous vous surprenez à dire « Je dois enregistrer les informations du [X] », il y a de fortes chances que [X] soit une entité. Chaque entité deviendra, dans votre outil no-code, une table ou une base de données distincte.

L'Attribut : l'adjectif

Si l'entité est le nom, l'attribut est l'adjectif ou la propriété qui la qualifie. Un *Client* a un *Nom*, un *Email*, une *Date de création*, un *Statut*. Un *Produit* a un *Prix*, un *SKU*, un *Poids*. Les attributs sont les colonnes de vos tables. Ils contiennent la donnée brute.

L'erreur classique du débutant est de confondre une entité et un attribut. Par exemple, faut-il créer une entité *Ville* ou simplement ajouter un attribut « Ville » (de type texte) dans la table *Client* ? La réponse dépend de votre besoin métier. Si vous devez gérer des codes postaux, des taxes régionales, ou des responsables de secteur par ville, *Ville* doit être une entité à part entière. Si c'est juste pour imprimer une étiquette d'expédition, un simple attribut texte suffira.

La Relation : le verbe

Les entités et leurs attributs ne vivent pas en vase clos. Elles interagissent. La relation est le « verbe » qui connecte deux entités entre elles. Un *Client* **passe** une *Commande*. Un *Employé* **appartient** à un *Département*. Un *Article* **est rangé** dans un *Entrepôt*.

C'est ici que la magie de la base de données relationnelle opère. Au lieu de répéter l'information, on crée des ponts. Comprendre ces relations, c'est passer d'un simple tableur plat (où chaque ligne est un

© 2026 Oussama Ghorbel

événement isolé et redondant) à un système d'information vivant, où la mise à jour d'un client se répercute instantanément sur l'historique de toutes ses commandes.

5.2. Un-à-un, un-à-plusieurs, plusieurs-à-plusieurs : exemples concrets

Une fois nos entités identifiées, il faut définir la nature exacte de leurs relations. En modélisation, on parle de **cardinalités**. Il existe trois grandes familles de relations qu'il est impératif de maîtriser pour configurer correctement vos outils comme Airtable, Bubble ou Xano.

La relation Un-à-Un (1:1)

C'est la relation la plus rare, mais elle est essentielle dans certains cas précis. Elle signifie qu'une occurrence de l'entité A est liée à une et une seule occurrence de l'entité B, et inversement.

Exemple concret : Un *Utilisateur* et son *Profil Détaillé*. Dans certaines architectures, on sépare les données de connexion (email, mot de passe) des données de profil (biographie, photo, préférences) pour des raisons de performance ou de sécurité. Un *Employé* peut aussi avoir une relation 1:1 avec un *Véhicule de fonction* attribué.

En no-code : Souvent, les outils no-code gèrent mal le 1:1 natif car ils préfèrent fusionner les deux concepts en une seule table avec plus de colonnes. Le 1:1 n'est vraiment utile que si vous utilisez un backend externalisé comme Supabase ou Xano pour des raisons de limites de colonnes ou de droits d'accès distincts.

La relation Un-à-Plusieurs (1:N)

C'est le pilier absolu de 90 % des bases de données. Une occurrence de l'entité A peut être liée à plusieurs occurrences de l'entité B, mais une occurrence de B n'est liée qu'à une seule occurrence de A.

Exemple concret : Un *Auteur* écrit plusieurs *Livres*, mais un *Livre* n'a qu'un seul *Auteur* principal. Un *Client* passe plusieurs *Commandes*, mais une *Commande* n'appartient qu'à un seul *Client*.

La mécanique : Dans une base de données relationnelle, c'est toujours le côté « Plusieurs » qui stocke la référence vers le côté « Un ».

© 2026 Oussama Ghorbel

La table *Commandes* contiendra donc une colonne « Client » qui pointe vers l'identifiant du client. En no-code, quand vous créez un champ de type "Lien" (Linked Record) sur Airtable, c'est exactement cette logique qui s'applique en arrière-plan.

La relation Plusieurs-à-Plusieurs (N:M)

C'est la relation qui donne le plus de fil à retordre aux débutants, car elle est contre-intuitive à représenter sur un tableur classique. Une occurrence de A peut être liée à plusieurs occurrences de B, et une occurrence de B peut être liée à plusieurs occurrences de A.

Exemple concret : Les *Étudiants* et les *Cours*. Un étudiant suit plusieurs cours, et un cours est suivi par plusieurs étudiants. Les *Articles* de blog et les *Tags* (Catégories). Un article a plusieurs tags, et un tag regroupe plusieurs articles.

La mécanique (La table de jointure) : En SQL classique, il est impossible de lier directement deux tables en N:M. Il faut créer une troisième table, appelée **table de jointure** (ou table d'association), qui ne contient que les identifiants des deux autres. Pour *Étudiants* et *Cours*, on créera une table *Inscriptions* qui dira « L'étudiant X est inscrit au cours Y ».

En no-code : C'est ici que les outils visuels brillent. Des plateformes comme Airtable ou Notion masquent cette table de jointure complexe. Quand vous créez un champ "Lien" en autorisant "Plusieurs enregistrements", l'outil crée et gère silencieusement la table d'association en arrière-plan, vous offrant une interface fluide pour ajouter des tags ou des étudiants en un clic. Cependant, si vous devez stocker des informations *sur la relation elle-même* (par exemple, la "Note" qu'un étudiant a obtenue à un cours spécifique, ou le "Prix d'achat" d'un produit spécifique dans une commande), vous devrez impérativement créer cette table de jointure manuellement (ex: une table *Lignes de Commande*).

5.3. Clés primaires, clés étrangères, identifiants uniques

Pour qu'une relation fonctionne, encore faut-il que chaque entité soit identifiable de manière absolue et infaillible. C'est le rôle des clés. Si vous utilisez Notion ou Airtable, vous n'avez probablement jamais vu

© 2026 Oussama Ghorbel

ces clés de vos propres yeux. Mais si vous vous orientez vers des outils plus robustes comme Bubble, Xano, Supabase ou Make (via des modules API), comprendre ce concept vous sauvera de nombreuses heures de débogage.

La Clé Primaire (Primary Key - PK)

La clé primaire est le numéro de sécurité sociale de votre enregistrement. C'est un identifiant unique, immuable et obligatoire qui permet de retrouver une ligne précise parmi des millions.

Le piège de l'identifiant "métier" : Beaucoup de débutants veulent utiliser l'adresse email, le numéro de téléphone ou le nom de l'entreprise comme clé primaire. C'est une erreur catastrophique. Les gens changent d'email, les entreprises changent de nom, les numéros de téléphone sont réattribués. Si votre clé primaire change, toutes les relations liées à cet enregistrement se brisent.

La solution : Utilisez des identifiants générés par le système. Soit un entier auto-incrémenté (1, 2, 3...), soit, et c'est la norme moderne dans le cloud, un **UUID** (Universally Unique Identifier). Un UUID ressemble à ceci : `550e8400-e29b-41d4-a716-446655440000`. La probabilité qu'un UUID soit généré deux fois dans l'univers est si infime qu'elle est considérée comme nulle. Bubble, Xano et Airtable (avec ses identifiants `recXXXXX`) utilisent tous des variantes de cette approche.

La Clé Étrangère (Foreign Key - FK)

Si la clé primaire sert à *identifier* l'enregistrement, la clé étrangère sert à *pointer* vers un autre enregistrement. C'est le fil d'Ariane qui matérialise la relation.

Dans notre exemple Un-à-Plusieurs entre *Client* et *Commande*, la table *Commande* possédera un attribut appelé `client_id` (la clé étrangère) qui contiendra la valeur de la clé primaire du client concerné. C'est ainsi que la base de données sait regrouper les factures : elle cherche toutes les commandes où `client_id` correspond à l'ID du client connecté.

L'importance de l'immutabilité

© 2026 Oussama Ghorbel

Une règle d'or qu'il est essentiel de rappeler : **on ne modifie jamais une clé primaire une fois qu'elle est créée.** Si vous devez fusionner deux fiches clients doublons, vous ne changez pas l'ID de l'un pour qu'il devienne l'autre. Vous créez un processus de migration des données (commandes, factures) vers le bon ID, puis vous archivez le mauvais. Comprendre la sacralité de l'identifiant unique est ce qui sépare un bricoleur no-code d'un véritable architecte de solutions.

5.4. Normalisation light : éviter la redondance sans sur-ingénierie

Dans le monde académique de l'informatique, la « normalisation » est un processus rigoureux (découpé en formes normales : 1NF, 2NF, 3NF, etc.) visant à éliminer toute redondance de données pour éviter les anomalies de mise à jour. Dans le no-code, nous allons adopter une approche de **normalisation light** : pragmatique, orientée métier, et consciente des limites de nos outils.

Le principe DRY (Don't Repeat Yourself) appliqué aux données

L'objectif principal de la normalisation est simple : une information ne doit exister qu'à un seul endroit.

Imaginons que vous stockiez le nom de la ville et le code postal directement dans la table *Commandes*. Si un client passe 50 commandes, vous aurez copié-collé 50 fois son adresse. Si le client déménage et que vous devez mettre à jour son historique pour des raisons de segmentation marketing, vous devrez modifier 50 lignes. Avec une base normalisée, l'adresse est stockée une seule fois dans la table *Clients* (ou *Adresses*), et les 50 commandes pointent simplement vers ce client. La mise à jour est instantanée et sans risque d'erreur.

Le piège de la sur-normalisation

Poussée à l'extrême, la normalisation devient un cauchemar d'expérience utilisateur. Si vous créez une table séparée pour les *Pays*, une autre pour les *Régions*, une autre pour les *Villes*, et une autre pour les *Rues*, vous obligerez l'utilisateur à faire quatre sélections en cascade juste pour remplir un formulaire d'inscription. De plus, les outils no-code comme Airtable ou Notion gèrent mal l'affichage de données réparties sur 6 tables différentes sans créer des usines à gaz de formules

© 2026 Oussama Ghorbel

`LOOKUP` (Recherches).

La règle d'or du no-code : Normalisez pour protéger l'intégrité des données critiques (Prix, Stocks, Identités, Statuts), mais acceptez une légère dénormalisation pour le confort de l'interface utilisateur et la performance de l'affichage.

L'exception cruciale : Le "Snapshot" (La photo à l'instant T)

Il existe un cas où la redondance n'est pas seulement acceptée, elle est **obligatoire** : les données historiques figées.

Prenons l'exemple d'une *Facture*. Une facture est un document légal qui reflète une réalité à un instant T. Si vous liez votre facture à la table *Produits* pour afficher le prix via une recherche (Lookup), et que vous augmentez le prix du produit l'année suivante, le montant de toutes vos anciennes factures va changer ! C'est illégal et désastreux pour votre comptabilité.

Dans ce cas, la table *Lignes de Facture* doit contenir un attribut `prix_unitaire_facturé` (un champ texte ou nombre simple, non lié) qui copie la valeur du prix au moment exact de la validation de la commande. On parle de "snapshot". Modéliser, c'est aussi savoir quand figer le temps.

5.5. Modéliser pour le no-code : les spécificités Airtable/Notion/Bubble

La théorie relationnelle est universelle, mais sa traduction pratique varie drastiquement selon l'outil no-code que vous avez choisi (selon la méthode vue au Chapitre 4). Chaque plateforme a sa propre "personnalité" face aux données.

Airtable : Le tableur qui a appris le relationnel

Airtable est sans doute le meilleur outil pour *apprendre* à modéliser visuellement. Son interface en grille force à penser en tables et en colonnes.

Sa force : Les champs `Linked Records` (Liens vers d'autres enregistrements) couplés aux `Lookups` (Recherches) et `Rollups` (Agrégations). Vous pouvez lier une table *Projets* à une table *Tâches*, et

© 2026 Oussama Ghorbel

utiliser un Rollup pour calculer automatiquement le "Pourcentage d'avancement" ou la "Somme des heures estimées" directement dans la table Projets.

Sa limite : Airtable n'est pas une vraie base de données relationnelle stricte. Il n'y a pas de contraintes d'intégrité référentielle (si vous supprimez un Client, ses Commandes orphelines ne sont pas automatiquement supprimées ou bloquées, le lien devient juste vide). Pour des systèmes critiques, cette souplesse peut devenir un danger.

Notion : Le document qui se prend pour une base

Notion a révolutionné la collaboration, mais sa gestion des données est fondamentalement différente. Dans Notion, chaque ligne d'une base de données est en réalité une *page* (un document).

Sa force : La fusion entre le texte riche et la donnée structurée. Parfait pour les CRM légers, les wikis, les suivis de contenus.

Sa limite : Les relations dans Notion sont coûteuses en performance et limitées en profondeur. Les Rollups sont basiques. Si vous essayez de modéliser un système de gestion de stocks complexe avec des tables de jointure multiples dans Notion, l'interface deviendra lente et la maintenance infernale. Notion est fait pour la connaissance, pas pour les transactions métier lourdes.

Bubble : Le SaaS builder et le débat des "List Fields"

Bubble offre une puissance de feu bien supérieure, se rapprochant des vraies bases de données. Cependant, Bubble propose une fonctionnalité controversée : les **List Fields** (Champs de liste).

Au lieu de créer une table séparée pour les *Tags* d'un *Article de blog*, Bubble vous permet de créer un champ "Liste de textes" directement dans la table *Article*.

Mon conseil : Fuyez les List Fields pour tout ce qui est amené à évoluer, être filtré, ou compter. Si vous stockez une liste d'emails dans un champ texte, vous ne pourrez pas facilement faire une recherche croisée ou envoyer un email groupé via un workflow. Dans Bubble, adoptez le réflexe SQL : créez des tables séparées et utilisez des champs de type "List of [Autre Table]". Cela alourdit légèrement votre base de

© 2026 Oussama Ghorbel

données (Database Tab), mais garantit que vos workflows backend resteront rapides et scalables.

Xano et Supabase : Le retour aux sources

Si votre stack inclut un backend externe comme Xano ou Supabase, vous n'êtes plus dans le "no-code data", mais dans le "low-code SQL". Ici, les concepts de clés étrangères, d'indexation, de types de données stricts (Integer, Varchar, Boolean, JSONB) et de contraintes `NOT NULL` reviennent en force. C'est l'environnement idéal pour les applications à fort trafic où la normalisation et la performance des requêtes API sont critiques.

5.6. Les pièges classiques (tables monstres, champs calculés mal placés)

Pour conclure cette plongée dans la modélisation, examinons les symptômes de ce que j'appelle le "syndrome du tableur". Ce sont les erreurs de modélisation les plus fréquemment corrigées lors d'audits d'applications no-code existantes.

1. La Table Monstre (The God Table)

C'est le péché capital. Vous ouvrez l'application d'un client et vous trouvez une table appelée `Data_Master` ou `Suivi_Global` avec 140 colonnes. On y trouve pêle-mêle : le nom du client, la date de la dernière commande, le nom de son chien, le statut du ticket de support n°45, et le chiffre d'affaires de l'année.

Le problème : Cette table est ingérable. Les formules de calcul ralentissent l'application, les vues sont illisibles, et les risques d'écraser la donnée d'un autre service sont immenses.

La solution : Appliquez le principe de responsabilité unique. Découpez cette table en entités métier claires (Clients, Commandes, Supports) et reliez-les.

2. Le champ "Tags" séparé par des virgules

Vous avez une colonne `Catégories` dans votre table *Produits* qui contient la valeur texte : `"Électronique, Soldes, Nouveau, Maison"`.

© 2026 Oussama Ghorbel

Le problème : Pour la base de données, c'est une simple chaîne de caractères. Elle ne sait pas ce qu'est une virgule. Si vous voulez filtrer votre application pour afficher "Tous les produits en Soldes", l'outil devra lire chaque ligne, analyser le texte, chercher la sous-chaîne "Soldes". C'est lent, sujet aux erreurs de frappe (ex: "Solde" vs "Soldes") et impossible à indexer correctement.

La solution : Créez une table *Catégories* et utilisez une relation Plusieurs-à-Plusieurs.

3. Les colonnes "Téléphone 1", "Téléphone 2", "Téléphone 3"

Hérité directement d'Excel, ce pattern consiste à créer plusieurs colonnes pour anticiper le fait qu'un client puisse avoir plusieurs numéros.

Le problème : Que se passe-t-il si le client a un quatrième numéro ? Vous devez modifier la structure de votre base de données, casser vos formulaires et refaire vos automatisations. Et comment chercher un numéro spécifique dans l'ensemble de la base s'il peut être dans n'importe quelle colonne ?

La solution : Créez une table *Coordonnées* ou *Téléphones* liée à la table *Client* (Relation 1:N). Un client peut avoir une infinité de numéros, chacun avec son propre type (Pro, Perso, Mobile).

4. Les champs calculés stockés (L'âge vs La date de naissance)

Ne stockez jamais l'âge d'une personne. L'âge est une donnée éphémère qui change chaque année. Si vous stockez "32 ans" dans une colonne, votre base de données sera fausse l'année prochaine.

La solution : Stockez la **Date de Naissance** (donnée source immuable). L'âge est un attribut calculé à la volée par l'interface utilisateur (via une formule `DATETIME_DIFF` sur Airtable ou une expression JavaScript sur WeWeb/Bubble).

La nuance : Comme vu avec le "Snapshot", si vous calculez le "Score de Crédit" d'un client à l'instant où il signe un prêt, ce score doit être figé dans la table *Contrats*, car il représente une vérité historique, contrairement à l'âge qui est une vérité biologique continue.

© 2026 Oussama Ghorbel

Modéliser ses données, c'est faire le tri entre ce qui est *source* (la vérité fondamentale), ce qui est *calculé* (la déduction logique), et ce qui est *historique* (la trace du passé). Maîtriser cette triade, c'est s'assurer que votre application no-code ne sera pas seulement une belle façade, mais un moteur robuste capable de supporter la croissance de votre entreprise.

Exercices de mise en pratique

La théorie ne vaut rien sans la confrontation au réel. Prenez une feuille de papier, un tableau blanc, ou votre outil de diagramme préféré (comme Miro ou Lucidchart), et attaquez-vous à ces trois exercices.

Exercice 1 : Dessiner le MCD d'une boutique e-commerce simple

Objectif : Modéliser le cœur transactionnel d'une boutique en ligne.

Consigne : Dessinez le Modèle Conceptuel de Données (MCD) en identifiant et reliant les **6 entités** suivantes : *Client, Produit, Commande, Ligne de Commande, Catégorie, Adresse de Livraison*.

Points d'attention :

• Définissez la cardinalité de chaque relation (ex: Un Client peut avoir plusieurs Adresses ?).

• Identifiez quelle entité joue le rôle de "table de jointure" indispensable entre les Produits et les Commandes.

• Listez 2 ou 3 attributs clés pour chaque entité (ex: pour *Ligne de Commande*, n'oubliez pas l'attribut "Snapshot" mentionné dans le chapitre).

Exercice 2 : Convertir un fichier Excel plat en 4 tables reliées

Contexte : Vous héritez d'un fichier Excel de suivi de projets d'agence web. Chaque ligne représente une tâche. Les colonnes sont : `ID_Tâche`, `Nom_Tâche`, `Statut`, `Date_Limite`, `Nom_Client`, `Email_Client`, `Téléphone_Client`, `Nom_Chef_de_Projet`, `Email_Chef_de_Projet`.

Consigne :

© 2026 Oussama Ghorbel

1. Identifiez les redondances et les violations du principe DRY.

2. Proposez une architecture éclatée en **4 tables distinctes** (ex: Tâches, Projets, Clients, Utilisateurs Internes).

3. Définissez les clés étrangères qui permettront de reconstituer l'information sans perdre aucune donnée.

Exercice 3 : Identifier 3 erreurs de modélisation dans un schéma fourni

Le schéma proposé (Description textuelle) :

Vous auditez la base de données Airtable d'une agence immobilière. Voici la structure de leur table unique `Biens_Immobilier` :

• `Nom_du_Bien` (Texte)

• `Adresse_Complète` (Texte long)

• `Prix_Actuel` (Nombre)

• `Historique_Prix` (Texte long contenant : "Janvier: 200k, Mars: 190k, Mai: 185k")

• `Agent_Responsable` (Texte : "Jean Dupont")

• `Visites_Programmées` (Texte : "12/05 à 14h - M. Smith; 14/05 à 10h - Mme Doe")

• `Nombre_de_Pièces` (Nombre)

• `Surface_m2` (Nombre)

• `Surface_Au_Sol` (Formule : `Surface_m2 * 0.9`)

Consigne : Identifiez au moins **3 erreurs majeures** de modélisation dans cette table en vous basant sur les concepts du chapitre (Table monstre, données non structurées, snapshot vs calcul, etc.). Pour chaque erreur, proposez la correction structurelle (création de nouvelles tables, changement de type de champ, etc.).

© 2026 Oussama Ghorbel

Chapitre 6 – Airtable en profondeur : le tableur qui a appris le relationnel

Quand j'ai découvert Airtable pour la première fois, j'ai cru voir un Excel avec des couleurs. J'avais tort. Airtable n'est pas un tableur amélioré. C'est une base de données relationnelle déguisée en tableur, et cette distinction change tout.

Si vous venez du monde des feuilles de calcul, vous allez retrouver vos repères : des lignes, des colonnes, des cellules. Mais très vite, vous allez réaliser que quelque chose de fondamentalement différent se passe ici. Les lignes ne sont pas de simples entrées. Ce sont des *records*, des enregistrements structurés qui peuvent se parler entre eux, se référencer, s'enrichir mutuellement.

Dans ce chapitre, nous allons disséquer Airtable pour en faire le backbone de données de votre stack no-code. Parce qu'une application no-code, aussi belle soit-elle, ne vaut que par la qualité des données qu'elle manipule.

6.1. Bases, tables, vues, champs : anatomie d'Airtable

Commençons par le vocabulaire. Airtable organise l'information selon une hiérarchie précise que vous devez maîtriser avant d'aller plus loin.

Une base est le conteneur principal. Pensez-y comme un fichier Excel, mais en plus puissant. Une base peut contenir plusieurs tables. Par exemple, vous pourriez avoir une base "Gestion commerciale" qui regroupe vos clients, vos opportunités et vos factures.

Une table est l'équivalent d'un onglet dans un tableur, mais avec une contrainte forte : chaque ligne (record) suit exactement la même structure définie par les colonnes (champs). Vous ne pouvez pas avoir une ligne avec 5 colonnes et la suivante avec 8. Cette rigueur est ce qui fait d'Airtable une vraie base de données.

Un champ définit le type d'information stockée dans une colonne. C'est ici qu'Airtable montre toute sa puissance : contrairement à Excel où une cellule peut contenir n'importe quoi, Airtable impose un type à

© 2026 Oussama Ghorbel

chaque champ. Texte, nombre, date, case à cocher, pièce jointe, lien vers un autre enregistrement... Cette discipline structurelle est votre meilleure alliée pour éviter le chaos.

Une vue est une façon de regarder vos données sans les modifier. C'est le concept le plus élégant d'Airtable. Vous avez 500 clients dans votre table ? Créez une vue "Clients VIP" qui filtre ceux dont le chiffre d'affaires dépasse 100 000 €. Créez une vue "À relancer cette semaine" qui affiche les clients sans contact depuis 30 jours. Les données restent les mêmes, mais vous les voyez sous un angle différent.

Un record est une ligne, un enregistrement. Chaque record a un identifiant unique (que vous voyez dans l'URL quand vous l'ouvrez) et contient des valeurs pour chaque champ de la table.

Cette architecture semble simple, mais elle pose les fondations d'un système de données robuste. La clé, c'est de penser vos tables comme des entités distinctes qui vont interagir entre elles, pas comme des feuilles de calcul isolées.

6.2. Les 12 types de champs essentiels (linked records, rollups, lookups)

Airtable propose plus de 25 types de champs, mais 12 d'entre eux constituent le socle de 95 % des usages. Commençons par les plus basiques avant d'attaquer les champs relationnels qui font toute la puissance de l'outil.

Les champs fondamentaux :

• **Single line text** : du texte sur une seule ligne. Idéal pour les noms, titres, codes.

• **Long text** : du texte multiligne avec possibilité de mise en forme riche. Parfait pour les descriptions, notes, commentaires.

• **Number** : un nombre avec contrôle du format (entier, décimal, monétaire, pourcentage).

• **Date** : une date avec ou sans heure, configurable selon vos besoins de précision.

• **Checkbox** : une case à cocher, donc un booléen (vrai/faux). Simple mais terriblement efficace.

© 2026 Oussama Ghorbel

• **Single select** : une liste déroulante où l'utilisateur choisit une seule option parmi celles que vous avez définies.

• **Multiple select** : comme le précédent, mais permet de choisir plusieurs options.

• **Attachment** : permet de joindre des fichiers (images, PDF, documents) directement à un enregistrement.

Ces huit types de champs couvrent la majorité des besoins de saisie. Mais c'est avec les quatre suivants qu'Airtable devient une vraie base de données relationnelle.

Les champs relationnels :

• **Link to another record** : c'est LE champ qui change tout. Il crée une relation entre deux tables. Quand vous liez un client à une opportunité, vous ne dupliquez pas l'information. Vous créez une référence. Modifiez le nom du client, et toutes les opportunités liées voient automatiquement le nouveau nom. C'est le principe de normalisation des bases de données relationnelles, rendu accessible sans SQL.

• **Lookup** : une fois que vous avez créé un lien entre deux tables, le lookup vous permet d'afficher un champ de la table liée. Exemple : dans votre table "Opportunités", vous avez lié le client. Avec un lookup, vous pouvez afficher directement l'email du client sans avoir à le ressaisir. L'information vient de la table "Clients" et se met à jour automatiquement.

• **Rollup** : c'est un lookup avec calcul. Vous pouvez agréger des données de la table liée. Par exemple, dans votre table "Clients", vous pouvez avoir un rollup qui calcule la somme de toutes les opportunités liées, ou qui compte le nombre de factures, ou qui trouve la date de la dernière interaction. Les formules disponibles sont nombreuses : SUM, COUNT, MAX, MIN, AVERAGE, et bien d'autres.

• **Formula** : un champ calculé qui combine des valeurs d'autres champs avec des opérateurs et fonctions. Vous pouvez concaténer du texte, faire des calculs conditionnels, manipuler des dates. La syntaxe est proche de celle d'Excel, mais avec quelques spécificités

© 2026 Oussama Ghorbel

Airtable.

Ces quatre types de champs relationnels transforment Airtable d'un simple outil de saisie en un véritable système d'information. La règle d'or : ne jamais dupliquer une information. Si vous vous retrouvez à saisir le même email de client dans trois tables différentes, c'est que vous n'avez pas encore créé les bons liens.

6.3. Vues personnalisées : grille, Kanban, calendrier, galerie, formulaire

Les vues sont l'interface entre vos données brutes et la façon dont vous voulez les consommer. Airtable propose cinq types de vues principaux, chacun adapté à un usage spécifique.

Grid view (vue grille) est la vue par défaut, celle qui ressemble le plus à un tableur. C'est la vue de travail, celle où vous saisissez, modifiez, triez, filtrez en masse. Chaque utilisateur peut créer ses propres vues grille avec ses filtres et tris personnalisés sans affecter les autres. C'est aussi la vue la plus puissante pour les opérations en lot : sélectionner 50 enregistrements et modifier un champ en une seule action.

Kanban view transforme vos données en tableau de type Trello. Pour l'utiliser, vous devez avoir un champ "Single select" qui servira de colonne. Typiquement, un champ "Statut" avec les valeurs "À faire", "En cours", "Terminé". Vous voyez alors vos enregistrements comme des cartes que vous pouvez glisser-déposer d'une colonne à l'autre. Idéal pour le suivi de projets, le pipeline commercial, ou tout workflow avec des étapes claires.

Calendar view affiche vos enregistrements sur un calendrier. Il faut au moins un champ de type date (deux si vous voulez des événements avec date de début et de fin). Parfait pour visualiser des deadlines, des rendez-vous, des publications planifiées. Vous pouvez superposer plusieurs calendriers et voir d'un coup d'œil la charge de travail d'une semaine ou d'un mois.

Gallery view présente vos enregistrements sous forme de cartes visuelles. C'est la vue idéale quand vos données incluent des images (via le champ Attachment). Utilisez-la pour un catalogue produits, un

© 2026 Oussama Ghorbel

trombinoscope d'équipe, un portfolio de projets. Vous contrôlez quels champs apparaissent sur la carte et dans quel ordre.

Form view est un cas particulier : ce n'est pas une vue pour consulter des données, mais pour en collecter. Vous créez un formulaire que vous pouvez partager via un lien. Les réponses alimentent automatiquement votre table. C'est l'équivalent de Google Forms, mais directement intégré à votre base de données. Vous pouvez configurer des champs obligatoires, personnaliser le message de confirmation, et même recevoir une notification à chaque nouvelle réponse.

La puissance des vues réside dans leur combinaison. Une même table peut avoir une vue grille pour l'administrateur qui veut tout voir, une vue Kanban pour le manager qui suit l'avancement, une vue calendrier pour l'équipe qui planifie, et un formulaire pour collecter de nouvelles entrées. Les données sont les mêmes, mais chaque persona voit ce dont il a besoin.

Astuce de pro : nommez vos vues de façon explicite. "Vue par défaut" ne veut rien dire. "Mes tâches de la semaine" ou "Clients à risque" indique immédiatement l'usage. Et n'hésitez pas à créer des vues personnelles qui ne sont visibles que par vous.

6.4. Interfaces Airtable : transformer les tables en apps internes

Les interfaces sont la fonctionnalité qui a fait passer Airtable du statut de base de données à celui de plateforme de création d'applications internes. Lancées en 2021, elles permettent de construire des tableaux de bord et des applications sur mesure à partir de vos tables, sans toucher à la structure sous-jacente.

Le principe est simple : vous partez d'une page blanche et vous y ajoutez des éléments (charts, tables, formulaires, boutons) que vous connectez à vos données. Chaque élément peut être filtré, personnalisé, et lié à d'autres éléments pour créer des interactions.

Prenons un exemple concret. Vous avez une base de gestion de projets avec trois tables : Projets, Tâches, Membres d'équipe. Vous pouvez créer une interface "Tableau de bord chef de projet" avec :

© 2026 Oussama Ghorbel

- Un graphique montrant la répartition des tâches par statut
- Une liste des tâches en retard filtrée automatiquement
- Un formulaire rapide pour ajouter une nouvelle tâche
- Un sélecteur de projet qui met à jour tous les autres éléments

L'utilisateur final n'a pas besoin de comprendre la structure de vos tables. Il voit une application cohérente, adaptée à son rôle. Vous pouvez créer plusieurs interfaces pour différents personas : une pour la direction avec des KPIs, une pour les opérationnels avec des listes de tâches, une pour les clients externes avec un suivi de livraison.

Les interfaces supportent aussi des éléments interactifs comme des boutons qui déclenchent des actions (lancer une automatisation, ouvrir un formulaire, modifier un enregistrement). Vous pouvez même intégrer des scripts JavaScript pour des comportements personnalisés, ce qui rapproche Airtable du low-code.

La limite actuelle des interfaces, c'est qu'elles restent dans l'écosystème Airtable. Vous ne pouvez pas les publier comme des applications web autonomes avec votre propre domaine. Pour ça, il faudra connecter Airtable à un outil comme Softr, Glide ou WeWeb (nous y reviendrons au chapitre 11). Mais pour des outils internes, les interfaces Airtable sont souvent suffisantes et beaucoup plus rapides à développer.

6.5. Automatisations natives : déclencheurs, actions, conditions

Airtable intègre un moteur d'automatisation qui vous permet de créer des workflows sans quitter la plateforme. Le principe est celui de tous les outils d'automatisation : un déclencheur (trigger) lance une ou plusieurs actions.

Les déclencheurs disponibles :

- Quand un enregistrement est créé
- Quand un enregistrement est modifié (vous pouvez préciser quels champs surveiller)

© 2026 Oussama Ghorbel

• Quand un enregistrement atteint une date spécifique (idéal pour les rappels)

• Quand un formulaire est soumis

• À intervalle régulier (tous les jours à 9h, par exemple)

Les actions natives :

• Envoyer un email (via l'email Airtable ou votre propre adresse)

• Envoyer une notification Slack ou Microsoft Teams

• Créer ou mettre à jour un enregistrement

• Exécuter un script JavaScript

• Appeler un webhook (pour connecter avec des outils externes)

Les conditions logiques :

Vous pouvez ajouter des blocs conditionnels pour que certaines actions ne s'exécutent que si des critères sont remplis. Par exemple : "Quand une opportunité passe au statut 'Gagnée', si le montant est supérieur à 50 000 €, envoyer un email au directeur commercial, sinon juste notifier le responsable d'équipe."

Voici un cas d'usage concret que l'on peut mettre en place pour un client. Dans une table "Support client", quand un ticket est créé avec la priorité "Urgente", une automatisation se déclenche : elle envoie un message Slack au canal #support-urgent, crée une tâche dans la table "Suivi" assignée au manager, et envoie un email de confirmation au client avec un délai de réponse garanti de 2 heures. Tout ça sans intervention humaine, en moins de 3 secondes.

La limite des automatisations natives, c'est leur simplicité. Vous ne pouvez pas faire de boucles, pas de logique complexe avec plusieurs branches, pas de manipulation de tableaux. Pour des workflows plus sophistiqués, il faudra passer par Make ou Zapier (chapitre 16). Mais pour 80 % des besoins d'automatisation interne, les automatisations Airtable suffisent et ont l'avantage d'être plus rapides à configurer et à maintenir.

6.6. API et extensions : quand Airtable devient un backend

© 2026 Oussama Ghorbel

Airtable expose une API REST complète qui permet à n'importe quelle application externe de lire et écrire des données. C'est ce qui en fait un backend idéal pour des applications no-code.

L'API suit les conventions REST standards. Vous pouvez lister des enregistrements, en créer, les modifier, les supprimer, le tout avec des requêtes HTTP simples. Chaque table a son propre endpoint, et vous pouvez filtrer, trier, paginer les résultats.

Exemple concret : vous construisez une application mobile avec Glide. Au lieu d'utiliser Google Sheets comme backend (limité en performance), vous connectez Glide à Airtable via l'API. Vos utilisateurs consultent et modifient des données stockées dans Airtable, bénéficiant de la puissance relationnelle et des automatisations natives.

Pour faciliter l'intégration, Airtable propose aussi des "extensions", des modules développés par la communauté ou par Airtable qui ajoutent des fonctionnalités. On trouve des extensions pour :

- Synchroniser avec Google Calendar
- Importer/exporter des données CSV
- Générer des documents PDF à partir de templates
- Créer des diagrammes de Gantt
- Connecter avec Stripe pour gérer des paiements

Vous pouvez aussi développer vos propres extensions en JavaScript si vous avez des besoins spécifiques. C'est du code, mais du code qui s'exécute dans le contexte d'Airtable, avec accès direct à vos données.

L'API a des limites de taux (5 requêtes par seconde par base), ce qui est suffisant pour la majorité des usages mais peut devenir contraignant pour des applications à fort trafic. Dans ce cas, vous devrez soit optimiser vos requêtes (batch operations), soit passer par un backend intermédiaire comme Xano ou Supabase qui servira de cache.

6.7. Limites (taille de base, performances, pricing) et contournements

Airtable est puissant, mais il n'est pas parfait. Connaître ses limites vous évitera des déconvenues et vous aidera à décider quand l'utiliser et

© 2026 Oussama Ghorbel

quand passer à autre chose.

Limite de taille des bases :

Chaque base a un nombre maximum d'enregistrements selon votre plan :

- Plan gratuit : 1 000 enregistrements par base
- Plan Team : 50 000 enregistrements par base
- Plan Business : 100 000 enregistrements par base
- Plan Enterprise : négociable

C'est souvent la limite la plus frustrante. Si vous gérez un catalogue de 200 000 produits, Airtable n'est pas adapté. Vous devrez soit archiver les anciens enregistrements dans une autre base, soit passer à une vraie base de données comme PostgreSQL (via Supabase).

Limites de performance :

Airtable ralentit quand vous avez beaucoup de champs calculés (formulas, rollups, lookups) sur de gros volumes. Chaque modification déclenche des recalculs en cascade. Si vous avez 50 000 enregistrements avec 10 rollups chacun, attendez-vous à des latences.

Contournement : minimisez les champs calculés, utilisez des automatisations pour figer des valeurs plutôt que de les recalculer en permanence. Par exemple, au lieu d'un rollup qui calcule le total des commandes en temps réel, créez une automatisation qui met à jour un champ "Total commandes" à chaque nouvelle commande.

Pricing :

Airtable est cher comparé à d'autres solutions. Le plan Team coûte 20 $ par utilisateur par mois. Si vous avez une équipe de 20 personnes qui ont besoin d'accéder à Airtable, ça fait 400 $ par mois, juste pour l'outil de données.

Contournement : limitez le nombre d'utilisateurs avec accès complet à Airtable. Pour les autres, créez des interfaces en lecture seule (qui ne consomment pas de licence) ou construisez des applications frontales avec Softr ou Glide qui se connectent à Airtable via l'API. Ainsi, seul l'administrateur a une licence Airtable, et les 50 utilisateurs finaux

© 2026 Oussama Ghorbel

accèdent via l'app frontale.

Pas de jointures complexes :

Airtable gère bien les relations simples (one-to-many, many-to-many), mais ne permet pas de requêtes SQL complexes avec multiples jointures et agrégations imbriquées. Si vous avez besoin de rapports analytiques poussés, vous devrez exporter vos données vers un outil de BI comme Metabase ou Google Data Studio.

Pas de transactions atomiques :

Quand vous modifiez plusieurs enregistrements via l'API, si une modification échoue, les autres ne sont pas annulées. Pour des opérations critiques qui nécessitent une cohérence transactionnelle, vous devrez implémenter votre propre logique de rollback ou passer par une base de données traditionnelle.

Malgré ces limites, Airtable reste mon choix par défaut pour 80 % des projets no-code. Sa combinaison unique de simplicité d'usage, de puissance relationnelle et d'intégrations natives en fait un backbone de données difficile à battre. Les 20 % restants, ce sont des cas où les volumes explosent ou où les besoins analytiques deviennent trop complexes. Dans ces cas-là, il est temps de migrer vers Supabase ou Xano.

Exercices pratiques

Maintenant, c'est à vous. Ces trois exercices vont consolider tout ce que nous venons de voir. Ne les sautez pas. La théorie ne vaut rien sans la pratique.

Exercice 1 : Construire une base CRM complète en 1 heure

Objectif : Créer un CRM fonctionnel avec trois tables liées.

Étapes :

1. Créez une base "Mon CRM" avec trois tables : Contacts, Entreprises, Activités

2. Dans Entreprises, ajoutez les champs : Nom (single line text), Secteur (single select), Site web (URL), Taille (single select : 1-10, 11-50, 51-200, 200+)

© 2026 Oussama Ghorbel

3. Dans Contacts, ajoutez : Prénom, Nom, Email, Téléphone, Poste, et un lien vers Entreprises

4. Dans Activités, ajoutez : Type (single select : Appel, Email, Réunion, Déjeuner), Date, Notes, et un lien vers Contacts

5. Créez un rollup dans Entreprises qui compte le nombre de contacts liés

6. Créez un rollup dans Contacts qui affiche la date de la dernière activité

7. Créez une vue Kanban dans Activités groupée par Type

8. Créez une vue Calendar dans Activités

Livrable : Une base CRM où vous pouvez voir d'un coup d'œil quelles entreprises ont le plus de contacts, quels contacts n'ont pas eu d'activité récente, et visualiser votre agenda de rendez-vous.

Exercice 2 : Créer une Interface pour le suivi de projets avec 3 vues

Objectif : Construire un tableau de bord de gestion de projets.

Prérequis : Avoir une base avec au moins une table "Projets" (Nom, Statut, Responsable, Date début, Date fin, Budget) et une table "Tâches" (Titre, Projet lié, Assigné à, Statut, Priorité).

Étapes :

1. Créez une nouvelle Interface "Suivi de projets"

2. Page 1 - Vue d'ensemble : ajoutez un graphique montrant la répartition des projets par statut, et une liste des 5 projets les plus récents

3. Page 2 - Mes tâches : ajoutez un filtre qui ne montre que les tâches assignées à l'utilisateur connecté, avec une vue Kanban par statut

4. Page 3 - Ajout rapide : créez un formulaire pour ajouter une nouvelle tâche en 3 clics

Livrable : Une interface que vous pourriez déployer demain matin pour votre équipe, avec trois vues complémentaires adaptées à différents besoins.

© 2026 Oussama Ghorbel

Exercice 3 : Configurer une automatisation qui notifie par email sur changement de statut

Objectif : Automatiser une notification contextuelle.

Étapes :

1. Dans votre table Projets, assurez-vous d'avoir un champ Statut (single select) avec au moins les valeurs "En cours", "En attente", "Terminé"
2. Créez une automatisation avec le déclencheur "Quand un enregistrement est modifié", en surveillant uniquement le champ Statut
3. Ajoutez une condition : si le nouveau statut est "En attente"
4. Ajoutez une action "Envoyer un email" à l'adresse du responsable du projet (vous devrez ajouter un champ Email dans votre table)
5. Personnalisez l'objet : "Projet [Nom du projet] en attente de validation"
6. Personnalisez le corps avec les détails du projet et un lien vers l'enregistrement
7. Testez l'automatisation en changeant le statut d'un projet

Livrable : Un workflow automatisé qui alerte immédiatement le responsable quand un projet nécessite son attention, sans qu'il ait à surveiller manuellement la base.

Ces trois exercices couvrent les fondamentaux : modélisation relationnelle, visualisation, et automatisation. Une fois maîtrisés, vous aurez les bases pour construire des systèmes beaucoup plus complexes. Airtable devient alors ce que vous en faites : un simple tableur coloré, ou le système nerveux central de votre organisation.

© 2026 Oussama Ghorbel

Chapitre 7 – Notion, Coda, SmartSuite : quand le document devient base de données

Au chapitre précédent, nous avons vu comment Airtable a transformé le tableur en base de données relationnelle. Il existe une autre famille d'outils, tout aussi légitime, qui part d'un point de départ radicalement différent : le **document**. Là où Airtable vous demande de penser « lignes et colonnes dès la première minute », Notion, Coda et SmartSuite vous demandent d'abord d'écrire, de structurer, de raconter — puis, presque par magie, de transformer ce que vous avez écrit en données interrogeables.

Cette approche « document-first » a une conséquence fondamentale : elle rend l'outil immédiatement familier pour n'importe quel collaborateur habitué à traitement de texte, tout en cachant une puissance relationnelle réelle derrière une interface rassurante. Mais elle a aussi un prix : des compromis sur la rigueur des données, la performance à grande échelle et la profondeur des vues.

Ce chapitre a un objectif simple : vous aider à comprendre où se situent les forces et les limites de ces bases documentaires, et à savoir lesquelles choisir — et surtout, *quand* les préférer à un vrai SGBD no-code comme Airtable.

7.1. Notion : blocs, bases, relations et rollups

Notion est devenu, en quelques années, l'outil le plus utilisé dans le monde du travail knowledge-first. Son concept de départ est le **bloc** : chaque élément d'une page — un paragraphe, un titre, une image, une liste à cocher, un tableau, une vue Kanban — est un bloc indépendant que l'on peut déplacer, dupliquer, transformer en un autre type de bloc. Le slogan mental à retenir est : *tout est bloc*.

Sur ce socle s'ajoute la notion de **base de données** (appelée aujourd'hui « base » ou « database »). Une base Notion est une collection de pages partageant les mêmes propriétés. Chaque ligne d'un tableau devient une page complète : vous pouvez y écrire un texte long, y insérer

© 2026 Oussama Ghorbel

des images, y ajouter des sous-pages. C'est exactement l'inverse de la logique Airtable, où la cellule est atomique et la fiche reste légère.

Les propriétés : le vocabulaire de la donnée

Notion propose une gamme de propriétés riche pour un outil documentaire :

- **Texte, nombre, date, URL, e-mail, téléphone** : les basiques.
- **Sélection** et **sélection multiple** : des listes de valeurs prédéfinies, équivalent aux *single select* et *multi select* d'Airtable.
- **Personne** : référence aux membres du workspace, utile pour attribuer des tâches.
- **Statut** : une propriété dédiée avec trois colonnes (À faire, En cours, Terminé), pratique pour les Kanban.
- **Relation** : le cœur du relationnel, nous y venons.
- **Rollup** : l'agrégation, l'autre pilier.
- **Formule** : un langage d'expressions maison, correct mais plus limité que celui de Coda.
- **Fichier, case à cocher, nombre de références, créé le/créé par, modifié le/modifié par** : propriétés système très utiles pour la gouvernance.

Relations et rollups : le relationnel sans SQL

Une **relation** est un lien entre deux bases. Si vous avez une base « Projets » et une base « Tâches », chaque tâche peut pointer vers son projet. Concrètement, vous créez une propriété de type Relation dans « Tâches », vous choisissez la base « Projets », et Notion crée en miroir une propriété dans « Projets » qui liste toutes les tâches liées. Ce lien est bidirectionnel : modifier un sens modifie l'autre.

Le **rollup** permet ensuite d'agréger les valeurs d'une relation. Exemple : dans « Projets », un rollup qui compte les tâches liées, un autre qui calcule le nombre de tâches au statut « En cours », un troisième qui additionne les estimations d'heures. Vous obtenez ainsi un tableau de bord projet sans écrire une ligne de code.

© 2026 Oussama Ghorbel

À ces deux briques s'ajoutent les **vues** : table, Kanban, calendrier, liste, chronologie (timeline), galerie, et depuis peu une vue « charge de travail » (workload). Toutes les vues portent sur les mêmes données ; filtrer, trier et grouper dans une vue n'affecte pas les autres.

Les forces de Notion

La première force est l'**adoption**. Notion ne fait pas peur. Un rédacteur, un chef de produit, un responsable RH s'y sentent chez eux parce que l'interface ressemble à un document. La seconde force est la **profondeur de contenu** : vos données vivent à côté de votre prose. Une page « Client Acme » peut contenir à la fois la fiche CRM structurée, le compte rendu de la dernière réunion, et la checklist de onboarding. Aucun outil tableaire ne permet cela aussi naturellement. La troisième force est l'**écosystème de templates** et une communauté massive qui produit modèles, hacks et tutoriels.

Les faiblesses de Notion

Soyons lucides. Notion souffre de **limites de performance** : au-delà d'une certaine volumétrie (généralement quelques milliers de lignes par base, selon la complexité des relations), les vues ralentissent, la synchronisation devient hésitante. Les **formules**, bien qu'améliorées, restent moins puissantes que celles de Coda ou d'Airtable : pas de vraies fonctions d'agrégation conditionnelles complexes sans bidouiller. Le **multi-select de relation** géré de façon imperfect (une page liée à plusieurs pages de l'autre base) crée des asymétries qui déroutent les débutants. Enfin, l'**automatisation native** est récente et encore superficielle comparée à Make ou Zapier.

Autre point souvent sous-estimé : Notion n'est **pas un SGBD**. Il n'y a pas de contrainte d'intégrité forte, peu de contrôle sur les types de données (un champ « nombre » acceptera du texte), et pas de validation à la saisie. C'est un choix de conception : Notion privilégie la souplesse sur la rigueur. Pour un wiki, une gestion de projet, un CRM de petite équipe, c'est acceptable. Pour tenir un stock, des transactions ou des données réglementées, ce ne sera pas suffisant sans couche externe.

7.2. Coda : formules puissantes et packs d'intégration

© 2026 Oussama Ghorbel

Si Notion est un document qui a découvert les données, **Coda** est un tableur qui a grandi jusqu'à devenir une application. Le pitch de Coda est le suivant : un seul document (un « doc ») peut à la fois être votre interface, votre base de données et votre moteur de logique.

La structure d'un doc Coda

Un doc Coda contient des **pages**, des **sections**, et surtout des **tables**. Une table Coda ressemble à un tableau, mais chaque colonne peut être de plusieurs types : texte, nombre, date, personne, et surtout **référence** (lien vers une autre table) et **formule**. Contrairement à Notion, la formule n'est pas une propriété parmi d'autres : elle est partout, et elle est de premier ordre.

Le langage de formule Coda

Coda dispose d'un langage d'expressions riche et cohérent, souvent comparé à celui de Google Sheets enrichi. On y trouve :

• Les fonctions classiques : `SUM`, `COUNT`, `AVERAGE`, `IF`, `SWITCH`.

• Les opérations sur les listes : `FILTER`, `MAP`, `SORT`, `UNIQUE`, `CONTAINS`, `HAS`.

• Les fonctions temporelles : `TODAY`, `DATEADD`, `DATEDIF`, `WEEKDAY`.

• Les fonctions de texte : `FIND`, `REPLACE`, `LEFT`, `SPLIT`, `JOIN`.

• Le « dot notation » qui rend tout très lisible : `Tasks.Filter(Status = "Done").Count()`.

Cette dernière ligne illustre l'esprit de Coda : on filtre une table comme on filtrerait une collection, puis on compte. Pas de jointure implicite, pas de sous-requête à assembler à la main. Pour quelqu'un qui maîtrise un peu les formules de tableur, la courbe d'apprentissage est rapide et gratifiante.

Les formules Coda peuvent être utilisées dans des **colonnes calculées**, dans des **filtres de vue**, dans des **règles de mise en forme conditionnelle**, et dans des **boutons**.

© 2026 Oussama Ghorbel

Les boutons et la logique applicative

Coda introduit un concept absent de Notion à ses débuts : le **bouton**. Un bouton peut exécuter une action : modifier des lignes, lancer une automatisation, ouvrir une page, appeler un pack. C'est ce qui permet de transformer un doc en véritable mini-application avec des interactions métier : « Marquer comme validé », « Dupliquer la commande », « Envoyer au responsable ».

À cela s'ajoutent les **automatisations intégrées** (triggers sur modification de ligne, sur date, sur format libre) et les **Packs**.

Les Packs : l'intégration native

Un **Pack** est un connecteur prêt à l'emploi. Coda propose des dizaines de packs : Gmail, Google Calendar, Slack, Jira, Salesforce, GitHub, OpenAI, et bien d'autres. Un pack expose des **formules** (pour lire des données : `Gmail.Threads(...)`) et des **actions** (pour écrire : `Gmail.Send(...)`). Résultat : vous pouvez interroger votre boîte mail directement dans une formule, ou envoyer un e-mail depuis un bouton, sans passer par un orchestrateur externe.

C'est un avantage concurrentiel réel : Coda réduit le besoin d'un Zapier ou d'un Make pour les scénarios courants. En revanche, l'écosystème de packs, bien que solide, reste moins étendu que celui de Zapier, et certains packs sont réservés aux plans payants.

Les forces et faiblesses de Coda

Forces : la puissance logique (formules, boutons, automatisations), la capacité à tenir des données relationnelles complexes dans un seul doc, la philosophie « tout dans un endroit » qui réduit la fragmentation des outils, et une excellente gestion des **vues filtrées et contrôlées** (on peut donner à un invité une section précise du doc, pas le doc entier).

Faiblesses : la courbe d'apprentissage est plus raide que Notion — un non-technique sera plus vite perdu. L'interface, bien que soignée, trahit une logique d'ingénieur. L'écosystème de templates est plus restreint. La **collaboration en temps réel** est bonne mais l'expérience perçue est parfois moins fluide que Notion sur les pages de long texte. Et surtout : Coda reste moins connu, ce qui pénalise l'adoption dans des équipes peu expérimentées en no-code.

© 2026 Oussama Ghorbel

Un point méthodologique : beaucoup d'équipes choisissent Coda *après* avoir buté sur les limites de formules d'Airtable ou de Notion. C'est souvent le bon ordre : commencer simple, puis migrer quand la complexité l'exige.

7.3. SmartSuite : le concurrent orienté processus

SmartSuite est le plus jeune des trois, et il le revendique. Né au début des années 2020, il se positionne explicitement comme l'outil « entre Airtable et les outils d'entreprise », avec une obsession : les **processus métier**.

Une philosophie différente

SmartSuite reprend le modèle base de données riche d'Airtable (champs relationnels, vues multiples, formulaires, interfaces) mais y ajoute deux dimensions fortes :

1. **La gestion de processus** : des vues dédiées aux flux de travail, des statuts avancés, des mécaniques de validation et d'assignation pensées pour des équipes opérationnelles (qualité, conformité, ressources humaines, industrie).

2. **La maturité « entreprise »** : permissions très fines par dossier et par champ, journaux d'audit, champs de type **signature**, **géolocalisation**, **fiche de temps**, **relation hiérarchique**.

Les spécificités notables

• **Plus de 20 types de champs**, dont certains introuvables chez Notion : signature électronique, code-barres/QR, géolocalisation, durée, progression, objet JSON.

• **Des vues riches** : table, Kanban, calendrier, timeline, galerie, carte (géographique), fiche (record), formulaire, et des vues de processus.

• **Les interfaces (Interfaces)** : comme Airtable, SmartSuite sépare les données des écrans. On construit des pages sur mesure pour les différents profils (un écran « opérateur », un écran « manager », un écran « administrateur ») sans exposer la structure de la base.

• **La hiérarchie de dossiers** : les solutions sont organisées en workspaces, dossiers et applications, ce qui donne une gouvernance

© 2026 Oussama Ghorbel

claire pour les grands comptes.

• **Les notifications et automatisations natives**, plus abouties que dans Notion : déclencheurs sur changement de champ, délais, conditions, actions multiples.

Forces et faiblesses de SmartSuite

Forces : un rapport fonctionnalités/prix souvent jugé très compétitif ; une prise en main plus rapide que Coda pour des processus classiques ; une couverture de cas d'usage « opérationnel » excellente (maintenance, contrôle qualité, gestion de dossiers clients, conformité) ; une communauté en croissance et une équipe produit réactive.

Faiblesses : l'écosystème de templates et de formations est moins fourni qu'Airtable ou Notion ; l'intégration dans le quotidien d'une équipe « bureau » est moins naturelle que Notion (moins de place pour la prose libre) ; la maturité de certaines fonctionnalités avancées reste en rattrapage ; et la notoriété reste limitée, ce qui peut poser problème en cas de recrutement ou de transmission.

SmartSuite brille quand vous avez un **processus récurrent, structuré, avec des responsabilités claires** et un besoin de traçabilité. Il brille moins quand vous voulez un espace de réflexion, un wiki vivant ou un document hybride.

7.4. Comparatif d'usage : wiki vs CRM vs gestion de projet

La question n'est pas « lequel est le meilleur ? » mais « lequel correspond à votre usage dominant ? ». Voici une grille de lecture couramment utilisée.

Cas d'usage 1 : le wiki et la base de connaissances

Vainqueur : Notion.

Un wiki a besoin de pages longues, de hiérarchie de dossiers, de liens croisés, de mise en page soignée, de recherche full-text performante, et d'une expérience d'écriture agréable. Notion excelle ici. Coda peut faire un wiki, mais son ADN reste le tableau, et l'écriture longue y est moins confortable. SmartSuite est mal adapté : ses pages sont d'abord des fiches, pas des documents.

© 2026 Oussama Ghorbel

Un wiki Notion devient plus puissant quand vous liez vos pages à des bases : par exemple, chaque page « Procédure » liée à une propriété « Propriétaire » qui référence la base « Équipe », et un rollup qui affiche le manager de l'auteur. Vous obtenez un wiki *structuré*, pas juste une pile de fichiers.

Cas d'usage 2 : le CRM (suivi commercial)

Vainqueur : dépend du volume et de la rigueur.

• **Notion** convient pour un CRM individuel ou une petite équipe qui veut centraliser notes et tâches avec les comptes rendus. Pipeline Kanban natif, relation Contacts <-> Entreprises <-> Opportunités, rollups de montant. Simple, visuel, rapidement opérationnel.

• **Coda** convient quand le CRM doit être *calculé* : scoring automatique, calcul de probabilité pondérée, relances conditionnelles, e-mails déclenchés par bouton. La puissance des formules change la donne.

• **SmartSuite** convient pour un CRM d'équipe structuré avec processus de validation, champs de conformité, et besoins de permissions fines.

• **Airtable** reste souvent le meilleur compromis si le CRM doit servir de source de vérité pour d'autres outils (via API et automatisations), grâce à sa structure très rigoureuse et son écosystème mature.

Mon conseil : si votre CRM doit alimenter un outil externe (facturation, marketing automation), privilégiez la rigueur structurelle. Si votre CRM doit vivre *à côté* de vos notes et de votre documentation, Notion sera plus adopté.

Cas d'usage 3 : la gestion de projet

Vainqueur : à trois, selon la nature du projet.

• Projets **créatifs ou documentaires** (rédaction, produit, stratégie) -> **Notion** : les tâches sont liées à des briefs, des comptes rendus, des specs.

• Projets **logiques et calculés** (planification avec charges, dépendances complexes, reporting automatique) -> **Coda**.

© 2026 Oussama Ghorbel

• Projets **opérationnels et répétitifs** (industrie, maintenance, RH, qualité) avec besoins de signatures, de preuves et de traçabilité -> **SmartSuite**.

Un tableau comparatif synthétique, à retenir comme heuristique :

Critère	Notion	Coda	SmartSuite
Expérience d'écriture	Excellente	Bonne	Moyenne
Puissance des formules	Moyenne	Excellente	Bonne
Types de champs métier	Corrects	Corrects	Très riches
Vues multiples	Bonnes	Bonnes	Très bonnes
Automatisations natives	Basiques	Bonnes	Bonnes
Intégrations (packs/API)	Correctes (API)	Très bonnes (Packs)	Bonnes
Permissions fines	Moyennes	Bonnes	Très bonnes
Performance à grande échelle	Faible	Moyenne	Bonne
Courbe d'apprentissage	Douce	Raide	Douce à moyenne
Idéal pour	Wiki, docs, PME	Mini-apps, logique	Processus, opérations

7.5. Notion API et Notion Automations : nouvelles possibilités

Longtemps critiqué pour son fermeture, Notion a considérablement ouvert son écosystème. Deux briques méritent d'être connues : **l'API officielle** et les **automatisations natives**.

L'API Notion

L'API REST officielle permet de **lire et écrire** les pages et bases de données de votre workspace depuis l'extérieur. Concrètement, vous pouvez :

• **Créer une page** dans une base à partir d'un événement externe (un formulaire web, un achat, un ticket support).

© 2026 Oussama Ghorbel

• **Rechercher des pages** et lire leurs propriétés pour alimenter un autre système.

• **Mettre à jour une propriété** (par exemple, passer un statut à « Terminé » depuis votre outil métier).

• **Déléguer l'accès** à une intégration : vous connectez une app (Zapier, Make, n8n, ou du code maison) à un espace précis, sans partager vos identifiants.

Points de vigilance : l'API travaille avec des **identifiants d'objets** (UUID) qu'il faut gérer soigneusement ; le **rate limiting** existe et il faut prévoir de la mise en file d'attente pour les gros volumes ; la **recherche de pages** repose sur des filtres et non sur du plein texte puissant ; et surtout, l'API reflète la souplesse de Notion : elle ne vous protège pas des incohérences de données que vous avez vous-même créées.

Un pattern très utile : utiliser un **identifiant stable** (un champ « Référence » que vous maîtrisez) plutôt que de dépendre des IDs Notion dans vos autres systèmes. Cela rend les migrations et les synchronisations bien plus robustes.

Notion Automations

Depuis peu, Notion embarque des **automatisations** directement dans les bases : déclencheurs (modification d'une propriété, ajout d'une page, arrivée d'une date, soumission d'un formulaire) associés à des actions (modifier des propriétés, ajouter une page dans une autre base, envoyer une notification, démarrer une intégration tierce).

Cela couvre les cas simples sans quitter l'outil : « Quand le statut passe à Validé, assigner à la personne responsable et ajouter une entrée dans la base Journal ». Pour les cas avancés — appeler une IA, interroger une API externe, traiter des lots — il faut passer par Make, Zapier, n8n ou du code.

Et l'IA ?

Notion a intégré des fonctions d'IA générative (résumés, traductions, extraction d'informations, réécriture) directement dans les pages. L'intérêt pour un no-code builder : faire de l'**extraction structurée** (résumer un compte rendu en trois champs propriétés) ou de la **génération assistée** de contenu. Nous y reviendrons en détail au chapitre

© 2026 Oussama Ghorbel

21 sur l'IA. Pour l'instant, retenez que l'IA de Notion est un accélérateur de contenu, pas un moteur de données : elle vous aide à écrire, moins à structurer avec rigueur.

7.6. Structurer un workspace d'équipe qui tient dans le temps

La plupart des échecs en no-code ne sont pas techniques : ils sont **organisationnels**. Un workspace Notion ou Coda commence toujours bien — quelques pages, un tableau de tâches — et finit souvent en cimetière de bases orphelines, de doublons et de « où est-ce qu'on range ça ? ». Voici les principes à appliquer pour construire un workspace durable.

1. Séparer l'espace de travail de l'espace documentaire

Organisez votre sidebar en deux zones claires :

• **Espace de référence** (stable) : wiki, procédures, organigramme, glossaire, base de connaissances.

• **Espace opérationnel** (évolutif) : projets en cours, CRM, tâches, rapports.

Cette séparation évite le syndrome de la page « Tableau de bord » qui accumule incohéremment les deux logiques. On doit pouvoir dire d'un élément : « c'est une référence » ou « c'est de l'opérationnel ».

2. Une source de vérité par type de donnée

Règle d'or : **une donnée, un endroit**. Si les personnes sont dans la base « Équipe », alors chaque autre base qui a besoin d'un responsable référence cette base — on ne recopie jamais le nom d'un collaborateur dans cinq endroins. Les doublons sont le premier facteur de corruption des bases documentaires, précisément parce qu'aucune contrainte d'intégrité ne les empêche.

Appliquez la même logique aux projets, aux clients et aux statuts. Les listes de valeurs (statuts, catégories, types) doivent être définies une fois et réutilisées partout.

3. Nommer avec discipline

© 2026 Oussama Ghorbel

Les noms de bases et de propriétés sont un investissement. Privilégiez des noms courts, stables et sans ambiguïté : « Tâches » plutôt que « Table Tâches V2 Finale ». Évitez les abréviations obscures. Définissez une convention pour les propriétés (par exemple, commencer par un préfixe pour les propriétés calculées : `Calc_`), et documentez-la dans une page « Convention du workspace ».

Pensez aussi à l'**avenir de la migration** : des noms propres et des identifiants stables facilitent l'export et l'intégration avec d'autres outils.

4. Concevoir pour les rôles, pas pour vous

Un workspace qui tient dans le temps est un workspace que les autres savent utiliser. Identifiez vos profils (administrateur, contributeur, lecteur, invité) et donnez à chacun :

• une **page d'accueil** personnalisée avec ses raccourcis,

• une **documentation courte** de ce qu'il doit faire (pas un manuel de 40 pages),

• des **vues filtrées** préparées (chaque rôle ne doit voir que ce qui le concerne).

Dans Notion, utilisez les permissions de page et les vues de formulaire ; dans Coda, les sections restreintes ; dans SmartSuite, les permissions par dossier et par champ.

5. Prévoir la vie des données : archivage et revues

Une base qui grossit indéfiniment devient illisible. Mettez en place :

• un **champ « Archivé le »** et une vue « Historique » masquée par défaut ;

• une **revue mensuelle** de 30 minutes : quelles bases sont mortes ? Quels doublons ? Quels statuts abandonnés ?

• un **journal des changements** (une simple base « Changelog ») où vous notez les évolutions majeures de structure. Ce journal vous sauvera la vie le jour où un collaborateur demandera « pourquoi ce champ existe ».

6. Documenter la structure elle-même

© 2026 Oussama Ghorbel

Créez une page « Cartographie du workspace » qui liste chaque base, son objectif, ses propriétés clés, ses relations et son propriétaire. Une ligne par base suffit. Mettez à jour cette page à chaque création ou suppression de base. C'est la garantie qu'un nouveau collaborateur — ou un futur repreneur du projet, nous y viendrons au chapitre 24 — pourra comprendre l'ensemble en quinze minutes.

7. Résister à la tentation du tout-faire

La plus grande menace pour un workspace documentaire, c'est l'ambition débordante. Notion et Coda rendent tout *possible* ; encore faut-il que ce soit *utile*. Commencez avec trois bases maximum, faites-les vivre deux mois, mesurez l'adoption, puis étendez. Un espace de travail sobre et utilisé vaut toujours mieux qu'une cathédrale no-code que personne n'ouvre.

Exercices

Exercice 1 — Construire un wiki d'équipe avec base de connaissances liée aux auteurs

Objectif : pratiquer les relations et rollups dans Notion, et structurer un wiki durable.

1. Créez une base **Équipe** avec les propriétés : Nom (texte), Rôle (sélection), Manager (relation vers Équipe elle-même, auto-référence), Disponibilité (sélection : Disponible / Occupé / Absent).

2. Créez une base **Articles** avec : Titre (texte), Auteur (relation vers Équipe), Catégorie (sélection), Dernière mise à jour (date, modifiée automatiquement), Statut (Brouillon / Publié / À revoir), Nombre de contributions de l'auteur (rollup : compte les articles liés à chaque auteur, affiché côté Équipe).

3. Créez une page d'accueil **Wiki** contenant : une vue galerie des articles au statut « Publié », groupée par Catégorie, triée par « Dernière mise à jour » décroissante.

4. Ajoutez une vue **Par auteur** (groupe par Auteur) pour visualiser la charge rédactionnelle.

© 2026 Oussama Ghorbel

5. Créez une page **Procédure de rédaction** qui référence (via une relation) les articles concernés par la procédure.

6. Documentez dans une page « Convention » : les noms des bases, le sens de chaque statut, et qui est responsable de la maintenance de chaque base.

Pour aller plus loin : ajoutez un rollup dans Équipe qui affiche le nombre d'articles « À revoir » de chaque auteur, et une vue filtre « Charge de rédaction ».

Exercice 2 — Créer un tracker de recrutement avec pipeline Kanban dans Coda

Objectif : exploiter les formules, les vues Kanban et les boutons de Coda.

1. Créez un doc avec une table **Candidats** : Nom, Poste (relation vers table Postes), Statut (Nouveau / Entretien 1 / Entretien 2 / Offre / Accepté / Refusé), Source, Date de candidation, Score (nombre), Notes (texte long), Jours depuis candidation (formule : `` `TODAY() - [Date de candidation]` ``).

2. Créez une table **Postes** : Intitulé, Responsable, Budget, et une formule qui compte les candidats liés (`` `ThisRow.[Candidats].Count()` ``), ainsi qu'un compte des candidats au statut « Offre ».

3. Ajoutez une vue **Kanban** groupée par Statut, filtrée sur les postes ouverts.

4. Ajoutez une colonne calculée **Priorité** : une formule `SWITCH` qui retourne « Haute » si Score >= 8 et Jours depuis candidation > 7, « Moyenne » sinon — pour repérer les dossiers qui dorment.

5. Créez un **bouton** « Passer à l'entretien 1 » sur chaque ligne, qui modifie le statut et incrémente un compteur d'étapes.

6. Ajoutez une formule ou un pack pour notifier un responsable (via Slack ou e-mail) lorsqu'un candidat atteint « Offre ».

7. Concevez une section **Tableau de bord** : nombre de candidats par poste, délai moyen par étape, taux de conversion entre statuts.

© 2026 Oussama Ghorbel

Pour aller plus loin : ajoutez une table **Entretiens** liée aux Candidats, avec date, intervenants et note globale, puis un rollup côté Candidats affichant la moyenne des notes d'entretien.

Exercice 3 — Migrer une base Airtable simple vers Notion (et mesurer les pertes)

Objectif : comprendre concrètement les compromis d'un passage d'une base rigoureuse vers un outil documentaire.

1. Prenez une base Airtable simple que vous connaissez (ou recréez-en une) : clients, contacts, opportunités, avec au moins une relation et un rollup, un champ « montant » et un champ « statut » à liste contrôlée.

2. Exportez-la en CSV. Créez dans Notion trois bases correspondantes, avec des propriétés équivalentes.

3. Importez les données (via copier-coller ou import CSV dans une base, puis réaffectation des propriétés). Recréez manuellement les relations et les rollups.

4. **Mesurez les pertes** dans un tableau Notion dédié, avec ces colonnes : Fonctionnalité Airtable / Équivalent Notion / Perte identifiée / Contournement possible / Impact (Faible / Moyen / Fort).

Portez attention en particulier à :

• la validation des données à la saisie (types stricts, listes contrôlées strictes) ;

• les vues de type « group by with summary » complexes ;

• les types de champs spécialisés (montant avec devise, barcode, etc.) ;

• la vitesse de chargement quand la base atteint plusieurs milliers de lignes ;

• la facilité de re-synchronisation vers un outil externe via API ;

• les formules conditionnelles multi-conditions.

5. Rédigez une conclusion d'une page : dans quel cas la migration est-elle **gagnante** (besoin de pages riches, collaboration rédactionnelle, adoption) ? Dans quel cas est-elle **perdante**

© 2026 Oussama Ghorbel

(intégrité, volumétrie, intégrations) ?

6. Formulez une recommandation : pour ce cas d'usage précis, rester sur Airtable, migrer sur Notion, ou adopter une architecture hybride (Airtable comme source de vérité, Notion comme couche de lecture et de documentation) ?

Pour aller plus loin : tentez l'inverse — exporter vers Coda — et comparez les pertes. Vous découvrirez que chaque migration n'est pas un simple transfert de données, mais un **changement de modèle mental**.

© 2026 Oussama Ghorbel

Chapitre 8 – Bases de données externes : Xano, Supabase, Google Sheets comme backend

Jusqu'ici, nous avons construit nos applications et nos processus avec des données logées dans les outils eux-mêmes : des tables Airtable derrière un Softr, une base Bubble derrière des workflows, une base Notion derrière un script Make. Cette approche est excellente pour démarrer. Elle devient vite un plafond. Ce chapitre est celui du passage de seuil : le moment où l'on décide de sortir les données de l'application pour les confier à un véritable backend — et le moment où l'on mesure ce que ce choix apporte, ce qu'il coûte, et ce qu'on doit maîtriser pour ne pas échanger une facilité contre une faille.

L'objectif est concret : savoir **quand** externaliser, **vers quoi**, et **comment** sécuriser ce qui n'est plus protégé par la bulle rassurante d'un outil tout-en-un.

8.1. Pourquoi sortir les données d'Airtable / Bubble : volumétrie, coût, latence

Le premier signal : la volumétrie

Chaque plateforme no-code a des limites explicites ou implicites sur le volume de données qu'elle manipule confortablement. Airtable, par exemple, publie des limites de lignes par base selon les plans — de quelques milliers de lignes en gratuit à quelques centaines de milliers en payant — et surtout des limites de **requêtes API par minute**. Bubble impose des « workflow units » et limite ce que ses suivi de recherche (*search*, *filtered list*) peuvent faire au-delà d'un certain volume. Ces limites ne sont pas des défauts : elles reflètent le modèle économique de l'outil. Mais elles deviennent un problème quand votre usage ressemble à celui d'une vraie application métier.

Trois symptômes apparaissent alors :

• **Les vues deviennent lentes** : afficher une liste de 5 000 lignes avec filtres croisés commence à prendre des secondes, puis des

© 2026 Oussama Ghorbel

dizaines de secondes.

• **Les synchronisations se télescopent** : un Make qui rapatrie 10 000 lignes Airtable toutes les heures consomme votre quota d'operations et ralentit tout le reste.

• **Les recherches « tout » deviennent impossibles** : chercher un client parmi 50 000 sans index revient à balayer la table entière à chaque frappe.

Le deuxième signal : le coût

Le coût d'un backend managé se compare toujours au coût de l'équivalent dans la stack actuelle. Prenons un raisonnement typique : si votre application traite 200 000 opérations par mois, un plan Airtable Business ou Enterprise, augmenté d'un plan Make Pro pour l'intégration, plus éventuellement des « credits » d'une base rapide, peut dépasser le coût d'une base Postgres managée dimensionnée pour la même charge. Le calcul n'est pas automatiquement en faveur de l'externalisation — tout dépend de vos volumes réels — mais il faut le faire **explicitement**, en listant :

1. le coût de la base de données aujourd'hui (abonnement, crédits, lignes) ;
2. le coût des automatisations liées au volume de données traitées ;
3. le coût de la latence, comptabilisé en temps perdu par les utilisateurs ;
4. le coût du développement compensé (le backend externe demande souvent plus de rigueur, mais permet aussi de déléguer des traitements à un moteur plus puissant).

Le troisième signal : la latence et la charge de calcul

Une base de données ne stocke pas seulement : elle **calcule**. Agrégations, jointures, tris sur plusieurs colonnes, pagination, recherche plein texte. Or beaucoup d'outils no-code poussent ce calcul vers le client ou vers l'application : c'est la base qui envoie 10 000 lignes, et c'est l'app (ou le navigateur) qui filtre. C'est le scénario le plus coûteux en latence.

© 2026 Oussama Ghorbel

En externalisant, vous renversez le raisonnement : la requête part vers le backend, le backend filtre, agrège, pagine, et ne renvoie que les 20 lignes utiles. Le gain est souvent brutal — de secondes à quelques dizaines de millisecondes — parce que le passage de témoin entre « je télécharge tout » et « je demande ce qu'il me faut » change tout.

Un modèle mental utile : séparer les trois rôles

Avant de choisir un outil, distinguez trois rôles que les stacks no-code tendent à fusionner :

- **Le store** : où vivent les données de référence (catalogue, clients, contrats).
- **Le compute** : où se font les calculs, règles métier, agrégations.
- **L'expérience** : l'interface que voit l'utilisateur (Bubble, Softr, WeWeb, Glide).

Un backend externe n'est utile que s'il clarifie cette frontière. Si vous sortez vos données sans savoir qui calcule quoi, vous aurez simplement déplacé la complexité — et parfois multiplié les allers-retours réseau.

Quand il est trop tôt

Soyons honnêtes : sortir ses données trop tôt est un contresens. En phase de découverte, où le schéma change chaque semaine, un backend managé ajoute de la friction (migrations, types, contraintes) sans bénéfice réel. Le bon moment est généralement celui où vous avez **un schéma relativement stable**, **un volume qui pèse**, et **une équipe (même d'une seule personne) capable d'assumer un outil plus exigeant**. Avant cela, Airtable, Notion ou la base interne de Bubble restent des choix rationnels.

8.2. Xano : backend no-code avec API REST/GraphQL automatique

Le principe

Xano se présente comme un **backend as a service visuel** : vous modélisez des tables, vous écrivez de la logique métier dans un éditeur de fonctions visuel, et Xano génère automatiquement des endpoints REST (et, selon les configurations, des API GraphQL) exploitables par

© 2026 Oussama Ghorbel

n'importe quel front. L'ambition est claire : donner aux non-développeurs ce qu'un développeur mettrait plusieurs jours à bâtir — API, base de données, authentification, tâches planifiées — sans écrire de code serveur.

Ce qu'on y trouve concrètement

- **Une base de données PostgreSQL** gérée, avec éditeur de tables, types, relations et index.
- **Un éditeur de fonctions en visual programming** : on compose des étapes (requête SQL, transformation de données, appel externe, condition, boucle, réponse) comme un workflow.
- **La génération d'endpoints** : chaque fonction exposée devient un endpoint REST avec sa méthode HTTP, ses paramètres, sa validation.
- **L'authentification** intégrée : utilisateurs, jetons (tokens), permissions par rôle.
- **Les tâches planifiées** (*cron*) pour les traitements de fond.
- **Un environnement de test** et des outils de déploiement.

Forces

La force de Xano est la **cohérence du modèle** : ce que vous modélisez côté base, vous le retrouvez dans l'API, sans couche de mappage manuelle. Pour quelqu'un qui vient de Make ou de Bubble, la courbe d'apprentissage est réelle mais courte : on reconnaît la logique « si ceci alors cela », on apprend simplement que chaque étape a un coût de performance et que les appels imbriqués mal maîtrisés se paient en latence.

Xano brille quand l'application front devient multiple : la même API peut servir une app Bubble, un site Softr, une app mobile et un partenaire externe. C'est le moment où un backend dédié cesse d'être un luxe.

Limites et points de vigilance

- **Le vendor lock-in de la logique** : vos fonctions Xano sont spécifiques à Xano. La base est PostgreSQL, exportable en théorie, mais la logique visuelle ne se transporte pas.

© 2026 Oussama Ghorbel

• **La tentation du « tout-fait »** : parce que c'est facile, on expose trop d'endpoints, on duplique de la logique, on finit avec un maquis d'API qu'aucun document ne décrit.

• **La performance n'est pas magique** : un endpoint qui fait dix requêtes séquentielles restera lent, quelle que soit la plateforme. Il faut penser comme un backend : indexer, paginer, minimiser les allers-retours.

Quand choisir Xano

Vous choisirez Xano quand vous voulez un **backend complet et visuel**, avec API exposée à plusieurs fronts, et que la logique métier est trop riche pour rester dans des automatisations Make/Zapier. C'est un choix « plateforme » : on l'adopte pour construire dessus plusieurs années.

8.3. Supabase : Postgres managé accessible aux non-développeurs

Le principe

Supabase part d'un postulat différent : plutôt que de créer un nouveau modèle visuel, il **habille PostgreSQL** — le SGBD open source le plus répandu — d'une interface web, d'une API automatique, d'une authentification, d'un stockage de fichiers et d'une fonction temps réel. Le point de départ n'est pas « un backend visuel », c'est « une base de données de production, en un clic ».

C'est une différence de posture importante : avec Xano, vous pensez en fonctions et endpoints ; avec Supabase, vous pensez en tables, en schéma et en politiques d'accès.

Ce qu'on y trouve concrément

• **Postgres managé** avec éditeur de tables et éditeur SQL pour les cas avancés.

• **Auto-API** : chaque table expose automatiquement des endpoints REST et GraphQL (PostgREST / pg GraphQL sous le capot), avec filtrage, tri, pagination via des paramètres d'URL.

© 2026 Oussama Ghorbel

• **Authentification** : inscriptions, connexion, jetons JWT, magic links, OAuth.

• **Row Level Security (RLS)** : des politiques SQL qui règlent, ligne par ligne, qui peut lire, écrire, supprimer. Nous y reviendrons en 8.6, car c'est le cœur de la sécurité Supabase.

• **Storage** pour les fichiers, **Realtime** pour les subscriptions, **Edge Functions** pour du code serveur léger (TypeScript/Deno).

• **Les migrations** : le schéma est versionnable, ce qui change radicalement la manière de travailler en équipe.

Forces

• **La puissance de Postgres** : jointures, index composites, vues, fonctions SQL, plein texte, JSONB. Ce que vous ne pouvez pas faire dans Airtable, vous le pouvez ici — avec la contrepartie qu'il faut un peu comprendre ce que l'on fait.

• **L'exportabilité** : sortir de Supabase est théoriquement simple, puisque c'est du Postgres standard.

• **Le plugin Bubble officiel** : l'intégration est mature et permet à Bubble de dialoguer avec Supabase sans passer par des hacks REST manuels.

• **La RLS** : un modèle de sécurité déclaratif, robuste, qui évite de devoir réimplémenter des gardiens dans chaque endpoint.

Limites et points de vigilance

• **La pente SQL** : les non-développeurs les plus à l'aise dans Make vont devoir franchir un palier. Heureusement, l'interface visuelle couvre l'essentiel du CRUD ; le SQL devient requis dès qu'on touche à la RLS fine ou aux requêtes complexes.

• **L'API auto est large par défaut** : sans RLS ou sans contrôle, une table exposée peut l'être trop. Activer une base, c'est aussi **fermer ce qu'on n'utilise pas**.

• **Le coût à l'usage** : le niveau gratuit est généreux pour apprendre, mais le passage à l'échelle (Compute, egress, points de sauvegarde) se surveille.

© 2026 Oussama Ghorbel

Quand choisir Supabase

Vous choisirez Supabase quand vos données sont le cœur du projet, quand vous voulez le contrôle et la portabilité de Postgres, et quand vous acceptez d'apprendre les bases du relationnel et de la sécurité déclarative. C'est le choix le plus « ingénieur » des options visuelles — au meilleur sens du terme.

8.4. Google Sheets : le « backend pauvre » et ses pièges

Pourquoi on y pense (à juste titre, parfois)

Google Sheets a des arguments séduisants : tout le monde sait l'utiliser, il est gratuit ou déjà payé via Google Workspace, il se connecte à tout (Make, Zapier, Apps Script, n8n, AppSheet), et il offre une immédiateté que nul SaaS ne rivalise. Pour un prototype, un tableau de bord interne ou un MVP à très faible volume, c'est parfois le bon outil. AppSheet, d'ailleurs, construit toute sa philosophie sur cette base : une feuille de calcul comme source de vérité.

Les pièges, un par un

1. La limite de cellules. Un sheet plafonne à 10 millions de cellules. Sur un tableau à 20 colonnes, cela fait 500 000 lignes — chiffre qui sonne bien, mais qui se heurte vite à la performance bien avant.

2. La latence des appels API. L'API Google Sheets n'est pas conçue pour du temps réel. Chaque lecture/écriture est un appel réseau coûteux, soumis aux quotas. Un Make qui écrit 500 lignes une par une peut mettre des minutes et déclencher des limites de débit.

3. Pas de contraintes d'intégrité. Rien n'empêche un utilisateur de saisir du texte dans une colonne de nombres, de supprimer un en-tête, de trier la colonne A sans les B… « Quelqu'un a touché le fichier » est l'incident le plus fréquent des projets construits sur Sheets.

4. La concurrence. Plusieurs automatisations écrivant simultanément dans la même feuille produisent des conflits, des doublons, des écritures perdues.

5. La sécurité par défaut est faible. Une feuille partagée « à tous avec le lien » est un data leak en puissance. Les droits Sheets sont

© 2026 Oussama Ghorbel

coarses : on ne peut pas dire « cet utilisateur ne voit que ses propres lignes » sans couche applicative.

6. L'absence d'historique fiable. L'historique de versions existe, mais il n'est pas pensé comme un audit trail métier. On ne saura pas facilement qui a modifié quoi, quand, depuis une automatisation.

Quand l'utiliser quand même

Google Sheets reste pertinent comme :

- **couche de saisie humaine** (formulaires, tableau de suivi) alimentant ensuite une vraie base ;
- **export/interchange** (un CSV vivant) ;
- **backend de prototype** dont on sait, dès le départ, qu'il sera remplacé ;
- **source pour AppSheet** dans un contexte contrôlé et à faible volumétrie.

Une règle éprouvée : **Sheets comme interface, jamais comme base de vérité au-delà de quelques milliers de lignes et d'une seule écriture à la fois.** Dès que deux automatisations écrivent dans la même feuille, il est temps de migrer.

Comment migrer proprement

Migrer de Sheets vers un vrai backend suit toujours le même chemin : 1) figer le schéma réel (colonnes, types, doublons cachés) ; 2) nettoyer et typer les données ; 3) créer la base cible avec contraintes ; 4) migrer par lots avec un script de vérification (nombre de lignes, sommes de contrôle) ; 5) basculer les lectures en double pendant une période d'observation ; 6) couper l'ancienne source. L'étape 4 est celle qu'on saute toujours — et qui évite les mauvaises surprises.

8.5. Firebase et Baserow : alternatives à connaître

Firebase (Google)

Firebase est le pionnier des BaaS modernes. Sa base phare, **Firestore**, est une base NoSQL orientée documents, conçue pour le temps réel : les clients (web, mobile) reçoivent les mises à jour sans recharger. À cela s'ajoutent Auth, Storage, Cloud Functions, Analytics

© 2026 Oussama Ghorbel

— tout un écosystème pensé pour les applications grand public.

Forces : temps réel natif, écosystème Google, SDK riches pour le mobile, modèle offline-first (l'application continue de fonctionner sans réseau puis se synchronise).

Faiblesses : le modèle documentaire pousse à dupliquer les données (les jointures n'existent pas, on « dénormalise » volontairement) ; les requêtes croisées complexes sont malaisées ; et surtout, **les règles de sécurité Firestore** (Security Rules) sont un langage à part entière qu'il faut maîtriser — une règle mal écrite expose potentiellement toute la base. Le coût, également, peut grimper vite avec la lecture de documents à forte volumétrie.

Quand y penser : applications temps réel (chat, collaboration, live), apps mobiles offline-first, projets où la synchronisation client importe plus que les requêtes analytiques.

Baserow

Baserow est une **base de données no-code open source**, proche d'Airtable dans l'ergonomie (grilles, vues, champs liés) mais auto-hébergeable. C'est son argument principal : vous pouvez l'installer sur votre propre serveur, garder les données chez vous, éviter l'abonnement SaaS.

Forces : souveraineté des données, coût prévisible (auto-hébergement), modèle familier, API REST automatique, plugins.

Faiblesses : l'écosystème d'intégrations est plus pauvre qu'Airtable ; les fonctions avancées (automatisations, formulaires, permissions fines) sont moins riches ; l'auto-hébergement vous rend responsable des sauvegardes, des mises à jour et de la disponibilité — expertise à acquérir si vous n'êtes pas à l'aise en serveur.

Quand y penser : quand la contrainte RGPD ou de souveraineté prime, quand vous voulez éviter un abonnement par utilisateur, quand votre usage est fondamentalement « tableur relationnel » sans besoin d'un moteur de calcul lourd.

Autres noms qu'il faut croiser sur la route

© 2026 Oussama Ghorbel

• **Pocketbase** : un backend tout-en-un léger (base SQLite, auth, storage, admin), idéal pour les petits projets et l'expérimentation ; moins adapté à une charge de production exigeante.

• **Xata**, **Neon**, **PlanetScale** : des bases serverless (Postgres ou MySQL) souvent utilisées *sous* un backend plutôt qu'à leur place — à connaître quand on passe en phase d'industrialisation.

• **Airtable comme backend via API** : techniquement possible, avec les limites de volumétrie et de débit évoquées plus haut.

Le bon réflexe n'est pas de tout connaître, mais de savoir **quel type de problème** chaque famille résout : documentaire temps réel (Firebase), relationnel managé (Supabase), backend visuel complet (Xano), tableur souverain (Baserow).

8.6. Sécuriser son backend : rôles, tokens, RLS (Row Level Security)

Sortir ses données, c'est sortir de la « clôture » de l'outil SaaS. Dès lors, la sécurité n'est plus une option de configuration : c'est une **architecture**. Trois piliers structurent l'essentiel.

Pilier 1 — Les rôles : qui peut faire quoi

Avant tout token, définissez une **matrice de rôles**. Non pas une liste vgue de « admins et users », mais des rôles métier précis :

• *Visiteur non authentifié* : lecture publique strictement nécessaire (catalogue, docs).

• *Utilisateur authentifié* : accès à ses propres données.

• *Collaborateur* : accès élargi à un périmètre (une équipe, un compte client).

• *Opérateur / support* : droits d'écriture élargis, mais pas l'accès aux données sensibles non nécessaires.

• *Admin* : gestion du système, idéalement protégée par une authentification renforcée.

Formalisez cela dans un simple tableau : **rôle × opération (créer, lire, modifier, supprimer) × portée (les siennes, celles de son équipe, tout)**. Ce tableau est le cahier des charges de votre sécurité ; tout ce qui

© 2026 Oussama Ghorbel

suit n'en est que l'implémentation.

Attention à un principe simple : **le rôle le plus large ne doit jamais être le défaut**. On part fermé, on ouvre ce qui est nécessaire — jamais l'inverse.

Pilier 2 — Les tokens : l'identité transportée

Les appels entre votre front et votre backend doivent être **authentifiés**. Le mécanisme dominant : les **jetons JWT** (JSON Web Tokens), émis à la connexion, envoyés dans l'en-tête HTTP `Authorization: Bearer …`, et vérifiés par le backend à chaque requête.

Points de vigilance, quel que soit l'outil (Xano, Supabase, Firebase, API custom) :

• **Durée de vie courte** pour les tokens d'accès, avec rafraîchissement (*refresh token*) ; un token volé ne doit pas ouvrir la porte des mois durant.

• **Portée** (*scope*) : un token d'automatisation serveur n'a pas besoin des mêmes droits qu'un token utilisateur final. Créez des clés distinctes par usage — une pour le site public, une pour l'app interne, une pour Make.

• **Rotation** : prévoyez de régénérer les clés, et documentez où elles sont stockées.

• **Ne jamais placer de clé secrète dans un front** : tout ce qui est embarqué dans un JavaScript ou une application mobile peut être extrait. Si le secret doit rester secret, il vit côté serveur ou dans une Edge Function.

• **HTTPS partout**, sans exception : un token en clair sur le réseau est un token compromis.

Un piège classique : réutiliser la même clé API pour trois applications. Le jour où l'une est compromise, vous révoquez tout — et tout tombe. Une clé par consommateur, c'est l'équivalent du principe du moindre privilège appliqué aux credentials.

Pilier 3 — La RLS : la sécurité dans la donnée elle-même

La **Row Level Security** est la grande idée de Postgres — et donc de Supabase : plutôt que de faire confiance à l'application pour filtrer, on

© 2026 Oussama Ghorbel

inscrit les règles **dans la base**. Une politique RLS s'exprime à peu près ainsi, en esprit :

> *« Sur la table `commandes`, un utilisateur authentifié peut lire les lignes où `client_id` est égal à son `id` ; un rôle `admin` peut tout lire. »*

Quel que soit le chemin d'accès — API automatique, requête SQL, script — la politique s'applique. C'est ce qu'on appelle une **défense en profondeur** : même si votre front oublie un filtre, la base refuse.

Les erreurs les plus fréquentes :

1. **Oublier d'activer la RLS** sur une table exposée — résultat : accès en lecture (voire écriture) à l'ensemble des données pour quiconque a l'URL. C'est l'équivalent du fichier Google Sheets « accessible à tous avec le lien ».
2. **Écrire une politique trop permissive** (par exemple `USING (true)`) « temporairement », et ne plus jamais y revenir.
3. **Négliger les opérations `INSERT` et `UPDATE`** en ne soignant que le `SELECT` — l'utilisateur peut alors modifier des lignes qui ne lui appartiennent pas.
4. **Oublier les politiques de deletion**, ce qui permet des pertes de données par un utilisateur lambda.
5. **Tester sans vérifier en conditions réelles** : tester avec un vrai token de rôle bas, pas seulement en tant qu'admin connecté.

Si Supabase fait de la RLS son mécanisme central, Xano et Firebase abordent la sécurité différemment : Xano par des **permissions d'endpoint et des vérifications dans la logique de fonction** ; Firebase par ses **Security Rules**. Dans tous les cas, la question est la même : *si mon front est compromis, que peut faire l'attaquant ?* Si la réponse est « tout », l'architecture est à revoir.

Au-delà des trois piliers

Quelques compléments à garder en tête :

- **Journalisation (logs)** : qui a appelé quoi, quand, avec quel identifiant. Sans logs, pas d'audit possible après incident.

© 2026 Oussama Ghorbel

- **Limitation de débit (rate limiting)** : empêcher qu'une automatisation mal configurée (ou un attaquant) ne noie votre backend en requêtes.
- **Validation des entrées** : même avec des types de colonnes, contrôlez longueurs, formats, valeurs autorisées côté serveur.
- **Sauvegardes chiffrées et restauration testée** : une sauvegarde qu'on n'a jamais restaurée n'existe pas.
- **Séparation des environnements** : développer sur une base de test, jamais sur la production — y compris pour tester des politiques de sécurité.

La sécurité n'est pas un chapitre que l'on coche une fois : c'est une discipline. Mais les trois piliers — **rôles, tokens, RLS** — couvrent, à eux seuls, la grande majorité des incidents évitables.

En synthèse

Externaliser ses données, ce n'est pas « mettre sa base ailleurs ». C'est choisir un modèle de données explicite, un moteur de calcul adapté à la charge, et une politique de sécurité que l'on peut démontrer. Airtable et la base de Bubble restent d'excellents points de départ ; Xano, Supabase, Firebase et Baserow sont des réponses différentes à des problèmes différents ; Google Sheets demeure un allié de prototypage dont il faut connaître les limites par cœur. Et quelle que soit la solution retenue, la question de la sécurité ne vient jamais à la fin : elle accompagne chaque table créée, chaque endpoint exposé, chaque jeton distribué.

Exercices

Exercice 1 — Créer un backend Xano avec 3 tables et des endpoints sécurisés

Objectif : modéliser un mini-backend et l'exposer de façon sécurisée.

1. Créez un compte Xano et un nouveau workspace.

© 2026 Oussama Ghorbel

2. Modélisez trois tables : `clients` (nom, email, statut), `projets` (nom, client_id, date_debut, statut), `taches` (titre, projet_id, assigne_a, fait : booléen). Définissez les relations (un client a plusieurs projets ; un projet a plusieurs tâches).

3. Créez une table `utilisateurs` ou activez l'authentification native, avec deux rôles : `client` et `interne`.

4. Générez les endpoints CRUD sur `projets` et `taches`. Pour chaque endpoint, ajoutez une validation d'entrée (champs requis, format email, valeurs de statut autorisées).

5. Sécurisez : l'endpoint de liste `projets` doit renvoyer uniquement les projets du client connecté pour le rôle `client`, et tous les projets pour le rôle `interne`. Testez avec deux tokens différents.

6. Documentez dans un tableau : endpoint, méthode, rôle autorisé, portée des données.

7. **Bonus** : ajoutez une tâche planifiée qui remet à `statut = en retard` tout projet dont la date de début dépasse 30 jours sans tâche réalisée.

Réussite : aucun endpoint n'est accessible sans token valide ; le rôle `client` ne voit jamais les données d'un autre client, même en modifiant manuellement les paramètres de la requête.

Exercice 2 — Connecter une app Bubble à Supabase via le plugin officiel

Objectif : faire dialoguer un front Bubble avec un backend Supabase sécurisé.

1. Créez un projet Supabase et une table `produits` (id, nom, prix, actif) avec quelques lignes de test.

2. Activez la RLS sur `produits` et créez une politique de lecture publique uniquement pour `actif = true`.

3. Dans Bubble, installez le plugin officiel Supabase et renseignez l'URL du projet et les clés adéquates (respectez la distinction entre clé anonyme et clé service — jamais de clé service côté client).

4. Créez une page Bubble listant les produits actifs via une requête Supabase, avec tri sur le nom.

© 2026 Oussama Ghorbel

5. Ajoutez un formulaire de création de produit. Vérifiez qu'il échoue proprement si vous désactivez la politique d'insertion, puis créez la politique d'insertion appropriée pour un rôle authentifié.

6. Testez le scénario « utilisateur non connecté tente de modifier un produit » : l'opération doit être refusée par la base, pas seulement masquée dans l'interface.

7. **Bonus** : ajoutez un champ `proprietaire_id`, connectez l'authentification Supabase à celle de Bubble, et faites en sorte qu'un utilisateur ne modifie que ses propres produits.

Réussite : une modification interdite échoue au niveau de Supabase, et les logs Supabase montrent le refus.

Exercice 3 — Auditer une architecture Google Sheets pour identifier les risques

Objectif : évaluer objectivement un backend « pauvre » et décider de sa suite.

1. Prenez (ou reconstituez) une architecture existante : un Google Sheets « Base clients » alimenté par un formulaire, synchronisé vers un Airtable, avec une automatisation Make qui envoie des emails et écrit les statuts dans la feuille.

2. Cartographiez : volume actuel de lignes, nombre de colonnes, nombre d'écritures automatiques par jour, nombre d'utilisateurs humains ayant accès en modification.

3. Répondez par écrit à ces questions :

• Combien de cellules le fichier utilise-t-il, et quel est le plafond théorique de lignes avec le schéma actuel ?

• Que se passe-t-il si deux automatisations écrivent au même instant ?

• Un utilisateur avec accès en lecture peut-il partager le fichier à l'externe ? Est-ce détectable ?

• Existe-t-il des colonnes contenant des données personnelles ? Où sont-elles, qui y a accès ?

• Quel est le temps de récupération si la feuille est supprimée ou corrompue ?

© 2026 Oussama Ghorbel

4. Identifiez au moins cinq risques, classez-les par criticité (forte/moyenne/faible) et proposez pour chacun une remediation : quick win (ex. : protection de la feuille, réduction des droits) ou migration.

5. Rédigez une recommandation d'une page : « Conserver / Encadrer / Migrer », avec un plan de migration en 5 étapes si vous optez pour la troisième option (voir la méthodologie de la section 8.4).

6. **Bonus** : établissez un seuil chiffré de déclenchement (nombre de lignes, nombre d'écritures automatiques, nombre d'utilisateurs) au-delà duquel la migration devient obligatoire.

Réussite : vous disposez d'un document de décision argumenté, opposable, qui transforme une intuition (« ça ne va plus ») en critères mesurables.

© 2026 Oussama Ghorbel

Chapitre 9 – Qualité, gouvernance et vie des données

Au début, vous aviez un tableur. Puis une base Airtable, une application Bubble branchée dessus, trois automatisations Make, un supplément Notion pour la documentation et soudain douze personnes qui saisissent des informations au même endroit — ou pire, à des endroits différents. La donnée n'est plus un actif : elle devient une zone de turbulences.

Ce chapitre est le plus important du livre après ceux qui traitent de la modélisation. Parce qu'une application mal construite se répare en quelques jours, alors qu'une donnée corrompue se rattrape rarement. Et parce que, dans un environnement no-code où chacun peut créer, la gouvernance ne peut pas être l'apanage d'un service central : elle doit être légère, partagée et intégrée au travail quotidien.

Je vais organiser ce chapitre comme une pyramide. À la base, l'hygiène : nommer, typer, valider. Au milieu, la pérennité : sauvegarder, archiver, documenter. Au sommet, la maîtrise : conformité, audit, indicateurs. Chaque niveau sans le suivant est fragile. Une stack sans sauvegarde mais avec des conventions parfaites est à une erreur humaine de la catastrophe. Une stack sauvegardée mais incompréhensible pour les nouveaux venus finit par être contournée.

9.1. Nommage, typage, conventions : l'hygiène de base

La qualité des données commence par un détail que tout le monde néglige : la façon dont on les nomme. Pas la façon dont on les stocke — c'est le sujet du chapitre 5 — mais la façon dont on les appelle.

Pourquoi le nommage est un choix d'architecture

Quand vous créez une table « Clients » avec un champ `Prénom client`, un autre `Nom du client`, un troisième `Email contact` et un quatrième `Mail pro`, vous venez d'inventer quatre représentations possibles de la même information. Six mois plus tard, une automatisation qui cherche le champ « email » n'en trouvera qu'un sur quatre. Votre assistant qui remplit la base inventera un cinquième.

© 2026 Oussama Ghorbel

Le nommage n'est pas de l'esthétique. C'est la condition de l'interopérabilité entre vos outils, entre vos collègues et entre vous-même dans six mois.

Les règles qui tiennent dans la durée

Voici un cadre sobre, applicable à Airtable, Notion, Supabase ou n'importe quelle source de données :

• **Un concept, un nom.** Le champ s'appelle `email` ou `e_mail` ou `adresse_email`, mais jamais les trois dans la même base. Choisissez une forme et éliminez les variantes.

• **Pas d'espace, pas d'accents, pas de caractères spéciaux dans les identifiants techniques.** Les labels visibles pour les humains peuvent rester en français avec accents (`Date de naissance`), mais si votre outil génère un identifiant technique, assurez-vous qu'il soit normalisé (`date_de_naissance` ou `dateDeNaissance`). Les accents cassent régulièrement les requêtes et les webhooks.

• **Format constant.** Si les dates sont `AAAA-MM-JJ`, elles le sont partout. Si les montants incluent la devise, c'est systématique. L'incohérence de format est la première cause d'échec silencieux dans les automatisations.

• **Préfixes pour les champs calculés ou dérivés.** `nb_participants`, `dernière_maj`, `est_actif`. Le préfixe signale à la lecture la nature du champ et évite qu'on tente de le saisir à la main.

• **Pas de jargon d'équipe isolé.** Si « deal » ne parle qu'à l'équipe commerciale, un champ `deal_status` sera mal renseigné par les autres. Nommez selon l'usage le plus large, ou documentez le terme.

Typage : le piège des champs textes universels

L'erreur la plus coûteuse de la donnée no-code, c'est le champ texte là où il aurait fallu un type. Un champ `Statut` en texte libre acceptera « en cours », « En Cours », «encours », « en attente » et « pending ». Vous venez de créer cinq valeurs distinctes pour deux états réels.

Les types à imposer systématiquement :

Information	Type correct	Pourquoi

© 2026 Oussama Ghorbel

Statut, catégorie, étiquette	Liste de choix (single/multi select)	Contrôle de vocabulaire
Identifiant, référence	Format texte court, unique	Ne jamais calculer ni convertir
Date	Champ date natif	Tri, calcul, fuseau horaire
Nombre, montant	Nombre (avec décimales si besoin)	Agrégations, totaux
Oui/non	Booléen	Pas de texte « oui » libre
Lien vers un autre objet	Relation / lookup	Intégrité référentielle

Le champ texte libre n'a sa place que pour ce qui est réellement du texte : commentaire, description, titre. Tout le reste est structuré ou ne devrait pas exister.

Un exemple concret de charte

Pour un CRM Airtable branché à un tunnel de vente, une charte minimale tient en quelques lignes :

Une page. Dix règles. Collée au-dessus de l'écran de la personne qui crée les nouveaux champs. C'est tout ce qu'il faut au départ, à condition de la faire respecter — ce qui est le sujet de la section 9.6.

9.2. Validations, champs obligatoires, valeurs par défaut

La convention, c'est bien. La contrainte, c'est mieux. Une donnée qu'on ne peut pas saisir incorrectement vaut mille audits.

Le principe : empêcher l'erreur à la saisie

Une donnée fausse coûte cher à corriger, surtout quand elle a déjà alimenté une facture, un email automatique ou un tableau de bord. Le bon moment pour la détecter, c'est la seconde où elle entre dans le système.

Les garde-fous de saisie, par ordre de robustesse :

1. **Champ obligatoire.** Le minimum. Un client sans email ne doit pas exister dans la base qui alimente votre séquence email. Dans Airtable, la vue de création peut exiger les champs critiques ; dans Supabase, `NOT NULL` ; dans Bubble, l'option « this input is required ».

© 2026 Oussama Ghorbel

2. **Liste de choix fermée.** Remplacez le texte libre par une énumération. C'est la validation la plus rentable : elle ne coûte rien à mettre en place et elle supprime une famille entière d'erreurs.

3. **Format validé.** Email, téléphone, numéro de référence, code postal : les formats ont des motifs reconnus. La plupart des outils proposent une validation par expression ou un type natif. Exiger que l'email contienne un `@` et un domaine semble naïf ; cela filtre quand même une part significative des saisies absurdes.

4. **Unicité.** Un email client, une référence produit, un numéro de facture : ces champs doivent être uniques. Airtable le fait par une vérification dans une vue ou via une automation ; les bases de données relationnelles, par contrainte `UNIQUE`.

5. **Validation croisée.** « La date de fin est postérieure à la date de début. » « Le montant total correspond à la somme des lignes. » Ces règles sont plus complexes, mais souvent exprimables en formule ou en automation. Ce sont elles qui attrapent les erreurs que personne ne voit.

Valeurs par défaut : votre alliée silencieuse

Un champ sans défaut est un champ que chacun remplit à sa manière — ou pas du tout. Trois usages systématiques :

• **Statut par défaut.** Toute nouvelle opportunité commence à `Qualification`. Toute tâche à `À faire`. Le défaut aligne tout le monde sur le même point de départ.

• **Horodatage automatique.** Date de création, dernière modification : dans la plupart des outils, ces champs sont gérés nativement. Dans les autres, une automation au moment de l'enregistrement suffit. Sans horodatage fiable, aucun audit de qualité n'est possible.

• **Propriétaire par défaut.** Le créateur de l'enregistrement comme responsable par défaut. Cela évite les enregistrements orphelins, ces lignes dont personne ne sait qui s'occupe.

Le cas délicat des champs obligatoires rétroactifs

Ajouter un champ obligatoire sur une base existante pose un problème : les anciens enregistrements ne l'ont pas. La bonne pratique

© 2026 Oussama Ghorbel

est un enchaînement en trois temps :

1. Ajouter le champ avec une valeur par défaut neutre.
2. Remplir rétroactivement, en priorisant les enregistrements actifs (les archivés peuvent rester vides, on les marquera).
3. Passer le champ en obligatoire une fois le rattrapage effectué.

Jamais l'inverse. Un champ obligatoire qui bloque la saisie sur des données historiques fera dire à vos collègues que « l'outil ne marche pas », et ils trouveront une façon de contourner le garde-fou — en général en remplissant au hasard.

Quand trop de validation nuit

La validation a un coût d'usage. Si saisir un dossier prend douze champs obligatoires et huit contrôles, les gens trouveront le chemin le plus court : saisir le minimum en force. Mieux vaut cinq champs obligatoires vraiment critiques et dix champs recommandés avec des valeurs par défaut intelligentes, que quinze obligations qui génèrent de la résistance.

Posez-vous la question pour chaque contrainte : « Que se passe-t-il concrètement si cette donnée est fausse ou absente ? » Si la réponse est « une facture partira au mauvais client », c'est obligatoire. Si c'est « le rapport mensuel sera un peu moins précis », c'est un défaut ou un avertissement.

9.3. Backups, exports, archivage : ne jamais perdre

Tout le monde croit ses données à l'abri parce qu'elles sont dans le cloud. Le cloud, c'est la responsabilité de quelqu'un d'autre — rarement la vôtre. Votre compte peut être supprimé, votre abonnement résilié, votre automatisation peut écraser une table en boucle, un collègue peut supprimer une vue. L'historique des versions n'est pas une sauvegarde.

La règle des trois copies

Formulation issue de l'administration système, parfaitement transposable au no-code : vos données critiques doivent exister en trois endroits — la production, une sauvegarde locale ou cloud que vous contrôlez, et une copie hors site ou chez un autre fournisseur.

© 2026 Oussama Ghorbel

Concrètement pour une base Airtable qui alimente votre activité :

• **Copie 1 — production.** Airtable lui-même.

• **Copie 2 — export automatique.** Un snapshot complet, chaque nuit ou chaque semaine, vers Google Drive, S3 ou un serveur.

• **Copie 3 — export différent.** Une sauvegarde vers un autre service (un export CSV local, un second bucket), pour couvrir le scénario où le service d'export tombe en panne ou perd l'accès à votre compte.

Mettre en place un backup réel

Prenons le cas d'Airtable vers un stockage objet, un scénario que vous déployerez en une heure :

1. **Créer une clé d'accès Airtable** avec le minimum de droits (lecture seule sur la base à sauvegarder).

2. **Choisir l'orchestrateur.** Make, n8n ou Zapier conviennent. Un scénario hebdomadaire, déclenché le dimanche matin.

3. **Récupérer les enregistrements.** Via l'API Airtable, par lots si votre base dépasse quelques milliers d'enregistrements — l'API limite le nombre d'enregistrements par appel.

4. **Générer un fichier.** Un JSON par table, ou un CSV consolidé, horodaté : `backup_airtable_2025-06-08T03-00.json`.

5. **Envoyer vers S3, Google Drive ou un autre bucket**, avec une politique de rétention (par exemple, conserver 8 semaines de sauvegardes hebdomadaires).

6. **Signaler l'échec.** Le point le plus souvent oublié : si le backup échoue silencieusement, il n'existe pas. Ajoutez une branche de scénario qui, en cas d'erreur, vous envoie un email ou un message Slack.

Le même principe s'applique à Notion (export natif planifié ou via API), à Supabase (les sauvegardes managées existent, mais exportez quand même une copie externe périodiquement) et à Google Sheets (un script programmé qui enregistre une version dans Drive).

Sauvegarde != archivage

Deux notions souvent confondues :

© 2026 Oussama Ghorbel

• La **sauvegarde** (backup) est un instantané réversible, destiné à restaurer après un incident. Elle a une durée de vie limitée.

• L'**archivage** vise la conservation longue de données que l'on n'utilise plus au quotidien mais que l'on doit conserver : anciens dossiers clients, factures des années précédentes, historiques d'activité.

Une base de production qui grossit sans archivage devient lente, cher et illisible. Le rythme que je recommande : une fois par trimestre, identifier les enregistrements plus vieux que 12 à 24 mois sans activité, les exporter vers une table ou un fichier d'archives, puis les marquer comme inactifs — plutôt que de les supprimer.

Suppression et archivage ne sont pas la même opération. Une suppression est définitive ; un archivage est réversible et documenté.

Tester la restauration

Un backup non testé est une hypothèse. Tous les trois à six mois, prenez le dernier export et restaurez-le dans un espace de test. Chronométrez le temps nécessaire. Vérifiez que les relations entre tables sont intactes, que les pièces jointes sont présentes, que les champs calculés se recalculent.

Cet exercice met presque toujours en évidence un problème : un export qui n'inclut pas les pièces jointes, un champ relationnel qui est exporté sous forme d'identifiants non résolus, une limite d'API qui a coupé la sauvegarde à 20 000 enregistrements sur 35 000. Mieux vaut le découvrir un mardi matin tranquille qu'un vendredi soir pendant un incident.

9.4. RGPD en contexte no-code : où sont stockées les données ?

La question de la conformité revient à chaque fois qu'une équipe est accompagnée sur un outil no-code. La réponse courte : vos données sont exactement là où votre stack les met — et cette liste est souvent plus longue qu'on ne le pense.

Cartographier le chemin de la donnée

© 2026 Oussama Ghorbel

Avant de parler conformité, faites l'inventaire. Prenez chaque donnée personnelle que vous collectez (nom, email, téléphone, adresse, identifiants, données de santé ou bancaires le cas échéant) et suivez-la :

1. **Saisie.** Où l'utilisateur tape-t-il ? (Formulaire Softr, page Bubble, Airtable partagé.)
2. **Stockage.** Dans quelle base, dans quelle région ?
3. **Traitement.** Quelles automatisations la manipulent ? (Make, Zapier, n8n.)
4. **Enrichissement.** Quels services tiers la reçoivent-ils ? (Emailing, CRM, analytics, IA.)
5. **Sortie.** Vers quels destinataires, quelles intégrations, quels exports ?

Cette cartographie en cinq temps est le préalable obligatoire. On ne peut pas répondre à « mes données sont-elles en règle ? » sans savoir où elles sont passées.

Les points de vigilance typiques d'une stack no-code

• **Les sous-traitants en cascade.** Quand Airtable appelle Make qui appelle SendGrid qui appelle OpenAI, vous avez potentiellement cinq sous-traitants. Chacun a sa politique de rétention, son lieu de stockage, ses conditions. Vous devez connaître au minimum les quatre premiers.

• **Les champs cachés.** Un champ `Notes` libre dans lequel on colle des extraits d'échanges contient souvent des données personnelles non déclarées. C'est la porte d'entrée classique des données non maîtrisées.

• **Les duplications non contrôlées.** Une base de test copiée à partir de la production contient de vraies données personnelles dans un environnement sans les mêmes protections. Règle simple : les environnements de test ne contiennent que des données fictives.

• **Les exports permanents.** Un CSV téléchargé chaque semaine sur l'ordinateur d'un prestataire fait partie de votre périmètre aussi longtemps qu'il existe.

Les obligations qui vous concernent concrètement

© 2026 Oussama Ghorbel

Sans entrer dans un commentaire juridique — cet ouvrage est informatif et ne remplace pas un avis d'avocat ou d'un délégué à la protection des données — voici les réflexes indispensables :

• **Licence légale.** Avoir une raison valable de collecter chaque donnée : contrat, intérêt légitime, consentement. Si vous ne savez pas pourquoi un champ existe, c'est qu'il ne devrait probablement pas exister.

• **Minimisation.** Ne collecter que le nécessaire. Collecter la date de naissance « au cas où » est un risque sans bénéfice.

• **Durée de conservation.** Définir combien de temps chaque catégorie de donnée reste active, puis archiver ou supprimer. Un tableau d'une dizaine de lignes suffit : catégorie, donnée, base, durée, action en fin de vie.

• **Droits des personnes.** Pouvoir exporter, corriger ou supprimer les données d'un individu. Testez le parcours : pouvez-vous retrouver toutes les données de « jean.martin@exemple.fr » en moins de trente minutes dans votre stack ? Si la réponse est non, votre stack est trop fragmentée ou vos index trop faibles.

• **Registre de traitement.** Même simplifié, un document listant vos traitements, leurs finalités et leurs sous-traitants est exigé et, plus important, il vous force à voir l'ensemble du système.

Choisir des outils avec des garanties

Lorsque vous évaluez un nouvel outil, posez ces questions dès la fiche produit :

• Où sont hébergées les données ? (Région UE pour les données européennes, c'est le réflexe par défaut.)

• Le fournisseur joue-t-il le rôle de sous-traitant ? Propose-t-il un accord de sous-traitance ?

• Existe-t-il des sauvegardes chiffrées, une politique de rétention, un registre des incidents ?

• Les données servent-elles à l'entraînement de modèles d'IA, sans expresse demande de votre part ? C'est devenu un critère de choix majeur pour les outils dotés de fonctions IA.

© 2026 Oussama Ghorbel

• Peut-on exporter l'intégralité des données à tout moment, dans un format ouvert ?

Un outil qui répond « non » ou « on verra » à trois de ces questions ne devrait pas héberger de données sensibles. Il peut rester pour des usages internes sans données personnelles.

9.5. Data dictionary : documenter pour les collègues futurs

La documentation en no-code a mauvaise réputation : on la promet, on ne la fait jamais, et on la refait de zéro à chaque départ de la personne qui savait. Le data dictionary est la forme la plus rentable de documentation, parce qu'il répond à la question que tout le monde se pose en ouvrant une base inconnue : « Qu'est-ce que ce champ veut dire ? »

Ce qu'il doit contenir

Un data dictionary n'est pas un manuel. C'est une table, un Notion, un Google Doc — au choix — avec, pour chaque champ significatif :

• **Nom technique et label affiché.**

• **Type.** Texte, liste, relation, date, nombre.

• **Description.** Une phrase. À quoi sert ce champ, quand on le remplit.

• **Valeurs possibles.** Pour une liste, la définition de chaque option. « En attente : le prospect n'a jamais répondu. » « En négociation : une proposition chiffrée a été envoyée. »

• **Obligatoire ou non**, et valeur par défaut.

• **Propriétaire.** Qui est responsable de la qualité de ce champ.

• **Source.** Saisie manuelle, alimentée par une automation, synchronisée depuis un autre outil.

Ajoutez une courte section pour chaque table : son rôle dans le système, ses liens avec les autres tables, et l'application ou l'automation qui la consomme.

Où le placer

© 2026 Oussama Ghorbel

Le data dictionary doit vivre là où les gens travaillent déjà. Deux options qui fonctionnent :

• **Dans Notion ou Coda**, à côté des autres procédures, avec une structure de page par table. L'avantage : on peut commenter, versionner, assigner des revues.

• **Dans un onglet dédié de la base elle-même**, utile si votre équipe vit dans Airtable et n'ouvre jamais Notion.

Le pire emplacement est un fichier sur le disque de l'administrateur. La documentation invisible n'existe pas.

Le rythme : documenter à la création, pas après coup

La règle qui change tout : dès qu'un champ ou une table est créé, il entre dans le data dictionary. C'est une étape du formulaire de création, comme donner un nom. Cela prend trente secondes au moment de la création ; cela prend une heure de reconstitution six mois plus tard, avec un risque d'erreur.

Pour les bases héritées, commencez par les dix champs les plus utilisés. Couvrez 80 % des consultations avec 20 % du dictionnaire. L'étendre ensuite progressivement, à chaque audit.

Un modèle prêt à l'emploi

Champ	Type	Description	Valeurs	Requis	Propriétaire	Source
`email`	Texte, unique	Adresse email principale du contact	—	Oui	Commercial	Saisie
`statut_lead`	Liste	Étape du parcours commercial	Nouveau, Qualifié, Proposé, Gagné, Perdu	Oui (défaut : Nouveau)	Commercial	Saisie
`dte_premier_contact`	Date	Date du premier échange réellement enregistré	Format ISO	Non	Commercial	Automation Make

© 2026 Oussama Ghorbel

`source`	Liste	Canal d'acquisition	Site, Recommandation, Salon, Autre	Oui	Marketing	Formulaire

Cinq lignes. Vous avez déjà plus de documentation que la plupart des équipes.

9.6. Qualité continue : audits mensuels et métriques

La qualité n'est pas un état, c'est un rythme. Sans récurrence, tout ce qui précède s'érode en quelques semaines : on ajoute un champ sans le documenter, une automation écrit une valeur inattendue, un export manuel casse un identifiant.

Le rituel de l'audit mensuel

Réservez une heure par mois. Une heure suffit si la check-list est stable. Voici une check-list type :

1. **Champs orphelins.** Tout champ créé depuis le dernier audit est-il documenté ? Son nom suit-il la charte ?

2. **Champs vides suspects.** Tout champ obligatoire a-t-il un taux de remplissage cohérent ? Un champ vide à 60 % signale un problème de saisie ou une contrainte contournée.

3. **Doublons.** Compter les doublons sur les clés uniques (emails, références). Un pic signale une intégration qui écrit deux fois.

4. **Automatisations en échec.** Passer en revue les scénarios Make, Zapier, n8n : combien d'exécutions en erreur ce mois-ci ? Une erreur récurrente silencieuse est une dette qui attend.

5. **Intégrités relationnelles.** Des enregistrements ont-ils perdu leur lien ? Des enfants sans parent, des factures sans client ?

6. **Dernière sauvegarde.** Le backup de la semaine a-t-il réussi ? Avez-vous vérifié qu'un export s'ouvre bien ?

7. **Droits d'accès.** Qui a encore accès à quelle base ? Les prestataires partis ont-ils été désactivés ? C'est souvent le point le plus urgent.

8. **Nouveaux outils.** Un collègue a-t-il branché un nouvel outil sur la donnée ? La stack a-t-elle grossi sans que personne ne l'ait noté ?

© 2026 Oussama Ghorbel

Ce dernier point est crucial : la dérive no-code se joue autant dans les nouveaux champs que dans les nouvelles applications qui apparaissent hors radar.

Les métriques qui valent la peine d'être suivies

Trop d'indicateurs tuent l'indicateur. Cinq suffisent pour démarrer :

- **Taux de remplissage des champs critiques.** Objectif : au-dessus de 95 %.
- **Taux d'enregistrements en doublon sur les clés uniques.** Objectif : proche de 0 %.
- **Taux d'échec des automatisations.** Objectif : inférieur à 1 % par mois.
- **Fraîcheur de la donnée.** Délai entre un événement terrain et sa présence dans la base. Utile pour les tableaux de bord dirigeants.
- **Âge de la dernière sauvegarde testée.** Objectif : moins de 90 jours.

Ces métriques n'exigent pas de data scientist. Une vue filtrée dans Airtable, un script léger, un tableau Notion à jour : c'est largement suffisant pour voir la dérive venir.

Le tri : est-ce du bruit ou du signal ?

Un audit produit des anomalies. Toutes ne se valent pas. Voici un tri simple :

- **Bloquant.** Risque de perte de donnée, de fuite, de mauvaise facturation. Traité dans la journée.
- **Structurant.** Incohérence de nommage, champ non documenté, automation non testée. Traité dans le sprint en cours.
- **Cosmétique.** Libellé améliorable, ordre de champ, couleur de vue. Noté, pas traité immédiatement.

Sans ce tri, l'audit devient une liste anxiogène que l'on repousse. Avec lui, il devient une file de travail réaliste.

Intégrer la qualité dans le flux de travail

© 2026 Oussama Ghorbel

L'audit mensuel ne suffit pas à lui seul. Trois réflexes à glisser dans le quotidien :

- **Une check-list de création.** Avant d'ajouter une table ou un champ : nom conforme ? type correct ? défaut défini ? entré dans le data dictionary ? propriétaire désigné ?
- **Une validation avant les actions à fort impact.** Une automation qui écrit en masse, un import CSV, un script de nettoyage : les exécuter d'abord sur un échantillon, toujours.
- **Un canal d'anomalie.** Un endroit unique où signaler « cette donnée a l'air bizarre ». Sans cela, les problèmes restent dans la tête des gens et disparaissent avec eux.

La donnée comme produit vivant

On parle souvent de « dette technique ». Il existe une dette de donnée, et elle est plus sournoise : elle ne casse pas le code, elle corrompt les décisions. Un tableau de bord construit sur des statuts saisis n'importe comment produira une lecture fausse de l'activité — avec la même apparence de fiabilité qu'une donnée saine.

Traiter ses données comme un produit, c'est accepter qu'elles aient un cycle de vie : création, usage, contrôle, archivage, suppression. C'est leur donner un propriétaire. C'est mesurer leur état. C'est prévoir leur sauvegarde. Rien de spectaculaire — mais l'accumulation de ces gestes modestes fait la différence entre une stack qui grandit sereinement et une stack qui, un beau jour, se révèle inexploitable.

Exercices

Exercice 1 — Rédiger une charte de nommage pour votre stack (1 page)

Sur une seule page, formalisez les règles que vous appliquerez à toutes vos bases et applications. Structure suggérée :

1. **Règles de nommage des tables** (pluriel, singulier ? langage ? préfixes par domaine ?).

© 2026 Oussama Ghorbel

2. **Règles de nommage des champs** (convention : snake_case, camelCase, libellés humains ; interdits : espaces, accents, caractères spéciaux).

3. **Conventions de type** (quel type pour quel usage ; texte interdit pour statuts et catégories).

4. **Formats normalisés** (dates, montants, identifiants, emails).

5. **Préfixes réservés** (`nb_`, `dte_`, `est_`, `est_`, etc.).

6. **Responsable de la charte** et modalités de dérogation.

Testez-la ensuite sur votre base la plus récente : identifiez au moins cinq champs qui violent votre propre charte, puis corrigez-les. Si vous n'en trouvez pas, votre charte est probablement trop vague.

Exercice 2 — Mettre en place un backup automatisé d'Airtable vers S3 ou Google Drive

Objectif : disposer d'un export automatique et horodaté de au moins une base critique, avec alerte en cas d'échec.

Étapes :

1. Créez une clé API Airtable en lecture seule, limitée à la base ciblée.

2. Choisissez votre orchestrateur (Make, n8n ou Zapier) et créez un scénario déclenché (hebdomadaire recommandé).

3. Récupérez les enregistrements de chaque table via l'API, en gérant la pagination.

4. Construisez un fichier JSON ou CSV horodaté par table.

5. Envoyez-le vers votre destination (S3 avec clé préfixée par la date, ou un dossier Drive dédié).

6. Ajoutez une branche d'erreur qui notifie par email ou Slack.

7. Conservez au moins 4 à 8 sauvegardes glissantes, puis supprimez plus ancien automatiquement.

8. Testez une restauration complète dans un espace de test et notez le temps qu'elle prend.

Critère de réussite : apr s trois semaines, vous avez trois sauvegardes consultables et vous avez déjà restauré une copie au moins

© 2026 Oussama Ghorbel

une fois.

Exercice 3 — Réaliser un audit RGPD rapide de vos 3 outils principaux

Pour chacun de vos trois outils les plus exposés aux données personnelles (par exemple Airtable, Make, votre outil email), répondez par écrit :

1. **Où sont stockées les données ?** Pays et région, au mieux.
2. **Quel est le rôle de l'outil ?** Responsable de traitement ou sous-traitant ?
3. **Quelles données personnelles transitent-elles ?** Liste précise, champ par champ.
4. **Quelle est la durée de conservation ?** Existe-t-elle, est-elle documentée ?
5. **Peut-on exporter ou supprimer l'ensemble des données ?** Décrivez la procédure réelle, pas celle de la brochure.
6. **Les conditions ont-elles changé depuis votre adoption ?** Revoyez la politique de confidentialité du fournisseur : date de mise à jour, usage éventuel des données pour l'entraînement d'IA.
7. **Qui a accès à l'outil aujourd'hui ?** Listez, et retirez les comptes inutiles.

Livrable : un tableau d'une page par outil, plus une liste d'au moins trois actions correctives priorisées (supprimer un compte, réduire une rétention, signer un accord, désactiver un export manuel non protégé).

© 2026 Oussama Ghorbel

Partie 3 — Construire des applications visuelles qui tiennent la route

© 2026 Oussama Ghorbel

Chapitre 10 – Bubble : le couteau suisse du SaaS no-code

Parmi tous les outils que nous avons traversés jusqu'ici, il en est un qui occupe une place à part dans l'écosystème no-code. Ce n'est ni le plus simple, ni le plus rapide à prendre en main, ni le plus élégant visuellement. Mais c'est celui vers lequel on revient, encore et encore, dès qu'un projet dépasse le stade du prototype pour devenir un véritable produit. Bubble n'est pas un builder de pages, un générateur de formulaires ou un habillage de base de données : c'est un environnement de développement complet, capable de produire des applications web fonctionnelles, avec leur propre logique métier, leur propre base de données et leurs propres integrations. Si Airtable est le tableur qui a appris le relationnel, Bubble est le SaaS qui a appris à se passer de développeurs.

Je dis cela sans angélisme. Bubble a une courbe d'apprentissage réelle, une dette technique possible, et des limites qu'il faut savoir identifier avant de s'engager. Mais pour quiconque souhaite lancer un produit, valider un marché, ou construire un outil métier sur mesure sans recruter une équipe technique, c'est aujourd'hui l'option la plus aboutie du marché. Ce chapitre est une immersion. Nous allons ouvrir l'éditeur, comprendre sa logique, et apprendre à construire des applications qui tiennent la route — pas des maquettes cliquables, mais de vrais produits.

10.1. Philosophie Bubble : pages, éléments, workflows, données

Bubble repose sur quatre piliers qui structurent toute application. Les comprendre, c'est comprendre 80 % de l'outil. Les 20 % restants, ce sont les cas particuliers, les optimisations, et les pièges que nous aborderons au fil du chapitre.

Les pages sont les écrans de votre application. Chaque page a une URL, peut recevoir des paramètres, et peut être protégée par des règles d'accès. Contrairement à une Single Page Application traditionnelle où tout se passe dans un seul fichier HTML, Bubble vous encourage à penser en termes d'écrans distincts : une page d'accueil, une page de

connexion, un tableau de bord, une page de détail, etc. C'est une approche plus proche du web classique que des frameworks modernes, et c'est un choix assumé : elle rend l'application plus lisible, plus facile à déboguer, et plus simple à faire évoluer.

Les éléments visuels (ou *visual elements*) sont tout ce que l'utilisateur voit et avec quoi il interagit : textes, images, boutons, groupes, inputs, repeating groups, maps, charts. Chaque élément possède des propriétés (couleur, taille, visibilité, condition d'affichage) et peut être relié à des données. C'est ici que réside la première grande force de Bubble : un élément n'est pas une simple boîte colorée, c'est un objet qui peut *afficher* une donnée, *déclencher* une action, et *changer d'état* en fonction du contexte. Un groupe peut contenir une liste de commandes, un bouton peut devenir invisible si l'utilisateur n'est pas administrateur, un texte peut afficher le nom du client connecté. Tout est dynamique par défaut.

Les workflows sont le cerveau de l'application. Un workflow se déclenche sur un événement (un clic, un chargement de page, une modification de données, un appel API) et exécute une série d'actions dans l'ordre. Ces actions peuvent être de plusieurs types : modification de la base de données, navigation, affichage d'alerte, envoi d'email, appel à une API externe, lancement d'un backend workflow. C'est dans les workflows que s'exprime la logique métier. Un bouton "Ajouter au panier" ne fait pas que changer une couleur : il crée une ligne dans la base de données, met à jour le total, déclenche une notification, et redirige l'utilisateur.

Les données (ou *data types*) constituent la colonne vertébrale. Dans l'onglet Data, vous définissez vos types d'objets (User, Product, Order, Comment…) et leurs champs. Chaque champ a un type (texte, nombre, date, image, oui/non, liste, ou référence vers un autre type). Bubble gère automatiquement la persistance, les relations, et les index. Pas de schéma SQL à écrire, pas de migration à orchestrer : vous ajoutez un champ, il existe. Vous changez son type, Bubble adapte. Cette flexibilité est une bénédiction en phase de conception, et un piège en phase de production si vous n'avez pas pensé votre modèle en amont — nous y reviendrons au chapitre 15 sur la maintenance.

© 2026 Oussama Ghorbel

La philosophie générale est donc la suivante : **l'interface est une projection de données, et les workflows sont les transformations qui relient les événements utilisateur aux modifications de ces données.** Tout le reste — plugins, API, JavaScript — n'est qu'extension de ce modèle.

10.2. Le responsive designer (nouveau moteur Flexbox)

Pendant des années, le responsive design a été le point faible de Bubble. L'ancien moteur, basé sur un positionnement fixe avec des options d'ancrage, produisait des interfaces qui se cassaient sur mobile, se chevauchaient sur tablette, et demandaient des heures de réglages manuels. En 2023, Bubble a introduit un nouveau moteur responsive basé sur Flexbox — le même modèle que celui utilisé par les développeurs front-end depuis des années. C'est un changement majeur, et il mérite qu'on s'y attarde.

Le principe de Flexbox est simple : un conteneur (appelé *container* dans Bubble) dispose ses enfants selon un axe (horizontal ou vertical), avec des règles d'alignement, d'espacement et de wrapping. Vous ne positionnez plus les éléments au pixel près : vous déclarez des intentions. "Ce groupe doit disposer ses enfants en ligne, avec 16 pixels d'écart, et passer en colonne sur mobile." L'outil s'occupe du reste.

Concrètement, chaque groupe peut maintenant être configuré comme un conteneur Flex. Vous définissez :

• **Layout type** : row (ligne) ou column (colonne).

• **Spacing** : l'espace entre les enfants, en pixels.

• **Alignment** : l'alignement sur l'axe principal et l'axe secondaire.

• **Wrapping** : si les enfants doivent passer à la ligne quand l'espace manque.

• **Padding** : l'espace intérieur du conteneur.

Chaque enfant, à son tour, possède des propriétés Flex : **min width**, **max width**, **flex grow** (la capacité à occuper l'espace disponible), **fixed width** (une largeur imposée). C'est cette combinaison qui permet de construire des mises en page complexes sans code : une barre latérale fixe à gauche, un contenu qui s'étend au centre, un panneau secondaire à

© 2026 Oussama Ghorbel

droite, le tout qui se réorganise en colonne sur mobile.

L'astuce qui a permis de sauver des dizaines de projets, c'est le **breakpoint mobile**. Plutôt que de tenter de faire tenir une interface desktop sur un écran de 375 pixels, vous créez un breakpoint à 768 pixels et vous basculez vers une disposition entièrement différente. Un conteneur qui était en row devient column, certains éléments disparaissent, d'autres apparaissent. Bubble gère cela via des conditions sur la largeur de la page (`Current page width < 768`), combinées à des conteneurs différents affichés selon le contexte.

Trois règles d'or pour un responsive réussi dans Bubble :

1. **Penser en conteneurs imbriqués, jamais en éléments flottants.** Un écran est une arborescence de conteneurs, pas une collection d'éléments positionnés librement.

2. **Utiliser des largeurs relatives et des max-width.** Un conteneur à `max-width: 1200px` centré avec des marges automatiques fonctionnera sur tous les écrans larges sans débordement.

3. **Tester systématiquement sur trois largeurs** : 1440 px (desktop), 768 px (tablette), 375 px (mobile). Si ces trois passent, le reste suivra.

Le nouveau moteur n'est pas parfait. Certaines mises en page avancées (grilles complexes, sticky elements sophistiqués) demandent encore des contorsions. Mais pour 95 % des applications SaaS, il est largement suffisant — et surtout, il rend le travail de design itératif au lieu de le rendre cauchemardesque.

10.3. Workflows : événements, actions, états, custom states

Si les données sont la colonne vertébrale d'une application Bubble, les workflows en sont le système nerveux. Chaque interaction utilisateur, chaque changement d'état, chaque appel externe passe par eux. Bien les concevoir, c'est construire une application maintenable. Les bâcler, c'est s'assurer des semaines de debug.

Un workflow se compose toujours d'un **déclencheur** (trigger) et d'une **suite d'actions**. Les déclencheurs les plus courants sont :

• **Button click** : l'utilisateur clique sur un bouton.

© 2026 Oussama Ghorbel

• **Page is loaded** : la page vient de s'ouvrir.

• **Input value changed** : un champ a été modifié.

• **Do when condition is true** : une condition devient vraie (utile pour les réactions automatiques).

• **Custom event** : un événement personnalisé que vous déclenchez vous-même depuis un autre workflow.

Les actions se classent en plusieurs catégories : **Data** (créer, modifier, supprimer des entrées en base), **Navigation** (changer de page, ouvrir un popup), **Elements** (modifier l'état d'un élément, afficher une alerte, réinitialiser un formulaire), **Plugins** (actions fournies par des plugins), **Backend** (lancer un workflow serveur). L'ordre des actions est crucial : elles s'exécutent séquentiellement, et une action peut dépendre du résultat de la précédente.

Un point souvent mal compris des débutants : **les custom states**. Un custom state est une variable temporaire attachée à un élément (ou à la page elle-même). Il permet de stocker une information sans toucher à la base de données. Exemples typiques : l'onglet actif d'une interface à onglets, un filtre sélectionné, un mode édition activé, une recherche en cours. Les custom states sont éphémères (ils disparaissent au rechargement de la page), rapides à modifier, et ne consomment pas de Workload Units. Ils sont indispensables pour construire des interfaces réactives sans surcharger la base.

Quelques patterns que je recommande systématiquement :

Le pattern "Custom event comme fonction". Plutôt que de dupliquer une suite d'actions dans dix workflows différents, isolez-la dans un custom event. Vous appelez ensuite cet événement depuis n'importe où, avec des paramètres. C'est l'équivalent no-code d'une fonction réutilisable, et cela réduit drastiquement la dette technique.

Le pattern "Do when condition is true pour les réactions dérivées". Si modifier une commande doit automatiquement mettre à jour le chiffre d'affaires du client, n'ajoutez pas cette action dans chaque workflow qui touche aux commandes. Créez un workflow `Do when Order's status is changed` qui s'en occupe. Votre logique devient déclarative plutôt que procédurale.

© 2026 Oussama Ghorbel

Le pattern "Only when pour les gardes". Chaque action peut avoir une condition `Only when`. Utilisez-la systématiquement pour éviter les actions inutiles : "Créer une notification *only when* le destinataire est différent de l'expéditeur", "Envoyer l'email *only when* l'adresse est renseignée". Ces gardes évitent des bugs silencieux et des consommations inutiles.

L'anti-pattern le plus fréquent, c'est le **workflow monstre** : un seul workflow de 40 actions qui gère l'inscription, la création du compte, l'envoi de l'email, la redirection, et la notification à l'admin. Illisible, impossible à déboguer, fragile à la moindre modification. Dès qu'un workflow dépasse 10-15 actions, c'est le signe qu'il faut le découper en custom events ou en backend workflows.

10.4. Backend workflows (tâches planifiées, API backend)

Les workflows que nous venons de voir s'exécutent côté client, dans le navigateur de l'utilisateur. Ils sont parfaits pour les interactions immédiates, mais inadaptés pour certaines tâches : traitement long, envoi massif d'emails, calculs lourds, synchronisations périodiques, exposition d'endpoints à des services externes. C'est là qu'interviennent les **backend workflows**.

Un backend workflow s'exécute sur les serveurs de Bubble, indépendamment de toute session utilisateur. Il peut être déclenché de trois manières :

1. **Depuis un workflow client**, via l'action *Schedule backend workflow* (exécution immédiate ou différée) ou *Schedule backend workflow on a list* (exécution sur chaque élément d'une liste).

2. **Par planification récurrente** (recursive workflow), où le workflow se programme lui-même à s'exécuter à nouveau après un délai donné. C'est la manière no-code de créer des cron jobs : synchroniser une base externe toutes les heures, envoyer un résumé quotidien, nettoyer des données obsolètes chaque nuit.

3. **Par appel API** (API endpoint), où le workflow expose une URL que des services externes peuvent appeler. Un webhook Stripe, une notification Shopify, un trigger Zapier peuvent ainsi déclencher directement une logique Bubble.

© 2026 Oussama Ghorbel

L'intérêt est triple. D'abord, **la performance** : un traitement lourd côté client bloque l'interface et peut faire planter le navigateur ; côté serveur, il tourne en arri re-plan. Ensuite, **la fiabilité** : un backend workflow n'est pas interrompu si l'utilisateur ferme l'onglet. Enfin, **la scalabilité** : Bubble répartit la charge sur ses serveurs, et vous pouvez suivre l'exécution dans les logs.

Deux outils complémentaires méritent d'être connus. Les **scheduled API workflows on a list** permettent d'exécuter un backend workflow sur chaque élément d'une liste, avec un intervalle configurable entre chaque exécution. Idéal pour envoyer 10 000 emails personnalisés sans exploser les quotas : vous programmez l'envoi à raison de 50 emails par minute, et Bubble s'occupe du reste. Les **recurring events**, définis dans l'onglet App Data, permettent de programmer des tâches à heure fixe (tous les jours à 3h, tous les lundis à 9h), sans passer par un workflow récursif.

Un piège classique : **oublier qu'un backend workflow n'a pas accès au contexte client**. Pas de `Current user`, pas de `Current page`, pas d'éléments visuels. Tout ce dont il a besoin doit lui être passé en paramètre. C'est une contrainte, mais aussi une discipline saine : elle force à concevoir des workflows autonomes et testables.

10.5. Plugins, API Connector et JavaScript-to-Bubble

Bubble, aussi complet soit-il, ne peut pas tout faire en natif. C'est là que les extensions entrent en jeu, selon trois modalités complémentaires.

Les plugins sont des extensions installées depuis le marketplace Bubble. Ils ajoutent des éléments visuels (charts, éditeurs riches, viewers PDF, cartes avancées), des actions de workflow (intégration Stripe, envoi via SendGrid, manipulation de fichiers), ou les deux. Le marketplace contient plusieurs milliers de plugins, gratuits ou payants. Avant d'en installer un, posez-vous trois questions : est-il maintenu (dernière mise à jour récente) ? a-t-il une base installée significative ? le développeur est-il réactif sur le forum ? Un plugin abandonné est une dette technique qui vous reviendra en pleine figure au prochain changement d'API du service sous-jacent.

© 2026 Oussama Ghorbel

L'API Connector est l'outil natif pour dialoguer avec n'importe quelle API REST. Vous définissez des appels (GET, POST, PUT, DELETE) en spécifiant l'URL, les headers, les paramètres, le body, et l'authentification. Une fois l'appel configuré, il devient une action de workflow utilisable comme n'importe quelle autre. Vous pouvez aussi l'utiliser comme source de données pour un repeating group ou un champ dynamique. L'API Connector gère l'OAuth2 pour les services qui le demandent, ce qui évite de manipuler manuellement les tokens de refresh.

La méthodologie que je recommande pour intégrer une API externe est la suivante : commencer par tester l'appel dans Postman ou équivalent, pour bien comprendre la structure de requête et de réponse. Reproduire ensuite cette configuration dans l'API Connector, en utilisant l'option *Initialize call* pour vérifier que la réponse est bien parsée. Enfin, créer les actions ou les data sources correspondantes, et les utiliser dans les workflows. Ne jamais tester une API pour la première fois directement dans un workflow de production.

JavaScript-to-Bubble est la porte de secours pour tout ce que Bubble ne sait pas faire nativement. Via l'élément *Javascript to Bubble* (fourni par le plugin officiel Toolbox, ou désormais en natif dans certains contextes), vous pouvez exécuter du code JavaScript et récupérer le résultat dans Bubble. C'est utile pour : des calculs complexes, des manipulations de dates non prévues, des appels à des librairies JavaScript spécifiques, des interactions avec le DOM pour des besoins ponctuels. Attention toutefois : chaque recours à du JavaScript est un aveu d'échec partiel du no-code. À utiliser avec parcimonie, à documenter rigoureusement, et à isoler dans des custom events pour faciliter une éventuelle réécriture future.

Une règle éprouvée, à se fixer et à appliquer systématiquement : **si vous écrivez plus de 20 lignes de JavaScript dans une application Bubble, c'est que vous avez mal choisi votre stack.** Soit Bubble n'est pas l'outil adapté à ce projet spécifique, soit vous n'avez pas assez cherché le plugin ou l'endpoint API qui fait le travail. Le code light est une béquille, pas un mode de développement.

10.6. Gestion des utilisateurs, rôles, Privacy Rules

© 2026 Oussama Ghorbel

Toute application sérieuse gère des utilisateurs et des permissions. Bubble fournit un type de données `User` préconfiguré, avec email et mot de passe, et un ensemble d'actions d'authentification (signup, login, logout, reset password, email confirmation). Mais la vraie puissance réside dans les **Privacy Rules**, un mécanisme souvent sous-utilisé et pourtant essentiel.

Le type `User` peut être enrichi de champs personnalisés : nom, prénom, rôle, organisation, préférences, etc. Pour gérer des rôles, deux approches coexistent. L'approche simple consiste à ajouter un champ texte `role` (admin, manager, member) ou un booléen `is_admin`. L'approche plus robuste consiste à créer un type `Role` séparé, avec des permissions granulaires, et à y relier les utilisateurs — surtout si vous avez plus de trois ou quatre rôles, ou si les permissions évoluent souvent.

Les **Privacy Rules** sont la couche de sécurité côté serveur. Elles définissent, pour chaque type de données, qui a le droit de lire, modifier, créer ou supprimer quoi. Par exemple : "Un utilisateur peut lire ses propres commandes, mais pas celles des autres", "Un admin peut tout lire", "Un manager peut lire les commandes de son équipe uniquement". Ces règles s'appliquent **quoi qu'il arrive**, même si votre interface affiche par erreur des données auxquelles l'utilisateur ne devrait pas avoir accès. C'est une garantie fondamentale : la sécurité ne repose pas sur l'interface, mais sur le serveur.

Configurer les Privacy Rules demande de la rigueur. Voici la méthodologie à suivre :

1. **Lister tous les types de données** de l'application.
2. **Pour chaque type, identifier les rôles** qui interagissent avec.
3. **Pour chaque couple type/rôle, définir les quatre permissions** : lecture, création, modification, suppression.
4. **Écrire les règles sous forme de conditions** auto-descriptives, en utilisant des noms explicites ("Owner can view own orders", "Admin can modify all users").
5. **Activer l'option "Ignore privacy rules" uniquement** dans les workflows backend légitimes (tâches d'administration, synchronisations), et jamais dans un workflow client.

© 2026 Oussama Ghorbel

L'erreur la plus fréquente est de désactiver les Privacy Rules "pour tester" et d'oublier de les réactiver. Ou pire : de contourner une règle en utilisant `Do a search for` avec `Ignore privacy rules` dans un workflow client, ce qui revient à ouvrir une brèche béante. Une Privacy Rule mal configurée, c'est une fuite de données qui s'ignore. Testez systématiquement vos règles avec plusieurs comptes de test de rôles différents avant chaque mise en production.

Un dernier point sur l'authentification : Bubble supporte nativement l'authentification sociale (Google, Facebook) et peut être complété par des plugins pour d'autres fournisseurs (Auth0, Firebase, etc.). Pour les applications B2B avec SSO d'entreprise, il faudra souvent passer par un plugin dédié ou un middleware. C'est un des rares cas où Bubble montre ses limites et où une réflexion d'architecture s'impose avant de s'engager.

10.7. Performance, WU (Workload Units) et optimisation

Depuis 2023, Bubble facture une partie de ses plans selon les **Workload Units (WU)**, une unité de mesure de la consommation serveur. Chaque action — recherche en base, création d'entrée, appel API, exécution de workflow — consomme un certain nombre de WU. Comprendre ce modèle n'est pas qu'une question de coût : c'est une discipline de conception qui produit des applications plus rapides et plus robustes.

Les principaux consommateurs de WU sont, par ordre décroissant :

• **Les recherches en base de données** (*Do a search for*), surtout quand elles portent sur de gros volumes, sans contrainte, ou avec des tris coûteux.

• **Les workflows récursifs** mal calibrés, qui s'exécutent trop souvent.

• **Les appels API** externes, qui consomment à la fois des WU et du temps de réponse.

• **Les opérations sur listes** (filter, sort, :extract, :count) appliquées à des résultats volumineux.

• **Les changements d'état fréquents** qui déclenchent des recalculs en cascade.

© 2026 Oussama Ghorbel

Quelques principes d'optimisation à appliquer systématiquement :

Filtrer au plus tôt. Une recherche qui ramène 10 000 entrées puis filtre côté client est une aberration. Ajoutez des contraintes dans le `Do a search for` pour ne récupérer que ce dont vous avez besoin. Si vous affichez les 20 dernières commandes d'un client, filtrez par client et limitez à 20, avec un tri par date décroissante.

Éviter les recherches imbriquées dans les repeating groups. Un repeating group qui affiche 50 commandes, et dont chaque ligne fait une recherche pour récupérer le nom du client, déclenche 50 requêtes. Utilisez plutôt des relations directes (la commande a un champ `client` qui pointe vers le User) ou des champs dénormalisés (stocker le nom du client directement dans la commande à la création). La dénormalisation, souvent considérée comme une hérésie en SQL, est une pratique courante et saine en no-code quand elle réduit significativement les recherches.

Utiliser les agrégats serveur plutôt que les calculs client. `:count`, `:sum`, `:average` appliqués à une recherche s'exécutent côté serveur et consomment peu. Recalculer ces valeurs dans un workflow en itérant sur une liste consomme énormément.

Mettre en cache via des custom states ou des champs dédiés. Si un tableau de bord affiche des statistiques coûteuses à calculer, recalculez-les une fois par jour via un backend workflow et stockez le résultat dans un champ. L'interface n'a plus qu'à lire une valeur précalculée.

Surveiller les logs. L'onglet Logs de Bubble détaille la consommation WU de chaque workflow, de chaque page, de chaque action. C'est votre thermomètre. Un workflow qui consomme 10 000 WU par exécution est un workflow à refactorer.

Au-delà des WU, la performance perçue compte tout autant. Un utilisateur qui attend 3 secondes avant de voir quelque chose à l'écran aura l'impression d'une application lente, même si le traitement total est rapide. Utilisez les états de chargement (loading indicators), les skeleton screens, et les actions *Display data* pour afficher immédiatement ce qui est disponible, quitte à compléter ensuite.

© 2026 Oussama Ghorbel

Exercices pratiques

Les concepts de ce chapitre ne prennent leur sens qu'une fois mis en pratique. Voici trois exercices progressifs, conçus pour être réalisés dans cet ordre, chacun consolidant les acquis du précédent.

Exercice 1 — Construire un clone simplifié de Product Hunt en 4 heures.

Objectif : mettre en œuvre pages, données, workflows et responsive dans un projet concret.

Spécifications minimales :

• Trois types de données : `User` (déjà existant, à enrichir avec username, bio, avatar), `Product` (name, tagline, url, creator, upvotes_count, created_date), `Upvote` (user, product, created_date).

• Quatre pages : accueil (liste des produits du jour triés par upvotes), détail produit, soumission d'un produit, profil utilisateur.

• Workflows : soumission, upvote (avec contrainte : un seul upvote par utilisateur et par produit), édition de profil.

• Responsive : interface utilisable sur mobile avec un conteneur Flex qui passe en colonne sous 768 px.

Livrable : une application fonctionnelle, déployée sur un domaine `bubbleapps.io`, testable par un tiers. Temps cible : 4 heures pour un lecteur ayant suivi les chapitres précédents.

Exercice 2 — Créer un workflow d'inscription avec email de confirmation.

Objectif : maîtriser les backend workflows, les custom events, et la gestion d'états temporaires.

Étapes :

1. Créer un type `EmailToken` avec les champs : user, token (texte), created_date, used (booléen).

2. À l'inscription, créer un User non confirmé, générer un token aléatoire, créer un EmailToken, et déclencher un backend workflow qui envoie l'email avec le lien de confirmation.

© 2026 Oussama Ghorbel

3. Créer une page `/confirm` qui reçoit le token en paramètre, vérifie qu'il existe et n'a pas été utilisé, marque le User comme confirmé, et marque le token comme utilisé.

4. Ajouter une Privacy Rule qui empêche la connexion des utilisateurs non confirmés.

5. Implémenter un backend workflow récursif qui supprime chaque nuit les comptes non confirmés de plus de 7 jours.

Livrable : un parcours complet testable avec deux adresses email différentes, et une démonstration du nettoyage automatique.

Exercice 3 — Optimiser une page lente en réduisant les recherches en base.

Objectif : appliquer les principes de performance dans un contexte réel.

Contexte : on utilise (ou vous construisez) un tableau de bord qui affiche, pour un utilisateur connecté, la liste de ses 100 dernières commandes, avec pour chacune le nom du client, le montant, la date, et le nombre d'items. Dans sa version non optimisée, cette page fait une recherche par ligne pour récupérer le nom du client, et un `:count` sur les items par ligne.

Tâches :

1. Mesurer la consommation WU initiale via les logs.

2. Dénormaliser le nom du client dans la commande (via un workflow de création qui stocke `client's name`).

3. Ajouter un champ `items_count` mis à jour à chaque ajout ou retrait d'item.

4. Remplacer le repeating group par une recherche contrainte et limitée.

5. Mesurer la nouvelle consommation WU et documenter le gain.

Livrable attendu : un avant/après chiffré, avec une réduction cible d'au moins 70 % des WU consommés.

Ces trois exercices couvrent l'essentiel de ce que Bubble permet de faire dans un cadre SaaS standard. Les maîtriser, c'est être capable de

© 2026 Oussama Ghorbel

construire la grande majorité des applications que l'on vous demandera. Les chapitres suivants nous emmèneront vers d'autres outils, pour d'autres cas d'usage — mais Bubble restera, pour beaucoup de vos projets, le point de départ naturel.

© 2026 Oussama Ghorbel

Chapitre 11 – Glide, Softr, WeWeb, Toddle : apps à partir de données existantes

Dans le chapitre précédent, nous avons exploré Bubble comme un couteau suisse du SaaS no-code : puissant, complet, capable de tout faire, mais exigeant en courbe d'apprentissage. Il existe une autre famille d'outils, moins tape-à-l'œil mais extraordinairement utile au quotidien, que j'appelle les **générateurs de front sur données existantes**. Leur philosophie tient en une phrase : vous avez déjà vos données quelque part — dans Airtable, Google Sheets, SmartSuite, Xano ou Supabase — et vous voulez poser dessus une interface utilisateur propre, fonctionnelle et agréable, sans repartir de zéro.

Cette approche change radicalement la donne pour trois raisons. D'abord, elle élimine le problème le plus fréquent en no-code : la dette de modélisation. Si vos données sont déjà structurées, propres et accessibles, vous n'avez plus qu'à construire la couche visuelle. Ensuite, elle autorise une séparation saine entre le backend (vos données, votre logique) et le frontend (ce que voient vos utilisateurs), ce qui facilite la maintenance et les évolutions futures. Enfin, elle réduit considérablement le temps de mise sur le marché : là où un projet Bubble complet prend souvent plusieurs semaines, un portail Softr ou une app Glide peut être opérationnel en quelques jours, voire quelques heures.

Dans ce chapitre, nous allons décortiquer quatre outils représentatifs de cette catégorie, chacun avec sa personnalité, ses forces et ses limites. Mon objectif n'est pas de vous convertir à l'un d'entre eux, mais de vous donner les clés pour choisir celui qui correspond à votre contexte — car c'est presque toujours le contexte, et non l'outil, qui dicte la bonne décision.

11.1. Glide : des apps mobiles pros en quelques heures

Glide est né d'une intuition simple : la majorité des besoins internes en entreprise — suivi de stock, annuaire d'équipe, gestion

© 2026 Oussama Ghorbel

d'interventions, catalogue produits — ne justifient pas le développement d'une application native coûteuse. Un tableur bien structuré, une interface mobile soignée, et le tour est joué. Ce positionnement a fait le succès de Glide auprès des petites équipes opérationnelles, des associations et des indépendants qui avaient besoin d'outiller rapidement un processus métier sans passer par la case développement.

Le principe fondateur repose sur une connexion directe à une source de données — historiquement Google Sheets, puis Airtable, et plus récemment Glide Tables, leur propre base de données interne. Vous pointez Glide vers votre feuille de calcul, il en infère une structure, et vous propose une application mobile ou web pré-générée que vous n'avez plus qu'à ajuster. Le résultat par défaut est déjà remarquablement propre : Glide applique un design system cohérent, des composants éprouvés et des patterns d'interaction standards. Vous n'avez pas à vous soucier du responsive, des marges, des tailles de police — tout est pris en charge.

Les forces de Glide sont nombreuses. La première est la vitesse de prototypage : en une après-midi, vous pouvez passer d'une idée griffonnée sur un coin de table à une application utilisable par votre équipe. La deuxième est l'expérience mobile de qualité, souvent supérieure à ce que produisent des outils plus complexes quand on les confie à des débutants. La troisième est la gestion intégrée des utilisateurs : Glide propose un système d'authentification par email, des rôles, des permissions ligne par ligne (row-level security), ce qui permet de construire des applications multi-utilisateurs sans se perdre dans la complexité.

Les limites sont tout aussi réelles. Glide reste contraint par son design system : si vous voulez une interface qui sort vraiment du cadre, vous serez rapidement frustré. Les possibilités de personnalisation visuelle existent mais demeurent encadrées. De même, la logique applicative est volontairement simple — des actions, des conditions, des colonnes calculées — et ne prétend pas rivaliser avec les workflows complexes de Bubble. Enfin, la tarification, longtemps basée sur le nombre de lignes, a évolué vers un modèle par utilisateur privé, ce qui peut devenir significatif pour des déploiements à grande échelle.

© 2026 Oussama Ghorbel

Les cas d'usage typiques où Glide excelle incluent les applications de terrain (techniciens, commerciaux, livreurs), les annuaires et catalogues interactifs, les outils de reporting léger, et plus généralement tout ce qui relève de la consultation et de la mise à jour simple de données structurées. De nombreuses PME entières tournent leur gestion opérationnelle sur Glide pendant des années, avec une satisfaction réelle et des coûts maîtrisés.

Un point important sur Glide Tables : face aux limites de Google Sheets (performances dégradées au-delà de quelques milliers de lignes, absence de vrais types de données, risques de corruption accidentelle), Glide a développé sa propre base de données interne. C'est un choix stratégique qui renforce la fiabilité mais vous lie davantage à la plateforme. Si la portabilité des données est une priorité absolue pour vous, conservez une source externe (Airtable ou Xano) et utilisez Glide uniquement comme couche visuelle.

11.2. Softr : portails clients et internes sur Airtable/SmartSuite

Si Glide mise sur la simplicité et le mobile, Softr mise sur la puissance du portail web. L'outil s'est construit autour d'une promesse claire : transformer vos bases Airtable (et plus récemment SmartSuite) en applications web professionnelles, avec un accent particulier sur les portails clients, les espaces membres et les outils internes d'entreprise.

La philosophie de Softr est celle du *block-based building*. Vous construisez votre page en assemblant des blocs préconçus : une liste d'éléments, un formulaire, un tableau, un graphique, un bandeau hero, une section de témoignages. Chaque bloc se connecte à une vue Airtable spécifique et hérite automatiquement de ses filtres et de son tri. Cette approche est moins flexible qu'un éditeur libre, mais elle garantit une cohérence visuelle et accélère considérablement la production.

L'authentification et les permissions sont le véritable point fort de Softr. L'outil gère nativement les comptes utilisateurs, les groupes, les niveaux d'accès, et permet de restreindre la visibilité des données en fonction de l'utilisateur connecté. C'est exactement ce qu'il faut pour construire un portail client où chacun ne voit que ses propres factures, ses propres projets, ses propres documents. De nombreuses agences et

© 2026 Oussama Ghorbel

cabinets de conseil ont remplacé des espaces clients bricolés avec Dropbox et Excel par des portails Softr connectés à Airtable, avec un gain d'image professionnelle considérable.

La gestion des paiements est également intégrée, via Stripe. Vous pouvez proposer des abonnements, des achats uniques, des périodes d'essai, et conditionner l'accès à certaines parties de votre application au statut de paiement. Pour les créateurs de contenus, les formations en ligne, les SaaS légers ou les marketplaces de services, c'est un atout majeur qui évite d'ajouter une couche d'outils supplémentaires.

Les limites de Softr tiennent principalement à deux aspects. D'abord, la dépendance à Airtable ou SmartSuite : si votre source de données atteint les limites de ces plateformes (nombre d'enregistrements, complexité des relations, performances des formules), Softr héritera automatiquement de ces contraintes. Ensuite, la personnalisation visuelle reste encadrée : vous pouvez ajuster les couleurs, les typographies, les espacements, mais vous ne redessinerez pas librement l'interface. Pour des besoins de branding très poussés, il faudra accepter un compromis ou envisager un autre outil.

Les cas d'usage typiques incluent les portails clients (agences, consultants, freelances), les espaces membres (associations, communautés, formations), les annuaires professionnels (réseaux de partenaires, catalogues de prestataires), les outils RH internes (onboarding, annuaire, demandes de congés) et les marketplaces légères. Softr brille particulièrement quand l'application est centrée sur l'accès personnalisé à des données et à des documents, avec une dimension transactionnelle modérée.

Un conseil pratique : avant de vous lancer dans Softr, structurez soigneusement votre base Airtable. Les vues que vous créez dans Airtable deviendront les blocs de votre application. Une base bien pensée, avec des vues nommées clairement et des filtres cohérents, donnera une application Softr limpide. Une base brouillonne donnera une application brouillonne — et le problème ne viendra pas de Softr.

11.3. WeWeb : front-end avancé connecté à Xano/Supabase

© 2026 Oussama Ghorbel

WeWeb représente une étape supplémentaire dans la sophistication. Là où Glide et Softr vous proposent des applications « clé en main » avec un design system imposé, WeWeb vous donne un éditeur visuel beaucoup plus libre, comparable à ce que fait Webflow pour les sites web, mais orienté applications. Et surtout, WeWeb a fait un choix stratégique fort : il ne gère pas les données lui-même. Il se connecte à des backends externes — Xano, Supabase, Airtable, des API REST, GraphQL ou même des bases SQL — et vous laisse construire l'interface que vous voulez par-dessus.

Cette séparation frontend/backend est un changement de paradigme pour beaucoup de makers no-code. Elle implique que vous devez maîtriser deux outils au lieu d'un, mais elle offre en retour une liberté et une robustesse incomparables. Votre backend peut évoluer, migrer, être partagé entre plusieurs frontends (web, mobile, application tierce) sans que votre interface WeWeb n'ait à être refaite. Votre frontend peut être complètement redesigné sans toucher à la structure de vos données. C'est la même architecture que celle utilisée par les équipes de développement professionnelles, rendue accessible en no-code.

L'éditeur de WeWeb ressemble à un outil de design. Vous positionnez des éléments librement sur une grille, vous gérez les flexbox, les grilles CSS, les états (hover, focus, actif), les variables, les conditions d'affichage. La courbe d'apprentissage est plus marquée que sur Glide ou Softr, mais le résultat peut être visuellement indiscernable d'une application développée sur mesure. Pour les projets où l'image de marque et l'expérience utilisateur sont critiques — levées de fonds, produits grand public, SaaS ambitieux — cette liberté a une valeur considérable.

La connexion à Xano est particulièrement aboutie. Xano, que nous avons évoqué au chapitre 8, est un backend no-code puissant qui gère les bases de données PostgreSQL, les endpoints API, l'authentification, la logique métier. Associé à WeWeb, il forme une stack capable de rivaliser avec des applications codées traditionnellement. Vous gardez la rapidité d'exécution du no-code, tout en disposant d'une architecture scalable, documentée et portable.

Supabase est l'autre backend privilégié. Open-source, basé sur PostgreSQL, Supabase séduit par sa transparence et son absence de

© 2026 Oussama Ghorbel

verrouillage propriétaire. La connexion avec WeWeb fonctionne bien, même si elle demande un peu plus de configuration manuelle qu'avec Xano. Pour les projets soucieux de souveraineté des données ou de coûts maîtrisés à grande échelle, le duo WeWeb + Supabase est une option très pertinente.

Les limites de WeWeb sont le reflet de ses forces. La liberté implique la responsabilité : si vous construisez mal votre interface, elle sera mauvaise, et l'outil ne vous sauvera pas. La gestion des états, des bindings de données, des actions complexes demande une vraie rigueur intellectuelle — c'est presque de la programmation visuelle. De plus, WeWeb ne gère pas nativement l'authentification utilisateur complète : vous devez vous appuyer sur Xano, Supabase ou un service tiers comme Auth0, ce qui ajoute une brique à maîtriser.

Les cas d'usage typiques incluent les SaaS ambitieux, les dashboards analytiques sur mesure, les applications métiers complexes (CRM, ERP légers, outils de gestion de projet), et plus généralement tout projet où la qualité de l'interface et la scalabilité du backend sont prioritaires. WeWeb s'adresse à des makers déjà à l'aise avec les concepts no-code, ou à des équipes mixtes associant un profil technique et un profil produit.

Une mise en garde : la tentation est grande, face à la liberté de WeWeb, de vouloir tout faire soi-même et de reproduire des patterns complexes. Résistez. Commencez par des composants simples, utilisez les templates proposés, et n'ajoutez de la complexité que lorsque le besoin est avéré. Une application WeWeb sobre et bien structurée battra toujours une application sophistiquée mais mal maintenue.

11.4. Toddle : le nouveau venu orienté performance

Toddle est un outil plus récent sur la scène no-code, et il mérite qu'on s'y arrête car il incarne une tendance de fond : la montée en gamme technique du no-code, avec une attention particulière portée à la performance, à la qualité du code généré et à l'expérience développeur. Là où certains outils historiques traînent une dette technique accumulée pendant des années, Toddle a pu faire des choix architecturaux modernes dès le départ.

© 2026 Oussama Ghorbel

La philosophie de Toddle est double. D'une part, proposer un éditeur visuel accessible à des non-développeurs, avec le drag-and-drop, les composants réutilisables, les bindings de données attendus. D'autre part, produire en sortie du code de qualité, rapide, optimisé pour le SEO quand c'est pertinent, et exportable. Cette promesse de « code propre généré visuellement » séduit particulièrement les profils techniques qui ont longtemps boudé le no-code par crainte du verrouillage propriétaire et des performances dégradées.

L'éditeur est volontairement proche des outils de développement professionnels. On y retrouve des concepts familiers pour les développeurs : composants, props, états, formules, variables, actions. La prise en main peut dérouter un pur maker no-code habitué aux interfaces plus guidées, mais elle parle immédiatement à quelqu'un qui a déjà fait du React ou du Vue sans avoir envie d'y revenir. Toddle assume cette position : le no-code n'est pas une régression par rapport au code, c'est une autre façon d'exprimer la même logique.

Les performances sont l'argument massue. Les applications Toddle se chargent vite, réagissent instantanément, et supportent bien la charge. C'est le résultat de choix techniques assumés : rendu optimisé, hydratation minimale, gestion fine des re-rendus. Pour des applications grand public où chaque milliseconde compte — e-commerce, médias, outils viraux — cet avantage est réel et mesurable.

Les connexions aux données se font via des API, ce qui laisse une grande liberté de backend. Xano, Supabase, Airtable, ou n'importe quel service exposant une API REST ou GraphQL peut être intégré. Toddle propose également un système de formules puissant pour manipuler les données côté client, avec une syntaxe qui rappelle les tableurs mais avec la rigueur d'un langage de programmation.

Les limites tiennent surtout à la jeunesse de l'outil. La communauté est plus petite que celle de Bubble, Glide ou Softr, ce qui signifie moins de templates, moins de tutoriels, moins de réponses toutes faites sur les forums. L'écosystème de plugins et d'intégrations natives est encore en construction. Enfin, la courbe d'apprentissage, plus technique, peut rebuter les profils purement métier qui veulent un résultat immédiat sans comprendre les concepts sous-jacents.

© 2026 Oussama Ghorbel

Les cas d'usage typiques incluent les applications web performantes grand public, les SaaS où la vitesse d'exécution est un argument concurrentiel, les projets portés par des équipes techniques ou mixtes qui veulent garder le contrôle sur la qualité du résultat, et les applications dont on anticipe qu'elles devront un jour être exportées ou reprises par une équipe de développement classique.

Mon conseil : ne choisissez pas Toddle parce que c'est nouveau, choisissez-le parce que votre projet a réellement besoin de ses forces — performance, qualité du code exporté, contrôle fin. Pour un portail client standard ou un outil interne, Glide ou Softr feront le travail plus vite et avec moins d'effort. Pour un produit grand public ambitieux, Toddle peut être le choix qui vous fera gagner des mois sur la route.

11.5. Comparatif : quand choisir lequel

Face à ces quatre outils, la tentation est grande de chercher « le meilleur ». Cette question n'a pas de sens en dehors d'un contexte. Voici plutôt une grille de lecture pour vous aider à trancher en fonction de votre situation.

Choisissez Glide si :

• Votre priorité est la rapidité de mise en production.

• Vous visez une expérience mobile de qualité sans effort.

• Votre logique applicative reste simple (consultation, mise à jour, filtres basiques).

• Votre équipe utilisatrice est opérationnelle, pas technique.

• Votre budget est contraint et votre périmètre bien défini.

Choisissez Softr si :

• Vous construisez un portail client, un espace membre ou un outil interne centré sur des données Airtable ou SmartSuite.

• L'authentification, les permissions fines et la gestion des paiements sont centrales.

• Vous acceptez un design system encadré en échange d'une production rapide et fiable.

• Votre équipe interne peut gérer Airtable comme source de vérité.

© 2026 Oussama Ghorbel

Choisissez WeWeb si :

• Vous avez besoin d'une interface sur mesure, visuellement aboutie, qui doit rivaliser avec des applications codées.

• Vous voulez séparer clairement frontend et backend pour des raisons de maintenabilité ou de scalabilité.

• Vous êtes prêt à investir du temps dans l'apprentissage d'un outil plus exigeant.

• Votre backend est (ou sera) Xano ou Supabase.

• Votre projet a une ambition produit forte — levée de fonds, lancement grand public, SaaS à long terme.

Choisissez Toddle si :

• La performance et la qualité du code généré sont des critères critiques.

• Vous avez des compétences techniques dans l'équipe ou vous êtes vous-même à l'aise avec les concepts de développement.

• Vous anticipez un besoin futur d'exporter le code ou de faire reprendre le projet par une équipe de développeurs.

• Votre projet vise un large public et la vitesse de chargement est un avantage concurrentiel.

Tableau de synthèse :

Critère	Glide	Softr	WeWeb	Toddle
Courbe d'apprentissage	Faible	Faible à moyenne	Moyenne à élevée	Moyenne à élevée
Liberté visuelle	Limitée	Encadrée	Très élevée	Très élevée
Backend natif	Glide Tables, Sheets, Airtable	Airtable, SmartSuite	Aucun (Xano, Supabase, API)	Aucun (API)
Authentification native	Oui	Oui (solide)	Via backend externe	Via backend externe
Performance	Bonne	Bonne	Très bonne	Excellente
Paiements intégrés	Limités	Oui (Stripe)	Via backend	Via backend
Export du code	Non	Non	Partiel	Oui

© 2026 Oussama Ghorbel

Écosystème et communauté	Large	Large	Croissant	Émergent

Une règle empirique qui a souvent fait ses preuves : commencez par l'outil le plus simple qui puisse répondre à votre besoin. Si Glide suffit, n'allez pas sur WeWeb sous prétexte qu'il est plus puissant. La puissance non utilisée est une dette qui pèse — sur votre temps d'apprentissage, sur votre maintenance, sur votre charge mentale. Passez à l'outil plus sophistiqué uniquement quand vous avez rencontré une limite concrète de l'outil simple, pas par anticipation.

11.6. Personnalisation visuelle sans casser la maintenabilité

Une question revient systématiquement quand on aborde ces outils : jusqu'où peut-on personnaliser l'interface sans compromettre la maintenabilité à long terme ? C'est une question cruciale, car une application no-code mal designée devient très vite un cauchemar à faire évoluer. Voici les principes que je recommande.

Commencez par un design system, pas par des pages. Avant même d'ouvrir l'outil, définissez votre palette de couleurs (trois à cinq couleurs maximum), votre typographie (deux familles, une pour les titres, une pour le corps), vos espacements (une échelle cohérente, par exemple 4, 8, 16, 24, 32 pixels), vos composants récurrents (boutons, cartes, formulaires). Notez tout cela dans un document simple, partagé avec toute l'équipe. Ce design system sera votre boussole : chaque page, chaque composant devra s'y conformer.

Utilisez les variables et les styles globaux. Les quatre outils proposent, à des degrés divers, la possibilité de définir des variables de couleur, de typographie, d'espacement. Utilisez-les systématiquement. Le jour où vous devrez changer le bleu principal de votre application, vous le ferez en un endroit, et toute l'application suivra. Si vous avez codé les couleurs en dur dans chaque composant, vous passerez des heures à chasser les occurrences oubliées.

Créez des composants réutilisables. Un bouton, une carte produit, un en-tête de section, un bloc de témoignage : chaque élément qui apparaît plusieurs fois dans votre application doit devenir un composant. Vous le modifierez une fois, et toutes ses instances se mettront à jour.

© 2026 Oussama Ghorbel

C'est un investissement initial, mais c'est ce qui différencie une application no-code maintenable d'un bricolage qui s'effondre à la troisième modification.

Résistez à la surpersonnalisation. Chaque écart au design system, chaque composant unique, chaque exception doit être justifié. Posez-vous la question : cette spécificité apporte-t-elle une vraie valeur à l'utilisateur, ou est-ce une préférence esthétique personnelle ? Dans 80 % des cas, la réponse est la seconde, et il vaut mieux revenir au standard. Les utilisateurs se moquent de savoir si votre bouton a un coin arrondi de 8 ou de 12 pixels ; ils veulent qu'il soit lisible, cliquable et cohérent avec le reste.

Documentez vos choix. Quand vous prenez une décision de design non triviale — un composant complexe, un pattern d'interaction particulier, une intégration spécifique — notez-la. Un simple document Notion avec captures d'écran et explications suffit. Dans six mois, quand vous reviendrez sur l'application ou que quelqu'un d'autre devra la maintenir, cette documentation vaudra de l'or.

Testez sur de vrais utilisateurs tôt. La tentation est grande de peaufiner l'interface pendant des semaines avant de la montrer. Résistez. Montrez une version imparfaite mais fonctionnelle à trois ou quatre utilisateurs réels dès la première semaine. Leurs retours vous éviteront des heures de travail sur des détails dont ils se moquent, et vous orienteront vers les vrais problèmes à résoudre.

Pensez responsive dès le début. Même si votre application est d'abord destinée au desktop, construisez-la en gardant en tête qu'elle devra fonctionner sur tablette et mobile. Les quatre outils gèrent le responsive, mais à des degrés divers : Glide le fait pour vous, Softr propose des vues adaptatives, WeWeb et Toddle vous laissent la main sur les breakpoints. Intégrer le responsive en fin de projet coûte toujours plus cher que de le prévoir dès la conception.

Séparez le design de la logique. Autant que possible, gardez les règles métier (qui voit quoi, ce qui déclenche quoi, les calculs) dans des endroits distincts de la présentation visuelle. Quand vous modifierez votre logique, vous ne voulez pas risquer de casser l'interface. Quand vous redesignerez l'interface, vous ne voulez pas risquer de casser la

© 2026 Oussama Ghorbel

logique. Cette séparation, naturelle sur WeWeb et Toddle grâce à la séparation frontend/backend, demande un effort conscient sur Glide et Softr.

En suivant ces principes, vous construirez des applications qui vieillissent bien. C'est peut-être le critère le plus sous-estimé en no-code : beaucoup d'outils permettent de produire vite, peu permettent de maintenir sereinement. La différence se joue dans la discipline de conception, pas dans la puissance de l'outil.

Exercices pratiques

Les trois exercices qui suivent sont conçus pour être réalisés dans l'ordre, en y consacrant entre deux et quatre heures chacun. Ils couvrent les trois grands cas d'usage abordés dans ce chapitre : l'app mobile de terrain, le portail client web, et le dashboard analytique avancé. Ne cherchez pas la perfection du premier coup ; cherchez à comprendre les mécanismes et à identifier les points de friction.

Exercice 1 — Générer une app mobile de gestion de stock depuis Google Sheets avec Glide

Objectif : construire une application utilisable par une équipe de magasiniers pour consulter et mettre à jour un stock en temps réel, avec scan de code-barres et historique des mouvements.

Étapes :

1. Créez une feuille Google Sheets avec trois onglets : `Produits` (référence, nom, catégorie, emplacement, stock actuel, seuil d'alerte), `Mouvements` (date, produit, type — entrée/sortie, quantité, utilisateur, commentaire), `Utilisateurs` (email, nom, rôle).
2. Remplissez avec une vingtaine de produits réalistes et quelques mouvements historiques.
3. Créez un compte Glide et connectez votre feuille Google Sheets comme source de données.
4. Configurez les relations : chaque mouvement est lié à un produit et à un utilisateur.

© 2026 Oussama Ghorbel

5. Construisez les écrans suivants :

• Liste des produits avec recherche, filtre par catégorie, indicateur visuel quand le stock est sous le seuil d'alerte.

• Fiche produit affichant le détail, l'historique des mouvements et un bouton pour enregistrer un nouveau mouvement.

• Formulaire d'ajout de mouvement avec sélection du produit, choix du type, saisie de la quantité.

• Écran de scan (utilisez le composant de scan de code-barres de Glide) qui préremplit le produit et ouvre directement le formulaire de mouvement.

6. Configurez une colonne calculée dans Glide qui met à jour automatiquement le stock actuel à partir de la somme des mouvements.

7. Activez l'authentification par email et limitez l'accès aux utilisateurs listés dans l'onglet `Utilisateurs`.

8. Testez sur votre téléphone avec deux ou trois utilisateurs réels et notez les frictions.

Aller plus loin : ajoutez une notification automatique (via un webhook vers Make ou Zapier) quand un produit passe sous son seuil d'alerte.

Exercice 2 — Construire un portail client Softr avec authentification et factures

Objectif : livrer à un cabinet de conseil un espace où chaque client peut consulter ses projets en cours, ses documents partagés et télécharger ses factures.

Étapes :

1. Créez une base Airtable avec quatre tables : `Clients` (entreprise, email, contact principal, statut), `Projets` (nom, client, statut, date début, date fin, description), `Documents` (nom, projet, type, URL fichier, date upload), `Factures` (numéro, client, montant, date émission, statut paiement, URL PDF).

2. Créez des vues dédiées dans Airtable : une vue `Projets du client` filtrée pour n'afficher que les projets actifs, une vue `Factures du

© 2026 Oussama Ghorbel

client` triée par date décroissante.

3. Remplissez avec trois clients fictifs, deux projets par client, quelques documents et factures.

4. Créez un compte Softr et connectez votre base Airtable.

5. Activez l'authentification par email et configurez les groupes d'utilisateurs (un groupe par client, basé sur le champ email).

6. Construisez les pages suivantes :

• Page d'accueil personnalisée avec le nom du client, un résumé de ses projets actifs et ses dernières factures.

• Page `Mes projets` listant les projets du client connecté, avec fiches détaillées incluant les documents associés.

• Page `Mes factures` avec tableau filtré sur le client, téléchargement des PDF, indicateur de statut de paiement.

• Page `Profil` permettant au client de modifier ses informations de contact.

7. Configurez les permissions pour que chaque client ne voie que ses propres données (filtres dynamiques basés sur l'email de l'utilisateur connecté).

8. Ajoutez un bloc de soumission de demande (formulaire Softr) permettant au client d'envoyer une demande qui crée un nouvel enregistrement dans une table `Demandes` Airtable.

9. Testez en vous connectant avec les trois emails clients et vérifiez l'isolation des données.

Aller plus loin : connectez Stripe pour permettre le paiement en ligne des factures impayées, avec mise à jour automatique du statut dans Airtable via Make.

Exercice 3 — Créer un dashboard WeWeb branché sur Xano avec filtres dynamiques

Objectif : construire un tableau de bord analytique pour une équipe commerciale, affichant les performances par vendeur, par région et par période, avec filtres interactifs.

Étapes :

© 2026 Oussama Ghorbel

1. Créez un projet Xano avec trois tables : `Vendeurs` (nom, région, email, objectif annuel), `Clients` (nom, région, segment, vendeur attribué), `Ventes` (date, client, vendeur, montant, produit, statut).

2. Importez un jeu de données réaliste : une trentaine de vendeurs, deux cents clients, mille ventes réparties sur les douze derniers mois.

3. Créez des endpoints API Xano qui retournent :

• Le chiffre d'affaires agrégé par mois, avec possibilité de filtrer par région, vendeur, segment.

• Le classement des vendeurs avec CA réalisé et pourcentage de l'objectif.

• L'évolution du CA par région sur douze mois.

• Le top 10 des clients par CA.

4. Créez un projet WeWeb et connectez-le à votre API Xano.

5. Configurez l'authentification via Xano (login, token JWT conservé en état global).

6. Construisez le dashboard avec :

• Une barre de filtres en haut : période (date début / date fin), région (menu déroulant multi-sélection), vendeur (menu déroulant).

• Quatre KPI principaux en haut de page (CA total, nombre de ventes, panier moyen, taux de conversion) qui se recalculent en fonction des filtres.

• Un graphique en courbes de l'évolution mensuelle du CA.

• Un tableau des vendeurs avec tri sur les colonnes et barre de progression vers l'objectif.

• Un graphique en barres du CA par région.

7. Implémentez la réactivité : quand un filtre change, tous les composants dépendants se rechargent automatiquement via des bindings WeWeb.

8. Ajoutez un état de chargement (skeleton ou spinner) pendant le rafraîchissement des données.

9. Gérez le cas d'erreur réseau avec un message explicite et un bouton de retry.

© 2026 Oussama Ghorbel

10. Testez les performances avec le jeu de données complet et optimisez les endpoints Xano si les temps de réponse dépassent 500 ms.

Aller plus loin : ajoutez une fonctionnalité d'export CSV des données filtrées et une option de sauvegarde des filtres favoris par utilisateur.

Ces trois exercices vous donneront une expérience concrète des trois grands paradigmes couverts dans ce chapitre : la rapidité de Glide, la solidité des portails Softr, la puissance du duo WeWeb + Xano. Chacun correspond à un type de projet que vous rencontrerez probablement dans votre parcours no-code. Gardez en tête que le choix de l'outil est rarement la décision la plus importante : la clarté du besoin, la qualité de vos données et la discipline de conception pèsent bien davantage dans la réussite finale.

© 2026 Oussama Ghorbel

Chapitre 12 – Power Apps, AppSheet, Retool : l'entreprise passe au low-code

Dans les chapitres précédents, nous avons exploré des outils comme Bubble, Glide ou Softr. Ces plateformes sont extraordinaires pour lancer un produit, créer une application client ou automatiser un processus métier de A à Z en partant de zéro. Mais lorsque l'on franchit les portes d'une grande entreprise, d'une ETI ou d'une administration, la réalité change radicalement. L'entreprise ne part jamais de zéro. Elle possède déjà des bases de données relationnelles massives, des annuaires d'employés stricts, des politiques de sécurité draconiennes et un écosystème logiciel verrouillé par sa Direction des Systèmes d'Information (DSI).

C'est ici qu'intervient le **low-code d'entreprise**. Contrairement au no-code pur qui vise à contourner l'IT pour créer de la valeur rapidement (ce qu'on appelle le *Shadow IT*), le low-code d'entreprise cherche à intégrer les "citizen developers" (les métiers) dans un cadre sécurisé, tout en offrant aux développeurs professionnels des accélérateurs de productivité. Les géants du secteur ont compris que l'avenir du développement interne ne résidait plus dans la rédaction manuelle de interfaces CRUD (Create, Read, Update, Delete) en React ou Angular, mais dans l'assemblage visuel de composants connectés aux données existantes.

Dans ce chapitre, nous allons décortiquer les trois piliers de cette révolution : l'omniprésence de Microsoft avec Power Apps, l'approche pragmatique et terrain de Google avec AppSheet, et la philosophie "developer-first" de Retool pour les équipes techniques et opérationnelles.

12.1. Power Apps : canvas vs model-driven, intégration Microsoft 365

Microsoft Power Apps n'est pas seulement un outil ; c'est une pièce maîtresse de la stratégie Microsoft 365. Si votre entreprise utilise déjà Teams, SharePoint, Azure Active Directory (désormais Entra ID) et Dynamics 365, Power Apps est presque inévitable. La plateforme se

© 2026 Oussama Ghorbel

divise en deux grandes philosophies de création, qu'il est crucial de comprendre pour ne pas faire fausse route dès le premier jour.

Les Canvas Apps (Applications Canevas)

Imaginez une toile blanche sur laquelle vous glissez-déposez des boutons, des galeries, des champs de texte et des images. Vous avez le contrôle absolu sur le design, le positionnement au pixel près et l'expérience utilisateur. Les Canvas Apps sont idéales pour des applications mobiles, des formulaires de saisie terrain, ou des interfaces très spécifiques destinées à un usage ponctuel. Elles s'intègrent nativement dans Microsoft Teams sous forme d'onglets ou d'applications personnelles. Leurs sources de données sont variées : SharePoint, Excel sur OneDrive, SQL Server, ou plus d'une centaine de connecteurs tiers. Cependant, cette liberté a un coût : la logique métier et le design sont étroitement couplés, ce qui rend la maintenance complexe si l'application grossit démesurément.

Les Model-Driven Apps (Applications Pilotées par Modèle)

Ici, on change de paradigme. Vous ne dessinez pas l'interface ; vous modélisez les données. Les Model-Driven Apps reposent exclusivement sur **Dataverse**, la base de données relationnelle et sécurisée de Microsoft (anciennement Common Data Service). Vous définissez vos tables, vos colonnes, vos relations (1-N, N-N) et vos règles métier. L'interface utilisateur (formulaires, vues, tableaux de bord) est ensuite générée automatiquement par le système en respectant les standards de design de Microsoft. C'est l'outil par excellence pour construire des back-offices complexes, des CRM sur mesure ou des systèmes de gestion d'actifs. L'UX est moins personnalisable, mais la robustesse, la scalabilité et la sécurité des données sont garanties par l'architecture sous-jacente.

La véritable force de Power Apps réside dans son intégration à l'écosystème Microsoft 365. Une Canvas App peut être déployée en un clic dans un canal Teams, utilisant l'authentification unique (SSO) de l'entreprise. L'utilisateur n'a pas besoin de créer un nouveau compte : son identité d'entreprise lui donne accès aux données auxquelles il a droit, et uniquement à celles-ci. C'est cette promesse de "prosumer" (producteur-consommateur) intégré qui a convaincu des milliers de DSI d'adopter la plateforme.

© 2026 Oussama Ghorbel

12.2. Formules Power Fx : le 'Excel des apps'

Si vous avez déjà utilisé Excel, vous connaissez déjà le langage de programmation de Power Apps. Microsoft a fait un choix stratégique brillant en créant **Power Fx**, un langage open-source, déclaratif et de bas niveau, directement inspiré des formules de tableur.

L'objectif est clair : réduire la courbe d'apprentissage pour les analystes financiers, les contrôleurs de gestion et les chefs de projet qui maîtrisent déjà `RECHERCHEV`, `SI`, `FILTRE` ou `SOMME`. Dans Power Fx, pour filtrer une galerie affichant des clients en fonction d'un champ de recherche et d'un menu déroulant, vous écrirez quelque chose comme :

`Filter(Clients, StartsWith(Nom, TextInput_Search.Text) && Status.Value = Dropdown_Status.Selected.Value)`

Cette familiarité cache cependant une complexité technique qu'il faut absolument maîtriser : **la délégation**.

Dans Excel, lorsque vous appliquez un filtre, le moteur de calcul parcourt toutes les lignes de votre feuille. Dans Power Apps, vos données peuvent résider sur un serveur SQL distant contenant des millions de lignes. Il est impossible de télécharger toutes ces lignes sur le téléphone d'un technicien pour les filtrer localement.

La délégation est le mécanisme par lequel Power Fx "traduit" votre formule en une requête native (comme une requête SQL) et l'envoie au serveur de base de données. Le serveur effectue le filtrage et ne renvoie que les résultats. Le piège ? Toutes les fonctions Power Fx ne sont pas "déléguables" avec toutes les sources de données. Si vous utilisez une fonction non délégable sur une liste SharePoint de 10 000 éléments, Power Apps ne téléchargera que les 500 premiers éléments en local, appliquera le filtre sur ces 500 éléments, et vous donnera un résultat faux. Comprendre les avertissements de délégation (ces fameux triangles jaunes dans le studio) est la compétence qui sépare le amateur du véritable architecte Power Platform.

Power Fx gère également les comportements asynchrones et les side-effects. Contrairement à une cellule Excel qui recalcule passivement, une formule Power Fx attachée à la propriété `OnSelect`

© 2026 Oussama Ghorbel

d'un bouton va déclencher des actions : `Patch()` pour modifier une donnée, `Navigate()` pour changer d'écran, ou `Launch()` pour ouvrir une URL. C'est un langage réactif qui lie l'interface à la donnée en temps réel.

12.3. AppSheet : la réponse Google pour le terrain

Si Microsoft domine le bureau et le back-office, Google a fait l'acquisition d'AppSheet pour conquérir un territoire très différent : le terrain, les entrepôts, les chantiers et les zones rurales. AppSheet est une plateforme no-code/low-code qui excelle dans la création d'applications mobiles "offline-first" (fonctionnant hors ligne).

L'approche d'AppSheet est radicalement différente de celle de Power Apps Canvas. Vous ne dessinez pas l'interface. Vous connectez une source de données (Google Sheets, Google BigQuery, Cloud SQL, ou même Excel sur OneDrive et Salesforce), et AppSheet "lit" votre schéma pour générer automatiquement une application mobile native.

La magie opère dans la configuration des **types de colonnes**. En changeant le type d'une colonne texte en `Image`, l'application ajoute automatiquement un bouton pour prendre une photo avec l'appareil photo du smartphone, l'horodate et la géolocalise. En la changeant en `Enum` avec des icônes, vous créez un menu de sélection visuel. En activant le type `Barcode`, le smartphone utilise sa caméra pour scanner des codes-barres ou des QR codes, idéal pour les inventaires.

L'offline et la synchronisation

Le véritable super-pouvoir d'AppSheet est sa gestion du mode hors ligne. Un inspecteur de lignes électriques en montagne n'a pas de réseau 4G. Avec AppSheet, l'application télécharge les données nécessaires localement sur l'appareil. L'utilisateur peut créer de nouveaux rapports, prendre des photos, signer sur l'écran et scanner des équipements. Dès que le téléphone capte à nouveau un réseau, l'application synchronise les modifications en arrière-plan vers la base de données centrale. La gestion des conflits (si deux utilisateurs modifient le même enregistrement hors ligne) est gérée nativement par le moteur de synchronisation.

AppSheet Automation

© 2026 Oussama Ghorbel

Google a également intégré un moteur d'automatisation puissant (anciennement appelé "Bots"). Il permet de déclencher des workflows basés sur des événements de données. Par exemple : "Quand un nouvel incident est créé avec une priorité 'Haute', envoyer une notification push au manager, créer un événement dans Google Calendar, et générer un PDF du rapport envoyé par email". Tout cela se configure via une interface visuelle sans écrire une seule ligne de code.

12.4. Retool et Internal : apps internes pour équipes tech et ops

Changeons de cible. Oubliez le citoyen développeur du service marketing. Ici, nous parlons aux ingénieurs, aux équipes de support client, aux data analysts et aux opérations. **Retool** n'est pas un outil no-code ; c'est un outil low-code conçu pour les développeurs qui détestent perdre du temps à construire des interfaces d'administration.

Le constat de Retool est simple : les développeurs passent des semaines à coder des outils internes (panels d'administration, dashboards de support, outils de modération, interfaces de gestion de base de données) en utilisant des frameworks frontend complexes, alors que ces outils n'ont pas besoin d'un design sur mesure. Retool fournit une bibliothèque de composants UI préconstruits (tableaux, formulaires, graphiques, arbres de décision) que l'on assemble par glisser-déposer.

Le SQL et le JavaScript comme moteurs

Là où Power Apps utilise le Power Fx et AppSheet utilise des configurations visuelles, Retool vous met face à votre base de données. Vous connectez Retool à votre PostgreSQL, MySQL, Snowflake, ou MongoDB. Ensuite, vous écrivez vos requêtes SQL directement dans l'éditeur.

Pour afficher les données dans un tableau, vous liez le composant à la requête : `{{ query1.data }}`. Pour rendre un bouton dynamique, vous utilisez du JavaScript : `{{ currentRow.id }}`.

C'est cette hybridation qui fait la force de Retool. Le développeur garde le contrôle total sur la logique de requêtage, les jointures complexes et les transactions, tout en s'affranchissant de la corvée du CSS et du state management frontend. La sécurité est également au cœur

© 2026 Oussama Ghorbel

de la proposition de valeur : Retool ne stocke pas vos données. Les requêtes transitent par les serveurs de Retool (ou par votre propre infrastructure si vous optez pour le déploiement On-Premise ou Docker), mais les données vont directement de votre base à votre navigateur. Les credentials de base de données sont chiffrés et les accès gérés via le SSO d'entreprise (Okta, Azure AD).

Internal : le virage no-code de Retool

Conscient que toutes les entreprises n'ont pas des développeurs à disposition pour chaque petit outil interne, Retool a récemment lancé **Internal**. Cette nouvelle offre se rapproche beaucoup plus du no-code pur, permettant aux équipes opérationnelles de créer des applications en se connectant à des APIs ou des bases de données, avec une interface de configuration des requêtes plus visuelle et moins axée sur le code brut. C'est la reconnaissance par le marché que la frontière entre l'outil du développeur et l'outil du métier est de plus en plus poreuse.

12.5. Power Pages : sites web externes connectés à Dataverse

Jusqu'à présent, nous avons parlé d'applications pour les employés. Mais que se passe-t-il lorsque vous devez exposer une partie de vos données à vos clients, vos partenaires ou vos fournisseurs ? C'est le rôle de **Microsoft Power Pages** (anciennement Power Apps Portals).

Power Pages permet de créer des sites web externes, responsive et sécurisés, qui s'appuient directement sur les données de Dataverse. Contrairement à un CMS classique comme WordPress qui stocke son contenu dans sa propre base, un site Power Pages est une "fenêtre" sur votre Dataverse.

Cas d'usage typiques

• **Portails de support client :** Un client se connecte, voit uniquement ses propres tickets d'incidents (grâce à la sécurité au niveau des lignes de Dataverse), et peut en ouvrir de nouveaux.

• **Portails partenaires :** Un réseau de franchisés accède à des catalogues de produits, soumet des commandes et suit ses performances.

© 2026 Oussama Ghorbel

• **Portails citoyens :** Une mairie permet aux habitants de soumettre des demandes d'urbanisme et de suivre leur statut.

La puissance du moteur Liquid

Pour personnaliser l'affichage des données sur ces pages web, Power Pages utilise **Liquid**, un langage de templating open-source créé par Shopify. Liquid permet d'injecter de la logique conditionnelle et de boucler sur des enregistrements Dataverse directement dans le code HTML de la page. Par exemple, pour afficher une liste de projets assignés à l'utilisateur connecté :

La gouvernance des identités externes est gérée via Azure AD B2C, permettant aux utilisateurs de s'inscrire avec leur email, leur compte LinkedIn, Facebook ou via un fournisseur d'identité d'entreprise. Le modèle de licence de Power Pages est basé sur le nombre d'utilisateurs authentifiés ou anonymes par mois, ce qui nécessite une modélisation financière rigoureuse avant le lancement, car les coûts peuvent s'envoler sur des portails grand public à fort trafic.

12.6. Gouvernance dans un tenant Microsoft/Google

Le plus grand ennemi du low-code en entreprise n'est pas la technique, c'est l'anarchie. Si vous donnez accès à Power Apps à 10 000 employés sans garde-fous, vous allez vous retrouver avec 5 000 applications abandonnées, des données sensibles copiées dans des fichiers Excel personnels, et des flux Power Automate qui saturent vos API. C'est ce qu'on appelle le *Shadow IT* toxique.

La gouvernance est ce qui transforme le low-code d'un jouet en un actif stratégique.

Le Center of Excellence (CoE) Starter Kit

Microsoft fournit gratuitement un ensemble d'outilles appelé le *CoE Starter Kit*. C'est une série d'applications Canvas et Model-Driven, de flux Power Automate et de tableaux de bord Power BI qui s'installent dans votre tenant. Le CoE scanne en permanence votre environnement pour vous dire : qui a créé quoi ? Quelles sont les applications les plus utilisées ? Quelles sont celles qui n'ont pas été modifiées depuis un an et qui devraient être archivées ? Il permet d'instaurer un processus

© 2026 Oussama Ghorbel

d'approbation : un employé peut créer une app pour son usage personnel, mais s'il la partage à plus de 50 personnes, une demande d'approbation est automatiquement envoyée à l'équipe IT.

Les politiques DLP (Data Loss Prevention)

C'est la pierre angulaire de la sécurité. Les politiques DLP permettent de classer les connecteurs en deux catégories : "Business" et "Non-Business". La règle d'or est simple : une application ou un workflow ne peut pas utiliser simultanément un connecteur Business (ex: SharePoint interne, SQL Server d'entreprise) et un connecteur Non-Business (ex: Twitter, Gmail personnel, Dropbox). Si un employé essaie de créer un flux qui copie les données clients de Dynamics 365 vers un Google Drive personnel, la politique DLP bloquera instantanément l'action et alertera la DSI.

ALM (Application Lifecycle Management)

Dans le monde du code traditionnel, on utilise Git, des branches et des pipelines CI/CD. Dans la Power Platform, l'ALM repose sur le concept de **Solutions**. Une solution est un conteneur logique qui regroupe des tables Dataverse, des applications, des flux et des rôles de sécurité. Vous développez dans un environnement "Dev", vous exportez la solution sous forme de fichier `.zip` managé (qui empêche les modifications directes), et vous l'importez dans un environnement "Test", puis "Prod". Les *Pipelines Power Platform* permettent désormais d'automatiser ces déploiements avec des approbations et des tests de régression, alignant ainsi le low-code sur les standards DevOps de l'entreprise.

Chez Google, la gouvernance d'AppSheet passe par le centre d'administration Google Workspace, permettant de restreindre la création d'applications à certains groupes d'utilisateurs, de forcer l'authentification à deux facteurs et d'auditer les accès aux données via les logs Google Cloud.

Exercices pratiques

© 2026 Oussama Ghorbel

La théorie des plateformes d'entreprise ne vaut que par la pratique. Les exercices suivants sont conçus pour être réalisés avec des comptes d'essai (Microsoft 365 Developer Program, compte Google personnel, et compte gratuit Retool).

Exercice 1 : Créer une Power App canvas de saisie de rapports terrain en 2 heures

Objectif : Permettre à des techniciens de saisir des interventions sur site via une application mobile connectée à SharePoint.

1. **Préparation de la donnée :**

• Allez sur un site SharePoint de test. Créez une liste personnalisée nommée "Interventions".

• Ajoutez les colonnes : `Client` (Texte), `Date_Intervention` (Date), `Type_Panne` (Choix : Électrique, Plomberie, Réseau), `Commentaire` (Texte multiligne), `Photo` (Image).

2. **Génération de l'application :**

• Dans le portail Power Apps, choisissez "Application canevas" > "Démarrer à partir de données" > "SharePoint".

• Connectez-vous à votre liste. Power Apps génère automatiquement une application à 3 écrans (Galerie de liste, Écran de détail, Écran d'édition).

3. **Personnalisation de l'UX (Canvas) :**

• Sur l'écran d'édition, remplacez le champ de saisie texte de `Type_Panne` par un menu déroulant visuel avec des icônes.

• Ajoutez un bouton "Prendre une photo" qui utilise la fonction `Camera` ou le contrôle `AddMediaButton` lié à la colonne Image.

4. **Logique Power Fx :**

• Sur le bouton "Enregistrer", modifiez la formule `OnSelect` pour ajouter une notification de succès et réinitialiser le formulaire :

`SubmitForm(EditForm1); Notify("Rapport envoyé avec succès", NotificationType.Success); ResetForm(EditForm1); Navigate(BrowseScreen1)`

5. **Déploiement :**

© 2026 Oussama Ghorbel

• Publiez l'application et partagez-la avec votre utilisateur de test. Ouvrez-la via l'application mobile "Power Apps" sur votre smartphone pour tester l'ergonomie tactile.

Piège à éviter : Ne modifiez pas la structure de la liste SharePoint directement pendant que l'application est ouverte dans le studio, sinon vous devrez rafraîchir la source de données et risquez de casser les liaisons des contrôles.

Exercice 2 : Déployer une app AppSheet avec scan de codes-barres pour inventaire

Objectif : Créer une application mobile d'inventaire fonctionnant hors ligne, capable de scanner des QR codes.

1. **Structure de la base :**

• Créez un Google Sheet nommé "Inventaire_Entrepot".

• Créez les en-têtes : `ID_Article` (Clé primaire), `Nom_Article`, `Emplacement`, `Quantite_Stock`, `Code_Barres`, `Derniere_MAJ`.

• Ajoutez quelques lignes de données fictives.

2. **Création et modélisation :**

• Allez sur AppSheet et créez une nouvelle app en pointant vers votre Google Sheet.

• Dans l'onglet **Data**, vérifiez que `ID_Article` est bien défini comme clé (Key) et que `Code_Barres` est de type `Text`.

3. **Activation du scanner :**

• Changez le type de la colonne `Code_Barres` pour `Barcode` (ou `Text` avec le mode de saisie défini sur "Scan barcode" dans les options de la colonne).

• Créez une "Action" nommée "Scanner et Ajouter" qui permet d'incrémenter la `Quantite_Stock` de 1 lorsque le code est scanné.

4. **Configuration Offline :**

• Dans **Settings > Offline**, activez le mode hors ligne. AppSheet va télécharger les données sur l'appareil.

• Dans la vue UX (écran de liste), activez la fonctionnalité "Quick Edit" pour permettre la modification rapide des quantités sur le

© 2026 Oussama Ghorbel

terrain.

5. **Test de synchronisation :**

• Ouvrez l'app sur votre smartphone. Coupez votre Wi-Fi et votre 4G (mode avion).

• Scannez un code-barres (vous pouvez en générer un sur votre écran d'ordinateur) et modifiez le stock.

• Réactivez le réseau. Observez l'icône de synchronisation en haut de l'app et vérifiez que votre Google Sheet s'est mis à jour dans le cloud.

Exercice 3 : Construire un admin panel Retool branché sur une base SQL existante

Objectif : Fournir à l'équipe support un outil pour rechercher, bloquer ou rembourser des utilisateurs dans une base PostgreSQL.

1. **Connexion à la donnée :**

• Créez un compte Retool. Utilisez la base de données de démonstration PostgreSQL fournie par Retool (ou connectez votre propre base locale via l'option "Connect to local database" en utilisant le bridge Retool).

2. **Création des requêtes (SQL) :**

• Créez une requête nommée `get_users` : `SELECT id, email, status, created_at FROM users WHERE email ILIKE '%{{ textInput_search.value }}%' ORDER BY created_at DESC;`

• Créez une requête nommée `update_status` : `UPDATE users SET status = {{ select_status.value }} WHERE id = {{ table_users.selectedRow.id }};`

3. **Assemblage de l'interface (Drag & Drop) :**

• Glissez un composant `Text Input` (nommé `textInput_search`) en haut de la page.

• Glissez un composant `Table` (nommé `table_users`). Dans ses propriétés, liez les données à `{{ get_users.data }}`.

• Configurez la table pour que la recherche se déclenche automatiquement quand l'utilisateur tape (propriété "Run query on

© 2026 Oussama Ghorbel

change").

4. **Création du panneau d'action :**

• Ajoutez un `Container` qui s'affiche uniquement si une ligne est sélectionnée dans la table (`{{ table_users.selectedRow != null }}`).

• À l'intérieur, ajoutez un `Select` (nommé `select_status`) avec les options "Active", "Suspended", "Banned".

• Ajoutez un `Button` "Mettre à jour". Sur son événement `onClick`, attachez la requête `update_status`.

5. **Gestion des permissions et UX :**

• Ajoutez une notification de succès après l'exécution de `update_status` : `utils.showNotification({ title: "Statut mis à jour", type: "success" })`.

• Dans les paramètres de la requête `update_status`, cochez "Require confirmation" pour éviter les clics accidentels sur des actions destructives.

Note de l'auteur : Retool illustre parfaitement la philosophie du low-code en entreprise. Vous n'avez pas codé le HTML du tableau, ni le CSS du bouton, ni la gestion de l'état de chargement (loading state). Vous avez simplement écrit le SQL métier et assemblé les briques. C'est ainsi que l'on construit des outils internes robustes en une après-midi au lieu de deux semaines.

© 2026 Oussama Ghorbel

Chapitre 13 – UX et design system en contexte no-code

Le plus grand piège du no-code n’est pas technique, il est ergonomique. Parce que ces outils permettent de construire une interface fonctionnelle en quelques heures, nous sommes souvent tentés de confondre « ce qui marche » avec « ce qui est bien conçu ». Une application qui exécute parfaitement une requête en base de données mais qui frustre l’utilisateur final est une application qui échoue. L'expérience utilisateur (UX) et l'interface utilisateur (UI) ne sont pas des couches de vernis que l'on ajoute à la fin du projet ; ce sont les fondations de l'adoption de votre produit.

Dans ce chapitre, nous allons démystifier le design. Vous n'avez pas besoin d'être un artiste ni d'avoir un budget agence pour produire des interfaces lisibles, accessibles et cohérentes. Il vous faut une méthode, des règles strictes et une boîte à outils pragmatique.

13.1. Les 7 principes d'UX qui sauvent 80% des apps no-code

L'UX ne repose pas sur l'intuition, mais sur la psychologie cognitive. En appliquant ces sept principes fondamentaux, vous éliminerez la grande majorité des frictions de vos applications, qu'elles soient construites sur Bubble, Glide, WeWeb ou Power Apps.

1. La loi de Hick : réduire la charge cognitive

Le temps qu'il faut à une personne pour prendre une décision augmente avec le nombre et la complexité des choix qui s'offrent à elle. En no-code, la tentation est grande d'afficher toutes les données disponibles sur un seul tableau de bord.

Application : Masquez les options avancées. Utilisez des menus déroulants, des onglets ou des modales pour révéler l'information progressivement. Si un utilisateur doit créer une commande, ne lui affichez pas en même temps les paramètres de son profil.

2. Le parcours critique : une vue, une action principale

© 2026 Oussama Ghorbel

Chaque écran de votre application doit avoir un objectif unique et indiscutable. L'action principale doit être visuellement dominante.

Application : Si votre page est dédiée à l'ajout d'un nouveau client, le bouton « Enregistrer le client » doit être le seul bouton coloré (Primaire). Les actions secondaires comme « Annuler » ou « Voir l'historique » doivent être visuellement en retrait (boutons fantômes ou liens textes).

3. Le feedback immédiat : la visibilité de l'état du système

Jakob Nielsen, pionnier de l'UX, l'a érigé en règle d'or : le système doit toujours informer l'utilisateur de ce qui se passe. En no-code, les requêtes API ou les workflows complexes (comme une génération de PDF via Make) peuvent prendre plusieurs secondes.

Application : Ne laissez jamais un écran figé. Déclenchez un *spinner* (loader), une barre de progression ou désactivez le bouton de soumission en affichant « Envoi en cours... » dès le clic de l'utilisateur.

4. La cohérence visuelle et fonctionnelle

Les utilisateurs passent 90% de leur temps sur *d'autres* applications. Ils s'attendent à ce que votre application respecte les standards du web. Un lien souligné en bleu doit mener quelque part, une icône de poubelle doit supprimer, et la navigation principale doit être prévisible.

Application : Ne réinventez pas la roue. Utilisez les conventions de placement : le logo en haut à gauche pour revenir à l'accueil, le profil utilisateur en haut à droite, les filtres à gauche ou en haut des tableaux de données.

5. La prévention des erreurs

Un bon design empêche l'erreur avant qu'elle ne se produise. C'est le principe du *Poka-Yoke* (détrompeur).

Application : Utilisez des masques de saisie pour les numéros de téléphone et les dates. Si un champ exige une adresse email, validez le format en temps réel (au moment où l'utilisateur quitte le champ) plutôt que de lui afficher une erreur globale après la soumission du formulaire. Proposez des valeurs par défaut intelligentes.

6. La hiérarchie visuelle

© 2026 Oussama Ghorbel

L'œil humain ne lit pas une interface, il la scanne. Il faut guider le regard de l'information la plus importante à la moins importante grâce à la taille, le contraste et l'espacement.

Application : Appliquez le modèle de lecture en « F » ou en « Z ». Le titre de la page doit être grand et sombre, le sous-titre plus petit et gris, et le corps de texte standard. Les éléments regroupés doivent être proches les uns des autres (loi de proximité), tandis que les sections distinctes doivent être séparées par un espace blanc généreux.

7. Le pardon et la réversibilité

L'erreur est humaine, votre application doit être clémente. La peur de faire une erreur irréversible paralyse l'utilisateur.

Application : Privilégiez le *soft delete* (mettre un statut "Archivé" ou "Corbeille" en base de données) plutôt que la suppression définitive. Pour les actions destructives, exigez une confirmation claire, voire une friction volontaire (comme taper le nom du projet pour le supprimer). Proposez toujours un bouton « Annuler » ou une notification avec option d'annulation (Undo) après une action sensible.

13.2. Construire un mini design system (couleurs, typo, composants)

Un design system n'est pas un document de 200 pages réservé aux équipes de Google ou d'Airbnb. Dans le contexte no-code, c'est un contrat visuel que vous passez avec vous-même avant même d'ouvrir votre éditeur. C'est ce qui transforme une interface « bricolée » en un produit professionnel.

La palette de couleurs : la règle du 60-30-10

Oubliez les arcs-en-ciel. Une interface professionnelle repose sur une palette restreinte et sémantique.

• **60% (Couleur dominante) :** C'est généralement le blanc, le gris très clair ou un gris très sombre (pour le mode nuit). C'est votre toile de fond, votre espace de respiration.

• **30% (Couleur secondaire) :** C'est la couleur de votre marque. Elle est utilisée pour les en-têtes, les éléments de navigation, les illustrations.

© 2026 Oussama Ghorbel

• **10% (Couleur d'accent) :** C'est la couleur de l'action. Elle doit être réservée exclusivement aux boutons d'appel à l'action (CTA) primaires et aux liens cliquables.

• **Les couleurs sémantiques :** Ajoutez quatre couleurs utilitaires universelles : Rouge (Erreur/Suppression), Vert (Succès/Validation), Jaune/Orange (Avertissement), Bleu (Information).

La typographie : la discipline de l'échelle

Limitez-vous à une ou deux polices maximum. Pour les interfaces (UI), privilégiez des polices sans-serif conçues pour la lisibilité à l'écran (Inter, Roboto, SF Pro,system-ui). Le secret ne réside pas dans le choix de la police, mais dans la définition d'une échelle stricte. Définissez ces styles dans l'onglet *Styles* de Bubble ou les *Variables* de Webflow :

• **H1 :** 32px / Bold (Titres de pages)

• **H2 :** 24px / Semi-Bold (Titres de sections)

• **H3 :** 18px / Medium (Titres de cartes ou modales)

• **Body :** 16px / Regular (Texte courant, formulaires)

• **Caption :** 12px ou 14px / Regular (Métadonnées, timestamps, aides)

Notez que la hauteur de ligne (line-height) est cruciale : comptez environ 1.4 à 1.6 fois la taille de la police pour le texte courant afin d'assurer le confort de lecture.

L'espacement : la grille de 8 points

L'erreur classique du débutant est d'utiliser des marges aléatoires (13px ici, 20px là). Adoptez le système de grille de 8 points (8pt grid). Tous vos espacements (marges, paddings, gaps) doivent être des multiples de 4 ou de 8 : 4px, 8px, 16px, 24px, 32px, 48px, 64px. Cela crée un rythme visuel inconscient mais profondément satisfaisant pour le cerveau humain.

Les composants réutilisables

Avant de construire vos pages, construisez vos briques. Dans votre outil no-code, créez les variantes de vos composants de base :

© 2026 Oussama Ghorbel

• **Boutons :** Primaire, Secondaire, Fantôme (Ghost), Danger. Pour chacun, définissez les états : *Default*, *Hover (survol)*, *Pressed (clic)*, *Disabled (inactif)*, *Loading*.

• **Champs de formulaire :** *Default*, *Focus (actif)*, *Error*, *Success*, *Disabled*.

• **Cartes (Cards) :** Avec ou sans ombre, avec image d'en-tête ou icône.

• **Badges/Tags :** Pour les statuts (ex: "En cours", "Terminé").

13.3. Figma pour maquetter, puis importer ou refaire

Il est souvent rappelé lors des accompagnements : construire directement dans Bubble ou Webflow sans maquette préalable, c'est comme construire une maison sans plan d'architecte, en posant les briques au fur et à mesure que l'on décide de la place des fenêtres. Figma est votre bac à sable, votre laboratoire où les erreurs ne coûtent rien en temps de développement.

Le mythe du "Figma to Code" magique

Il existe de nombreux plugins promettant d'exporter votre design Figma directement vers Webflow, Bubble ou React. Mon conseil est catégorique : **méfiez-vous-en pour la production**. Ces outils génèrent souvent du code ou des structures de groupes lourdes, non responsives et cauchemardesques à maintenir. Ils sont utiles pour extraire des assets (icônes, images) ou des valeurs CSS, mais pas pour la structure logique de votre application.

Le bon workflow : Maquetter, puis Reconstruire

1. **Wireframing (Basse fidélité) :** Utilisez des rectangles gris et du texte brut pour valider l'architecture de l'information et le parcours utilisateur. Ne pensez pas aux couleurs. Si l'application n'est pas compréhensible en noir et blanc, elle ne le sera pas en couleur.

2. **Mockups (Haute fidélité) :** Appliquez votre mini design system. C'est ici que vous validez l'esthétique, les contrastes et l'espacement.

3. **Prototypage :** Reliez les écrans dans Figma pour simuler les transitions et les flux. Faites tester ce prototype cliquable à vos utilisateurs potentiels.

© 2026 Oussama Ghorbel

4. **Handoff et Intégration :** Ouvrez votre outil no-code à côté de Figma. Utilisez le *Dev Mode* ou le panneau d'inspection de Figma pour copier les valeurs exactes (codes hexadécimaux, paddings en pixels, tailles de police) et recréez manuellement vos composants et pages. Cette reconstruction manuelle vous oblige à penser en termes de *Groupes*, *Repeating Groups* et *Flexbox*, ce qui est la véritable sémantique de votre outil no-code.

13.4. Accessibilité : contrastes, clavier, lecteurs d'écran

L'accessibilité (souvent abrégée a11y) n'est pas une option morale ou légale réservée aux sites gouvernementaux. C'est un marqueur de qualité technique. Une application accessible est une application mieux structurée, plus robuste et plus facile à utiliser pour *tous*, y compris dans des conditions sous-optimales (plein soleil sur un écran de smartphone, bras cassé, fatigue visuelle).

Les contrastes de couleurs

Le texte doit se détacher clairement de son arrière-plan. Les normes WCAG (Web Content Accessibility Guidelines) exigent un ratio de contraste d'au moins 4.5:1 pour le texte normal, et 3:1 pour le texte large.

L'erreur classique : Le texte gris clair (#AAAAAA) sur fond blanc. Il est illisible pour les personnes malvoyantes ou sur des écrans peu lumineux. Utilisez des outils comme le *WebAIM Contrast Checker* pour valider chaque association couleur de texte / couleur de fond de votre design system. N'utilisez jamais la couleur comme seul vecteur d'information (ex: ne dites pas "cliquez sur le bouton vert", ajoutez une icône ou du texte).

La navigation au clavier

De nombreux utilisateurs (notamment les personnes souffrant de troubles moteurs ou les professionnels saisissant des données à la chaîne) n'utilisent pas la souris, mais la touche `Tab` pour naviguer.

Application : Testez votre application no-code uniquement au clavier. Pouvez-vous atteindre tous les champs ? Pouvez-vous valider un formulaire avec la touche `Entrée` ? Surtout, l'élément actuellement sélectionné doit être visuellement identifiable grâce à un **Focus State**

© 2026 Oussama Ghorbel

(un anneau ou une bordure colorée autour du bouton ou du champ). Ne supprimez jamais le `outline` CSS par défaut sans le remplacer par un indicateur visuel fort.

Lecteurs d'écran et sémantique

Les personnes aveugles utilisent des lecteurs d'écran (comme VoiceOver ou NVDA) qui lisent le code de la page.

• **Images :** Ajoutez toujours un texte alternatif (`alt text`) aux images porteuses de sens. Les icônes purement décoratives doivent être ignorées par les lecteurs.

• **Hiérarchie des titres :** N'utilisez pas un texte en H3 juste parce que vous aimez sa taille de police. Respectez l'ordre logique : H1 (un seul par page), puis H2, puis H3.

• **Les limites du no-code :** Certains outils no-code génèrent des structures DOM complexes (beaucoup de `div` imbriquées) qui peuvent perturber les lecteurs d'écran. Soyez vigilant sur les formulaires : assurez-vous que chaque champ `<input>` est bien associé à un `<label>` explicite, et non pas seulement à un texte placé visuellement au-dessus.

13.5. Micro-copy : textes d'erreur, empty states, confirmations

L'UX, c'est aussi ce que l'on lit. La *micro-copie* (micro-copy) désigne tous les petits textes de l'interface qui guident, rassurent ou alertent l'utilisateur. C'est la voix de votre produit. Une micro-copie maladroite peut ruiner des heures de travail sur le design visuel.

Les Empty States (États vides)

L'état vide est la première chose que voit un nouvel utilisateur lorsqu'il n'a encore créé aucune donnée. Ne laissez jamais un tableau vide avec pour seul message "Aucune donnée trouvée". C'est une opportunité manquée.

• **L'Empty State éducatif :** Expliquez à quoi sert la fonctionnalité. ("Le CRM centralise tous vos échanges clients.")

© 2026 Oussama Ghorbel

• **L'Empty State d'action :** Proposez un bouton clair pour démarrer. ("Créer mon premier client" ou "Importer depuis un CSV").

• **L'illustration :** Une illustration vectorielle légère et bienveillante dédramatise l'absence de contenu.

Les messages d'erreur

Le principe fondamental est de ne jamais blâmer l'utilisateur. Bannissez les messages robotiques ou accusateurs.

• *Mauvais :* "Erreur de saisie." ou "Format invalide."

• *Mauvais :* "Vous avez entré un mauvais mot de passe."

• *Bon :* "Le mot de passe doit contenir au moins 8 caractères, dont une majuscule et un chiffre."

• *Bon :* "Nous ne reconnaissons pas cette adresse email. Vérifiez l'orthographe ou créez un compte."

L'erreur doit être spécifique, placée juste à côté du champ concerné, et proposer une solution.

Les confirmations et la réassurance

Lorsqu'une action est réussie, confirmez-la. L'utilisation de *Toast notifications* (ces petites bulles qui apparaissent en haut ou en bas à droite de l'écran et disparaissent après 3 secondes) est idéale pour les actions courantes ("Profil mis à jour").

Pour les actions irréversibles ou destructives, créez de la friction. Une modale de confirmation avec un bouton rouge et un bouton gris ("Êtes-vous sûr de vouloir supprimer ce projet ? Cette action est définitive.") est indispensable.

Le ton et la voix

Adaptez votre micro-copie à votre cible. Une application B2B de gestion financière requiert un ton sobre, précis et rassurant. Une application B2C de réservation de loisirs peut se permettre un ton plus enjoué, des emojis parcimonieux et des formulations plus directes.

13.6. Tests utilisateurs low-cost (5 utilisateurs, 30 min chacun)

© 2026 Oussama Ghorbel

Vous n'avez pas besoin d'un laboratoire avec miroir sans tain pour tester votre application. Jakob Nielsen a démontré qu'ajouter plus d'utilisateurs à un test n'augmente que marginalement le nombre de problèmes découverts, tout en explosant les coûts. **Cinq utilisateurs suffisent pour déceler 85% des problèmes d'utilisabilité.**

Le recrutement

Ne testez pas avec vos développeurs, ni avec votre équipe produit, ni avec votre mère. Ils en savent trop ou sont trop bienveillants. Trouvez des collègues d'autres départements, des amis qui correspondent à votre persona, ou proposez un café à des inconnus dans un espace de coworking en échange de 30 minutes de leur temps.

Le protocole "Think Aloud" (Penser à voix haute)

C'est le cœur du test. Demandez à l'utilisateur de verbaliser tout ce qu'il pense, ce qu'il cherche, ce qu'il ne comprend pas, pendant qu'il interagit avec l'application.

Exemple : "Je cherche le bouton pour ajouter un devis... Je m'attendais à ce qu'il soit en haut à droite, mais je ne le vois pas. Ah, il est dans le menu latéral, je n'avais pas vu l'icône."

Votre rôle est de vous taire. Ne guidez pas, ne justifiez pas vos choix de design, ne défendez pas votre application. Observez la friction.

Le script de test (30 minutes)

1. **L'accueil (5 min) :** Mettez la personne à l'aise. Précisez que *c'est l'application qui est testée, pas elle*. "Il n'y a pas de mauvaise réponse, et si vous bloquez, c'est la faute du design, pas la vôtre."

2. **Le contexte (5 min) :** Posez 2 ou 3 questions sur ses habitudes actuelles pour comprendre son état d'esprit.

3. **Les scénarios (15 min) :** Donnez des tâches réalistes mais ne dites pas *comment* les faire.

• *Mauvais :* "Cliquez sur le bouton bleu 'Nouveau' puis remplissez le formulaire."

• *Bon :* "Imaginez que vous venez de signer avec un nouveau fournisseur. Comment faites-vous pour l'ajouter dans le système et lui attribuer la catégorie 'IT' ?"

© 2026 Oussama Ghorbel

4. **Le debrief (5 min) :** Demandez son ressenti global. "Qu'est-ce qui a été le plus frustrant ? Si vous aviez une baguette magique, que changeriez-vous ?"

La synthèse

Filmez l'écran et la voix (avec l'accord de la personne) via des outils comme Loom ou Zoom. Ensuite, listez tous les points de friction. Classez-les en trois catégories :

• **Bloquants :** L'utilisateur ne peut pas terminer la tâche. (À corriger immédiatement).

• **Majeurs :** L'utilisateur termine la tâche mais avec beaucoup de frustration ou de temps perdu. (À planifier dans la prochaine itération).

• **Mineurs :** Gêne esthétique ou micro-friction. (À traiter quand le temps le permettra).

Le no-code vous donne le super-pouvoir de la vitesse. Utilisez-le pour corriger les problèmes bloquants identifiés le matin même, et déployez la nouvelle version pour un nouveau test l'après-midi. C'est là que réside la véritable agilité du produit.

Exercices pratiques

Pour ancrer les concepts de ce chapitre, réalisez ces trois exercices sur l'application no-code de votre choix (un projet en cours ou un prototype dédié).

Exercice 1 : Créer un design system de 10 composants réutilisables

Ouvrez votre outil no-code (ou Figma en amont) et créez une page "Style Guide". Modélisez les 10 composants suivants avec toutes leurs variantes d'états (Default, Hover, Active, Disabled, Error/Focus) :

1. Bouton Primaire
2. Bouton Secondaire
3. Champ de texte simple (Input)

© 2026 Oussama Ghorbel

4. Zone de texte multiligne (Textarea)
5. Menu déroulant (Select)
6. Case à cocher (Checkbox)
7. Carte de contenu (Card)
8. Badge de statut (Tag)
9. Alerte/Notification (Toast)
10. Modale de confirmation

Objectif : Ne plus jamais avoir à "styler" un bouton manuellement lors de la création d'une nouvelle page.

Exercice 2 : Auditer l'accessibilité d'une app avec l'outil Contrast Checker

Prenez trois écrans clés de votre application (ex: page de login, tableau de bord, formulaire).

1. Relevez les codes hexadécimaux de vos textes et de vos fonds.
2. Passez-les dans le *WebAIM Contrast Checker*.
3. Identifiez les éléments qui échouent au standard WCAG AA.
4. Corrigez les couleurs de votre design system pour atteindre un ratio minimum de 4.5:1.
5. Testez la navigation de votre page de login et de votre formulaire principal en utilisant uniquement la touche `Tab` et `Entrée` de votre clavier. Ajoutez des styles de *Focus* clairs si nécessaire.

Exercice 3 : Mener 3 tests utilisateurs filmés et en extraire 5 apprentissages

Préparez un script de test comportant 3 scénarios clés de votre application.

1. Recrutez 3 personnes correspondant à votre cible (ou s'en approchant).
2. Menez les tests en appliquant le protocole "Think Aloud" et enregistrez les sessions (écran et voix).
3. Visionnez les enregistrements et notez chaque hésitation, erreur ou remarque.

© 2026 Oussama Ghorbel

4. Synthétisez vos observations en extrayant exactement **5 apprentissages majeurs** (ex: "L'icône de filtre n'est pas comprise", "Le bouton de validation est caché sous la ligne de flottaison sur mobile").

5. Implémentez les correctifs pour les 2 problèmes les plus bloquants dans les 48 heures.

Le fossé entre le fonctionnel et l'exceptionnel

L'un des plus grands malentendus concernant le no-code est de croire que la vitesse de développement est son seul véritable atout. C'est une erreur. La véritable révolution du no-code réside dans la réduction du temps de latence entre une idée et son test face à la réalité du marché. Mais pour que ce test soit valide, l'interface doit être irréprochable.

Aujourd'hui, n'importe qui peut assembler une base de données Airtable, la connecter à Softr ou WeWeb et publier un portail client en un week-end. La barrière à l'entrée technique a disparu. Par conséquent, ce qui différencie désormais un produit no-code qui convertit et fidélise d'un simple "bricolage interne", c'est la qualité de l'expérience utilisateur. L'UX n'est plus un luxe réservé aux startups ayant levé des fonds ; c'est le minimum viable pour être pris au sérieux par vos utilisateurs, vos clients et vos collaborateurs.

En appliquant rigoureusement les principes de ce chapitre, vous ne faites pas que du "joli". Vous réduisez les coûts de support client (car les utilisateurs comprennent l'outil), vous accélérez l'adoption en interne (car l'application ne génère pas de frustration) et vous créez de la confiance. Dans le monde du B2B comme du B2C, la confiance se mesure en pixels, en contrastes, en temps de réponse et en clarté des messages d'erreur.

La dette de design : le mal silencieux des projets no-code

En développement traditionnel, on parle souvent de "dette technique". En no-code, le danger qui guette chaque créateur est la **dette de design**.

Lorsque vous construisez une application sans design system préétabli, vous prenez des micro-décisions esthétiques à chaque nouvelle

© 2026 Oussama Ghorbel

page. "De quelle couleur devrait être ce bouton ?" "Quelle taille de police pour ce nouveau tableau ?" "Où placer cette nouvelle action ?" Au fil des semaines, ces micro-décisions s'accumulent et divergent. Votre application finit par ressembler à un patchwork de bonnes idées isolées, créant une expérience globale incohérente, fatigante et peu professionnelle.

Le mini design system que nous avons défini plus haut n'est pas une contrainte créative, c'est votre bouclier contre la dette de design. Il automatise vos choix visuels pour que vous puissiez consacrer votre énergie cognitive à la logique métier, à l'architecture des données et à l'orchestration des workflows. Quand le design est systématisé, la création d'une nouvelle page n'est plus un exercice de style, mais un simple assemblage de blocs certifiés conformes.

N'oubliez jamais cette règle d'or : une fonctionnalité terminée à 90% mais dont l'interface est confuse sera perçue comme "cassée" par l'utilisateur. Une fonctionnalité basique mais dont l'UX est fluide, rassurante et cohérente sera perçue comme "magique". Le no-code vous donne les briques ; l'UX est le ciment qui les transforme en forteresse.

Ce qu'il faut retenir

Avant de passer à la suite, ancrez ces vérités fondamentales qui doivent guider chacune de vos sessions de construction visuelle :

1. **L'utilisateur ne lit pas, il scanne.** Votre hiérarchie visuelle doit faire le travail de guidage à sa place.
2. **Le silence de l'interface est une angoisse.** Chaque action, chaque chargement, chaque soumission doit générer un feedback immédiat.
3. **L'erreur est inévitable, la punition est optionnelle.** Concevez des parcours qui pardonnent, qui préviennent et qui expliquent avec bienveillance.
4. **L'accessibilité n'est pas une niche.** C'est la garantie que votre application fonctionne dans le monde réel, avec ses imperfections, ses handicaps et ses contextes d'usage variés.
5. **Testez tôt, testez petit.** Cinq utilisateurs armés d'un protocole "Think Aloud" vous en apprendront plus qu'un mois de débats internes sur la couleur d'un bouton.

© 2026 Oussama Ghorbel

Vous avez désormais entre les mains les clés pour concevoir des interfaces qui respectent l'humain qui se cache derrière l'écran. Vous savez comment structurer l'information, comment créer un langage visuel cohérent et comment valider vos hypothèses sur le terrain sans vous ruiner.

Mais une interface belle, fluide et validée par des tests utilisateurs n'est qu'une coquille vide si elle n'est pas capable de protéger ce qu'elle contient. Car derrière ces boutons primaires, ces tableaux de données élégants et ces workflows automatisés se cache la ressource la plus précieuse et la plus vulnérable de votre entreprise : la donnée.

Que se passe-t-il si un utilisateur non autorisé accède à ce tableau de bord ? Comment s'assurer que ce collaborateur ne puisse modifier que les fiches clients de sa propre région ? Comment gérer les sessions, les mots de passe et les permissions granulaires sans réinventer la roue de la cybersécurité ?

C'est exactement l'objet de notre prochaine étape. Après avoir appris à rendre vos applications désirables et utilisables, il est temps d'apprendre à les rendre impénétrables. Bienvenue dans les coulisses de la sécurité, de l'authentification et de la gestion des accès.

© 2026 Oussama Ghorbel

Chapitre 14 – Sécurité, authentification et gestion des accès

Il existe un mythe tenace dans l'écosystème no-code : l'idée que la plateforme choisie s'occupe de l'intégralité de la sécurité. C'est une illusion dangereuse. Lorsque vous utilisez Bubble, WeWeb, Xano ou Make, ces éditeurs sont effectivement responsables de la sécurité de leur infrastructure, de la protection de leurs serveurs contre les attaques DDoS et du chiffrement des données au repos. En revanche, la sécurité de votre logique métier, de vos règles d'accès et de vos intégrations repose entièrement sur vos épaules.

Dans le développement traditionnel, les ingénieurs passent des années à étudier les failles OWASP, les injections SQL et les vulnérabilités XSS. Le no-code vous protège nativement de certaines de ces menaces bas niveau (vous ne manipulerez pas directement de requêtes SQL brutes, ce qui élimine le risque d'injection SQL classique). Mais il expose de nouvelles surfaces d'attaque, souvent liées à une mauvaise compréhension de la distinction entre ce qui relève de l'expérience utilisateur (UX) et ce qui relève de la sécurité pure.

Dans ce chapitre, nous allons structurer la sécurité de vos applications no-code autour de trois piliers fondamentaux : prouver son identité (authentification), définir ses droits (autorisation), et protéger les coulisses (secrets et abuse management). Nous terminerons par une méthodologie d'audit et un plan de réponse à incident, car en sécurité, la question n'est pas de savoir *si* une faille sera exploitée, mais *quand* et *comment* vous y réagirez.

14.1. Authentification : native, SSO, OAuth, Auth0/Clerk

L'authentification répond à une question simple : « Qui êtes-vous ? ». C'est la porte d'entrée de votre application. Si cette porte est mal conçue, aucune des serrures intérieures ne servira à quoi que ce soit.

L'authentification native

La majorité des outils no-code (Bubble, Glide, Softr) proposent un système d'authentification natif basé sur le couple email/mot de passe.

© 2026 Oussama Ghorbel

C'est rapide à mettre en place et suffisant pour des prototypes ou des applications internes simples. Cependant, cette approche vous rend responsable de la gestion des mots de passe, des flux de réinitialisation et de la prévention des attaques par force brute. De plus, pour les utilisateurs finaux, multiplier les mots de passe est une source de friction cognitive majeure.

OAuth et Social Login

Pour réduire cette friction, l'intégration de l'OAuth (Open Authorization) via des fournisseurs comme Google, Apple, Microsoft ou LinkedIn est devenue un standard. L'utilisateur délègue la vérification de son identité à un tiers de confiance. Dans une application B2C, le Social Login augmente drastiquement les taux de conversion à l'inscription. Dans Bubble, cela se configure en quelques clics via l'onglet SEO / Meta tags et les plugins natifs. Attention toutefois à bien gérer le premier lien : si un utilisateur s'inscrit avec Google, puis tente plus tard de se connecter avec un mot de passe sur la même adresse email, votre application doit être capable de fusionner les comptes ou de bloquer la tentative pour éviter la création de doublons.

Le SSO (Single Sign-On) pour l'entreprise

Dès que vous vendez une application à des entreprises (B2B), le SSO n'est plus une option, c'est une exigence. Les DSI (Directions des Systèmes d'Information) exigent que leurs employés se connectent via leur annuaire d'entreprise (Azure AD, Okta, Google Workspace) en utilisant des protocoles comme SAML ou OIDC. Le SSO permet à l'entreprise de révoquer l'accès d'un employé en un clic depuis son propre système, sans avoir à vous contacter pour supprimer son compte dans votre application.

Les fournisseurs d'identité externes (IdP) : Auth0, Clerk, WorkOS

Lorsque votre application no-code gagne en complexité, ou lorsque vous découplez votre frontend (WeWeb, Toddle) de votre backend (Xano, Supabase), déléguer l'authentification à un service spécialisé devient une nécessité architecturale.

© 2026 Oussama Ghorbel

• **Clerk** s'est imposé comme la référence pour les startups grâce à ses composants UI préconçus (profils utilisateurs, gestion des organisations) et son excellente expérience développeur.

• **Auth0** (racheté par Okta) reste le standard pour les besoins enterprise complexes, offrant des règles de gestion des identités extrêmement granulaires.

• **WorkOS** est particulièrement brillant pour intégrer le SSO Enterprise (SAML/OIDC) sans avoir à négocier avec chaque protocole spécifique.

L'intégration de ces services dans une stack no-code se fait généralement via des webhooks. Lorsqu'un utilisateur se connecte via Clerk, un webhook est envoyé à Make ou directement à votre backend (Xano) pour créer ou mettre à jour la session et générer un token JWT (JSON Web Token) qui servira de passeport pour toutes les requêtes API ultérieures.

14.2. Autorisation : rôles, permissions, Privacy Rules Bubble

Si l'authentification est le vigile à l'entrée de la boîte de nuit, l'autorisation est la liste VIP qui détermine qui a le droit d'entrer dans la salle réservée. Une erreur classique et catastrophique en no-code consiste à confondre l'interface utilisateur (UI) avec l'autorisation.

Masquer un bouton « Supprimer » avec une condition visuelle du type *« Ce bouton est visible uniquement si le rôle de l'utilisateur est Admin »* n'est **pas** une mesure de sécurité. C'est une mesure d'UX. Un utilisateur malveillant peut intercepter la requête API, utiliser la console de son navigateur ou un outil comme Postman pour envoyer l'ordre de suppression directement au serveur. La sécurité doit toujours être évaluée côté serveur.

RBAC et ABAC

Le modèle le plus courant est le RBAC (Role-Based Access Control). Vous définissez des rôles (Admin, Manager, Contributeur, Lecteur) et vous attribuez des permissions à ces rôles. Pour des systèmes plus complexes, on utilise l'ABAC (Attribute-Based Access Control), où l'accès dépend d'attributs dynamiques (ex: un manager ne peut voir que les documents dont le département correspond à son propre

© 2026 Oussama Ghorbel

département).

Les Privacy Rules de Bubble : le cœur du réacteur

Si vous utilisez Bubble, les *Privacy Rules* (Règles de confidentialité) sont le concept le plus critique à maîtriser. Elles s'exécutent strictement côté serveur. Par défaut, dans Bubble, si aucune règle n'est définie, les données sont accessibles. C'est une erreur de débutant. La bonne pratique absolue est de configurer le système pour que, par défaut, tout soit privé, et de créer des règles d'exception explicites.

Une Privacy Rule se lit ainsi : « L'utilisateur courant peut lire cette donnée SI [Condition] ». Par exemple : *« Current User is This Thing's Creator »* ou *« Current User's Company is This Invoice's Company »*. Si vous oubliez de protéger une base de données contenant des informations de santé ou des données financières, n'importe qui pouvant deviner l'URL de l'API de votre application pourra aspirer l'intégralité de votre base de données en quelques secondes.

Row-Level Security (RLS) dans les backends externes

Si vous utilisez Supabase ou Xano comme backend, le concept équivalent est la Row-Level Security (RLS). Supabase permet de définir des politiques SQL qui s'appliquent automatiquement à chaque requête. Xano gère cela via des *API Endpoints* sécurisés où chaque requête doit inclure un token d'authentification, et où la logique de l'endpoint vérifie systématiquement que l'ID de l'utilisateur correspond au propriétaire de la ressource demandée. Ne faites jamais confiance au frontend pour filtrer les données ; exigez que le backend ne renvoie que les données strictement autorisées.

14.3. Secrets, clés API : où les stocker (jamais dans l'app !)

Une clé API est l'équivalent numérique d'un mot de passe et d'un compte bancaire combinés. Elle permet à votre application de communiquer avec des services tiers (Stripe pour les paiements, SendGrid pour les emails, OpenAI pour l'IA).

La règle d'or, absolue et non négociable, est la suivante : **un secret ne doit jamais être exposé au navigateur de l'utilisateur.**

© 2026 Oussama Ghorbel

Lorsque vous construisez une application web, tout ce qui s'exécute côté client (dans le navigateur) peut être inspecté. Si vous configurez un appel API dans le *Workflow* de Bubble (qui s'exécute côté client) ou dans le frontend de WeWeb, et que vous incluez une clé API dans les en-têtes (Headers) de la requête, n'importe quel utilisateur peut ouvrir l'onglet « Réseau » (Network) des outils de développement de son navigateur, cliquer sur la requête, et copier votre clé API.

Le principe du moindre privilège

De nombreux services fournissent deux types de clés : une clé publique (ex: `pk_live_...` chez Stripe) conçue pour être utilisée côté frontend, et une clé secrète (ex: `sk_live_...`) qui doit rester côté serveur. N'utilisez jamais la clé secrète côté client. De plus, appliquez le principe du moindre privilège : si votre application n'a besoin que d'envoyer des emails via SendGrid, créez une clé API restreinte à l'action "Send" uniquement, et non une clé avec des droits d'administration sur votre compte.

Où stocker les secrets ?

1. **Dans les variables d'environnement du backend :** Sur Bubble, utilisez les *Backend Workflows* et stockez les clés dans les paramètres de l'API Connector en les marquant comme "Private" (ce qui les empêche d'être utilisées dans les workflows côté client). Sur Xano ou Supabase, utilisez les variables d'environnement sécurisées.

2. **Dans les orchestrateurs :** Make et Zapier stockent les credentials de manière chiffrée. Faites transiter les actions sensibles (comme la création d'un paiement ou l'envoi d'un SMS) par Make plutôt que de les exécuter directement depuis le frontend.

3. **Gestionnaires de secrets :** Pour les architectures complexes, des outils comme HashiCorp Vault ou AWS Secrets Manager peuvent être interrogés via des API par votre backend no-code au moment de l'exécution, évitant ainsi de stocker les clés en dur dans les configurations de la plateforme.

Enfin, planifiez la rotation de vos clés. Si un développeur quitte votre équipe ou si vous soupçonnez une fuite, vous devez pouvoir régénérer la clé chez le fournisseur et mettre à jour votre backend en quelques minutes sans casser l'application.

© 2026 Oussama Ghorbel

14.4. Protection contre les abus : rate limiting, captchas, logs

Une application no-code parfaitement sécurisée contre les fuites de données peut tout de même être mise à genoux par des abus de ressources. Les attaques par déni de service (DoS), le *credential stuffing* (tenter des milliers de mots de passe volés sur votre page de login) ou le scraping de vos données sont des menaces quotidiennes.

Rate Limiting (Limitation de débit)

Le rate limiting consiste à restreindre le nombre de requêtes qu'un utilisateur ou une adresse IP peut effectuer dans un laps de temps donné. Bubble impose des limites natives sur les plans inférieurs, mais pour les API exposées, vous devez configurer des limites strictes. Si vous exposez une API via Xano, configurez des quotas par token utilisateur. Si vous utilisez Cloudflare en proxy devant votre application, ses règles de rate limiting sont extrêmement efficaces pour bloquer les bots avant qu'ils n'atteignent votre infrastructure no-code et ne consomment vos unités de travail (Workload Units).

CAPTCHAs et défis cognitifs

Pour protéger les endpoints sensibles qui ne nécessitent pas d'authentification (inscription, mot de passe oublié, formulaire de contact), l'ajout d'un CAPTCHA est indispensable. Oubliez les anciennes grilles de feux de circulation qui ruinent l'UX. Privilégiez des solutions invisibles ou à faible friction comme Cloudflare Turnstile ou hCaptcha. Ils analysent la télémétrie du navigateur et le comportement de la souris pour attribuer un score de confiance, ne présentant un défi que si le comportement semble automatisé.

Logs et monitoring

Vous ne pouvez pas vous défendre contre ce que vous ne voyez pas. Les plateformes no-code fournissent des logs basiques, mais ils sont souvent épurés après quelques jours et difficiles à analyser.

• **Auth Logs :** Chaque tentative de connexion (réussie ou échouée) doit être tracée.

• **Action Logs :** Les modifications critiques (changement de rôle, suppression de base de données, export de données) doivent être enregistrées dans une table d'audit dédiée, immuable, que même les

© 2026 Oussama Ghorbel

administrateurs ne peuvent pas modifier.

Pour les applications à fort trafic, exportez les logs de votre backend (Xano, Supabase) et de votre orchestrateur (Make) vers un outil d'observabilité comme Datadog, Logtail ou Sentry. Configurez des alertes pour les anomalies : par exemple, si plus de 10 tentatives de connexion échouent sur un même compte en 5 minutes, déclenchez une alerte et bloquez temporairement le compte.

14.5. Audit de sécurité d'une app no-code en 30 points

Pour structurer votre démarche, voici une checklist d'audit en 30 points à appliquer systématiquement avant la mise en production de toute application no-code, et à réviser trimestriellement.

Authentification et Sessions

1. Les mots de passe exigent une complexité minimale et une longueur d'au moins 12 caractères.

2. L'authentification à deux facteurs (2FA/MFA) est disponible et fortement recommandée pour les rôles administrateurs.

3. Les sessions expirent automatiquement après une période d'inactivité définie.

4. Les tokens de session (JWT) sont stockés dans des cookies sécurisés (HttpOnly, Secure, SameSite=Strict) et non dans le localStorage.

5. Le flux de réinitialisation de mot de passe utilise des liens à usage unique avec une durée de vie courte (ex: 15 minutes).

6. Les connexions sociales (OAuth) sont configurées pour vérifier que l'email est bien validé par le fournisseur.

7. Les comptes sont verrouillés temporairement après 5 tentatives de connexion échouées.

Autorisation et Données

8. Le principe du « Default Deny » (tout est privé par défaut) est appliqué sur toutes les bases de données.

9. Les Privacy Rules (Bubble) ou RLS (Supabase/Xano) sont testées avec des comptes de différents rôles.

© 2026 Oussama Ghorbel

10. L'accès aux données n'est jamais validé uniquement par une condition visuelle côté frontend.

11. Les identifiants d'objets (IDs) ne sont pas séquentiels et devinables (privilégier les UUIDs).

12. Les endpoints API vérifient systématiquement que l'utilisateur authentifié est propriétaire de la ressource qu'il tente de modifier (prévention de l'IDOR - Insecure Direct Object Reference).

13. Les données sensibles (PII, santé, finance) sont identifiées, classifiées et isolées dans des tables spécifiques.

14. Les droits d'administration globale sont limités à un nombre restreint de personnes physiques identifiées.

API, Webhooks et Intégrations

15. Aucune clé API secrète n'est présente dans le code frontend ou les workflows côté client.

16. Les webhooks entrants sont sécurisés par une vérification de signature (HMAC) pour garantir leur origine.

17. Les appels API sortants utilisent exclusivement le protocole HTTPS.

18. Les CORS (Cross-Origin Resource Sharing) sont configurés pour n'accepter que les requêtes provenant de vos domaines officiels.

19. Les orchestrateurs (Make/Zapier) n'ont accès qu'aux scopes API strictement nécessaires.

20. Les erreurs API ne renvoient jamais de stack traces ou d'informations techniques sensibles au client.

Frontend et Expérience Utilisateur

21. Les champs de saisie sensibles (numéros de carte, mots de passe) ont les attributs HTML `autocomplete="off"` ou les gestionnaires de mots de passe sont correctement guidés.

22. Les formulaires sont protégés contre les soumissions multiples (désactivation du bouton après clic).

23. Les CAPTCHAs invisibles protègent les formulaires publics (contact, inscription).

© 2026 Oussama Ghorbel

24. Les liens externes générés par les utilisateurs s'ouvrent avec l'attribut `rel="noopener noreferrer"` pour prévenir le phishing par tabnabbing.

Infrastructure, Environnement et Gouvernance

25. Les environnements de développement/staging utilisent des bases de données fictives (aucune donnée de production réelle).

26. L'accès à l'éditeur de l'application (Bubble editor, Xano backend) est restreint par IP ou protégé par SSO.

27. Les sauvegardes de la base de données sont automatisées, chiffrées et testées régulièrement.

28. Un registre des traitements de données personnelles (RGPD) est tenu et à jour.

29. Les dépendances (plugins tiers, modules Make) sont auditées et mises à jour.

30. Un plan de réponse à incident est documenté et connu de l'équipe.

14.6. Réponse à incident : qui fait quoi quand ça casse

Malgré vos précautions, un incident peut survenir : une clé API Stripe fuite sur un repository GitHub public, un plugin tiers est compromis, ou une faille logique permet à un utilisateur de télécharger la base de données de vos clients. La panique est votre pire ennemie. Vous devez disposer d'un Plan de Réponse à Incident (IRP).

1. Détection et Confinement (Les premières minutes)

L'objectif est de stopper l'hémorragie. Si une clé API est compromise, révoquez-la immédiatement depuis la console du fournisseur et générez-en une nouvelle, même si cela casse temporairement l'application. Si un compte administrateur est compromis, désactivez-le. Si un workflow Make s'emballe et consomme des milliers d'opérations, mettez le scénario en pause. Coupez l'accès au point de vulnérabilité, même si cela implique de mettre l'application en mode maintenance.

2. Investigation et Éradication

© 2026 Oussama Ghorbel

Une fois le système stabilisé, identifiez la cause racine. Utilisez les logs d'authentification et d'API pour retracer les actions de l'attaquant. A-t-il exploité une faille IDOR ? A-t-il utilisé une technique d'ingénierie sociale pour obtenir les identifiants d'un employé ? Corrigez la faille (ajoutez une Privacy Rule manquante, forcez la réinitialisation des mots de passe de l'équipe) avant de rétablir le service.

3. Communication et Obligations Légales

C'est ici que la réalité juridique rattrape la technique. Si l'incident implique une fuite de données personnelles, le Règlement Général sur la Protection des Données (RGPD) en Europe vous impose de notifier l'autorité de contrôle (la CNIL en France) dans un délai de **72 heures** après avoir pris connaissance de la violation, si celle-ci présente un risque pour les droits et libertés des personnes. Tenez un registre précis de la chronologie de l'incident. Communiquez avec vos utilisateurs de manière transparente, sans jargon technique, en expliquant ce qui s'est passé, quelles données sont touchées, et quelles mesures vous prenez pour les protéger.

4. Post-Mortem

La culture de la sécurité doit être une culture sans blâme (*blameless*). L'objectif du post-mortem n'est pas de trouver un coupable à licencier, mais de comprendre comment le système a permis à l'erreur humaine de se transformer en catastrophe. Mettez à jour votre audit en 30 points, ajoutez de nouvelles règles de monitoring, et partagez les apprentissages avec toute l'équipe.

Exercices pratiques

Pour ancrer ces concepts, je vous propose trois exercices pratiques à réaliser sur vos propres environnements de développement.

Exercice 1 : Implémenter une authentification SSO Google sur une app Bubble

1. Rendez-vous sur la Google Cloud Console et créez un nouveau projet.

© 2026 Oussama Ghorbel

2. Configurez l'écran de consentement OAuth et créez des identifiants (Client ID et Client Secret) de type "Application Web".

3. Ajoutez l'URL de votre application Bubble dans les "Origines JavaScript autorisées" et l'URL de callback Bubble dans les "URI de redirection autorisés".

4. Dans Bubble, allez dans l'onglet *Settings > SEO / Meta tags*, activez Google OAuth et collez vos identifiants.

5. Créez un bouton "Se connecter avec Google" sur votre page de login.

6. **Le défi sécurité :** Créez un workflow qui, à la première connexion via Google, vérifie si l'email appartient à un domaine d'entreprise spécifique (ex: `@monentreprise.com`). Si ce n'est pas le cas, déconnectez immédiatement l'utilisateur et affichez une alerte d'accès refusé.

Exercice 2 : Configurer Clerk pour gérer rôles et invitations sur une app WeWeb

1. Créez un compte sur Clerk et configurez une nouvelle application. Activez Google et Email/Password comme méthodes de connexion.

2. Dans WeWeb, installez le plugin officiel Clerk et configurez-le avec vos clés publiques.

3. Créez un système d'invitations : depuis le dashboard Clerk, générez un lien d'invitation pour un nouvel utilisateur en lui attribuant un rôle personnalisé (ex: `org:editor`).

4. Configurez un webhook dans Clerk qui se déclenche à chaque création d'utilisateur (`user.created`).

5. Connectez ce webhook à un endpoint API dans Xano. Le endpoint doit recevoir les données de Clerk, créer l'utilisateur dans votre base de données Xano, et mapper le rôle Clerk dans un champ "Role" de votre table Users.

6. **Le défi sécurité :** Dans WeWeb, créez une page "Paramètres de l'organisation" et utilisez une condition de route (Frontend) ET une vérification dans l'endpoint API Xano (Backend) pour s'assurer que seul un utilisateur avec le rôle `org:admin` peut modifier les paramètres.

© 2026 Oussama Ghorbel

Exercice 3 : Réaliser l'audit de sécurité en 30 points sur une app existante

1. Sélectionnez une application no-code que vous avez construite (ou utilisez un template complexe que vous avez dupliqué).
2. Copiez la liste des 30 points de la section 14.5 dans un tableur ou une base Notion.
3. Évaluez chaque point avec un statut : Conforme, Partiel, Non conforme, Non applicable.
4. Pour chaque point "Non conforme", rédigez un ticket d'action corrective avec une estimation du temps de résolution.
5. **Le défi sécurité :** Simulez une attaque IDOR. Connectez-vous avec le Compte A, copiez l'URL ou l'identifiant d'une ressource privée (ex: une facture). Déconnectez-vous, connectez-vous avec le Compte B, et tentez d'accéder à cette même ressource via l'URL ou en forçant l'appel API via Postman. Documentez le résultat et corrigez la Privacy Rule ou la logique backend si l'accès est autorisé.

La sécurité n'est pas une fonctionnalité que l'on ajoute à la fin d'un sprint de développement. C'est une posture, une manière de penser l'architecture de votre produit dès la première base de données créée. En maîtrisant l'authentification, en verrouillant vos autorisations côté serveur et en protégeant vos secrets, vous transformez vos applications no-code de simples prototypes en véritables actifs d'entreprise, capables de résister aux réalités du monde numérique.

© 2026 Oussama Ghorbel

Chapitre 15 – Tests, déploiement et maintenance continue

Il existe une illusion tenace dans l'univers du no-code, une sorte d'ivresse de la rapidité qui nous pousse à croire qu'une application est terminée dès lors qu'elle fonctionne sur notre propre écran. En tant que créateurs, nous sommes souvent les pires testeurs de nos propres œuvres. Nous connaissons par cœur le « chemin du bonheur » (le *happy path*), cette suite logique d'actions qui mène l'utilisateur du point A au point B sans aucune embûche. Mais la réalité du terrain, celle de vos utilisateurs, est faite de connexions instables, de données mal formatées, de clics frénétiques sur les boutons de validation et de navigateurs obsolètes.

Passer du statut de « maker » bricolant une solution interne à celui de véritable éditeur de logiciel exige l'adoption d'une discipline de delivery rigoureuse. Le no-code nous affranchit de la syntaxe du code, mais il ne nous dispense absolument pas des exigences de l'ingénierie logicielle. Déployer, tester, monitorer et maintenir : ce sont les piliers qui séparent les prototypes fragiles des produits robustes capables de supporter la charge de milliers d'utilisateurs. Dans ce chapitre, nous allons structurer votre approche de la mise en production pour bannir la peur des régressions et instaurer un cycle d'amélioration continue.

15.1. Environnements dev/staging/prod dans les outils no-code

L'une des erreurs les plus coûteuses observées dans les équipes no-code est le test en production. Modifier une base de données, ajuster un workflow critique ou changer une règle de privacy directement sur l'environnement utilisé par vos clients est une recette pour le désastre. La séparation des environnements n'est pas un luxe réservé aux développeurs traditionnels ; c'est une nécessité absolue pour la survie de votre produit.

Historiquement, les outils no-code ont tardé à proposer de véritables environnements de staging. Aujourd'hui, la maturité de l'écosystème permet de mettre en place une architecture à trois niveaux : développement (dev), pré-production (staging) et production (prod).

© 2026 Oussama Ghorbel

Sur des plateformes comme **Bubble**, la distinction est native mais nécessite une discipline de fer. L'éditeur vous offre un environnement de développement (`dev`) et un environnement de production (`live`). Le déploiement se fait par un bouton « Deploy to live ». Cependant, le piège réside dans la base de données : par défaut, les données de dev et de live sont séparées, mais la synchronisation peut être complexe. Pour créer un véritable staging, les équipes avancées dupliquent l'application entière (App A pour Dev, App B pour Staging, App C pour Prod) et utilisent des outils comme l'API de Bubble ou des scripts personnalisés pour copier les données de production vers le staging (en les anonymisant, bien entendu) avant de tester de nouvelles fonctionnalités majeures.

Sur les backends comme **Xano** ou **Supabase**, la notion d'environnement est beaucoup plus proche du développement traditionnel. Xano propose des branches (branches de développement) qui permettent de travailler sur de nouvelles structures de données sans impacter l'API de production. Supabase offre des environnements de branching complets, incluant la base de données PostgreSQL et l'authentification, permettant de valider des migrations SQL ou des changements de schéma avant de les fusionner (*merge*) vers la production.

Pour les outils de frontend comme **WeWeb** ou **Toddle**, l'intégration avec Git permet de gérer des environnements de prévisualisation (preview environments) pour chaque *pull request*. Vous pouvez ainsi tester une nouvelle interface avec des données réelles (mais isolées) avant de la fusionner.

La règle d'or de la gestion des environnements est l'isolation des données. Votre environnement de staging ne doit jamais envoyer de vrais emails à vos clients, ni déclencher de vraies facturations via Stripe. Utilisez des webhooks de test, des clés API sandbox (comme le mode test de Stripe ou Twilio), et des bases de données dédiées. Penser ses environnements, c'est se donner le droit à l'erreur sans en faire payer le prix à ses utilisateurs.

15.2. Tests manuels structurés : checklist de recette

Même à l'ère de l'automatisation, le test manuel reste irremplaçable pour évaluer le « ressenti » d'une application (le *feel*), la fluidité de l'UX

© 2026 Oussama Ghorbel

et la logique métier complexe. Cependant, un test manuel ne s'improvise pas. Il se structure autour d'une checklist de recette rigoureuse, souvent appelée UAT (*User Acceptance Testing*).

L'improvisation face à une nouvelle release conduit inévitablement à des oublis. Pour structurer vos tests, je vous recommande d'adopter le format BDD (*Behavior-Driven Development*) simplifié, basé sur la structure « Étant donné / Quand / Alors » (Given / When / Then).

Prenons l'exemple d'une fonctionnalité de génération de devis PDF dans une application no-code :

• **Étant donné** que je suis un utilisateur connecté avec le rôle « Commercial » et que le panier contient 3 articles.

• **Quand** je clique sur le bouton « Générer le devis » et que je sélectionne le format « Détaillé ».

• **Alors** le système doit afficher un loader, générer un PDF en moins de 3 secondes, le stocker dans la base de données, et m'envoyer un email avec le lien de téléchargement.

Votre checklist de recette doit couvrir quatre dimensions fondamentales :

1. **Le Happy Path (Chemin nominal) :** L'utilisateur fait tout correctement. Les données sont valides, les permissions sont accordées, le réseau est stable. C'est le test de base qui valide que la fonctionnalité existe et répond au besoin principal.

2. **L'Unhappy Path (Chemins d'erreur) :** C'est ici que se joue la robustesse de votre app. Que se passe-t-il si l'utilisateur soumet un formulaire vide ? S'il upload un fichier de 50 Mo alors que la limite est de 5 Mo ? S'il perd sa connexion internet au milieu d'un processus de paiement ? Votre application doit gérer ces erreurs avec élégance (messages d'erreur clairs, absence de crash, données non corrompues).

3. **Les permissions et la sécurité (RBAC) :** Testez l'application en vous connectant avec chaque rôle défini. Un utilisateur standard peut-il accéder à une URL d'administration en la tapant manuellement ? Un client peut-il voir les données d'un autre client en manipulant les paramètres de l'URL ? Les règles de privacy de

© 2026 Oussama Ghorbel

votre backend (Bubble, Xano, Supabase) doivent être impitoyablement testées.

4. **Le Responsive et la compatibilité :** Vérifiez les points de rupture (breakpoints) de votre design. L'application est-elle utilisable sur un smartphone de petite taille ? Les tableaux de données débordent-ils de l'écran ? Testez sur Chrome, Safari (notamment sur iOS qui a des comportements spécifiques avec le stockage local et les dates), et Firefox.

Pour gérer ces checklists, abandonnez les fichiers Excel perdus sur un bureau partagé. Utilisez des outils visuels comme Notion, Airtable, ou des plateformes dédiées au QA comme TestRail ou Qase. Chaque cas de test doit avoir un statut (À faire, En cours, Validé, Échoué), une priorité (Bloquant, Majeur, Mineur) et un espace pour capturer des captures d'écran ou des vidéos en cas de bug.

15.3. Tests automatisés : Synthetic, BlazeMeter, outils dédiés

Le test manuel a un défaut majeur : il ne passe pas à l'échelle. À chaque nouvelle fonctionnalité, votre checklist s'allonge, et le temps de recette explose. C'est ici que l'automatisation des tests entre en jeu. L'ingénieur logiciel Martin Fowler résumait cette philosophie par une phrase que j'affectionne : *« If it hurts, do it more often, and bring the pain forward »* (Si ça fait mal, faites-le plus souvent, et avancez la douleur). Autrement dit, si les tests de régression sont longs et pénibles, automatisez-les pour les exécuter à chaque modification.

Cependant, l'automatisation des tests UI (interface utilisateur) en no-code est notoirement complexe. Des outils comme Selenium ou Playwright reposent sur l'inspection du DOM (la structure HTML de la page). Or, des outils comme Bubble ou Glide génèrent des identifiants de classes CSS dynamiques et aléatoires à chaque compilation. Un test automatisé qui clique sur le bouton `#button-8f7a9` cassera dès la prochaine mise à jour de l'éditeur.

Pour contourner ce problème, l'automatisation en no-code doit se concentrer sur deux niveaux :

1. Les tests de workflows et d'API (Backend Testing)

© 2026 Oussama Ghorbel

C’est le niveau le plus critique et le plus stable. Plutôt que de tester l’interface, testez la logique métier. Si vous utilisez Make, Zapier ou n8n, vous pouvez créer des scénarios de test qui envoient des fausses données (via des webhooks) à vos API et vérifient que les bases de données sont mises à jour correctement. Pour les backends comme Xano ou Supabase, vous pouvez utiliser des outils comme **Postman** ou **Bruno** pour créer des suites de tests automatisés qui interrogent vos endpoints API, vérifient les codes de réponse (200 OK, 400 Bad Request, 403 Forbidden) et valident la structure des données JSON renvoyées. Ces tests s’exécutent en quelques secondes et garantissent que votre logique métier n’a pas régressé.

2. Les tests UI intelligents (AI-Driven Testing)

Une nouvelle génération d’outils de test automatisé utilise l’intelligence artificielle et la vision par ordinateur pour s’affranchir de la fragilité du DOM. Des plateformes comme **Octomind**, **Autify** ou **Apify** permettent d’enregistrer des parcours utilisateurs. L’outil « regarde » l’écran comme un humain le ferait, identifiant le bouton « Valider la commande » par son texte et sa position visuelle plutôt que par son code source sous-jacent. Si le designer change la couleur ou le padding du bouton, le test ne casse pas.

3. Les tests de charge et de performance

Votre application no-code tiendra-t-elle la charge si 500 utilisateurs se connectent simultanément pour un événement en direct ? Des outils comme **BlazeMeter** ou **k6** permettent de simuler des milliers de requêtes concurrentes sur vos endpoints API ou vos pages publiques. C’est indispensable pour identifier les goulots d’étranglement, comme les limites de taux d’appels (rate limits) d’Airtable ou l’explosion des Workload Units sur Bubble.

L’automatisation ne doit pas chercher à couvrir 100 % de l’application. Visez la règle du 80/20 : automatisez les 20 % de fonctionnalités qui sont utilisées 80 % du temps et qui génèrent le revenu critique de votre entreprise.

15.4. Versioning : branches, rollbacks, documentation des releases

© 2026 Oussama Ghorbel

Dans le développement traditionnel, Git est le maître du temps. Il permet de voyager dans le passé, de créer des réalités parallèles (branches) et de fusionner le travail de plusieurs personnes. En no-code, le versioning a longtemps été le parent pauvre, se limitant à un historique linéaire et parfois opaque. Aujourd'hui, la discipline du versioning est accessible et doit être appliquée avec rigueur.

Le versioning remplit trois fonctions vitales : la traçabilité, la collaboration et la réversibilité.

La nomenclature des versions

Adoptez le versioning sémantique (SemVer), même pour vos apps no-code. Une version s'écrit `Majeure.Mineure.Patch` (ex: `2.4.1`).

• **Majeure** : Changement cassant (breaking change), refonte de l'UX, nouvelle architecture de données.

• **Mineure** : Nouvelle fonctionnalité rétrocompatible (ajout d'un tableau de bord exportable).

• **Patch** : Correction de bug, ajustement mineur de design.

Nommer vos déploiements dans l'historique de Bubble ou dans les releases de WeWeb avec ces numéros permet à toute l'équipe de savoir exactement de quoi on parle. « Le bug est apparu dans la version 2.4.0 » est infiniment plus clair que « Le bug est apparu après les modifications de mardi dernier ».

La stratégie de Rollback

Le rollback (retour en arrière) est votre filet de sécurité. Lorsqu'une mise en production introduit un bug bloquant, votre objectif numéro un n'est pas de comprendre *pourquoi* ça a cassé, mais de *rétablir le service* pour vos utilisateurs.

Dans Bubble, cela signifie identifier la version précédente stable dans l'onglet « History » et cliquer sur « Restore ». Dans des outils connectés à Git comme Toddle ou WeWeb, cela implique de reverser le commit ou de redéployer la branche `main` précédente.

Cependant, le rollback de données est beaucoup plus périlleux que le rollback de code. Si votre nouvelle version a modifié la structure de votre base de données (par exemple, fusionner les champs "Prénom" et

© 2026 Oussama Ghorbel

"Nom" en un seul champ "Nom Complet"), un rollback simple de l'interface laissera votre base de données dans un état corrompu. C'est pourquoi toute migration de données complexe doit être précédée d'un backup complet (via des outils comme Salto pour Salesforce, ou des scripts d'export JSON pour Xano/Airtable) et doit idéalement être conçue pour être réversible (migrations *up* et *down*).

La documentation des releases (Changelog)

Chaque déploiement doit être accompagné d'un changelog. Ce document n'est pas seulement pour les utilisateurs finaux (bien qu'une page « Nouveautés » dans votre app soit excellente pour l'adoption), il est crucial pour l'équipe interne. Utilisez des outils comme **LaunchNotes** ou simplement une base Notion pour documenter : ce qui a été ajouté, ce qui a été corrigé, et les points d'attention pour le support client. Si vous changez le libellé d'un bouton critique, l'équipe de support doit le savoir avant que le premier ticket d'assistance n'arrive.

15.6. Monitoring : uptime, erreurs, performance (Sentry, Better Uptime)

Vous ne pouvez pas améliorer ce que vous ne mesurez pas, et vous ne pouvez pas corriger ce que vous ne voyez pas. Le monitoring (ou l'observabilité) est le système nerveux de votre application en production. Attendre qu'un utilisateur furieux vous envoie un email pour découvrir que votre passerelle de paiement est hors ligne depuis trois heures est une situation que vous devez éviter à tout prix.

L'observabilité en no-code repose sur trois piliers :

1. Le monitoring de disponibilité (Uptime & Synthetic Monitoring)

Des outils comme **Better Uptime** (maintenant partiellement intégré à Datadog), **Pingdom** ou **UptimeRobot** vérifient régulièrement que votre application répond. Mais ne vous contentez pas de pinguer la page d'accueil. Configurez des moniteurs synthétiques qui simulent un parcours critique : se connecter, ajouter un article au panier, initier le checkout. Si l'étape du checkout renvoie une erreur 500 ou met plus de 4 secondes à répondre, vous devez être alerté immédiatement.

© 2026 Oussama Ghorbel

2. Le tracking d'erreurs et la rejouabilité (Error Tracking & Session Replay)

Quand un utilisateur vous dit « ça ne marche pas », il est souvent incapable de décrire les étapes pour reproduire le bug. C'est ici que des outils comme **LogRocket** ou **PostHog** deviennent magiques. En insérant un simple snippet JavaScript dans les paramètres SEO/méta de votre outil no-code, ces services enregistrent l'écran de l'utilisateur (de manière anonymisée et sécurisée), ses clics, et ses mouvements de souris. Vous pouvez littéralement « rejouer » la session de l'utilisateur qui a rencontré le bug et comprendre exactement où le workflow a échoué.

Pour les erreurs techniques (JavaScript errors, échecs d'appels API), l'intégration de **Sentry** (souvent possible via des plugins natifs ou l'injection de code HTML) permet de regrouper les erreurs, de les classer par fréquence et d'alerter l'équipe technique avant que l'impact utilisateur ne devienne massif.

3. Le monitoring de performance et des limites plateformes

Chaque outil no-code a ses limites physiques. Airtable limite le nombre d'appels API par seconde. Bubble facture à la « Workload Unit » (WU) et peut ralentir si les bases de données ne sont pas optimisées. Make peut mettre en pause vos scénarios si vous dépassez votre quota d'opérations.

Mettez en place des tableaux de bord (via Make, Xano ou des outils de BI comme Metabase) qui surveillent la consommation de vos ressources. Créez des alertes préventives : « Alerte Slack lorsque la consommation de WU de la journée atteint 80 % du quota » ou « Alerte lorsque la latence moyenne des requêtes Xano dépasse 800ms ».

La gestion des alertes (Alerting Fatigue)

Le piège du monitoring est la fatigue d'alerte. Si votre canal Slack `#alerts-prod` sonne 50 fois par jour pour des erreurs mineures ou des timeouts réseaux transitoires, votre équipe finira par ignorer les notifications, y compris celle qui signale un crash critique. Soyez impitoyable sur la configuration des seuils. Une alerte PagerDuty ou un appel téléphonique au milieu de la nuit doit être réservé aux incidents de

© 2026 Oussama Ghorbel

sévérité 1 (P1) : perte de données, faille de sécurité, ou interruption totale du service générateur de revenus.

15.6. Roadmap produit et rétro utilisateurs mensuelle

La maintenance continue ne se limite pas à garder les serveurs allumés et à corriger les bugs. C'est aussi un processus d'évolution constante, guidé par la réalité du terrain et les retours de ceux qui utilisent votre outil au quotidien. Le no-code permet une itération rapide, mais cette rapidité peut devenir un défaut si elle n'est pas canalisée par une vision produit claire.

La centralisation du feedback

Les retours utilisateurs arrivent de partout : emails, messages sur Intercom, discussions Slack, appels avec le service client. Si ce feedback n'est pas centralisé, il se perd. Utilisez des outils de gestion de roadmap et de feedback comme **Canny**, **Productboard** ou même un tableau **Trello/Notion** bien structuré. Chaque demande de fonctionnalité ou rapport de bug doit être transformé en une « carte » unique, liée à l'utilisateur qui l'a demandée. Cela vous permet, lorsque la fonctionnalité est enfin déployée trois mois plus tard, de recontacter automatiquement les 15 personnes qui l'avaient réclamée. C'est un levier d'engagement et de fidélisation extraordinairement puissant.

La rétrospective mensuelle

Instituez une réunion mensuelle de rétrospective produit et technique, réunissant les créateurs no-code, les responsables métier et le support client. Cette réunion doit répondre à trois questions :

1. **Qu'est-ce qui a bien fonctionné ce mois-ci ?** (Ex: Le nouveau workflow d'onboarding a augmenté la rétention de 15 %).

2. **Qu'est-ce qui a cassé ou ralenti l'équipe ?** (Ex: Les timeouts récurrents sur l'API de notre fournisseur de SMS, ou la dette technique accumulée sur la page de profil utilisateur).

3. **Quelles sont les priorités pour le mois prochain ?**

Prioriser avec méthode

© 2026 Oussama Ghorbel

Face à une liste de 50 demandes d'évolutions, comment choisir ? Appliquez des frameworks de priorisation comme la méthode **RICE** (Reach, Impact, Confidence, Effort) ou **MoSCoW** (Must have, Should have, Could have, Won't have). Le no-code fausse souvent notre perception de l'effort : on a tendance à vouloir tout développer parce que « ça va vite à faire ». Mais une fonctionnalité, même codée en deux heures sur Bubble, ajoute de la complexité à l'interface, nécessite des tests, de la documentation et de la maintenance future. L'effort réel inclut toujours le coût de la maintenance à long terme. Apprenez à dire non, ou à repousser les fonctionnalités « nice-to-have » pour protéger la stabilité et la simplicité de votre application.

Exercices pratiques

Pour ancrer les concepts de ce chapitre, je vous invite à réaliser ces trois exercices concrets sur l'une de vos applications no-code en cours de développement ou en production.

Exercice 1 : Rédiger une checklist de recette de 25 points

Ouvrez votre outil de gestion de projet (Notion, Airtable, Jira) et créez une base de données « Cas de Test ». Rédigez 25 cas de test structurés pour la fonctionnalité la plus critique de votre application (ex: le tunnel d'achat, la création d'un compte complexe, la génération d'un rapport).

• **Consigne :** Sur ces 25 points, 10 doivent couvrir le *Happy Path*, 10 doivent explorer des *Edge Cases* (données invalides, limites de caractères, upload de fichiers corrompus), et 5 doivent tester spécifiquement les règles de sécurité et de permissions (tenter d'accéder à une donnée sans être authentifié, tenter de modifier l'ID d'un objet dans l'URL).

Exercice 2 : Mettre en place un monitoring d'uptime avec alerte Slack/Teams

Ne vous fiez plus uniquement à vos utilisateurs pour savoir si votre app est en ligne.

© 2026 Oussama Ghorbel

• **Action :** Créez un compte gratuit sur **Better Uptime** (ou UptimeRobot). Configurez un moniteur HTTP qui ping l'URL principale de votre application toutes les 3 minutes.

• **Intégration :** Connectez le webhook d'alerte de l'outil de monitoring à un canal Slack ou Teams dédié (`#incidents-prod`).

• **Aller plus loin :** Configurez un deuxième moniteur de type « Mot-clé » (Keyword check) qui vérifie non seulement que la page renvoie un code 200, mais que le texte « Bienvenue sur votre tableau de bord » est bien présent dans le HTML, garantissant que l'application n'affiche pas une page blanche ou une erreur de base de données tout en renvoyant un code de succès.

Exercice 3 : Simuler un rollback complet d'une app après une mauvaise release

La théorie du rollback est simple, la pratique sous le stress l'est moins.

• **Simulation :** Sur votre environnement de développement ou staging, effectuez volontairement une modification « cassante » (ex: supprimer un champ critique dans la base de données, ou casser volontairement une condition dans un workflow Make/Xano). Déployez cette version.

• **Action :** Constatez l'erreur. Sans chercher à « corriger » le bug en avançant, exécutez la procédure de rollback native de votre outil (restauration de l'historique de version sur Bubble, revert de commit sur WeWeb, restauration d'un snapshot de base de données sur Xano).

• **Mesure :** Chronométrez le temps qu'il vous faut entre le moment où vous constatez l'erreur et le moment où l'application est de nouveau fonctionnelle pour l'utilisateur final. Documentez ce temps et les frictions rencontrées (ex: « J'ai perdu 15 minutes car je ne savais pas quel snapshot de base de données restaurer »). L'objectif est de réduire ce temps à moins de 5 minutes lors d'un véritable incident.

© 2026 Oussama Ghorbel

Partie 4 — Automatiser : connecter, orchestrer, fluidifier

© 2026 Oussama Ghorbel

Chapitre 16 – Zapier, Make, n8n, Power Automate : la guerre des orchestrateurs

Si les bases de données sont la mémoire de votre entreprise et les applications visuelles son visage, les orchestrateurs d'automatisation en sont le système nerveux. C'est ici, dans l'ombre des interfaces utilisateur, que la magie opère : des données circulent, des décisions sont prises, des notifications sont envoyées et des processus métier s'exécutent sans la moindre intervention humaine.

Dans la première partie de cet ouvrage, nous avons démystifié le no-code. Dans les parties suivantes, nous avons appris à modéliser des données et à construire des interfaces. Mais une application isolée, aussi belle soit-elle, n'est qu'un silo. La véritable puissance du no-code réside dans la capacité à faire dialoguer ces silos entre eux. Bienvenue dans l'ère de l'orchestration.

Aujourd'hui, le marché des plateformes d'automatisation est en pleine effervescence. Ce qui a commencé comme de simples outils de « tuyauterie » pour connecter un formulaire web à une adresse email est devenu un champ de bataille technologique où s'affrontent des géants aux philosophies radicalement différentes. Zapier, Make, n8n, Power Automate… Comment s'y retrouver ? Faut-il privilégier la simplicité, la puissance visuelle, la souveraineté des données ou l'intégration corporate ?

Dans ce chapitre, nous allons décortiquer l'anatomie d'une automation, analyser en profondeur les forces et les faiblesses de chaque grand acteur du marché, et explorer la nouvelle vague d'outils propulsés par l'intelligence artificielle. À l'issue de cette lecture, vous saurez exactement quel outil choisir pour chaque facette de votre stack.

16.1. Anatomie d'une automation : trigger, action, condition, boucle

Avant de comparer les outils, il est impératif de maîtriser le vocabulaire et la logique universelle de l'automatisation. Peu importe la plateforme que vous utiliserez, toute automation (ou workflow) repose

sur une séquence de quatre concepts fondamentaux. Comprendre ces concepts, c'est comprendre comment les APIs communiquent entre elles.

Le Trigger (Déclencheur)

C'est l'étincelle, l'événement qui donne le top départ de votre workflow. Il existe principalement deux familles de déclencheurs :

1. **Le Polling (Interrogation régulière)** : L'orchestrateur demande à l'application source toutes les X minutes : « Y a-t-il de nouveaux éléments ? ». C'est simple à mettre en place, mais cela consomme des ressources et crée un délai (latence) entre l'événement réel et son traitement.
2. **Le Webhook (Instant Push)** : L'application source est configurée pour envoyer instantanément un paquet de données (généralement au format JSON) à une URL spécifique fournie par votre orchestrateur dès qu'un événement se produit. C'est instantané, écologique en ressources, et c'est la marque d'une architecture mature.

L'Action

Une fois le trigger activé, l'action est la tâche effectuée sur une application cible. Cela peut être la création d'une ligne dans un CRM, l'envoi d'un email, la mise à jour d'un statut dans un tableau de bord, ou la génération d'une facture en PDF. Chaque action consomme une requête API.

La Condition (Filtre et Routeur)

Le monde réel n'est pas linéaire ; il est fait d'embranchements. Les conditions permettent d'évaluer les données reçues du trigger pour décider de la suite du parcours. *Si le montant de la commande est supérieur à 1000 €, alors envoyer une alerte au directeur commercial ; sinon, archiver la commande.* Les plateformes avancées permettent de créer des routeurs complexes, divisant un seul flux entrant en multiples flux sortants basés sur des règles booléennes, des expressions régulières ou des correspondances de texte.

La Boucle (Itérateur et Agrégateur)

© 2026 Oussama Ghorbel

C'est souvent là que les débutants trébuchent. Les APIs renvoient fréquemment des données sous forme de tableaux (arrays), c'est-à-dire des listes d'objets. Si votre trigger récupère 50 nouvelles commandes d'un coup, vous ne pouvez pas simplement les envoyer en bloc vers votre CRM si celui-ci n'accepte qu'une commande par requête.

- **L'Itérateur** prend un tableau et le découpe pour traiter chaque élément un par un, créant une boucle.
- **L'Agrégateur** fait l'inverse : il attend que la boucle soit terminée pour rassembler les résultats individuels en un seul tableau structuré, prêt à être envoyé vers une action finale (comme l'envoi d'un récapitulatif sur Slack).

Maîtriser ces quatre piliers vous permet de modéliser n'importe quel processus métier, de la simple notification à la synchronisation bidirectionnelle complexe entre un ERP et un CRM.

16.2. Zapier : simplicité, 7000+ apps, pricing à la tâche

Zapier est le pionnier incontesté de l'automatisation no-code. Fondé en 2011, l'outil a démocratisé le concept du « If This Then That » pour les applications métier. Sa philosophie est claire : rendre l'automatisation accessible à tous, même aux profils les moins techniques, en masquant la complexité des APIs derrière une interface linéaire et rassurante.

La promesse et l'expérience utilisateur

L'interface de Zapier est volontairement contrainte. Vous construisez des « Zaps » sous forme de listes verticales. Cette linéarité force la simplicité et empêche de créer des usines à gaz illisibles. Zapier brille par son catalogue : avec plus de 7000 applications natives intégrées, il est presque certain que l'outil que vous utilisez s'y trouve. De plus, la qualité de ses intégrations (les « Apps ») est généralement excellente, avec des mappings de données clairs et une gestion robuste des erreurs basiques.

Les limites de la linéarité

Dès que votre logique métier nécessite des boucles complexes, des transformations de données lourdes (manipulation de JSON, regex avancées) ou des embranchements multiples, Zapier montre ses limites.

© 2026 Oussama Ghorbel

L'outil a introduit les « Paths » (chemins conditionnels) et les « Looping » (boucles), mais l'interface verticale rend la visualisation globale d'un processus complexe très difficile. On finit souvent par devoir découper un processus métier en plusieurs Zaps qui se déclenchent les uns les autres, créant une dette technique invisible.

Le modèle économique : le piège de la « Task »

C'est le point de friction majeur de Zapier. Le pricing est basé sur la consommation de « Tasks » (tâches). Une tâche est comptabilisée à chaque fois qu'une action s'exécute avec succès.

Le piège : Si vous utilisez un filtre pour arrêter un Zap, Zapier a historiquement évolué sur sa façon de facturer ce filtre, mais la manipulation des données (via l'outil natif *Formatter*) ou les recherches dans des tableurs consomment des tâches. Pour une startup qui traite des dizaines de milliers d'événements par mois, la facture Zapier peut rapidement dépasser le coût d'un serveur dédié ou d'un employé à temps partiel. Zapier reste l'outil roi pour les solopreneurs, les MVPs et les flux linéaires à faible volume, mais il devient un gouffre financier à l'échelle.

16.3. Make (ex-Integromat) : scénarios visuels, pricing à l'opération

Si Zapier est le grand public de l'automatisation, Make en est le studio de création professionnel. Racheté et rebaptisé, Make a bouleversé le marché en proposant une approche radicalement différente : le canevas visuel infini.

La puissance du canevas et de la manipulation de données

Sur Make, vous ne construisez pas des listes, vous dessinez des organigrammes. Les modules (les applications) sont des bulles reliées par des flux de données. Cette représentation spatiale permet de visualiser d'un seul coup d'œil des processus métier extrêmement complexes, avec des routeurs multiples, des boucles imbriquées et des gestions d'erreurs spécifiques.

Make est un rêve pour quiconque doit manipuler des données. Son éditeur de mapping permet de glisser-déposer des variables, de créer des collections (tableaux), d'utiliser des fonctions mathématiques, textuelles

© 2026 Oussama Ghorbel

et de date extrêmement poussées sans avoir à écrire la moindre ligne de code. La gestion des erreurs (Ignorer, Reprendre, Rollback, Envoyer vers un chemin d'erreur) est native et visuelle, ce qui en fait un outil incroyablement robuste pour la production.

Le modèle économique : l'opération

Make facture à l'« opération ». Contrairement à la tâche de Zapier, une opération est comptabilisée *dès qu'un module s'exécute*, même s'il ne fait que filtrer ou échouer.

Le piège de l'Itérateur : C'est ici que les architectes no-code doivent être vigilants. Si vous recevez un tableau de 100 lignes et que vous utilisez un Itérateur suivi de 3 modules de traitement, vous venez de consommer 1 (trigger) + 100 (itérateur) + 300 (actions) = 401 opérations en une seule exécution. Make est beaucoup moins cher que Zapier à volume égal, mais une mauvaise conception de vos boucles peut faire exploser votre compteur d'opérations en quelques minutes.

Make est l'outil de prédilection des agences no-code, des ops managers et des équipes qui ont besoin de construire des moteurs de transformation de données complexes tout en gardant un contrôle visuel total.

16.4. n8n : open-source, self-hosted, pour équipes techniques

n8n (prononcé "n-eight-n", pour nodemation) représente une philosophie fondamentalement différente. C'est un outil « fair-code » (souvent assimilé à l'open-source) qui permet une approche hybride : no-code pour l'orchestration visuelle, low-code pour la transformation des données.

L'approche nodale et le code light

L'interface de n8n ressemble à celle de Make (des nœuds reliés entre eux sur un canevas), mais elle est conçue pour les profiles techniques. Là où Make vous force à utiliser ses fonctions natives pour manipuler du texte ou des tableaux, n8n vous tend les bras avec le nœud « Code ». Vous pouvez y écrire du JavaScript ou du Python natif pour transformer vos structures JSON. Pour un développeur ou un ingénieur données, c'est un gain de temps et de flexibilité inestimable. De plus, n8n intègre

© 2026 Oussama Ghorbel

nativement des nœuds basés sur LangChain, permettant de construire des workflows d'IA générative (RAG, agents) avec une facilité déconcertante.

La souveraineté des données et le Self-Hosting

L'argument massue de n8n est la possibilité de l'héberger sur vos propres serveurs (Self-Hosting). Dans des secteurs hautement régulés (santé, finance, droit) ou pour les entreprises soumises à des exigences strictes de conformité (RGPD, HDS, ISO 27001), envoyer les données clients sur les serveurs cloud de Zapier ou Make aux États-Unis est souvent un point de blocage légal.

En hébergeant n8n sur un VPS (Virtual Private Server) en Europe ou sur votre cloud privé (AWS, Scaleway), les données ne quittent jamais votre infrastructure.

Le modèle économique : le coût serveur

Puisque vous l'hébergez vous-même, n8n ne vous facture pas à la tâche ou à l'opération. Votre seule limite est la puissance de calcul (CPU/RAM) de votre serveur. Pour le prix d'un serveur à 20 € par mois, vous pouvez exécuter des millions d'exécutions qui vous coûteraient des milliers d'euros sur Zapier.

Le coût caché : Le temps DevOps. Vous êtes responsable des mises à jour, de la sécurité du serveur, de la configuration des bases de données Postgres sous-jacentes et de la gestion des certificats SSL. n8n est l'outil parfait pour les startups tech, les scale-ups et les équipes disposant d'un minimum de culture serveur.

16.5. Power Automate : intégration Microsoft et RPA desktop

Microsoft Power Automate (anciennement Microsoft Flow) est le mastodonte de l'écosystème corporate. Il ne joue pas tout à fait dans la même cour que les acteurs précédents, car il est rarement choisi pour sa beauté ou sa simplicité, mais pour son omniprésence dans les environnements d'entreprise.

L'écosystème Microsoft 365

© 2026 Oussama Ghorbel

Si votre entreprise respire via Outlook, SharePoint, Teams, Dynamics 365 et Excel Online, Power Automate est un choix par défaut presque obligatoire. Les connecteurs natifs vers l'écosystème Microsoft sont profonds, rapides et souvent inclus dans les licences Microsoft 365 Enterprise (E3/E5) que les DSI paient déjà. La gouvernance y est stricte : les administrateurs peuvent bloquer le transfert de données sensibles (DLP - Data Loss Prevention) et auditer chaque flux.

Le RPA (Robotic Process Automation) : l'arme fatale

La plus grande force de Power Automate réside dans sa capacité à combler le fossé entre les applications modernes (API) et les applications legacy (sans API). Avec *Power Automate Desktop*, vous pouvez enregistrer vos clics de souris et vos frappes au clavier pour automatiser de vieux logiciels comptables, des terminaux bancaires ou des interfaces web obsolètes. C'est ce qu'on appelle le RPA.

Le RPA permet de créer des « robots logiciels » qui interagissent avec l'interface utilisateur (UI) exactement comme le ferait un humain. Bien que fragile (si le bouton change de place, le robot casse), c'est souvent la *seule* solution pour automatiser des processus bloqués sur des logiciels patrimoniaux que la DSI n'a pas les moyens de remplacer.

Les limites : UX et Licensing

L'interface de Power Automate Cloud est souvent jugée lourde, verbeuse et peu intuitive pour les créateurs no-code habitués à l'élégance de Make. De plus, le modèle de licence de Microsoft est un labyrinthe : licences par utilisateur, licences par flux, connecteurs « Premium » (comme SQL Server ou HTTP) qui nécessitent des surcoûts… Power Automate est l'outil de la transformation digitale imposée par le haut (top-down), plutôt que celui de l'agilité bottom-up des équipes opérationnelles.

16.6. Bardeen, Relay.app, Relevance AI : la nouvelle vague

Le marché ne s'est pas arrêté à la guerre des API. Une nouvelle génération d'outils émerge pour résoudre des problèmes que les orchestrateurs classiques (basés sur des règles déterministes « Si X alors Y ») ne savent pas gérer.

© 2026 Oussama Ghorbel

Bardeen : l'automatisation du navigateur

Bardeen a compris une vérité fondamentale : beaucoup de données n'ont pas d'API, mais existent dans votre navigateur web. Bardeen se présente comme une extension Chrome puissante qui permet de scraper des données de n'importe quelle page web (LinkedIn, annuaires, sites immobiliers) et de les injecter directement dans un CRM ou un Notion. C'est l'outil roi pour les équipes commerciales (Sales Ops) qui font de la prospection et de l'enrichissement de leads manuellement.

Relay.app : l'Humain dans la boucle (Human-in-the-loop)

Les orchestrateurs classiques sont binaires : ils s'exécutent ou s'arrêtent. Mais les vrais processus métier nécessitent souvent une validation humaine. Relay.app a été conçu autour du concept de « Human-in-the-loop ». Un workflow peut s'interrompre pour envoyer un message Slack interactif à un manager, attendre son approbation (avec boutons Oui/Non et champs de commentaires), et reprendre son cours en intégrant la décision humaine dans les données du flux. C'est une approche collaborative de l'automatisation, parfaite pour les processus RH, juridiques ou financiers.

Relevance AI et la vague des Agents IA

Avec l'avènement des LLM (Large Language Models), nous passons de l'automatisation déterministe à l'orchestration probabiliste. Des plateformes comme Relevance AI (ou les fonctionnalités AI de Make et n8n) permettent de créer des « Agents ». Au lieu de dire à l'outil *« Si l'email contient le mot 'remboursement', alors classer dans la catégorie A »*, vous donnez à l'Agent un objectif et des outils : *« Lis cet email, comprends l'intention du client, consulte notre base de connaissances Notion, et rédige un projet de réponse empathique que tu soumettras à l'équipe support »*. L'orchestrateur ne se contente plus de router des données ; il délègue des tâches cognitives.

16.7. Matrice de choix selon volume, complexité, budget, souveraineté

Choisir son orchestrateur n'est pas une décision anodine ; c'est un choix d'infrastructure qui impactera votre agilité et vos coûts pour les années à venir. Voici une matrice de décision pour vous guider.

© 2026 Oussama Ghorbel

Critère / Plateforme	Zapier	Make	n8n (Self-hosted)	Power Automate
Courbe d'apprentissage	Très faible	Moyenne	Élevée (DevOps + JS)	Moyenne à Élevée
Complexité logique	Faible (Linéaire)	Très élevée (Visuel)	Très élevée (Code + Visuel)	Élevée (Verbeux)
Modèle de coût	Élevé (à la Task)	Moyen (à l'Opération)	Fixe (Coût serveur)	Complexe (Licences MS)
Souveraineté des données	Faible (Cloud US)	Moyenne (Cloud EU/US)	Totale (Votre serveur)	Forte (Cloud MS / On-prem)
Cas d'usage idéal	MVP, Solopreneurs, Linéaire	Scale-ups, Agences, Data Ops	Tech, IA, Haut volume, RGPD	Corporate, Microsoft, RPA

L'arbre de décision rapide :

1. **Avez-vous des contraintes légales strictes (Santé, Banque, HDS) ?**

n8n en self-hosted (ou Power Automate On-Premise).

2. **Votre stack est-elle 100 % Microsoft 365 et devez-vous automatiser un vieux logiciel Windows ?**

Power Automate.

3. **Devez-vous manipuler des tableaux complexes, router des données et visualiser des processus métiers denses ?**

Make.

4. **Voulez-vous connecter deux apps en 5 minutes sans réfléchir à la structure des données ?**

Zapier.

5. **Votre équipe technique veut-elle intégrer des LLMs et écrire du JavaScript au milieu du workflow ?**

n8n.

Dans la pratique, les entreprises matures n'utilisent pas un seul orchestrateur. Elles adoptent une approche polyglotte : Zapier pour les petites tâches marketing isolées, Make pour le moteur opérationnel

© 2026 Oussama Ghorbel

central (synchronisation CRM/ERP), et n8n pour les pipelines de données lourds et l'IA. L'important est de définir des frontières claires pour éviter de créer un « plat de spaghettis » d'automatisations ingouvernable.

Exercices pratiques

La théorie de l'orchestration ne s'acquiert qu'en se confrontant à la réalité des APIs et des modèles de facturation. Ces trois exercices sont conçus pour vous faire sentir physiquement la différence entre les plateformes.

Exercice 1 : Le duel de l'enrichissement (Zapier vs Make)

Objectif : Comprendre l'impact de l'interface sur la lisibilité et la gestion des erreurs.

Scénario : Un prospect remplit un formulaire (Typeform/Tally). Vous devez enrichir son email via une API (ex: Clearbit ou Apollo), vérifier si son entreprise compte plus de 50 employés, et si oui, créer une opportunité dans HubSpot et notifier le canal Slack `#sales-hot-leads`.

1. **Sur Zapier :** Construisez le Zap. Utilisez l'outil *Filter* pour la condition des 50 employés. Observez comment l'interface linéaire vous oblige à empiler les étapes. Simulez une erreur sur l'API d'enrichissement (ex: email invalide) et observez comment Zapier gère l'arrêt du Zap.
2. **Sur Make :** Créez le scénario. Utilisez un *Router* après l'enrichissement pour séparer les "Hot Leads" des "Cold Leads". Ajoutez un *Error Handler* (Break or Ignore) sur le module d'enrichissement.
3. **Analyse :** Comparez le temps de construction, la clarté visuelle du processus global, et la façon dont chaque outil vous alerte lorsque l'API d'enrichissement renvoie un code HTTP 404.

Exercice 2 : Installation de n8n en self-hosted (Guide pas-à-pas)

Objectif : Démystifier le self-hosting et comprendre l'infrastructure sous-jacente.

© 2026 Oussama Ghorbel

Prérequis : Un VPS (ex: chez Scaleway, Hetzner ou DigitalOcean) avec Ubuntu 22.04 et Docker installé.

1. Connectez-vous à votre serveur en SSH.

2. Créez un dossier dédié : `mkdir n8n && cd n8n`

3. Créez un fichier `docker-compose.yml` avec la configuration suivante :

4. Lancez le conteneur : `docker compose up -d`

5. **Pour aller plus loin (DevOps) :** Configurez un reverse proxy (comme Traefik ou Nginx) devant ce conteneur pour gérer automatiquement les certificats SSL (Let's Encrypt), car les webhooks de nombreuses applications (comme Stripe ou Shopify) refusent de communiquer avec des URLs en HTTP non sécurisé.

Exercice 3 : Le choc des factures (Calcul de coûts)

Objectif : Savoir modéliser le coût d'une automation avant de la mettre en production.

Scénario mensuel : Vous recevez 10 000 commandes e-commerce par mois.

• Étape 1 : Webhook de réception (Trigger).

• Étape 2 : Itération sur les articles du panier (en moyenne 3 articles par commande, soit 30 000 itérations).

• Étape 3 : Pour chaque article, mise à jour du stock dans l'ERP (Action).

• Étape 4 : Envoi d'un récapitulatif sur Slack (Action).

Calcul sur Zapier (Modèle Task) :

• Le trigger ne compte pas.

• L'action de mise à jour ERP (via itération) : 10 000 commandes * 3 articles = 30 000 tasks.

• L'action Slack : 10 000 tasks.

• *Total Zapier : 40 000 tasks/mois.* (Consultez la grille tarifaire de Zapier : vous serez probablement sur le plan "Company" ou "Enterprise", coûtant plusieurs centaines, voire milliers d'euros).

© 2026 Oussama Ghorbel

Calcul sur Make (Modèle Opération) :

• Webhook : 10 000 ops.

• Itérateur (le module lui-même s'exécute pour chaque commande) : 10 000 ops.

• Action ERP (à l'intérieur de la boucle) : 30 000 ops.

• Action Slack : 10 000 ops.

• *Total Make : 60 000 operations/mois.* (Sur Make, le plan à 29 $/mois inclut 10 000 ops. Il vous faudra le plan à 99 $/mois ou 299 $/mois selon les packs d'opérations additionnels).

Calcul sur n8n (Self-Hosted) :

• *Total n8n : 0 € de coût de plateforme.*

• Coût réel : Un VPS avec suffisamment de RAM pour absorber les pics de charge (ex: 40 €/mois) + 2 heures de maintenance serveur par an.

Conclusion de l'exercice : L'automatisation à haut volume de données imbriquées (tableaux/itérations) punit sévèrement les modèles SaaS à la consommation. C'est dans ces cas précis que le passage à n8n ou à une architecture low-code custom devient une nécessité économique vitale pour la startup.

© 2026 Oussama Ghorbel

Chapitre 17 – Concevoir des workflows robustes : patterns et anti-patterns

Il existe un biais cognitif bien connu chez les créateurs d'automatisations, qu'ils soient développeurs traditionnels ou makers no-code : le biais du « chemin heureux » (happy path). Lorsque nous concevons un scénario sur Make, Zapier ou n8n, nous imaginons le cas idéal. Le webhook est reçu, l'API répond en 200 millisecondes, la base de données accepte l'écriture, et l'e-mail de confirmation part sans encombre. Sur le papier, et lors des trois premiers tests, tout fonctionne à la perfection.

Puis vient la réalité de la production. Les APIs subissent des temps d'arrêt, les limites de taux d'appels (rate limits) sont atteintes, les utilisateurs double-cliquent sur les boutons de validation, et les formats de données changent sans préavis. Un workflow qui n'a jamais échoué est souvent un workflow qui ne tourne pas assez souvent pour avoir rencontré la réalité des systèmes distribués.

L'objectif de ce chapitre est de vous faire franchir un cap décisif : passer du statut de « bricoleur no-code » à celui d'architecte de processus métier. Nous allons explorer les patterns (modèles de conception) qui rendent vos automatisations résilientes, capables de survivre aux pannes, aux changements d'API et à la croissance de votre entreprise, tout en identifiant les anti-patterns qui fragilisent votre infrastructure invisible.

17.1. Idempotence : exécuter deux fois ne doit rien casser

L'idempotence est un concept emprunté aux mathématiques et à l'informatique distribuée, qui stipule qu'une opération peut être appliquée plusieurs fois sans changer le résultat au-delà de l'application initiale. En no-code, c'est la règle d'or absolue.

Pourquoi est-ce crucial ? Parce que le web n'est pas fiable. Si votre scénario Make reçoit un webhook de Shopify pour une nouvelle commande, mais que la connexion à votre CRM échoue au milieu du processus, l'orchestrateur pourrait tenter de relancer le webhook ou vous

© 2026 Oussama Ghorbel

pourriez décider de relancer manuellement le bundle. Si votre scénario n'est pas idempotent, vous venez de créer deux commandes dans votre CRM, de générer deux factures dans Stripe et d'envoyer deux e-mails de bienvenue au client.

Le pattern « Upsert » (Update / Insert)

L'approche la plus courante pour garantir l'idempotence dans des bases de données comme Airtable, Notion ou Supabase est le pattern de recherche préalable. Au lieu d'utiliser aveuglément un module « Créer un enregistrement », votre workflow doit d'abord « Chercher un enregistrement » basé sur un identifiant unique externe (comme l'``order_id` de Shopify).

• Si l'enregistrement existe : le workflow le met à jour (ou ne fait rien, selon la logique métier).

• S'il n'existe pas : le workflow le crée.

Les clés d'idempotence API

Certaines APIs modernes, comme celle de Stripe, intègrent nativement ce concept via l'en-tête HTTP `Idempotency-Key`. Lorsque vous appelez l'API pour créer un paiement, vous générez un identifiant unique (par exemple, un UUID lié à l'ID de la commande). Si votre workflow plante et réessaie l'appel API avec la même clé, Stripe comprendra qu'il s'agit de la même requête et ne débitera le client qu'une seule fois. En tant que concepteur no-code, vous devez toujours lire la documentation de l'API cible pour vérifier si elle supporte ces clés, et configurer vos modules HTTP en conséquence.

Le verrouillage par statut

Une autre technique consiste à utiliser un champ de statut dans votre base de données source. Dès que le webhook est reçu, le workflow met à jour le statut de la commande en « En cours de traitement ». Si un deuxième webhook arrive pour la même commande, le scénario vérifie le statut au tout premier module. S'il lit « En cours de traitement » ou « Traité », il s'arrête immédiatement (via un routeur ou un filtre).

17.2. Gestion des erreurs : retries, fallbacks, alertes

© 2026 Oussama Ghorbel

Dans les systèmes distribués, l'échec n'est pas une exception, c'est une certitude. Le réseau tombe, les serveurs redémarrent, les bases de données se verrouillent. Votre architecture doit s'attendre à l'échec et savoir comment y réagir.

Les directives de reprise (Retries)

La plupart des orchestrateurs comme Make proposent des directives de gestion des erreurs (Error Handlers). L'erreur classique est de laisser le scénario planter (Break) dès qu'une API renvoie une erreur 500 (Erreur interne du serveur) ou 503 (Service indisponible). Ces erreurs sont souvent transitoires.

La bonne pratique consiste à attacher un gestionnaire d'erreur avec une directive « Retry » (Réessayer). Configurez un nombre maximal de tentatives (par exemple, 3) avec un délai d'attente exponentiel (ex: attendre 10 secondes, puis 1 minute, puis 5 minutes). Cela laisse le temps à l'API cible de se rétablir sans saturer son serveur.

Attention aux erreurs 400 (Bad Request) ou 401 (Unauthorized) : celles-ci sont définitives. Réessayer ne servira à rien car le problème vient de votre payload ou de votre clé API. Il faut distinguer les erreurs transitoires (5xx) des erreurs fatales (4xx).

Les chemins de secours (Fallbacks)

Que se passe-t-il si votre passerelle SMS principale (ex: Twilio) subit une panne mondiale ? Votre processus d'alerte critique ne doit pas s'arrêter. Le pattern de fallback consiste à router l'erreur vers un service secondaire. Si le module Twilio échoue après ses tentatives, le flux est redirigé vers un module SendGrid pour envoyer un e-mail d'alerte, ou vers un webhook Slack pour notifier l'équipe technique.

Alertes contextuelles

Envoyer un message Slack disant « Le scénario X a échoué » est inutile si vous ne pouvez pas agir. Une alerte robuste doit contenir :

1. Le nom du scénario et le module fautif.
2. Le message d'erreur exact renvoyé par l'API.
3. Le `payload` (les données) qui étaient en cours de traitement, pour pouvoir les rejouer manuellement ou comprendre quel client est

© 2026 Oussama Ghorbel

impacté.

Dans Make, l'utilisation du module « Resume » (Reprendre) permet de capturer l'erreur, de l'envoyer dans une base de données de suivi des bugs, et de terminer le scénario proprement sans le marquer comme « en échec » dans l'historique global, évitant ainsi de fausser vos métriques de santé.

17.3. Logs structurés et traçabilité (execution ID, correlation ID)

Lorsque vous gérez une dizaine de scénarios interconnectés, déboguer une erreur revient souvent à chercher une aiguille dans une botte de foin. Si une commande n'arrive pas dans votre ERP, est-ce la faute du webhook Shopify, du scénario de nettoyage des données, ou du module de création dans l'ERP ?

Pour répondre à cette question, vous devez implémenter l'observabilité via deux identifiants clés :

• **L'Execution ID (ou Bundle ID) :** C'est l'identifiant technique généré par l'orchestrateur pour cette exécution précise (ex: `runId` sur Make). Il est utile pour retrouver le log brut dans l'historique de l'outil.

• **Le Correlation ID :** C'est un identifiant métier qui traverse l'ensemble de votre système. Par exemple, le `order_number` de Shopify.

Le pattern du journal centralisé

Ne laissez pas les logs éparpillés dans les historiques de Make, Zapier ou n8n, qui sont souvent purgés après quelques jours ou semaines selon votre forfait. Créez une base de données dédiée (sur Airtable, Supabase ou Xano) qui servira de « Tour de contrôle ».

À la fin de chaque scénario (et dans chaque gestionnaire d'erreur), ajoutez un module qui crée une ligne dans cette base de logs. Cette ligne doit contenir :

• Le Correlation ID (ex: `CMD-9932`)

• Le nom du scénario

© 2026 Oussama Ghorbel

- Le statut (Succès, Échec, Fallback)
- L'Execution ID technique
- L'horodatage (Timestamp)

Ainsi, lorsqu'un client appelle le support en disant « Je n'ai pas reçu ma confirmation pour la commande CMD-9932 », votre équipe n'a qu'à chercher cet ID dans la base de logs pour voir exactement à quelle étape du pipeline le blocage a eu lieu, quel scénario a échoué, et pourquoi. C'est la différence entre une équipe qui subit son no-code et une équipe qui le pilote.

17.4. Pattern 'queue' pour absorber les pics de charge

Imaginez que vous lanciez une campagne marketing massive ou que vous participiez au Black Friday. Soudainement, 10 000 webhooks frappent votre scénario Make en l'espace de cinq minutes. Votre scénario tente de traiter les 10 000 commandes en parallèle. Résultat : vous explosez les limites de taux d'appels (rate limits) de votre API CRM, votre base de données Airtable se verrouille (Error 429 : Too Many Requests), et le scénario s'effondre sous son propre poids.

Le découplage par file d'attente (Queue)

La solution architecturale est de séparer la *réception* du *traitement*. C'est le pattern de la file d'attente (asynchrone).

1. **Le Scénario A (Ingestion) :** Il reçoit le webhook, valide la structure minimale des données, et les insère immédiatement dans une table « File d'attente » (sur Xano, Supabase, ou même Airtable avec un champ statut « À traiter »). Il répond ensuite instantanément `200 OK` à l'émetteur du webhook. Ce scénario est ultra-léger et peut absorber des milliers de requêtes par minute.
2. **Le Scénario B (Traitement) :** Il s'exécute selon une planification (ex: toutes les 5 minutes) ou via un polling continu. Il interroge la base de données pour récupérer les 50 premiers enregistrements « À traiter », les traite à un rythme contrôlé qui respecte les limites de l'API cible, puis met à jour leur statut en « Traité ».

Ce pattern protège vos systèmes en aval. Peu importe la violence du pic de charge entrant, votre système de traitement avancera à son propre

© 2026 Oussama Ghorbel

rythme, de manière stable et prévisible, sans perdre une seule donnée.

17.5. Pattern 'fan-out / fan-in' pour le parallélisme

Parfois, vous devez traiter un grand volume de données de manière individuelle, par exemple enrichir 1 000 contacts avec l'API Clearbit, ou générer 500 PDFs personnalisés. Faire cela de manière séquentielle (l'un après l'autre) prendrait des heures et dépasserait le temps d'exécution maximal autorisé par votre orchestrateur (timeout).

Le Fan-Out (Dispersion)

Le fan-out consiste à prendre un tableau de données et à le diviser pour traiter chaque élément en parallèle. Dans Make, on utilise le module « Itérateur ». Cependant, l'itérateur a des limites de mémoire et de temps.

Pour les très gros volumes, le pattern robuste consiste à utiliser un sous-scénario. Le scénario principal itère sur le tableau et, pour chaque élément, appelle un webhook qui déclenche un *sous-scénario* indépendant. Vous passez ainsi d'un traitement séquentiel à un traitement massivement parallèle.

Le Fan-In (Agrégation)

Le défi du parallélisme est de savoir quand tout est terminé pour passer à l'étape suivante (ex: envoyer le rapport final de synthèse). C'est le fan-in.

Dans des outils comme n8n, le nœud « Merge » ou « Wait » permet d'attendre que toutes les branches parallèles soient terminées. Dans Make, si vous utilisez des sous-scénarios asynchrones, le fan-in est plus complexe : il faut souvent utiliser un compteur dans une base de données. Chaque sous-scénario, une fois terminé, incrémenté un compteur lié au `batch_id`. Un troisième scénario surveille ce compteur ; lorsqu'il atteint le nombre total d'éléments du lot, il déclenche l'action finale de synthèse.

17.6. Anti-patterns : workflows monolithiques, données codées en dur

© 2026 Oussama Ghorbel

Pour bâtir du no-code robuste, il est tout aussi important de savoir ce qu'il ne faut *pas* faire. Lors des audits d'architectures no-code en entreprise, on retrouve invariablement les mêmes anti-patterns qui transforment les outils visuels en cauchemars de maintenance.

Anti-pattern 1 : Le scénario spaghetti (Monolithe)

Un scénario qui dépasse 50 modules sur un seul canevas est un anti-pattern. Il devient illisible, impossible à tester par morceaux, et si un module au milieu échoue, relancer l'ensemble est périlleux.

La solution : La modularisation. Utilisez les sous-scénarios (ou routines). Un scénario principal agit comme un chef d'orchestre : il reçoit la donnée, appelle le sous-scénario « Enrichissement », puis le sous-scénario « Création CRM », puis le sous-scénario « Notification ». Chaque sous-scénario a une responsabilité unique, peut être testé indépendamment et réutilisé dans d'autres processus.

Anti-pattern 2 : Les données codées en dur (Hardcoding)

Mapper directement l'ID d'un dossier Google Drive (`1aBcD_EfGh...`) ou l'ID d'un canal Slack (`#C01234XYZ`) directement dans les paramètres d'un module est une bombe à retardement. Le jour où vous dupliquez ce scénario pour l'environnement de production, ou si le canal Slack est supprimé et recréé, tout casse.

La solution : Le pattern de la « Table de Configuration ». Au tout début de votre scénario, ajoutez un module qui lit une table de configuration (stockée dans votre base de données ou via des variables d'environnement sur n8n) pour récupérer les ID dynamiques selon l'environnement (Dev, Staging, Prod) ou le client.

Anti-pattern 3 : Ignorer la pagination

Vous utilisez un module « Lister les enregistrements » sur Airtable ou HubSpot. Vous mappez le résultat et passez à la suite. Problème : la plupart des APIs limitent les réponses à 100 ou 250 éléments par page. Si vous avez 1 000 clients, vous venez d'en ignorer 750 sans même le savoir.

© 2026 Oussama Ghorbel

La solution : Toujours vérifier si l'API renvoie un curseur de pagination (un `offset` ou un `next_page_url`) et configurer l'itération ou la boucle de votre orchestrateur pour continuer à requêter jusqu'à ce que toutes les pages soient consommées.

Anti-pattern 4 : Le biais du type de données

Traiter un numéro de téléphone ou un code postal comme un « Nombre » (Integer) plutôt que comme du « Texte » (String). Un code postal commençant par `0` (comme `06000` à Nice) perdra son premier zéro s'il est traité mathématiquement par un JSON mal configuré, créant des erreurs en cascade dans les outils de routage logistique. En no-code, forcez toujours le typage explicite (Texte, Booléen, Date ISO 8601) avant d'envoyer des données à une API externe.

17.7. Documenter un scénario pour qu'un collègue le reprenne

Le no-code est du code. Et comme tout code, il a besoin de documentation. Le « Bus Factor » (le risque que votre processus métier s'arrête si vous êtes renversé par un bus, ou plus prosaïquement, si vous partez en vacances) est le plus grand ennemi de l'industrialisation no-code. Un scénario non documenté est une dette technique invisible.

Chaque scénario critique doit posséder son « Readme », idéalement stocké dans un wiki interne (Notion, Confluence) ou directement dans les notes de l'orchestrateur si l'outil le permet (comme les annotations sur Make ou les notes sur n8n).

Le template de documentation d'un workflow :

1. **Objectif Métier :** En une phrase, que fait ce scénario et pourquoi existe-t-il ? (ex: « Synchronise les nouveaux leads qualifiés de Typeform vers Salesforce et alerte le SDR de garde »).

2. **Déclencheur (Trigger) :** Webhook, planning, polling ? Quelle est la source ?

3. **Entrées / Sorties :** Quelles sont les données attendues en entrée (payload JSON) et ce qui est créé en aval.

4. **Dépendances et Accès :** Lister les APIs tierces utilisées, les comptes de service impliqués, et où sont stockées les clés (ex: « Clé

© 2026 Oussama Ghorbel

API Stripe stockée dans le coffre-fort 1Password, liée via le compte Make 'Finance-Prod' »).

5. **Gestion des erreurs :** Expliquer la logique de fallback. (ex: « Si Salesforce renvoie une erreur 400, le lead est créé dans la table Airtable 'Fallback Leads' et une alerte Slack est envoyée sur #tech-support »).

6. **Limites connues :** (ex: « Ne gère pas les mises à jour de numéros de téléphone, seulement les créations »).

Prendre 15 minutes pour rédiger cette fiche après la conception d'un workflow vous sauvera des heures d'ingénierie inverse (reverse engineering) paniquée dans six mois, lorsque l'API changera de version et que vous aurez oublié comment vous aviez contourné le problème.

Exercices pratiques

Pour ancrer ces concepts d'architecture robuste, je vous propose trois exercices progressifs à réaliser sur votre orchestrateur préféré (Make, n8n ou Zapier).

Exercice 1 : Ajouter retry + fallback sur un scénario Make existant

Prenez un scénario simple que vous avez déjà créé (par exemple, un formulaire qui crée une ligne dans un CRM).

1. Identifiez le module de création dans le CRM.

2. Attachez-lui un gestionnaire d'erreur.

3. Configurez une directive « Retry » avec 3 tentatives et un délai progressif (10s, 30s, 1m) pour absorber les erreurs 500.

4. Ajoutez un second gestionnaire d'erreur (Fallback) qui, en cas d'échec définitif (après les retries), envoie un message formaté sur Slack ou par e-mail contenant le `bundleID` et les données brutes du formulaire, pour traitement manuel.

Exercice 2 : Refactoriser un workflow monolithique en 3 sous-scénarios

© 2026 Oussama Ghorbel

Trouvez (ou créez) un « scénario spaghetti » d'au moins 30 modules qui fait trois choses distinctes : réception/validation, enrichissement via une API externe, et écriture en base de données.

1. Découpez ce scénario en trois sous-scénarios indépendants (ex: `SUB_01_Validation`, `SUB_02_Enrichissement`, `SUB_03_Stockage`).

2. Créez un scénario principal « Chef d'orchestre » qui reçoit le webhook initial et appelle séquentiellement ces trois sous-scénarios en leur passant les données nécessaires.

3. Testez le sous-scénario `SUB_02` de manière isolée avec des données fictives pour valider sa robustesse indépendamment du reste du flux.

Exercice 3 : Écrire la doc d'un scénario en 1 page

Choisissez le scénario le plus critique de votre organisation actuelle.

1. Ouvrez un document vierge (Notion, Word, Markdown).

2. Dessinez un diagramme de flux simple (vous pouvez utiliser des outils comme Mermaid.js, Whimsical, ou simplement des blocs de texte).

3. Remplissez le template de documentation proposé dans la section 17.7 (Objectif, Trigger, Dépendances, Gestion des erreurs, Limites).

4. Partagez cette documentation à un collègue qui ne connaît pas ce workflow et demandez-lui s'il serait capable de comprendre où chercher en cas de panne. Ajustez le document selon ses retours.

Conclusion : De l'artisan à l'architecte de l'ombre

Au terme de ce chapitre, une vérité fondamentale doit s'être ancrée dans votre esprit : la robustesse d'une automatisation ne se mesure pas à sa complexité visuelle, mais à sa capacité à absorber le chaos du monde réel. Le no-code nous a vendu la promesse de la rapidité, de l'agilité et de l'accessibilité. Mais cette vitesse d'exécution est un couteau à double tranchant. Elle nous permet de déployer en quelques heures ce qui prenait autrefois des mois de développement traditionnel, mais elle nous expose aussi au risque de multiplier les dettes techniques à une échelle industrielle si nous ne posons pas de garde-fous architecturaux.

© 2026 Oussama Ghorbel

Concevoir des workflows robustes, c'est accepter de passer du temps sur ce qui ne se voit pas. L'utilisateur final, le directeur commercial ou le client ne verront jamais votre gestionnaire d'erreur avec fallback, votre table de configuration dynamique ou votre système de file d'attente asynchrone. Ils ne verront que le résultat : un processus qui fonctionne, tout le temps, sans interruption, sans perte de données et sans nécessiter l'intervention humaine constante d'un administrateur épuisé. C'est là que réside la véritable valeur du maker no-code en entreprise. Vous n'êtes plus seulement celui qui « connecte des applications entre elles ». Vous êtes l'architecte de l'ombre, le garant de la continuité des processus métier.

La Checklist de pré-déploiement (La « Pre-Flight » No-Code)

Avant de basculer n'importe quel scénario de votre environnement de test vers la production, je vous invite à instaurer une discipline de fer inspirée de l'aéronautique : la checklist de pré-vol. Imprimez-la, gardez-la dans votre Notion ou votre wiki interne, et refusez de publier un workflow critique tant que ces cases ne sont pas cochées.

1. **Test du chemin malheureux (Unhappy Path) :** Avez-vous volontairement provoqué une erreur 500, une erreur 401 et une rupture de payload pour observer la réaction du scénario ?

2. **Idempotence vérifiée :** Si je relance le webhook d'entrée trois fois de suite avec les mêmes données, mon système cible crée-t-il bien un seul et unique enregistrement ?

3. **Gestion des erreurs active :** Chaque module critique (API externe, écriture en base) possède-t-il un *error handler* avec une logique de *Retry* ou de *Fallback* ?

4. **Traçabilité assurée :** Un *Correlation ID* métier est-il généré ou capturé, et un log est-il écrit dans la base de supervision centrale ?

5. **Zéro donnée en dur (Hardcoding) :** Tous les identifiants de dossiers, de canaux ou de bases de données sont-ils sourcés depuis une table de configuration ou des variables d'environnement ?

6. **Pagination gérée :** Si la source de données dépasse la limite par défaut de l'API (souvent 100 lignes), mon itérateur ou ma boucle

© 2026 Oussama Ghorbel

est-il configuré pour consommer toutes les pages ?

7. **Documentation à jour :** Le *Readme* du scénario est-il rédigé, incluant le schéma de flux, les dépendances, les accès et les limites connues ?

Si vous pouvez répondre « oui » à ces sept questions, votre automatisation n'est plus un simple script fragile. C'est un actif numérique pérenne, prêt à supporter la croissance de votre organisation et à résister aux inévitables tempêtes du paysage logiciel moderne.

Vers le catalogue des possibles

Nous avons désormais posé les fondations, maîtrisé la donnée, appris à construire des interfaces et, avec ce chapitre, acquis la rigueur architecturale nécessaire pour orchestrer des flux complexes et résilients. Vous possédez la méthode et les outils pour survivre aux pannes, anticiper les changements d'API et absorber la charge.

Mais une question demeure, peut-être la plus vertigineuse de toutes pour le créateur no-code : *que construire maintenant ?*

Face à la toile blanche de Make, de n8n ou de Zapier, le syndrome de la page blanche n'est généralement pas lié à un manque de compétences techniques, mais à un manque de vision face à l'infinité des processus métier automatisables. On sait *comment* bâtir, mais on hésite sur *quoi* bâtir pour générer un impact immédiat. C'est pour répondre à ce vertige que nous allons aborder la prochaine étape.

Dans le chapitre suivant, nous laisserons de côté la théorie architecturale pour plonger dans le pragmatisme pur. Je vous présenterai le catalogue des automatisations métier types : les recettes qui génèrent un retour sur investissement immédiat, classées par département, par maturité technique et par stack. Préparez-vous à cartographier votre propre entreprise et à identifier, dès demain matin, les premiers goulots d'étranglement que vous allez pouvoir faire disparaître sans écrire la moindre ligne de code.

© 2026 Oussama Ghorbel

Chapitre 18 – Automatisations métier types : le catalogue des 50 recettes

Automatisations métier types : le catalogue des 50 recettes

Après avoir exploré les orchestrateurs et leurs patterns de conception, il est temps de passer à la pratique. Ce chapitre vous propose un catalogue de recettes d'automatisation éprouvées, classées par fonction métier. Chaque recette est conçue pour être adaptée à votre contexte spécifique, que vous utilisiez Make, Zapier, n8n ou Power Automate.

L'objectif n'est pas de copier-coller aveuglément, mais de comprendre les mécanismes sous-jacents pour les appliquer à vos propres processus. Une bonne automatisation commence toujours par une compréhension claire du besoin métier avant de choisir les outils.

18.1. Ventes : du lead capté à la signature électronique

Le cycle de vente se prête particulièrement bien à l'automatisation, car il combine des étapes répétitives (qualification, suivi) avec des moments critiques (signature, facturation). Voici les recettes essentielles.

Recette 1 : Capture et qualification automatique de leads

Le scénario classique : un formulaire web capture un lead, et vous devez le router vers le bon commercial, enrichir ses données, et déclencher une séquence de nurturing.

Le point clé ici est la segmentation précoce. Un lead qualifié chaud doit être traité immédiatement, tandis qu'un lead tiède entre dans un parcours de maturation. L'erreur fréquente consiste à envoyer tous les leads au même endroit sans distinction.

Recette 2 : Suivi automatique post-démo

Après une démonstration produit, le délai de suivi est critique. Voici comment automatiser sans perdre la touche personnelle.

© 2026 Oussama Ghorbel

Cette recette illustre le principe de l'escalade progressive. Chaque étape ajoute de la valeur ou de l'urgence sans être intrusive. Le commercial garde le contrôle mais bénéficie d'un filet de sécurité automatique.

Recette 3 : Génération et envoi de devis avec signature électronique

L'étape de signature est souvent un goulot d'étranglement. Automatisons-la.

La clé de cette recette est la synchronisation bidirectionnelle entre votre CRM et votre outil de signature. Le commercial voit en temps réel si le client a ouvert, consulté ou signé le document.

Recette 4 : Alertes d'engagement pour les deals stagnants

Comment détecter qu'un deal s'enlise avant qu'il ne soit trop tard ?

Cette recette fonctionne sur le principe de l'exception. Plutôt que de notifier sur tout, elle signale uniquement les situations anormales qui nécessitent une intervention humaine.

18.2. Marketing : nurturing, scoring, synchronisation CRM/ESP

Le marketing moderne repose sur la capacité à délivrer le bon message, à la bonne personne, au bon moment. L'automatisation est au cœur de cette promesse.

Recette 5 : Synchronisation bidirectionnelle CRM <-> ESP (Email Service Provider)

Vos contacts existent dans votre CRM et votre outil d'emailing. Gardons-les synchronisés.

La synchronisation bidirectionnelle évite les doublons et garantit que vos campagnes ciblent les bonnes personnes. Attention toutefois aux boucles infinies : utilisez des timestamps ou des flags pour éviter qu'une mise à jour dans un système déclenche une mise à jour dans l'autre, qui déclenche à nouveau le premier système.

Recette 6 : Scoring comportemental dynamique

© 2026 Oussama Ghorbel

Le scoring ne doit pas être statique. Il doit refléter l'engagement réel du prospect.

Cette recette transforme des signaux faibles en insights actionnables. Le commercial ne contacte pas quelqu'un qui a juste ouvert un email, mais intervient quand l'ensemble des comportements indique un intérêt sérieux.

Recette 7 : Nurturing multi-canal adaptatif

L'email seul ne suffit plus. Combinons les canaux.

L'adaptativité est la clé. Un prospect engagé reçoit du contenu avancé, tandis qu'un prospect passif reçoit des rappels doux. Cette recette nécessite un outil capable de gérer des branches conditionnelles complexes (Make ou n8n excellents ici).

Recette 8 : Nettoyage automatique des listes email

Les listes email se dégradent naturellement (changements d'emploi, adresses invalides). Automatisons leur maintenance.

Une liste propre améliore votre délivrabilité et réduit vos coûts. Cette recette de maintenance préventive devrait tourner en continu sans intervention humaine.

18.3. Support : ticket, SLA, escalation, CSAT automatique

Le support client est un terrain fertile pour l'automatisation, car il combine volume élevé et processus structurés.

Recette 9 : Création et routage intelligent de tickets

Chaque demande entrante doit être catégorisée et assignée correctement dès le départ.

L'analyse IA en amont évite le tri manuel et réduit les erreurs de routage. Même avec une précision de 85%, le gain de temps est substantiel.

Recette 10 : Monitoring SLA et escalade automatique

Les délais de réponse contractuels doivent être respectés. Automatisons leur surveillance.

© 2026 Oussama Ghorbel

Cette recette fonctionne sur deux temporalités : prévention (avant dépassement) et réaction (après dépassement). L'escalade automatique garantit qu'aucun ticket ne tombe dans les oubliettes.

Recette 11 : Résolution automatique des questions fréquentes

Certaines questions reviennent sans cesse. Automatisons leurs réponses.

Cette recette réduit la charge sur les agents pour les questions simples, tout en gardant une porte de sortie quand la réponse automatique ne suffit pas.

Recette 12 : Collecte CSAT post-résolution

La satisfaction client se mesure à chaud, juste après la résolution.

La collecte CSAT automatisée fournit des données précieuses tout en identifiant immédiatement les clients mécontents qui nécessitent une attention particulière.

18.4. RH : onboarding, congés, évaluations, offboarding

Les processus RH sont hautement structurés et chronophages. L'automatisation libère du temps pour l'humain.

Recette 13 : Onboarding complet J-7 à J+30

L'intégration d'un nouvel employé implique de multiples services. Coordonnons-les.

Cette recette orchestre des actions sur plusieurs semaines et systèmes (IT, facilities, RH, management). Sans automatisation, des éléments sont inévitablement oubliés.

Recette 14 : Gestion des demandes de congés avec validation

Le processus de congés semble simple mais implique plusieurs validations et synchronisations.

Cette recette élimine les allers-retours par email et garantit que toutes les parties prenantes sont informées automatiquement.

Recette 15 : Rappels évaluations annuelles

© 2026 Oussama Ghorbel

Les évaluations sont souvent retardées par manque de rappels systématiques.

Cette recette transforme un processus annuel souvent chaotique en un workflow fluide avec rappels progressifs.

Recette 16 : Offboarding sécurisé

Le départ d'un employé nécessite de révoquer les accès et transférer les responsabilités.

L'offboarding est critique pour la sécurité. Cette recette garantit qu'aucun accès n'est oublié, ce qui serait une faille de sécurité majeure.

18.5. Finance : facturation, relances, prévisionnel, clôture

La finance exige précision et traçabilité. L'automatisation réduit les erreurs humaines tout en accélérant les cycles.

Recette 17 : Facturation automatique post-livraison

Dès qu'une prestation est livrée, la facture doit partir.

Cette recette élimine le délai entre livraison et facturation, améliorant ainsi votre cash flow.

Recette 18 : Relances automatiques des impayés

Les relances sont inconfortables mais nécessaires. Automatisons-les avec tact.

L'escalade progressive maintient la relation client tout en montrant que vous prenez le paiement au sérieux. Chaque étape est documentée automatiquement.

Recette 19 : Reporting financier automatique

Les rapports financiers mensuels prennent des heures à préparer manuellement.

Cette recette fournit une vision financière à jour sans effort manuel, permettant à la direction de prendre des décisions éclairées rapidement.

Recette 20 : Clôture mensuelle assistée

La clôture comptable mensuelle implique de nombreuses vérifications.

© 2026 Oussama Ghorbel

Cette recette transforme la clôture d'un exercice stressant en un processus contrôlé avec alertes précoces sur les problèmes.

18.6. Ops : inventaire, reporting, alertes, data quality

Les opérations quotidiennes génèrent de nombreuses tâches répétitives parfaites pour l'automatisation.

Recette 21 : Monitoring de stock avec réapprovisionnement automatique

Les ruptures de stock coûtent cher. Anticipons-les.

Cette recette équilibre automatisation et contrôle humain. Les commandes sont préparées automatiquement mais validées manuellement, évitant les erreurs coûteuses.

Recette 22 : Reporting opérationnel multi-sources

Les données opérationnelles sont souvent dispersées dans plusieurs outils.

Cette recette crée une source de vérité unique à partir de données fragmentées, facilitant la prise de décision.

Recette 23 : Alertes de qualité des données

Les données se dégradent sans maintenance proactive.

Cette recette traite la qualité des données comme un processus continu plutôt qu'un nettoyage ponctuel, améliorant la fiabilité globale de vos systèmes.

Recette 24 : Synchronisation inter-systèmes avec réconciliation

Quand plusieurs systèmes doivent rester synchronisés, les écarts sont inévitables.

Cette recette maintient la cohérence entre systèmes sans intervention manuelle constante, tout en escaladant les cas complexes.

18.7. Composer ses propres recettes à partir des briques

Vous avez maintenant vu 24 recettes concrètes, mais le catalogue complet en contient 50. Plutôt que de toutes les lister, comprenons comment les composer vous-même à partir de briques élémentaires.

© 2026 Oussama Ghorbel

Les 7 briques fondamentales

Toute automatisation, aussi complexe soit-elle, se construit à partir de ces briques :

1. **Trigger** : L'événement déclencheur (webhook, planning, changement d'état)
2. **Extraction** : Récupérer des données depuis une source
3. **Transformation** : Modifier, enrichir, filtrer les données
4. **Condition** : Brancher le flux selon des critères
5. **Action** : Écrire, envoyer, créer quelque chose
6. **Boucle** : Répéter pour chaque élément d'une liste
7. **Attente** : Pause temporelle avant de continuer

Pattern de composition : la chaîne de valeur

La plupart des recettes suivent ce pattern :

Exemple de composition : alerte de churn prédictif

Combinons les briques pour créer une recette sophistiquée.

Cette recette combine extraction multi-sources, transformation complexe (scoring), conditions multiples, et actions différenciées. Pourtant, elle n'utilise que les 7 briques fondamentales.

Principes de composition robuste

Quand vous composez vos propres recettes, respectez ces principes :

1. **Atomicité** : Chaque action doit être indépendante et idempotente. Si l'automatisation échoue à mi-parcours et redémarre, elle ne doit pas créer de doublons.

2. **Observabilité** : Loggez chaque étape importante. Quand quelque chose échoue, vous devez pouvoir retracer exactement où et pourquoi.

3. **Graceful degradation** : Si une action échoue, l'automatisation doit continuer autant que possible plutôt que de s'arrêter brutalement.

© 2026 Oussama Ghorbel

4. **Circuit breaker** : Si une action échoue plusieurs fois consécutivement, arrêtez l'automatisation et alertez un humain plutôt que de répéter indéfiniment.

5. **Reversibility** : Quand c'est possible, concevez vos actions pour qu'elles soient réversibles. Créer un enregistrement est facile, mais pouvez-vous le supprimer proprement si nécessaire ?

Le catalogue complet en synthèse

Voici les 26 recettes supplémentaires qui complètent le catalogue de 50, organisées par domaine :

Ventes (6-10) : Lead scoring prédictif, assignation territoriale automatique, prévision de closing, analyse win/loss automatique, alertes concurrentielles.

Marketing (11-15) : Attribution multi-touch, A/B testing automatisé, personnalisation site web dynamique, gestion campagnes multi-canal, veille concurrentielle.

Support (16-20) : Chatbot avec escalation humaine, détection fraude, knowledge base auto-enrichie, gestion communauté, analyse sentiment temps réel.

RH (21-25) : Recrutement (tri CV automatique), formation (assignation adaptative), engagement (pulse surveys), mobilité interne, gestion notes de frais.

Finance (26-30) : Réconciliation bancaire, détection anomalies, forecasting automatique, gestion budget par département, conformité réglementaire.

Ops (31-35) : Monitoring infrastructure, gestion incidents, déploiement continu, backup automatique, gestion licences logicielles.

Transverse (36-40) : Gestion projets (allocation ressources), communication interne (newsletters automatiques), sécurité (détection accès suspects), conformité RGPD (gestion consentements), veille réglementaire.

Avancé (41-50) : Orchestration multi-workflows, synchronisation temps réel bidirectionnelle, machine learning intégré, traitement langage

© 2026 Oussama Ghorbel

naturel, génération contenu IA, analyse prédictive, optimisation continue, gouvernance données, audit trail complet, disaster recovery.

Chacune de ces recettes suit les mêmes principes que celles détaillées précédemment. La complexité augmente, mais les briques fondamentales restent identiques.

Quand ne pas automatiser

Toutes les recettes du monde ne justifient pas l'automatisation de tout. Voici quand résister à l'envie d'automatiser :

• **Volume trop faible** : Si une tâche prend 5 minutes par mois, l'automatisation n'est pas rentable.

• **Variabilité extrême** : Si chaque exécution est radicalement différente, l'automatisation sera plus complexe que le travail manuel.

• **Jugement humain critique** : Certaines décisions nécessitent nuance et contexte qu'une automatisation ne peut capturer.

• **Réglementation stricte** : Certains processus exigent une traçabilité humaine explicite pour des raisons légales.

L'automatisation est un outil, pas une fin en soi. Les meilleures recettes sont celles qui libèrent du temps humain pour des tâches à plus forte valeur ajoutée.

Exercices pratiques

Exercice 1 : Implémenter la recette "lead -> CRM -> email -> task commercial"

Implémentez dans votre orchestrateur préféré (Make, Zapier, ou n8n) le workflow suivant :

1. Créer un formulaire web (Typeform, Google Forms, ou Tally) capturant : nom, email, entreprise, taille entreprise, message

2. Configurer un trigger sur soumission de formulaire

3. Enrichir automatiquement les données entreprise via une API (Clearbit, Apollo, ou Hunter)

© 2026 Oussama Ghorbel

4. Créer un contact et une opportunité dans votre CRM (HubSpot, Pipedrive, ou Notion comme CRM)

5. Envoyer un email de remerciement personnalisé avec contenu adapté à la taille d'entreprise

6. Créer une tâche assignée au commercial avec priorité basée sur taille entreprise (>100 employés = haute priorité)

7. Notifier l'équipe commerciale via Slack ou Teams

Critères de réussite :

• Le lead apparaît dans le CRM dans les 30 secondes suivant la soumission

• L'email part automatiquement avec personnalisation correcte

• La tâche commerciale inclut toutes les informations de contexte

• Le workflow gère correctement les erreurs (email invalide, CRM indisponible)

Extension : Ajoutez une condition de scoring. Si la taille d'entreprise est >500 employés ET le message contient certains mots-clés (budget, urgent, projet), marquez le lead comme "Hot" et déclenchez une notification SMS au commercial.

Exercice 2 : Construire un onboarding RH automatisé de J-7 à J+30

Concevez et implémentez un workflow complet d'onboarding couvrant la période J-7 (une semaine avant l'arrivée) à J+30 (un mois après).

Étapes à automatiser :

J-7 :

• Commander matériel IT (créer ticket dans système IT ou envoyer email structuré)

• Créer comptes utilisateur (email, Slack, outils métier) via API ou formulaires

• Préparer documentation d'accueil (générer PDF personnalisé)

J-1 :

© 2026 Oussama Ghorbel

• Envoyer email de bienvenue avec informations pratiques (adresse, horaires, dress code, agenda jour 1)

• Notifier l'équipe de l'arrivée imminente

• Créer tâche manager "Préparer accueil"

J0 (jour d'arrivée) :

• Envoyer programme de la journée

• Créer tâches récurrentes (daily standup, 1:1 manager, formations obligatoires)

• Assigner un buddy automatiquement

J+7 :

• Envoyer survey "Première semaine" (5 questions max)

• Analyser réponses et alerter RH si score < 3/5

J+30 :

• Envoyer survey "Premier mois" plus détaillé

• Vérifier completion des formations obligatoires

• Planifier point de suivi RH

• Générer rapport d'intégration pour le manager

Critères de réussite :

• Toutes les actions se déclenchent aux bonnes dates

• Les emails sont personnalisés avec nom, rôle, équipe

• Les surveys sont envoyés automatiquement et les réponses analysées

• Les alertes fonctionnent (manager notifié si survey négatif)

• Le workflow est résilient (si un système est down, les autres actions continuent)

Extension : Ajoutez une branche pour les managers. Le manager du nouvel employé reçoit également des tâches et rappels (préparer l'accueil, planifier des 1:1, donner du feedback à J+7 et J+30).

Exercice 3 : Créer un dashboard de relances clients impayés synchronisé avec la compta

© 2026 Oussama Ghorbel

Construisez un système complet de gestion des impayés avec synchronisation bidirectionnelle entre votre outil de relance et votre logiciel comptable.

Composants à implémenter :

1. **Extraction des impayés** :

• Quotidiennement, récupérer depuis votre logiciel comptable (ou Google Sheets simulé) toutes les factures échues

• Pour chaque facture : montant, date échéance, client, historique de paiements

2. **Scoring de risque** :

• Calculer un score de risque pour chaque client basé sur :

• Retard actuel (jours)

• Historique de paiement (toujours à l'heure ? souvent en retard ?)

• Montant total dû

• Ancienneté relation client

3. **Stratégie de relance adaptative** :

• Score faible (client fiable, petit retard) : Email courtois J+3

• Score moyen : Email J+1, appel J+7

• Score élevé (client difficile, gros montant) : Email J+1, appel J+3, courrier recommandé J+10

4. **Tracking des actions** :

• Logger chaque relance envoyée (email, appel, courrier)

• Tracker les réponses et promesses de paiement

• Mettre à jour le statut dans le logiciel comptable

5. **Dashboard de pilotage** :

• Créer un dashboard (Google Data Studio, Notion, ou Airtable) affichant :

• Montant total des impayés

• Répartition par ancienneté (0-30 jours, 31-60 jours, 61-90 jours, >90 jours)

© 2026 Oussama Ghorbel

• Top 10 clients débiteurs

• Taux de recouvrement (montant récupéré / montant total dû)

• Efficacité des relances (% factures payées après 1ère, 2ème, 3ème relance)

6. **Alertes et escalades** :

• Alerte direction si impayés totaux > seuil critique

• Escalade automatique si facture > montant X reste impayée > 60 jours

• Notification hebdomadaire au responsable financier avec résumé

Critères de réussite :

• Les relances partent automatiquement selon la stratégie définie

• Le dashboard se met à jour en temps réel

• La synchronisation avec la comptabilité est bidirectionnelle (quand un paiement arrive, la relance s'arrête)

• Les escalades fonctionnent correctement

• L'historique complet de chaque facture est traçable

Extension : Ajoutez une couche d'IA. Utilisez GPT ou Claude pour personnaliser le ton des emails de relance selon l'historique relationnel avec le client (ton plus ferme pour les mauvais payeurs récurrents, plus compréhensif pour les clients habituellement fiables ayant un retard exceptionnel).

Ces trois exercices couvrent les trois grands domaines d'automatisation : acquisition (ventes), opérations internes (RH), et gestion financière. Maîtriser ces patterns vous donnera les fondations pour automatiser pratiquement n'importe quel processus métier dans votre organisation.

© 2026 Oussama Ghorbel

Chapitre 19 – RPA et automatisation desktop : quand l'API n'existe pas

Depuis le début de cet ouvrage, nous avons célébré la magie des API, des webhooks et des connecteurs natifs. Nous avons vu comment Make, Zapier ou n8n permettent de faire dialoguer des applications en quelques clics, en échangeant des données structurées via des portes de service invisibles et ultra-rapides. Mais il est temps de confronter cette vision idéalisée à la réalité du terrain. Le monde de l'entreprise n'est pas peuplé uniquement de SaaS modernes dotés de documentations Swagger impeccables. Il est aussi rempli de logiciels vieux de vingt ans, d'ERP on-premise, de portails gouvernementaux capricieux et d'applications de bureau isolées.

Face à ces systèmes, les API n'existent pas. Ou pire, elles existent mais sont réservées à une équipe de développeurs internes qui n'a pas le temps de vous aider. C'est ici qu'entre en scène le RPA, ou *Robotic Process Automation*. Le RPA est l'art de créer des « travailleurs numériques » capables d'interagir avec l'interface utilisateur d'un logiciel exactement comme le ferait un être humain : en déplaçant la souris, en cliquant sur des boutons, en lisant l'écran et en tapant au clavier. Dans ce chapitre, nous allons explorer cette discipline fascinante, ses outils, ses limites, et la manière de l'intégrer dans votre stack no-code et low-code pour combler les lacines de l'automatisation traditionnelle.

19.1. RPA vs API automation : complémentaires, pas concurrents

Pour bien comprendre le RPA, il faut d'abord le distinguer de l'automatisation par API. L'analogie la plus simple pour illustrer cela est celle du restaurant. Utiliser une API, c'est comme passer commande directement en cuisine via un système de ticket numérique : c'est instantané, structuré, et peu importe la décoration de la salle ou la disposition des tables. Le RPA, en revanche, c'est comme engager une personne pour s'asseoir à table, lire le menu physique, appeler le serveur, lui parler, et vous rapporter le plat. C'est plus lent, plus sujet aux erreurs (si le serveur change le menu ou si la table est déplacée), mais cela

© 2026 Oussama Ghorbel

fonctionne partout, même là où il n'y a pas de système de ticket numérique.

L'automatisation par API travaille sur la couche des données (le *backend*). Elle est robuste, rapide, et peut traiter des milliers de requêtes par seconde. Le RPA travaille sur la couche de présentation (le *frontend* ou l'interface utilisateur). Il est intrinsèquement plus lent, car il doit respecter le temps de chargement des pages et les délais de rendu graphique.

Pourtant, ces deux approches ne s'opposent pas ; elles se complètent. Dans une architecture d'entreprise mature, on applique la règle d'or suivante : **toujours privilégier l'API si elle existe et est accessible**. Le RPA doit être considéré comme un outil de dernier recours, ou comme un pont temporaire (*bridge*) en attendant qu'une API soit développée.

On distingue généralement deux modes d'opération en RPA :

1. **Unattended (Non assisté) :** Le robot s'exécute en arrière-plan, sur une machine virtuelle ou un serveur, sans intervention humaine. Il est déclenché par un planificateur (ex: tous les jours à 3h du matin) ou par un événement système. C'est l'idéal pour les traitements de masse nocturnes.

2. **Attended (Assisté) :** Le robot s'exécute sur le poste de travail de l'utilisateur, en parallèle de ses actions. Il agit comme un copilote. Par exemple, lorsqu'un agent du service client ouvre une fiche client dans un vieux CRM, le robot assisté détecte l'action et va automatiquement chercher les dernières factures dans un autre logiciel pour les afficher sur un écran secondaire.

Le RPA est la solution ultime pour ce que les Anglo-Saxons appellent le *swivel-chair integration* (l'intégration de la chaise tournante) : cette tâche épuisante où un employé doit copier des données d'un écran, tourner sa chaise (ou changer de fenêtre), et les coller dans un autre système.

19.2. Power Automate Desktop : le RPA accessible de Microsoft

© 2026 Oussama Ghorbel

Pendant longtemps, le RPA était une chasse gardée des grandes entreprises, nécessitant des licences coûteuses et des équipes de développement dédiées. Microsoft a radicalement changé la donne en acquérant la société WinAutomation en 2019 et en intégrant sa technologie directement dans Windows 10 et 11 sous le nom de **Power Automate Desktop (PAD)**.

Power Automate Desktop est une application native qui permet de créer des flux de travail automatisant n'importe quelle application de bureau ou web. Son interface repose sur un canevas visuel où l'on assemble des blocs d'actions (plus de 400 actions natives sont disponibles).

L'un des atouts majeurs de PAD est son **enregistreur d'actions** (Recorder). Vous lancez l'enregistrement, vous effectuez votre tâche manuellement dans Excel, sur Chrome ou dans votre logiciel de comptabilité, et PAD traduit vos clics et frappes au clavier en une séquence d'actions logiques. Bien sûr, l'enregistrement brut nécessite presque toujours un nettoyage et une optimisation (pour ajouter des boucles, des conditions ou gérer les erreurs), mais il accélère considérablement la phase de découverte du processus.

L'intégration dans l'écosystème Power Platform

La véritable force de PAD ne réside pas seulement dans ses capacités locales, mais dans son pont avec le cloud. Un flux desktop peut être déclenché par un flux cloud (via Power Automate classique). Imaginez le scénario suivant :

1. Un client remplit un formulaire sur votre site web (déclencheur cloud).
2. Les données sont stockées dans une base SharePoint ou Dataverse.
3. Power Automate Cloud déclenche un flux Power Automate Desktop sur une machine virtuelle de l'entreprise.
4. Le robot desktop ouvre le vieil ERP local (ex: Sage ou Cegid), se connecte, et saisit la commande client.
5. Le robot desktop capture le numéro de commande généré par l'ERP et le renvoie au flux cloud, qui envoie un email de

© 2026 Oussama Ghorbel

confirmation au client.

Avantages et limites de PAD

L'avantage principal est son coût : PAD est gratuit pour un usage personnel (automatiser ses propres tâches sur son propre poste). Pour un usage entreprise (partage de flux, déclenchement cloud, files d'attente), il s'intègre dans les licences Microsoft 365 et Power Platform existantes. De plus, ses connecteurs natifs pour Excel, Outlook et les applications Windows sont d'une fiabilité redoutable.

Sa limite principale est son attache à l'environnement Windows. Contrairement à des solutions basées sur le cloud ou des extensions de navigateur, PAD nécessite une machine Windows allumée, avec une session active (ou au moins déverrouillée, selon les actions), ce qui complexifie le déploiement en mode *unattended* sans infrastructure serveur adaptée.

19.3. UiPath Community, Automation Anywhere, Axiom.ai

Si Power Automate Desktop a démocratisé l'accès au RPA pour les utilisateurs de Windows, le marché enterprise et les approches alternatives offrent des outils aux philosophies très différentes.

UiPath : le géant de l'industrie

UiPath est le leader historique et incontesté du marché RPA mondial. Leur offre *Community Edition* est gratuite et donne accès à la quasi-totalité des fonctionnalités de leur studio de développement professionnel. UiPath propose deux environnements : *StudioX*, pensé pour les citoyens développeurs (proche de la philosophie no-code), et *Studio*, conçu pour les développeurs RPA professionnels, permettant d'écrire du code VB.NET ou C# en parallèle des blocs visuels.

La grande force d'UiPath réside dans son *REFramework* (Robotic Enterprise Framework). C'est un modèle de conception (template) qui impose une structure rigoureuse à vos robots : initialisation, lecture des transactions, traitement, et gestion des exceptions. C'est lourd pour un petit script, mais indispensable pour déployer des dizaines de robots en production sans qu'ils ne plantent au moindre imprévu.

Automation Anywhere : l'approche Cloud-Native

© 2026 Oussama Ghorbel

Avec sa plateforme A360, Automation Anywhere a pris le pari du *Cloud-Native RPA*. Contrairement aux outils traditionnels où le studio de développement est une application desktop lourde, le *Bot Builder* d'Automation Anywhere s'exécute directement dans le navigateur web. Cela facilite la collaboration en équipe et la gestion centralisée des bots. Leur approche est très orientée vers l'intégration de l'IA (IQ Bot) pour la lecture de documents non structurés (factures PDF, emails) avant de les injecter dans des systèmes legacy.

Axiom.ai : le RPA léger par extension de navigateur

Parfois, vous n'avez pas besoin de contrôler l'ensemble du bureau Windows. Votre "vieux logiciel" est en fait un portail web legacy, mal codé, sans API, mais accessible via Chrome ou Edge. C'est là qu'intervient Axiom.ai.

Axiom n'est pas une application desktop, mais une extension de navigateur. Son interface de *builder* visuel permet de créer des scripts qui manipulent le DOM (la structure de la page web) sans écrire une ligne de JavaScript ou de Selenium. Vous pouvez lui demander de "cliquer sur l'élément qui contient le texte 'Factures'", de "télécharger le CSV", ou de "remplir le tableau HTML ligne par ligne". C'est l'outil parfait pour le web scraping avancé et l'automatisation de portails web administratifs, avec l'avantage de pouvoir s'exécuter en arrière-plan (headless) sans bloquer votre écran.

19.4. Cas d'usage : ERP legacy, portail admin, téléchargements

Pour ancrer ces concepts, explorons trois cas d'usage classiques où le RPA brille et sauve des centaines d'heures de travail manuel.

1. L'ERP Legacy (SAP GUI, AS400, vieux SaaS)

De nombreuses entreprises industrielles ou logistiques tournent encore sur des interfaces AS400 (écran vert sur fond noir) ou sur la version desktop de SAP GUI. Ces systèmes sont extrêmement puissants mais hermétiques aux intégrations modernes.

Le scénario : Chaque vendredi, le contrôleur de gestion doit extraire la liste des bons de commande non réceptionnés, la croiser avec un

© 2026 Oussama Ghorbel

fichier Excel de prévisions, puis créer des alertes de retard dans le système.

La solution RPA : Le robot se connecte à SAP GUI, navigue dans les menus arborescents (transaction ME2N), exporte la liste en CSV, ouvre Excel, exécute les macros de croisement, puis retourne dans SAP pour flaguer les commandes problématiques. Le robot ne "comprend" pas ce qu'est un bon de commande ; il sait juste que le champ "Numéro" est à la coordonnée X,Y ou possède l'identifiant UIA `txtField_01`.

2. Les portails administratifs et gouvernementaux

Les interactions avec les URSSAF, les impôts, ou les portails de fournisseurs d'énergie sont souvent un cauchemar d'automatisation. Ces sites sont conçus pour des humains, avec des CAPTCHAs, des sessions qui expirent après 10 minutes d'inactivité, et des boutons "Suivant" qui chargent la page en 4 secondes de manière aléatoire.

Le scénario : Récupérer les avis d'imposition ou les attestations de TVA chaque mois pour les archiver dans le système de Gestion Électronique des Documents (GED) de l'entreprise.

La solution RPA : Un robot *unattended* se connecte avec les identifiants de l'entreprise, gère les délais d'attente (wait for element), télécharge les PDF, les renomme selon une nomenclature stricte, et les pousse via API (cette fois-ci !) vers Dropbox ou SharePoint.

3. Téléchargements et Web Scraping sur portails fournisseurs

Le scénario : Vous travaillez avec 50 fournisseurs différents. Chacun possède son propre portail B2B où vous devez vous connecter pour télécharger les factures du mois. Aucun ne propose d'envoi automatique par email ou d'API EDI.

La solution RPA : Plutôt que de payer un employé pour faire le tour des 50 portails, on crée un "robot collecteur". Il lit une base de données (ex: Airtable) contenant les URLs de connexion, les identifiants (stockés de manière sécurisée), et les sélecteurs spécifiques à chaque portail. Le robot enchaîne les connexions, télécharge les fichiers, et met à jour le statut dans Airtable.

© 2026 Oussama Ghorbel

19.5. Fragilité du RPA : comment la compenser (sélecteurs robustes, monitoring)

Il est essentiel de garder cette métaphore en tête : **le RPA est un verre de cristal**. C'est beau, c'est utile, mais si vous le posez sur une table bancale, il finira par se briser. La fragilité inhérente au RPA vient de sa dépendance à l'interface utilisateur. Si un développeur web change l'ID d'un bouton, si Windows met à jour l'échelle d'affichage de 100% à 125%, ou si le logiciel legacy affiche une pop-up inattendue "Votre mot de passe expire dans 3 jours", le robot échouera.

Pour compenser cette fragilité, le développeur RPA doit maîtriser trois concepts fondamentaux :

1. L'art des sélecteurs robustes

Un sélecteur est la chaîne de caractères qui permet au robot d'identifier un élément à l'écran (comme le sélecteur CSS pour un développeur web, mais appliqué à tout l'OS).

Un mauvais sélecteur : `id="btn_submit_84729"` (si l'ID est généré dynamiquement à chaque chargement de page, le robot plantera le lendemain).

Un bon sélecteur : Chercher un bouton, dont le texte est "Valider", situé à l'intérieur de la fenêtre nommée "Création de facture".

Dans UiPath ou PAD, on utilise des **wildcards** (jokers) comme `*` ou `?` pour absorber les variations. On utilise aussi l'**ancrage** (anchoring) : au lieu de chercher le champ "Montant" par sa position absolue, on demande au robot de trouver le texte "Montant TTC" et de cliquer sur le champ vide situé exactement à sa droite.

2. La gestion des environnements virtuels (Citrix / VMware)

Dans les grandes entreprises, les utilisateurs n'ont pas les logiciels installés sur leur machine. Ils se connectent à un bureau virtuel (Citrix, Remote Desktop). Pour le robot, cet environnement est une "boîte noire" : il ne voit pas les boutons ou les champs de texte, il ne voit qu'une **image** (un flux vidéo de pixels).

Dans ce cas, les sélecteurs UIA ne fonctionnent plus. Il faut recourir à la **reconnaissance d'image** (Image Automation) et à l'**OCR**

© 2026 Oussama Ghorbel

(Reconnaissance Optique de Caractères). Le robot va chercher l'image du bouton "Enregistrer" à l'écran et cliquer au centre de ces pixels. C'est extrêmement lent et fragile (une simple modification du thème de couleur de Windows peut casser la reconnaissance), mais c'est parfois la seule issue.

3. Architecture défensive : Try/Catch et Monitoring

Un robot ne doit jamais être conçu en supposant que tout se passera bien. Il faut structurer le flux avec des blocs *Try/Catch* (Essayer/Attraper).

Exemple : Le robot essaie de cliquer sur "Télécharger". Si l'élément n'apparaît pas dans les 15 secondes (Timeout), il attrape l'exception, prend une capture d'écran de l'état actuel de la fenêtre, ferme proprement l'application, envoie la capture d'écran et le log d'erreur sur un canal Slack ou Teams dédié au support, et passe à la transaction suivante.

Le monitoring est crucial. Un robot qui échoue en silence est pire qu'un processus manuel, car personne ne s'en rend compte avant que le désastre ne soit consommé (ex: 500 factures non payées).

19.6. Éthique et conformité : ne pas violer les CGU des sites

L'automatisation desktop et le web scraping soulèvent des questions légales et éthiques qu'il est impossible d'ignorer. Si vous automatisez des logiciels internes à votre entreprise (votre propre ERP, vos fichiers Excel locaux), vous êtes chez vous. Vous pouvez casser vos propres fenêtres, il n'y a aucun problème légal.

La situation change radicalement dès que votre robot interagit avec des **portails externes, des sites publics ou des plateformes tierces**.

Le respect des CGU (Conditions Générales d'Utilisation)

La quasi-totalité des sites web modernes incluent dans leurs CGU une clause interdisant l'utilisation de "robots, spiders, scrapers ou autres outils automatisés pour accéder au service". Violer ces CGU peut entraîner la suspension immédiate de votre compte, la rupture d'un contrat commercial, voire des poursuites judiciaires pour accès non autorisé ou parasitisme économique. Avant de lancer un robot sur le portail d'un fournisseur ou d'un concurrent, lisez les CGU et, si

© 2026 Oussama Ghorbel

nécessaire, demandez une autorisation écrite au service juridique ou commercial de la cible.

La gestion sécurisée des identifiants

Une erreur classique des débutants en RPA est de stocker les mots de passe des portails cibles en clair dans les variables du flux ou dans un fichier Excel. C'est une faille de sécurité critique. Les outils RPA professionnels s'intègrent avec les gestionnaires d'identifiants (Windows Credential Manager, CyberArk, Azure Key Vault, ou le coffre-fort natif de Make/UiPath). Le robot demande l'identifiant au coffre-fort au moment de l'exécution, l'utilise en mémoire vive, et ne le stocke jamais sur le disque dur.

Le respect des serveurs cibles et des CAPTCHAs

Un humain clique sur "Suivant" toutes les 5 secondes. Un robot peut le faire 50 fois par seconde. Si vous ne configurez pas de délais aléatoires (*human-like delays*) dans votre robot, vous risquez de lancer une attaque par déni de service (DDoS) involontaire sur le serveur de votre fournisseur, ce qui fera bannir votre adresse IP.

Quant aux CAPTCHAs, ils ont été inventés spécifiquement pour vous arrêter. Essayer de les contourner avec des services de résolution automatisée (où des humains sous-payés résolvent les CAPTCHAs en arrière-plan) se situe dans une zone grise très sombre, tant sur le plan éthique que légal. Si un portail externe critique impose un CAPTCHA à chaque connexion, c'est le signe qu'il faut abandonner le RPA et exiger une intégration API ou SFTP officielle.

Comme le résume un adage bien connu dans l'industrie de la donnée : *"Ce n'est pas parce que vous pouvez scraper une donnée que vous avez le droit de le faire."* Le RPA est un super-pouvoir, et comme tout super-pouvoir, il exige une grande responsabilité.

Exercices pratiques

Pour maîtriser les concepts de ce chapitre, je vous invite à réaliser les trois exercices suivants. Ils sont conçus pour être effectués avec **Power Automate Desktop** (gratuit sur Windows) ou **Axiom.ai** (pour les parties

© 2026 Oussama Ghorbel

web).

Exercice 1 : Automatiser la récupération quotidienne de rapports depuis un portail

Objectif : Créer un robot capable de se connecter à un portail web (utilisez un site de test ou le portail de votre banque/fournisseur d'énergie), de naviguer vers la section "Documents", et de télécharger le dernier relevé disponible.

Étapes clés :

1. Ouvrez Power Automate Desktop et créez un nouveau flux.
2. Utilisez l'action "Lancer le navigateur" (Chrome ou Edge) et naviguez vers l'URL.
3. Utilisez l'enregistreur web pour capturer la saisie de vos identifiants et le clic sur le bouton de connexion.
4. Ajoutez une action "Attendre que le contenu de la page web contienne" (pour gérer le temps de chargement après la connexion).
5. Identifiez le lien du dernier document PDF et utilisez l'action "Télécharger le fichier" ou "Cliquer sur le lien".
6. *Bonus :* Ajoutez une action pour renommer le fichier téléchargé avec la date du jour et le déplacer dans un dossier spécifique de votre disque dur.

Exercice 2 : Créer un robot qui remplit un formulaire legacy depuis Excel

Objectif : Simuler une saisie de données en masse. Vous allez utiliser un formulaire web de test (comme `demoqa.com` ou un formulaire interne de votre entreprise) et un fichier Excel contenant 10 lignes de données (Nom, Prénom, Email, Département).

Étapes clés :

1. Préparez votre fichier Excel avec les en-têtes et les 10 lignes de données.
2. Dans PAD, utilisez l'action "Lire à partir d'une feuille de calcul Excel" pour charger les données dans une variable de type Table.

© 2026 Oussama Ghorbel

3. Créez une boucle "Pour chaque" (For Each) qui itère sur chaque ligne de votre table.

4. À l'intérieur de la boucle, utilisez des actions "Remplir le champ de texte sur la page web" pour mapper les colonnes Excel aux champs du formulaire.

5. Ajoutez un clic sur le bouton "Soumettre".

6. **Point critique :** Ajoutez une action "Attendre" ou "Attendre que l'élément de la page web contienne" pour vous assurer que le message de confirmation "Succès" est apparu avant de passer à la ligne suivante et de recharger le formulaire. Sans cela, votre robot ira plus vite que le serveur et les saisies se chevaucheront.

Exercice 3 : Instrumenter un robot avec logs + alertes en cas d'échec

Objectif : Transformer un robot fragile en processus robuste et supervisable.

Étapes clés :

1. Reprenez le flux de l'Exercice 1 ou 2.

2. Encadrez les actions critiques (connexion, téléchargement, soumission) dans un bloc **On block error** (l'équivalent du Try/Catch dans PAD).

3. Dans la section "En cas d'erreur" (Catch), ajoutez les actions suivantes :

• "Prendre une capture d'écran" et l'enregistrer dans un dossier "Logs_Erreurs".

• "Écrire dans un fichier journal" (un fichier .txt ou .csv) l'heure de l'erreur, le message d'exception et le nom du fichier qui posait problème.

• "Envoyer un e-mail" (via Outlook ou Gmail) à votre propre adresse avec la capture d'écran en pièce jointe et un objet explicite : "[ALERTE RPA] Échec du robot de téléchargement".

4. Testez votre gestion d'erreur en forçant un échec (par exemple, en changeant l'URL de connexion pour une URL invalide, ou en débranchant votre câble réseau au moment du téléchargement). Vérifiez que le robot ne plante pas brutalement, mais qu'il s'arrête

© 2026 Oussama Ghorbel

proprement en vous alertant.

© 2026 Oussama Ghorbel

Chapitre 20 – Webhooks, APIs et code light : l'extension inévitable

Il existe un paradoxe fondamental dans l'appellation « no-code ». Ce terme, bien qu'excellent pour le marketing et pour rassurer les débutants, est techniquement inexact. Ce que nous faisons, en réalité, c'est du « visual-first » ou du « low-code assumé ». Tôt ou tard, dans votre parcours de maker, vous atteindrez la limite native de votre outil visuel. Vous voudrez connecter un service obscur qui ne possède pas de module natif sur Zapier ou Make. Vous devrez manipuler une structure de données imbriquée que votre base Airtable refuse d'ingérer. Vous ferez face à un mur.

Ce mur n'est pas une fatalité, c'est une invitation. L'extension inévitable du no-code, c'est la compréhension des protocoles qui régissent le web moderne. Le code n'est pas l'ennemi du no-code ; c'est son filet de sécurité. Savoir lire une documentation d'API, comprendre la mécanique d'un webhook, et être capable d'écrire dix lignes de JavaScript ou de Python pour débloquer une situation, c'est ce qui sépare le maker amateur de l'architecte no-code professionnel. Dans ce chapitre, nous allons franchir la frontière. Nous n'allons pas apprendre à développer des applications entières en lignes de commande, mais nous allons acquérir le « code light » : ce strict minimum vital qui vous donnera le pouvoir absolu sur vos automatisations.

20.1. Qu'est-ce qu'une API REST ? Verbes, payloads, statuts HTTP

Une API (Application Programming Interface) est un contrat de service entre deux applications. L'architecture REST (Representational State Transfer) est le standard qui domine le web aujourd'hui. Pour la comprendre, utilisez l'analogie du restaurant. Vous (le client) êtes assis à une table. La cuisine (le serveur/base de données) possède les ingrédients et sait préparer les plats. Vous ne pouvez pas entrer dans la cuisine. Vous devez passer par le serveur (l'API), qui prend votre commande, la transmet à la cuisine, et vous rapporte le plat.

© 2026 Oussama Ghorbel

Pour passer commande, vous devez utiliser un vocabulaire précis : les **verbes HTTP**.

• **GET** : « Donne-moi cette information. » (Ex: Récupérer la liste de mes clients). C'est une action de lecture, qui ne modifie rien.

• **POST** : « Prends ces données et crée quelque chose de nouveau. » (Ex: Créer une nouvelle facture).

• **PUT** : « Remplace entièrement cette ressource existante par ce que je te donne. »

• **PATCH** : « Modifie juste quelques champs de cette ressource existante. » (Ex: Changer le statut d'une commande de 'en cours' à 'livrée').

• **DELETE** : « Supprime cette ressource. »

Votre commande (la **requête**) est composée de plusieurs éléments. L'**URL** est l'adresse du restaurant et de la table (ex: `https://api.monservice.com/v1/invoices/123`). Les **Query Parameters** sont des options ajoutées à l'URL après un point d'interrogation (ex: `?status=paid&limit=50`). Les **Headers** (en-têtes) sont des métadonnées invisibles mais cruciales : c'est là que vous glissez votre clé API (`Authorization: Bearer VOTRE_CLE`) pour prouver votre identité, et que vous précisez le format de votre message (`Content-Type: application/json`). Enfin, le **Body** (corps) contient les données brutes que vous envoyez, généralement formatées en JSON, lors d'un POST ou d'un PATCH.

Une fois la commande traitée, le serveur vous renvoie une réponse accompagnée d'un **code de statut HTTP**. En tant que maker, vous devez connaître ces codes par cœur pour debugger vos scénarios :

• **200 OK** : Tout s'est bien passé (généralement pour un GET ou un PATCH).

• **201 Created** : La ressource a été créée avec succès (généralement pour un POST).

• **204 No Content** : Succès, mais aucune donnée n'est renvoyée (souvent pour un DELETE).

• **400 Bad Request** : C'est votre faute. Le format de votre JSON est invalide, ou il manque un champ obligatoire.

© 2026 Oussama Ghorbel

• **401 Unauthorized** : Problème d'authentification. Votre clé API est fausse, expirée, ou mal placée dans les headers.

• **403 Forbidden** : Vous êtes authentifié, mais vous n'avez pas les droits pour accéder à cette ressource spécifique.

• **404 Not Found** : L'URL est correcte, mais la ressource demandée (ex: l'ID 123) n'existe pas.

• **429 Too Many Requests** : Vous avez dépassé le quota de l'API (Rate Limiting). Il faut ralentir ou implémenter un système de file d'attente.

• **500 Internal Server Error** : C'est la faute du serveur distant. Leur code a planté. Réessayez plus tard.

Quand un module natif dans Make ou Zapier échoue, il masque souvent ces détails derrière un message d'erreur générique. C'est pourquoi comprendre la mécanique REST brute est indispensable.

20.2. Webhooks entrants/sortants : le système nerveux du no-code

Si les API REST fonctionnent sur le mode du « polling » (vous appelez le serveur toutes les 15 minutes pour demander « Y a-t-il de nouvelles commandes ? »), les webhooks fonctionnent sur le mode du « pushing » (le serveur vous envoie un SMS dès qu'une commande arrive). Le polling est coûteux en opérations et lent. Le webhook est instantané et économique. C'est le système nerveux de l'automatisation moderne.

Un **webhook sortant** (outgoing) est une URL unique générée par votre outil d'automatisation (comme le module « Custom Webhook » dans Make). Vous copiez cette URL et la collez dans les paramètres de l'application source (ex: Stripe, Typeform, Shopify). Vous dites à l'application source : « Dès qu'un événement X se produit, envoie les données à cette adresse ».

Un **webhook entrant** (incoming) est l'inverse. Votre application no-code (ex: une app Bubble) génère une URL de webhook pour notifier un système externe qu'une action s'est produite chez vous.

La mécanique sous-jacente est simple : un webhook n'est rien d'autre qu'une requête HTTP POST automatisée. Quand l'événement se

© 2026 Oussama Ghorbel

déclenche, le service source construit un payload JSON et l'envoie à votre URL. Votre scénario Make reçoit ce payload et s'exécute.

Cependant, cette simplicité cache des pièges. Le premier est la **fiabilité**. Si votre scénario Make est en pause, ou si Airtable subit une panne au moment exact où le webhook est reçu, la donnée est perdue. Contrairement à une file d'attente (message broker), un webhook standard est une livraison « best effort ». Pour les données critiques (comme un paiement Stripe), vous devez configurer des systèmes de retry ou utiliser des outils tampons comme Hookdeck ou Svix.

Le second piège est la **sécurité**. N'importe qui peut trouver l'URL de votre webhook et lui envoyer de fausses données. C'est pourquoi les services sérieux signent leurs webhooks. Ils ajoutent un header (ex: `X-Signature`) contenant un hash cryptographique (HMAC) généré à partir du payload et d'une clé secrète que vous seul connaissez. Dans votre outil no-code, la première étape de votre scénario doit toujours être de recalculer ce hash et de le comparer à celui reçu. S'ils ne correspondent pas, vous bloquez l'exécution. Ne laissez jamais un webhook financier ouvert aux quatre vents.

20.3. Postman/Insomnia pour tester avant d'intégrer

L'erreur classique du maker débutant est de tester une nouvelle API directement dans son scénario Make ou Zapier. Quand l'erreur survient, le diagnostic devient un cauchemar : est-ce mon scénario qui est mal configuré ? Est-ce le module Make qui a un bug ? Est-ce l'API distante qui rejette ma requête ?

Pour isoler les variables, il faut tester l'API dans un environnement neutre. C'est le rôle de clients API comme **Postman** ou **Insomnia**. Ce sont des logiciels dédiés à la formulation et à l'inspection de requêtes HTTP.

Voici ma règle d'or : **si la requête ne fonctionne pas dans Postman, elle ne fonctionnera jamais dans Make.**

La plupart des documentations d'API modernes fournissent des exemples de requêtes au format `cURL` (une commande en ligne de commande). Postman et Insomnia permettent d'importer directement une commande cURL en un clic. Cela remplit automatiquement l'URL, les

© 2026 Oussama Ghorbel

headers, les paramètres et le body.

Une fois la requête exécutée dans Postman, vous obtenez la réponse brute. Vous pouvez inspecter le code de statut, lire le message d'erreur exact renvoyé par le serveur (souvent beaucoup plus détaillé que ce que Make vous affiche), et observer la structure exacte du JSON de réponse. C'est uniquement une fois que vous avez obtenu un `200 OK` dans Postman que vous devez transposer cette configuration dans votre orchestrateur no-code, en recopiant méticuleusement les headers et le body. Postman agit comme votre laboratoire de test ; Make est votre usine de production. Ne mélangez jamais les deux.

20.4. Lire du JSON : la compétence minimale du maker

Le JSON (JavaScript Object Notation) est la lingua franca du web. Si vous ne savez pas lire et naviguer dans une structure JSON, vous serez constamment bloqué lors du mapping de vos données.

Le JSON repose sur deux structures fondamentales :

1. **L'objet** : Une collection de paires clé/valeur, entourée d'accolades `{}`. C'est l'équivalent d'une ligne dans un tableur, ou d'un dictionnaire.

2. **Le tableau (Array)** : Une liste ordonnée de valeurs, entourée de crochets `[]`. C'est l'équivalent d'une colonne ou d'une liste d'éléments.

La complexité survient quand ces structures s'imbriquent. Une API renvoie rarement des données plates. Elle renvoie des objets contenant des tableaux, qui contiennent d'autres objets.

Pour extraire le nom du premier client dans Make ou Zapier, vous devez utiliser la **notation par points** (dot notation) et l'**indexation des tableaux**. Les tableaux commencent toujours à l'index `0`.

Le chemin (path) pour obtenir "Entreprise A" sera donc : `data.invoices[0].client.name`.

Apprenez à repérer visuellement les crochets `[]`. Si vous voyez des crochets, cela signifie que vous avez affaire à une liste. Vous ne pourrez pas mapper directement cette liste vers un champ texte unique dans

© 2026 Oussama Ghorbel

Airtable ; vous devrez utiliser un itérateur (Array Aggregator / Iterator dans Make) pour traiter chaque élément de la liste un par un. Maîtriser la lecture du JSON, c'est maîtriser la topographie de vos données.

20.5. JavaScript dans Make, Bubble, Retool : 10 snippets indispensables

Les outils no-code intègrent de plus en plus de blocs permettant d'exécuter du JavaScript personnalisé (le module "Tools" ou "Set Multiple Variables" avec des fonctions dans Make, les workflows "Run JavaScript" dans Bubble, les JS Queries dans Retool). Voici 10 extraits de code (snippets) indispensables pour combler les lacunes des outils visuels.

1. Formater une date ISO en format français lisible

Les API renvoient des dates au format ISO 8601 (`2026-10-11T14:30:00Z`). Pour les afficher proprement :

2. Extraire un tableau d'IDs à partir d'un tableau d'objets

Utile quand une API renvoie une liste d'objets complets, mais que vous avez juste besoin des IDs pour faire une requête ultérieure.

3. Filtrer un tableau pour ne garder que les éléments actifs

4. Aplatir un objet JSON imbriqué (Flattening)

Parfois, Airtable ou Notion refusent les objets imbriqués. Ce code transforme `{ user: { name: 'A' } }` en `{ 'user.name': 'A' }`.

5. Générer un identifiant unique (UUID)

Pratique pour créer des clés primaires ou des noms de fichiers uniques avant de les envoyer à une base de données.

6. Encoder / Décoder en Base64

Indispensable pour certains headers d'authentification (Basic Auth) ou pour manipuler des images.

7. Extraire une donnée avec une expression régulière (Regex)

Par exemple, extraire le premier code postal français (5 chiffres) trouvé dans une longue chaîne de texte non structurée.

© 2026 Oussama Ghorbel

8. Découper un tableau en petits morceaux (Chunking)

Essentiel pour respecter les limites de taux (rate limits) des API. Si vous avez 500 éléments à envoyer et que l'API limite à 100 par minute, vous créez des paquets de 100.

9. Sommer les valeurs d'un champ spécifique dans un tableau

Pour calculer le chiffre d'affaires total à partir d'un tableau de commandes.

10. Supprimer les doublons d'un tableau

20.6. Python light : pandas pour nettoyer, requests pour appeler

Si JavaScript est le roi du navigateur et des fonctions serverless rapides, Python reste le souverain incontesté de la manipulation de données lourdes. Quand vous devez traiter un export CSV de 100 000 lignes, nettoyer des données corrompues, ou effectuer des calculs statistiques avant de les injecter dans votre application no-code, les blocs JS de Make montreront leurs limites (timeouts, limites de mémoire). C'est là qu'intervient un petit script Python.

La bibliothèque **`requests`** est l'équivalent de Postman, mais en code. Elle rend les appels API d'une simplicité enfantine :

Mais la vraie magie opère avec **`pandas`**. Pandas est une bibliothèque qui transforme vos données en "DataFrames" (des tableaux ultra-performants). Supposons qu'une API vous renvoie un JSON complexe et mal structuré, et que vous deviez l'aplatir, filtrer les lignes où le chiffre d'affaires est nul, et formater les dates avant de les importer dans Airtable via l'API Airtable.

Ce type de script s'exécute en quelques secondes sur une machine modeste et vous évite de consommer des dizaines de milliers d'opérations dans Zapier ou Make.

20.7. Cloud Functions (Cloudflare Workers, Google Cloud Functions) comme glue

Il arrive un moment où l'orchestrateur no-code devient un goulot d'étranglement financier ou technique. Imaginez que vous receviez 50

© 2026 Oussama Ghorbel

000 webhooks par jour d'un capteur IoT. Chaque webhook doit subir une petite transformation (arrondir une décimale, ajouter un timestamp) avant d'être envoyé à votre base de données. Faire transiter ces 50 000 événements par Make vous coûtera une fortune en opérations. C'est le cas d'usage parfait pour les **Cloud Functions**, le "glue" (la colle) de l'architecture moderne.

Les Cloud Functions sont des micro-serveurs qui s'allument à la demande, exécutent quelques lignes de code, et s'éteignent. Vous ne payez que pour le temps d'exécution à la milliseconde près.

Cloudflare Workers est mon outil favori pour les transformations rapides en JavaScript. Le code s'exécute à la "edge" (à la périphérie du réseau, au plus près de l'utilisateur), ce qui le rend incroyablement rapide. Voici à quoi ressemble un Worker qui intercepte un webhook, modifie le payload, et le transfère :

Google Cloud Functions (ou AWS Lambda) est préférable si vous avez besoin de l'écosystème Python, d'accéder à des bibliothèques lourdes (comme Pandas ou des modèles d'IA légers), ou d'interagir avec les services Google (BigQuery, Google Drive).

L'intégration de ces fonctions dans votre stack no-code est transparente. Pour l'outil émetteur, votre Cloud Function n'est qu'une URL de webhook classique. Pour l'outil récepteur, c'est une source de données comme une autre. Vous avez créé un micro-service sur mesure, scalable à l'infini, pour une fraction de centime par appel. C'est la preuve que le code light n'est pas là pour remplacer le no-code, mais pour le propulser dans la cour des grandes architectures logicielles.

Exercices pratiques

Pour ancrer ces concepts, je vous propose trois exercices progressifs. Ne passez pas à la suite du livre avant de les avoir réalisés.

Exercice 1 : Appeler une API publique (OpenWeather) depuis Make et afficher le résultat

1. Créez un compte gratuit sur OpenWeatherMap et récupérez votre clé API.

© 2026 Oussama Ghorbel

2. Dans Make, créez un scénario avec un déclencheur manuel (ou une Webhook).

3. Ajoutez le module "HTTP" (et non un module OpenWeather natif, le but est de faire l'appel brut).

4. Configurez une requête **GET** vers `https://api.openweathermap.org/data/2.5/weather`.

5. Ajoutez les *Query Parameters* : `q` (votre ville, ex: "Paris,FR"), `appid` (votre clé API), et `units` ("metric").

6. Exécutez le scénario. Inspectez le JSON de sortie (Bundle).

7. Ajoutez un module "Tools > Set Multiple Variables" pour extraire et formater la température (`main.temp`) et la description du ciel (`weather[0].description`) en une phrase complète : "À Paris, il fait X degrés avec un ciel Y".

Exercice 2 : Écrire un webhook qui reçoit un Stripe event et met à jour Airtable

1. Dans Make, créez un scénario débutant par "Custom Webhook". Copiez l'URL générée.

2. Créez une base Airtable avec une table "Paiements" (Colonnes : ID Stripe, Montant, Statut).

3. Dans le Stripe Dashboard (en mode test), allez dans la section Webhooks et ajoutez votre URL Make. Abonnez-vous à l'événement `payment_intent.succeeded`.

4. Simulez un paiement réussi via l'outil de test Stripe ou Postman.

5. Dans Make, mappez les données du webhook (`data.object.id` et `data.object.amount_received`). *Attention : Stripe envoie les montants en centimes, utilisez une formule Make pour diviser par 100.*

6. Ajoutez le module Airtable "Create a Record" ou "Search and Update".

7. **Bonus sécurité** : Ajoutez un module "Tools > Set Variable" au tout début du scénario pour vérifier la signature HMAC du webhook Stripe avant de traiter la donnée.

© 2026 Oussama Ghorbel

Exercice 3 : Déployer une Cloudflare Worker qui transforme un payload avant renvoi

1. Créez un compte gratuit sur Cloudflare et accédez à la section "Workers & Pages".
2. Créez un nouveau Worker avec l'éditeur en ligne.
3. Copiez-collez le code JavaScript fourni dans la section 20.7.
4. Modifiez la logique de transformation : ajoutez un champ `source_system: "IoT_Sensor"` et convertissez toutes les clés du JSON en majuscules.
5. Déployez le Worker et récupérez son URL (`votre-worker.votre-compte.workers.dev`).
6. Utilisez Postman pour envoyer une requête POST avec un JSON fictif contenant une température en Celsius.
7. Vérifiez dans Postman que la réponse (ou la redirection vers un webhook de test comme webhook.site) contient bien le payload transformé avec les clés en majuscules et la température en Fahrenheit.

Vous venez de franchir un cap. Vous n'êtes plus limité par les blocs préfabriqués des éditeurs de logiciels. Le web tout entier, dans sa structure la plus brute et la plus puissante, est désormais à votre disposition pour bâtir des systèmes robustes, sur mesure, et sans aucune friction.

© 2026 Oussama Ghorbel

Partie 5 — Industrialiser : IA, équipe, gouvernance et passage à l'échelle

© 2026 Oussama Ghorbel

Chapitre 21 – IA générative intégrée : GPT, Claude, agents dans vos apps

Lors des premières explorations des APIs de génération de texte, l'intégration de l'intelligence artificielle dans une application no-code relevait presque de l'expérimentation de laboratoire. On envoyait une requête, on croisait les doigts pour que le format de réponse soit correct, et on priait pour que le coût en tokens n'explose pas le budget mensuel. Aujourd'hui, en 2026, la donne a radicalement changé. L'IA générative n'est plus une fonctionnalité « gadget » que l'on ajoute en fin de projet pour impressionner un client ; elle est devenue un composant natif de notre stack, au même titre qu'une base de données ou un module d'authentification.

L'objectif de ce chapitre n'est pas de faire de vous un data scientist ou un ingénieur en machine learning. En tant que bâtisseur no-code, votre force réside dans votre capacité à orchestrer des outils, à comprendre les flux de données et à concevoir des expériences utilisateur fluides. Nous allons voir comment intégrer des capacités cognitives (compréhension du langage, vision, analyse vocale) dans vos workflows et applications, en gardant le contrôle sur la qualité, les coûts et la robustesse de vos systèmes.

21.1. Ce que les LLM font bien, mal, et très mal en 2026

Avant de connecter le moindre module dans Make ou Zapier, il est crucial de calibrer nos attentes. Les Grands Modèles de Langage (LLM) sont des moteurs probabilistes, pas des bases de données de faits. Comprendre leurs limites est la première étape pour concevoir des applications qui tiennent la route.

Ce qu'ils font remarquablement bien :

• **Extraction et structuration :** Transformer un email désordonné, un PDF complexe ou une transcription vocale en un objet JSON parfaitement formaté. C'est le cas d'usage roi en entreprise.

• **Synthèse et résumé :** Condenser des dizaines de pages de documentation, des transcripts de réunions Zoom ou des historiques

de support client en quelques points clés.

• **Compréhension multimodale :** En 2026, les modèles comme GPT-4o ou Claude analysent nativement des images, des captures d'écran d'erreurs, des plans architecturaux ou des graphiques avec une précision stupéfiante.

• **Traduction et adaptation de ton :** Localiser une interface ou adapter un message marketing à différentes personas culturelles.

• **Génération de code et de formules :** Écrire des regex, des formules Excel complexes ou des scripts JavaScript pour vos outils low-code.

Ce qu'ils font mal (et nécessitent des outils externes) :

• **Les mathématiques complexes et la logique déterministe :** Un LLM peut se tromper sur une multiplication à quatre chiffres ou sur un enchaînement logique strict. C'est pourquoi on utilise désormais le *Code Interpreter* ou des outils de calcul externes (via des APIs Python ou des modules de calcul dans vos workflows) pour déléguer la partie mathématique.

• **La connaissance en temps réel :** Sans outil de recherche web (Search API), un LLM est figé dans le temps de son dernier entraînement. Il faut lui donner accès à des outils de *browsing* pour qu'il puisse sourcer des informations actuelles.

Ce qu'ils font très mal (les zones de danger) :

• **La garantie de vérité absolue (Hallucinations) :** Le modèle préfère inventer une réponse plausible plutôt que de dire « je ne sais pas ».

• **Les prédictions de marchés et la finance :** *Notez que les analyses financières générées par l'IA sont purement informatives et ne constituent en aucun cas un conseil en investissement.*

• **Le raisonnement causal profond et la politique d'entreprise non écrite :** Ils ne comprennent pas les nuances humaines non documentées de votre organisation.

© 2026 Oussama Ghorbel

21.2. Connecter OpenAI, Anthropic, Mistral depuis Make/Zapier

Intégrer un LLM dans un orchestrateur comme Make (ex-Integromat) ou Zapier est aujourd'hui un jeu d'enfant, à condition de maîtriser la gestion des données structurées.

Les modules natifs vs le module HTTP

Zapier et Make proposent des modules natifs pour OpenAI et Anthropic. Ils sont parfaits pour débuter : vous choisissez le modèle, vous collez votre prompt, et l'outil gère l'authentification. Cependant, pour des applications en production, je recommande souvent d'utiliser le module **HTTP** (ou Webhooks). Pourquoi ? Parce que les modules natifs sont parfois en retard sur les nouvelles fonctionnalités des APIs (comme le *JSON mode* forcé, les *tool calls*, ou la gestion fine du *streaming*).

L'architecture d'un appel API standard dans Make :

1. **Requête HTTP (POST) :** Vous appelez l'endpoint de chat (ex: `https://api.openai.com/v1/chat/completions`).

2. **Headers :** `Authorization: Bearer VOTRE_CLE_API` et `Content-Type: application/json`.

3. **Body (JSON) :** C'est ici que vous structurez votre tableau de messages (System, User).

4. **Parsing de la réponse :** La réponse de l'API n'est pas directement le texte. Elle est encapsulée. Dans Make, vous devrez extraire le texte via le chemin `choices[0].message.content`.

Le choix du fournisseur en 2026 :

• **OpenAI (GPT-4o, o1) :** Le standard pour la polyvalence, la vision et le raisonnement complexe.

• **Anthropic (Claude) :** Exceptionnel pour le traitement de documents longs (fenêtres de contexte massives) et pour le respect strict des instructions de sécurité et de ton.

• **Mistral AI :** L'alternative européenne incontournable. Si votre application no-code traite des données de santé ou des données personnelles de citoyens européens soumises à des contraintes RGPD strictes, les modèles de Mistral (hébergés sur des

© 2026 Oussama Ghorbel

infrastructures souveraines) sont souvent le choix le plus sage pour la conformité.

Gestion des timeouts : Les LLM peuvent mettre plusieurs secondes à répondre, surtout sur des prompts longs ou des modèles de raisonnement (comme la série *o1*). Dans Make, pensez à augmenter le timeout de votre module HTTP à 120 secondes, et configurez des reprises sur erreur (retries) avec un délai exponentiel en cas de surcharge de l'API (erreur 429).

21.3. Prompt engineering pour apps : système, utilisateur, few-shot

Oubliez la façon dont vous parlez à ChatGPT dans votre navigateur. Dans une application no-code, vous ne cherchez pas une conversation, vous cherchez une **exécution déterministe**. Le prompt engineering pour les applications repose sur une structure rigoureuse.

Le System Prompt : Le cadre légal

C'est l'instruction globale qui définit le comportement, le persona et les contraintes de sortie. Il ne change jamais d'une requête à l'autre.

Exemple : « Tu es un assistant de qualification de leads B2B. Ton objectif est d'analyser le texte fourni et d'extraire des informations structurées. Tu ne dois jamais inventer d'informations. Si une donnée est absente, retourne la valeur `null`. »

Le User Prompt : La donnée dynamique

C'est ici que vous injectez les variables de votre workflow (le contenu de l'email, le texte du PDF, etc.).

Exemple : `Voici le texte à analyser : {{email_body}}`

Le Few-Shot Prompting : L'apprentissage par l'exemple

C'est la technique la plus puissante pour garantir la qualité sans avoir à écrire des consignes de 1000 mots. Vous donnez au modèle 2 ou 3 exemples de paires « Entrée / Sortie attendue » directement dans le prompt. Le modèle comprend le pattern et le reproduit.

Forcer le JSON (JSON Mode)

© 2026 Oussama Ghorbel

En 2026, la plupart des APIs proposent un paramètre `response_format: { "type": "json_object" }`. C'est une bouée de sauvetage pour le no-code. Cela force le LLM à ne renvoyer que du JSON valide, évitant ainsi les erreurs de parsing dans Make ou Zapier quand le modèle a la mauvaise idée d'ajouter « Voici le JSON demandé : ```json ... ``` » autour de sa réponse.

Temperature et Top_p :

• Pour de l'extraction, de la classification ou du JSON : `temperature = 0` (ou très proche). Vous voulez que le modèle soit froid, logique et reproductible.

• Pour de la génération de contenu marketing, des brainstormings ou de la rédaction créative : `temperature = 0.7`. Vous lui donnez le droit d'être créatif et d'explorer des chemins sémantiques variés.

21.4. RAG simple : donner sa documentation à un LLM

Le RAG (*Retrieval-Augmented Generation*, ou Génération Augmentée par Recherche) est le concept qui a fait passer l'IA du statut de « causeur sympathique » à celui de « véritable assistant métier ».

Le problème du LLM seul est qu'il ne connaît pas votre entreprise, vos produits, ou vos procédures internes. Le RAG consiste à lui fournir un « livre ouvert » pendant l'examen. Plutôt que de lui demander de répondre de mémoire, on va d'abord chercher les informations pertinentes dans votre base de connaissances, puis les lui donner en contexte pour qu'il formule la réponse.

Les 3 étapes du RAG en no-code :

1. **L'ingestion et le Chunking (Découpage) :**

Vous ne pouvez pas envoyer un manuel de 300 pages dans le prompt (limite de tokens et coût exorbitant). Il faut découper vos documents en « chunks » (paragraphes ou sections logiques de 300 à 500 mots). Des outils comme Make peuvent lire un Notion ou un Google Drive, découper le texte et l'envoyer à une API d'embedding.

2. **L'Embedding et la Base Vectorielle :**

© 2026 Oussama Ghorbel

L'embedding est le processus qui transforme votre texte en une suite de nombres (un vecteur) représentant son *sens sémantique*. Deux phrases qui veulent dire la même chose auront des vecteurs très proches, même si les mots sont différents. Ces vecteurs sont stockés dans une base de données vectorielle. En no-code, **Pinecone**, **Weaviate** ou **Supabase** (avec son extension pgvector) sont les standards. Ils s'intègrent nativement dans Make.

3. **La Retrieval (Recherche) et la Génération :**

Quand l'utilisateur pose une question dans votre app (ex: via Bubble ou WeWeb), la question est elle-même transformée en vecteur. La base vectorielle cherche les 3 ou 4 chunks les plus proches sémantiquement. Vous récupérez ces textes, vous les injectez dans le System Prompt du LLM (« Utilise uniquement le contexte suivant pour répondre... »), et vous générez la réponse finale.

L'alternative no-code native :

Si construire un pipeline RAG sur Make vous semble complexe, des plateformes comme **Voiceflow**, **Botpress** ou **Stack AI** proposent des interfaces visuelles où vous uploadez vos PDFs ou connectez votre Notion, et l'outil gère le chunking, l'embedding et la recherche vectorielle en arrière-plan. C'est idéal pour déployer un chatbot de support en quelques heures.

21.5. Agents IA : AutoGen, CrewAI, n8n AI agents, Relevance AI

Jusqu'ici, nous avons parlé de chaînes linéaires : une entrée, un appel LLM, une sortie. Mais les problèmes métier complexes nécessitent souvent de la réflexion, de la planification et l'utilisation d'outils. C'est là qu'interviennent les **Agents IA**.

Comme le souligne Andrew Ng, pionnier de l'IA, les *workflows agentiques* (où l'IA itère, réfléchit à ses erreurs et utilise des outils) surpassent aujourd'hui les modèles monolithiques les plus puissants. Un agent n'est pas juste un prompt ; c'est une entité dotée d'un objectif, d'une mémoire et d'une boîte à outils.

Les frameworks de code (et comment les utiliser en no-code) :

© 2026 Oussama Ghorbel

Des frameworks comme **AutoGen** (Microsoft) ou **CrewAI** permettent de créer des équipes d'agents (ex: un agent "Chercheur", un agent "Rédacteur", un agent "Critique"). Bien que ce soient des bibliothèques Python, vous n'avez pas besoin de savoir coder pour les utiliser. Vous pouvez les héberger sur une plateforme comme **Render** ou **Railway**, ou les emballer dans une API via **FastAPI**, puis appeler cette API depuis votre application Bubble ou votre scénario Make.

Les plateformes Agentiques No-Code / Low-Code :

L'écosystème a explosé pour rendre les agents accessibles visuellement :

• **Relevance AI :** C'est le couteau suisse de l'agentique no-code. Vous pouvez créer des "Tools" (appels API, recherches web, requêtes SQL) et les assigner à des agents. L'interface visuelle permet de définir le persona de l'agent, ses outils, et de le déployer via API ou widget web.

• **n8n AI Agents :** n8n (l'alternative open-source à Make) a intégré des nœuds dédiés aux agents. Vous pouvez créer une boucle où l'agent décide lui-même quel nœud exécuter (ex: "Ai-je besoin de chercher dans la base de données SQL, ou d'envoyer un email ?"). C'est du low-code pur, mais d'une puissance redoutable.

• **FlowiseAI / Langflow :** Des constructeurs visuels de chaînes LangChain. Vous glissez-déposez des nœuds (LLM, Vector Store, Outils de recherche) pour créer des architectures agentiques complexes, que vous exposez ensuite comme des endpoints API pour vos apps.

Le pattern de l'agent "Réflexion" :

Un pattern simple à implémenter dans Make consiste à faire appel à deux LLM en séquence. Le premier génère une réponse ou un plan. Le second (avec un prompt de "Critique") évalue la sortie du premier, pointe les erreurs, et lui renvoie pour correction. Cette boucle d'auto-correction améliore drastiquement la qualité des outputs sans intervention humaine.

21.6. Contrôler les coûts, la qualité et les hallucinations

© 2026 Oussama Ghorbel

Déployer de l'IA en production, c'est s'exposer à trois risques majeurs : la facture qui explose, la qualité qui dérive, et les hallucinations qui nuisent à votre réputation. Voici comment les mitiger.

1. Le contrôle des coûts et le Semantic Caching

Appeler GPT-4o pour classifier un email en "Spam" ou "Non-Spam" est un gaspillage financier et écologique.

• **La règle du bon modèle :** Utilisez les modèles "nano" ou "mini" (très peu chers et rapides) pour la classification, le routage et l'extraction simple. Gardez les modèles de raisonnement avancé pour la synthèse complexe ou la génération de code.

• **Le Semantic Cache :** Contrairement à un cache traditionnel qui cherche une correspondance exacte, le cache sémantique utilise les embeddings. Si un utilisateur pose la question "Comment réinitialiser mon mot de passe ?" et qu'un autre demande "J'ai oublié mon password, je fais quoi ?", le système reconnaît l'intention identique et sert la réponse déjà générée et stockée (dans un Redis ou une base Airtable), économisant ainsi un appel API coûteux.

2. L'évaluation de la qualité (LLM-as-a-Judge)

Comment tester automatiquement si votre agent répond bien ? En utilisant un autre LLM. Dans votre workflow de test, vous pouvez envoyer la question, la réponse de votre app, et le document source à un modèle tiers avec ce prompt : *« Sur une échelle de 1 à 5, la réponse est-elle fidèle au document source et répond-elle à la question ? Réponds en JSON. »* Cela vous permet de créer des tableaux de bord de qualité automatisés.

3. Tuer les hallucinations

• **Grounding strict :** Forcez le modèle à citer ses sources. « Réponds uniquement en te basant sur le contexte fourni. Si l'information n'est pas dans le contexte, réponds exactement : *Je n'ai pas l'information dans ma base de connaissances.* »

• **Validation par schéma JSON :** Ne laissez jamais le LLM inventer les clés de votre JSON. Fournissez-lui un schéma strict (JSON Schema) et utilisez les *Tool Calls* (ou *Function Calling*) de l'API pour le forcer à remplir les champs spécifiques avec les types

© 2026 Oussama Ghorbel

de données attendus (string, boolean, integer).

• **Human-in-the-loop (HITL) :** Pour les actions à haut risque (ex: rembourser un client, envoyer une proposition commerciale), l'agent IA ne doit pas avoir le bouton "Valider". Il prépare le brouillon, le pousse dans un tableau Notion ou un CRM avec le statut "À valider", et c'est un humain qui déclenche l'action finale via un bouton no-code.

21.7. Cas d'usage : support, rédaction, extraction, classification

Pour ancrer ces concepts, voici quatre architectures no-code typiques que l'on peut observer ou déployer en entreprise.

Le Support Client (Triage et Brouillons)

• *Le flux :* Un ticket arrive dans Zendesk ou Intercom. Make intercepte le webhook. Un premier LLM (rapide et peu coûteux) classifie l'intention (Bug, Facturation, Demande de feature) et analyse le sentiment (Urgent/Colérique, Neutre).

• *L'action :* Si c'est une question fréquente, le workflow fait une recherche RAG dans la base Notion des FAQ. Un second LLM (plus lent, plus empathique) rédige une proposition de réponse. Le tout est renvoyé dans Zendesk comme "Réponse interne suggérée" pour l'agent humain.

La Rédaction et la Personnalisation à l'échelle

• *Le flux :* Vous lancez une campagne d'outreach B2B. Votre base de données (HubSpot ou Airtable) contient le site web du prospect et son dernier post LinkedIn (récupéré via une API d'enrichissement comme Apollo ou Clay).

• *L'action :* Le LLM analyse le post LinkedIn pour en extraire les centres d'intérêt, puis génère un email d'approche hyper-personnalisé qui fait le lien entre leur actualité et votre solution. Le taux de réponse explose comparé aux templates statiques.

L'Extraction de Données Non Structurées

© 2026 Oussama Ghorbel

• *Le flux :* Votre entreprise reçoit 500 CVs ou factures par mois au format PDF.

• *L'action :* Un outil comme Make lit le PDF (ou utilise l'API de vision de GPT-4o si la mise en page est complexe). Le prompt demande l'extraction de champs précis (Nom, Entreprise, Montant HT, TVA, Date d'échéance) en JSON. Le module JSON de Make parse la réponse et crée automatiquement les lignes dans votre ERP ou votre base de données.

La Classification et le Routage Sémantique

• *Le flux :* Des centaines d'avis clients tombent sur Trustpilot, App Store et réseaux sociaux.

• *L'action :* Un agent IA analyse chaque avis. Il ne fait pas que du "positif/négatif". Il extrait les entités (ex: "Livraison", "Interface", "Prix") et qualifie le problème. Si un avis mentionne un bug technique critique, l'agent crée automatiquement un ticket Jira et alerte l'équipe produit sur Slack.

Exercices pratiques

La théorie ne vaut pas la pratique. Pour maîtriser l'intégration de l'IA dans vos stacks no-code, réalisez ces trois exercices progressifs.

Exercice 1 : Créer un chatbot support branché sur une base Notion via RAG

1. Créez une base Notion contenant 10 pages de documentation technique (votre FAQ).

2. Inscrivez-vous sur **Voiceflow** ou **Botpress** (les versions gratuites suffisent).

3. Connectez votre Notion à la plateforme pour qu'elle indexe et "chunk" vos documents (création de la base vectorielle).

4. Concevez le flow de conversation : Salutation -> Capture de la question -> Appel au Knowledge Base (RAG) -> Formulation de la réponse par le LLM.

© 2026 Oussama Ghorbel

5. Intégrez le widget de chat sur une page web simple (Carrd ou Webflow) et testez avec des questions dont les réponses ne sont *pas* dans la base pour vérifier que le bot admet son ignorance sans halluciner.

Exercice 2 : Automatiser l'extraction de données de 1000 PDFs avec GPT-4o

1. Placez 10 PDFs de factures (anonymisées) dans un dossier Google Drive.

2. Dans **Make**, créez un scénario qui itère sur les fichiers du dossier.

3. Utilisez le module OpenAI "Analyze Image/PDF" (ou le module HTTP avec l'API Vision).

4. Configurez le prompt pour extraire : `Numéro de facture`, `Nom du fournisseur`, `Montant Total`, `Date`. Forcez le mode JSON.

5. Ajoutez un module "Parse JSON" et envoyez les données structurées dans un Google Sheet ou une table Airtable.

6. *Bonus :* Ajoutez un module de vérification mathématique (formule Make) pour vous assurer que la somme des lignes extraites correspond bien au Montant Total, et marquez la ligne en "Rouge" dans Airtable si le LLM a fait une erreur de lecture.

Exercice 3 : Construire un agent qui qualifie les leads entrants avant CRM

1. Simulez l'arrivée d'un lead via un webhook Make (contenant : Nom, Email, Message du formulaire de contact, URL de son site web).

2. Ajoutez un module HTTP pour scraper le texte de la page d'accueil du site web du lead (via une API comme ScraperAPI ou le module natif).

3. Envoyez le message du lead ET le texte de son site web à un LLM avec un System Prompt de "Qualificateur BANT" (Budget, Authority, Need, Timing).

4. Demandez au LLM de retourner un score de lead (1 à 100) et un résumé de 2 phrases sur la pertinence du prospect pour votre entreprise.

© 2026 Oussama Ghorbel

5. Utilisez un routeur Make : si le score est > 70, créez le deal dans HubSpot/Pipedrive et notifiez les ventes sur Slack. Si < 70, envoyez un email automatisé de nurturing et archivez le lead.

© 2026 Oussama Ghorbel

Chapitre 22 – Travailler en équipe sur du no-code

Il existe un mythe tenace dans l'univers du no-code : celui du « loup solitaire », ce maker génial qui, depuis son canapé, bâtit en un week-end un SaaS complet capable de concurrencer des entreprises établies. Si cette image est séduisante et sert souvent de catalyseur pour découvrir ces outils, elle s'effondre rapidement dès qu'il s'agit de faire croître une entreprise, de maintenir une application critique ou de gérer des processus métier complexes.

Le no-code ne dispense pas du travail d'équipe ; au contraire, il l'exige. Parce que la barrière à l'entrée technique est faible, le risque de voir émerger un chaos architectural est immense si les règles de collaboration ne sont pas strictement définies. Passer du maker solo à l'équipe de makers, c'est passer de l'artisanat à l'ingénierie. C'est apprendre à construire à plusieurs sur le même édifice visuel sans que les murs ne s'effondrent. Dans ce chapitre, nous allons structurer cette collaboration : des rôles de chacun aux conventions de nommage, de la revue de pairs à la création d'un véritable Centre de Compétences (CoE).

22.1. Rôles : product owner no-code, maker, admin plateforme, IT partner

Dans le développement traditionnel, les rôles sont figés par des décennies de pratique (Business Analyst, Développeur, DevOps, QA). En no-code, les frontières sont plus poreuses, mais les *fonctions* restent indispensables. Même si une seule personne endosse plusieurs casquettes au début, il est crucial de conceptualiser ces rôles pour savoir quel chapeau l'on porte à un instant T.

Le Product Owner (PO) No-Code

C'est le gardien du « pourquoi » et du « quoi ». Le PO no-code ne se contente pas de rédiger des user stories classiques ; il doit posséder une literacy visuelle. Il comprend les limites de la stack choisie. S'il demande une fonctionnalité, il sait intuitivement si cela nécessite une simple vue filtrée dans Airtable ou un workflow complexe dans Make avec des

© 2026 Oussama Ghorbel

appels API. Son rôle est de traduire le besoin métier en spécifications visuelles (maquettes, schémas de processus) et de prioriser la backlog en fonction de la valeur métier et de la dette technique potentielle.

Le Maker (Builder / Développeur Visuel)

C'est l'architecte et le maçon du « comment ». Le Maker conçoit la base de données, dessine les interfaces, et orchestre les automatisations. Son super-pouvoir n'est pas de taper du code, mais de penser en systèmes. Un excellent Maker no-code anticipe les cas limites (edge cases), optimise les requêtes pour ne pas épuiser les quotas d'API, et construit des interfaces résilientes. En équipe, les Makers se répartissent souvent par domaines fonctionnels (ex: un Maker sur le backend Xano, un autre sur le frontend WeWeb, un troisième sur les automatisations Make).

L'Admin Plateforme (Platform Admin)

C'est le gardien du temple. Dans l'euphorie de la création, on oublie souvent la gestion des licences, des clés API, des webhooks et des facturations. L'Admin Plateforme centralise ces informations critiques dans un coffre-fort numérique (comme 1Password ou Bitwarden). Il gère les invitations dans les workspaces, surveille la consommation des opérations (les fameux "tasks" ou "operations" qui dictent la facture mensuelle de vos outils) et s'assure que les comptes ne sont pas liés à l'adresse email personnelle d'un employé qui pourrait quitter l'entreprise.

Le Partenaire IT (IT Partner / DSI)

L'IT n'est pas l'ennemi du no-code, c'est son allié pour le passage à l'échelle. Le partenaire IT intervient pour configurer l'authentification unique (SSO/SAML), ouvrir les ports de pare-feu, valider la conformité RGPD des outils choisis, et intégrer les applications no-code dans l'écosystème legacy de l'entreprise via des API gateways sécurisés. Impliquer l'IT dès la phase de conception évite les blocages dramatiques au moment du déploiement en production.

22.2. Conventions de nommage et documentation vivante

Avez-vous déjà essayé de déchiffrer un scénario d'automatisation de 80 modules, créé par un collègue parti en vacances, où chaque variable

© 2026 Oussama Ghorbel

s'appelle `array1`, `data2` ou `test_final` ? C'est ce que j'appelle le « spaghetti no-code ». Contrairement au code traditionnel où l'IDE (l'environnement de développement) impose une certaine rigueur, les outils no-code vous laissent une liberté totale. Cette liberté est un piège mortel en équipe.

La charte de nommage

Une convention de nommage doit être décidée avant la création du premier projet et appliquée avec une rigueur militaire. Voici une structure éprouvée pour les bases de données et les workflows :

• **Bases de données (ex: Airtable, Supabase) :** Utilisez des préfixes clairs. `tbl_` pour les tables, `vw_` pour les vues, `fld_` pour les champs. Nommez les tables au singulier (`tbl_Client` et non `tbl_Clients`).

• **Workflows (ex: Make, Zapier) :** Le nom du scénario doit raconter son histoire. Adoptez le format `[APP_SOURCE] - [TRIGGER] - [ACTION] - [APP_DEST]`. Exemple : `[Stripe] - New Payment - Format & Filter - [Notion]`.

• **Variables et Custom Properties (ex: Bubble, WeWeb) :** Utilisez le camelCase ou le snake_case, mais soyez cohérent. Préfixez les variables temporaires (`tmp_userList`) et les variables d'état (`state_isLoading`).

• **Éléments visuels (UI) :** Nommez vos boutons et groupes pour pouvoir les cibler facilement dans les workflows. `btn_submit_checkout` est infiniment meilleur que `Button 14`.

La documentation vivante

La documentation traditionnelle (un fichier Word de 40 pages mis à jour une fois par an) est obsolète avant même d'être publiée. En no-code, la documentation doit être *vivante* et contextualisée.

1. **Documentation in-app :** Utilisez massivement les blocs de notes (notes) dans Make, les commentaires dans les formules Airtable, et les infobulles dans Bubble. Un module complexe doit toujours être précédé d'une note expliquant *pourquoi* il est là, et pas seulement *ce qu'il fait*.

© 2026 Oussama Ghorbel

2. **Architecture visuelle :** Maintenez un schéma d'architecture (sur Miro, Whimsical ou Lucidchart) qui cartographie les flux de données entre vos applications. Ce schéma est la boussole de l'équipe.

3. **Le Readme du Maker :** Chaque projet complexe doit avoir un espace Notion ou Confluence lié, contenant : le lien vers l'app, le lien vers le schéma d'architecture, la liste des variables d'environnement requises, et les "gotchas" (les pièges connus, comme "Ne jamais modifier le champ X car il est synchronisé par le workflow Y").

22.3. Revue de pairs : checklist avant mise en production

Dans le développement logiciel, la "Pull Request" et la revue de code sont des standards. En no-code, nous devons instaurer la **Revue de Flux** (Flow Review) ou la **Revue d'App**. Puisque la plateforme élimine les erreurs de syntaxe (le code ne "compile" pas s'il est mal écrit), la revue de pairs en no-code se concentre sur la logique métier, la performance, la sécurité et la maintenabilité.

Ne déployez jamais un workflow critique ou une nouvelle page d'application sans qu'un autre maker l'ait examiné. Voici la checklist de revue de pairs que je recommande d'imposer dans votre équipe :

1. Gestion des erreurs et résilience

• *Automatisations :* Y a-t-il des routeurs d'erreur (Error Handlers) sur les modules critiques (appels API, création de base de données) ? Si l'API renvoie une erreur 500, le scénario plante-t-il silencieusement ou alerte-t-il l'équipe (via Slack/Teams) et met-il les données en quarantaine ?

• *Applications :* Que se passe-t-il si l'utilisateur clique deux fois très vite sur le bouton de paiement ? Le bouton se désactive-t-il pendant le chargement ? Y a-t-il des messages d'erreur clairs pour l'utilisateur final ?

2. Performance et optimisation des quotas

• *Boucles et Itérations :* Le workflow fait-il des recherches dans une base de données *à l'intérieur* d'une boucle (le fameux problème

© 2026 Oussama Ghorbel

N+1) ? Si oui, peut-on récupérer les données en amont et les filtrer en mémoire ?

• *Requêtes :* Utilise-t-on des recherches ciblées (Search/Get) plutôt que de télécharger toute la base de données pour filtrer côté client ?

• *Webhooks :* Les webhooks sont-ils sécurisés par des tokens ou des signatures pour éviter qu'un tiers ne déclenche le scénario avec des données malveillantes ?

3. Sécurité et Privacy Rules

• *Données sensibles :* Les mots de passe, tokens API ou données personnelles (PII) sont-ils stockés dans des variables d'environnement et non en dur dans les champs de texte ?

• *Règles de confidentialité (Privacy Rules) :* Dans des outils comme Bubble ou Supabase, a-t-on vérifié que l'utilisateur A ne peut absolument pas requêter les données de l'utilisateur B via l'inspecteur réseau du navigateur ?

4. Maintenabilité et conventions

• La convention de nommage (vue en 22.2) est-elle respectée ?

• Les modules complexes sont-ils commentés ?

• Y a-t-il du "code mort" ou des scénarios désactivés qui polluent l'espace de travail et devraient être archivés ?

Conseil de mise en œuvre : La revue peut se faire de manière asynchrone. Le maker enregistre une vidéo Loom de 5 minutes où il "joue" son workflow, explique ses choix architecturaux et soumet la vidéo à son pair. Cela crée en plus une excellente base de documentation vidéo !

22.4. Gestion des accès et des environnements en équipe

L'un des plus grands chocs pour les développeurs traditionnels qui arrivent dans le no-code est l'absence fréquente de natifs CI/CD (Intégration Continue / Déploiement Continue) et de gestion d'environnements (Dev / Staging / Prod) aussi rigoureuse que Git. Pourtant, en équipe, travailler directement sur l'environnement de production est une recette pour le désastre.

© 2026 Oussama Ghorbel

Le principe du moindre privilège

Par défaut, les outils no-code ont tendance à donner les droits d'administration à quiconque est invité dans le workspace. C'est une faille de sécurité majeure.

- Les métiers (Marketing, RH) qui utilisent l'application au quotidien doivent avoir des accès "Éditeur de contenu" ou "Utilisateur final", mais jamais "Admin de la structure".
- Seuls les Makers confirmés ont le droit de modifier le schéma de la base de données ou les workflows backend.
- L'Admin Plateforme est le seul à pouvoir gérer les intégrations OAuth, les clés API globales et les paramètres de facturation.

Stratégies d'environnements (Dev, Staging, Prod)

Puisque le versioning natif (type Git) commence seulement à apparaître sur quelques plateformes avancées, voici comment les équipes no-code professionnelles gèrent leurs environnements :

1. **L'approche native (quand elle existe) :** Des outils comme Bubble proposent un environnement "Development" et un bouton "Deploy to Live". C'est le minimum syndical. Toute nouvelle fonctionnalité est construite en Dev, testée, puis déployée en Live.
2. **L'approche par duplication (Le "Poor Man's Staging") :** Pour des bases de données comme Airtable ou des backends comme Xano, l'équipe maintient une base "Dev" et une base "Prod". Les Makers construisent et testent sur la base Dev. Une fois validé, les modifications de structure (nouvelles colonnes, nouveaux endpoints) sont répliquées manuellement ou via des scripts de migration sur la base Prod.
3. **Les variables d'environnement :** C'est le nerf de la guerre. Vos workflows Make ou Zapier ne doivent jamais contenir en dur l'URL de votre base de données de production ou les clés API de test. Utilisez les fonctionnalités de "Variables" ou "Environments" de vos orchestrateurs pour basculer d'un environnement à l'autre en un clic, sans avoir à rééditer 50 modules.
4. **Le Feature Branching visuel :** Pour les très grosses modifications, certains Makers dupliquent le scénario Make, le

© 2026 Oussama Ghorbel

nomment `[DEV] - [Nom du scénario]`, font leurs tests, puis, une fois validé, remplacent l'ancien scénario en production et archivent le dev.

22.5. Onboarding d'un nouveau maker en 3 jours

L'un des arguments de vente du no-code est la rapidité. Cette rapidité doit aussi s'appliquer à l'intégration d'un nouveau membre dans votre équipe de makers. Un onboarding structuré permet à un profil technique (ou un profil métier très à l'aise avec la logique) d'être productif et de pousser sa première modification en production en moins de 72 heures.

Jour 1 : Immersion, Architecture et Sécurité

• *Matin :* Accueil, présentation de la vision produit et du métier. Le nouveau maker doit comprendre *pourquoi* l'application existe et qui sont ses utilisateurs finaux.

• *Après-midi :* Octroi des accs (via l'Admin Plateforme, en respectant le moindre privilège). Lecture de la documentation vivante (le Readme du Maker).

• *Fin de journée :* Exploration guidée de l'architecture. Le maker senior partage son écran et navigue dans le schéma Miro, puis montre comment les données circulent de l'interface (ex: WeWeb) vers le backend (ex: Xano) et les automatisations (ex: Make).

Jour 2 : Shadowing et "Good First Issues"

• *Matin :* Pair-building. Le nouveau maker observe le maker senior résoudre un bug ou construire une petite fonctionnalité. L'accent est mis sur la méthodologie : comment le senior cherche dans les logs, comment il isole le problème, comment il nomme ses variables.

• *Après-midi :* Le nouveau maker prend les commandes sur une tâche simple et isolée (une "Good First Issue"). Exemple : ajouter un nouveau champ dans un formulaire, créer une vue filtrée dans Airtable, ou ajouter une étape de formatage de date dans un workflow Make existant.

• *Fin de journée :* Première soumission à la revue de pairs. Le senior corrige avec bienveillance, en expliquant les conventions de nommage et les règles de sécurité.

© 2026 Oussama Ghorbel

Jour 3 : Première fonctionnalité et déploiement

• *Matin :* Le nouveau maker reçoit une spécification complète pour une petite feature autonome (ex: créer un scénario qui alerte Slack lorsqu'un client VIP remplit un formulaire de support).

• *Après-midi :* Construction, tests unitaires (le maker teste lui-même avec des données fictives), et enregistrement de la vidéo Loom pour la revue de pairs.

• *Fin de journée :* Validation par le pair, déploiement en environnement de production (ou promotion du Dev au Live), et célébration de cette première contribution. Le maker est désormais opérationnel.

22.6. Centre de compétences no-code interne (No-Code CoE)

Lorsque le no-code se propage dans une organisation de plus de 50 personnes, vous faites face à un phénomène inévitable : le Shadow IT. Le marketing achète un abonnement Zapier sur la carte bleue du directeur, les RH construisent une base Airtable contenant des données médicales sans respecter le RGPD, et la finance utilise un outil d'IA non sécurisé pour analyser des bilans comptables.

Pour transformer ce chaos potentiel en avantage concurrentiel, les entreprises performantes créent un **Centre de Compétences No-Code (No-Code Center of Excellence ou CoE)**.

Qu'est-ce qu'un CoE No-Code ?

Ce n'est pas une tour d'ivoire qui interdit et bloque. C'est une équipe transversale (souvent composée d'un lead Maker, d'un représentant de la DSI, et d'un Product Owner métier) dont la mission est de *faciliter* la création tout en *sécurisant* l'écosystème. C'est le concept de "démocratisation avec des garde-fous".

Les 4 piliers du CoE :

1. **La gouvernance de la Stack :** Le CoE définit la liste des outils approuvés (le "Catalogue de Service"). Si un employé veut utiliser un nouvel outil, il soumet une demande au CoE qui évalue la sécurité, la conformité RGPD et la pérennité de l'éditeur.

© 2026 Oussama Ghorbel

2. **La bibliothèque de templates (Assets) :** Le CoE maintient des modèles prêts à l'emploi. Un template de base Airtable pour la gestion de projet, un template Make pour l'envoi d'emails transactionnels via l'API de l'entreprise, un design system pour les applications internes. Cela évite que chaque équipe ne réinvente la roue.

3. **L'animation de la communauté (Guildes) :** Le CoE organise des "No-Code Cafés" mensuels où les makers de différents départements présentent leurs réalisations. Il gère un canal Slack/Teams d'entraide où les débutants peuvent poser leurs questions sans peur du jugement.

4. **Le support et l'escalade IT :** Le CoE sert de traducteur entre les makers métiers et la DSI. Si un maker a besoin d'accéder à une API interne sécurisée, le CoE prépare le dossier technique pour l'IT, garantissant que les standards de sécurité sont respectés.

Le CoE est la preuve ultime que le no-code a atteint sa maturité dans l'entreprise. Il n'est plus un jouet expérimental, mais un véritable levier de transformation digitale, encadré, mesuré et soutenu par la direction.

Exercices pratiques

Pour ancrer les concepts de ce chapitre, je vous invite à réaliser ces trois exercices, seul ou avec votre équipe. Ils constituent les fondations de votre future organisation de makers.

Exercice 1 : Rédiger la charte de nommage et de documentation de l'équipe

• **Objectif :** Créer un document de référence de 2 pages maximum qui servira de loi pour toute création no-code dans votre entreprise.

• **Action :** Définissez les règles de nommage pour vos bases de données (tables, champs), vos workflows (scénarios, modules, variables) et vos interfaces (boutons, groupes).

• **Livrable :** Un tableau synthétique avec des exemples "À faire" et "À ne pas faire". Ajoutez-y la liste des outils de documentation obligatoires (ex: "Tout scénario de plus de 10 modules doit avoir un

© 2026 Oussama Ghorbel

schéma Miro associé"). Partagez ce document dans votre espace Notion ou Confluence et faites-le signer symboliquement par tous les makers actuels.

Exercice 2 : Concevoir le parcours d'onboarding d'un maker junior (3 jours)

• **Objectif :** Préparer l'arrivée de votre prochain recruit ou la montée en compétence d'un collègue métier qui souhaite devenir maker.

• **Action :** Reprenez la structure du Jour 1, Jour 2, Jour 3 proposée dans la section 22.5 et adaptez-la à *votre* stack spécifique.

• **Livrable :** Un agenda détaillé heure par heure. Identifiez à l'avance 3 "Good First Issues" (bugs mineurs ou petites évolutions) réelles et isolées dans votre application actuelle, que le nouveau maker pourra résoudre le Jour 2. Préparez les accès et les licences qui devront être créés la veille de son arrivée.

Exercice 3 : Mener une revue de pair sur un workflow réel avec la checklist

• **Objectif :** Instaurer la culture de la revue de flux (Flow Review) et débusquer des erreurs cachées.

• **Action :** Sélectionnez le workflow le plus complexe et le plus critique actuellement en production dans votre entreprise (ex: le scénario qui génère les factures ou qui synchronise le CRM avec l'outil de support).

• **Livrable :** Le maker qui n'a *pas* construit ce workflow doit l'auditer en utilisant la checklist de la section 22.3. Il doit rédiger un rapport d'étonnement et une liste de recommandations (gestion des erreurs, optimisation des requêtes, ajout de commentaires). Planifiez une réunion de 45 minutes pour que le créateur original et l'auditeur corrigent ensemble les failles identifiées.

Pour aller plus loin : rendre la collaboration irréversible

Les outils et les processus décrits dans ce chapitre n'ont de valeur que s'ils s'ancrent dans les habitudes de l'équipe. Une charte de nommage que personne ne consulte après sa rédaction, une checklist de revue que l'on bâcle sous la pression d'une livraison, un onboarding

© 2026 Oussama Ghorbel

improvisé le matin même de l'arrivée d'un nouveau collègue : autant de bonnes intentions qui s'effacent face à l'urgence quotidienne. Le défi n'est donc pas technique, il est culturel.

Trois principes simples permettent de rendre cette collaboration irréversible.

Commencer petit, mais commencer maintenant. Vous n'avez pas besoin d'un Centre de Compétences à quatre personnes pour appliquer ce chapitre. Si vous êtes deux makers, appliquez la convention de nommage dès le prochain projet et imposez-vous une revue croisée systématique, même si elle dure vingt minutes. La rigueur précoce coûte peu ; la rigueur tardive, quand la base de données compte deux cents tables et que trois équipes dépendent du même scénario d'automatisation, coûte très cher.

Rendre les règles visibles, pas seulement connues. Une convention enfouie dans un onglet Notion lointain n'existe pas. Les équipes qui réussissent affichent leurs règles là où le travail se fait : un rappel en haut de chaque base de données, un template de scénario pré-commenté, une vidéo d'onboarding de dix minutes que le nouveau maker regarde avant même d'accéder aux outils. La documentation vivante n'est pas un document que l'on écrit une fois, c'est un environnement que l'on entretient.

Mesurer pour améliorer, jamais pour sanctionner. Les indicateurs d'une équipe de makers efficace ne sont pas le nombre de scénarios créés ou le nombre de clics épargnés. Ce sont des indicateurs de robustesse : combien de workflows ont une gestion d'erreur active, combien de projets ont un schéma d'architecture à jour, combien de temps met un nouveau maker pour livrer sa première fonctionnalité validée. Ces métriques servent à repérer les frictions, pas à classer les individus.

Il faut également anticiper les tensions inhérentes au travail collectif. Le premier point de friction fréquent est le sentiment de possession : un maker qui a bâti seul une application pendant des mois peut percevoir la revue de pairs comme une remise en cause de son travail. Il faut nommer les choses clairement dès le départ : on ne relit pas le travail d'un collègue pour le juger, on le relit pour protéger l'équipe contre les

© 2026 Oussama Ghorbel

risques qu'un seul regard ne peut pas voir. La meilleure analogie reste celle du pilotage en binôme en aviation : deux pilotes expérimentés ne se relirent pas par méfiance, mais parce que la charge cognitive d'un vol entier dépasse toujours les capacités d'un seul cerveau.

Le deuxième point de friction est le rythme. Les métiers voudront souvent aller plus vite que l'équipe de makers, surtout une fois les premiers gains de productivité constatés. C'est précisément le rôle du Product Owner no-code de arbitrer, en rappelant qu'une dette technique accumulée en no-code se paie avec le même intérêt qu'en développement classique : par des applications lentes, des automatisations fragiles et un turnover des makers qui ne supportent plus de travailler sur un héritage illisible.

Enfin, gardez à l'esprit que le no-code transforme la frontière entre métiers et techniques, mais ne l'efface pas. Un commercial brillant qui construit son premier outil de suivi de pipeline n'est pas pour autant un architecte de systèmes, et un développeur chevronné qui découvre Bubble n'est pas automatiquement un expert des parcours utilisateurs. L'équipe de makers se construit précisément à l'intersection de ces compétences complémentaires : c'est leur diversité de regards qui fait la qualité du résultat, à condition que des conventions communes leur donnent une langue partagée.

Le passage du maker solo à l'équipe de makers n'est donc pas une contrainte administrative imposée d'en haut. C'est une décision stratégique. Une application maintenue par une seule personne est un risque opérationnel : elle disparaît avec elle, ou elle l'emmène dans sa dépendance à un outil non documenté. Une application maintenue par une équipe, avec des rôles clairs, des conventions respectées, des revues systématiques et une documentation vivante, devient un actif de l'entreprise : elle survit aux départs, elle se transmet, elle s'étend.

C'est cette transformation — de l'outil personnel au patrimoine collectif — qui distingue les expérimentations no-code qui restent anecdotiques de celles qui deviennent, durablement, des applications d'entreprise à part entière.

© 2026 Oussama Ghorbel

Chapitre 23 – Gouvernance, shadow IT et relation avec la DSI

Gouvernance, shadow IT et relation avec la DSI

Il existe une tension fondamentale dans toute organisation qui découvre le no-code. D'un côté, les équipes métier ressentent une liberté inédite : elles peuvent enfin construire les outils dont elles ont besoin sans attendre six mois qu'un ticket IT remonte en priorité. De l'autre, la direction des systèmes d'information observe avec inquiétude cette prolifération d'applications, d'automatisations et de bases de données qui échappent à son contrôle traditionnel. Cette tension n'est pas un bug du système. C'est le symptôme d'une transformation profonde qui nécessite un nouveau contrat de confiance entre ceux qui construisent et ceux qui gouvernent.

Dans ce chapitre, nous allons explorer comment transformer cette tension en collaboration productive. La gouvernance no-code n'est pas une question de verrous et de restrictions. C'est l'art de créer un cadre qui encourage l'innovation tout en protégeant l'organisation contre les risques systémiques. Nous verrons comment le shadow IT, longtemps considéré comme une menace, peut devenir un indicateur précieux des besoins non satisfaits. Nous construirons ensemble les mécanismes qui permettent de savoir quelles applications existent, qui les possède, et comment elles traitent les données. Enfin, nous définirons les indicateurs qui vous permettront de mesurer la maturité no-code de votre organisation et d'identifier les prochaines étapes de votre transformation.

23.1. Shadow IT : risque ou opportunité ? Les deux, selon la gouvernance

Le shadow IT désigne l'ensemble des systèmes informatiques, applications et processus mis en place par les employés sans l'approbation explicite de la DSI. Pendant des décennies, ce phénomène a été perçu comme un problème à éradiquer. Les directions informatiques multipliaient les politiques restrictives, bloquaient l'installation de logiciels non autorisés, et tentaient de centraliser toutes

© 2026 Oussama Ghorbel

les initiatives technologiques. Cette approche avait sa logique : comment garantir la sécurité, la conformité et la maintenabilité d'un parc applicatif quand personne ne sait ce qui tourne vraiment dans l'organisation ?

Le no-code a radicalement changé la donne. Aujourd'hui, un responsable marketing peut créer en quelques heures une application de gestion de campagnes qui remplace avantageusement le tableur Excel partagé par email depuis trois ans. Un chef de projet peut automatiser le routage des demandes de validation sans écrire une ligne de code. Ces initiatives ne sont pas du shadow IT malveillant. Ce sont des réponses pragmatiques à des besoins réels que l'IT traditionnel n'avait pas la capacité de traiter avec la rapidité requise.

Mais le shadow IT no-code porte aussi des risques bien réels. On peut observer des organisations où des dizaines de workflows Make tournent en production, connectés à des APIs critiques, sans aucune documentation ni supervision. Quand le collaborateur qui les a créés quitte l'entreprise, personne ne sait comment les maintenir. On voit régulièrement des bases Airtable contenant des données clients sensibles partagées avec des permissions trop larges, exposant l'organisation à des violations de conformité. On rencontre souvent des équipes qui ont construit des applications redondantes, chacune résolvant le même problème avec des approches incompatibles, créant des silos de données impossibles à réconcilier.

La question n'est donc pas d'éliminer le shadow IT. C'est de le rendre visible, compréhensible et gouvernable. Une organisation mature ne cherche pas à empêcher les initiatives bottom-up. Elle crée les conditions pour que ces initiatives émergent dans un cadre qui protège à la fois les individus et l'organisation. Cela commence par reconnaître que le shadow IT est souvent le symptôme d'un besoin légitime mal servi par les canaux officiels. Plutôt que de punir, il faut comprendre : pourquoi cette équipe a-t-elle ressenti le besoin de construire son propre outil ? Quelle friction dans le processus officiel a poussé à cette initiative ? Quelles données sont impliquées, et quels risques cela représente-t-il ?

Cette posture d'écoute et d'accompagnement transforme le shadow IT d'une menace en un laboratoire d'innovation. Les applications construites dans l'ombre deviennent des prototypes qui, une fois validés et sécurisés, peuvent être intégrés au patrimoine applicatif officiel. Les

© 2026 Oussama Ghorbel

makers qui ont pris l'initiative deviennent des ambassadeurs du no-code, capables de former leurs collègues et de partager les bonnes pratiques. La gouvernance ne tue pas l'innovation. Elle lui donne les moyens de passer à l'échelle.

23.2. Catalogue d'apps internes : qui possède quoi

La première étape vers une gouvernance efficace consiste à savoir ce qui existe. Cela semble évident, mais c'est souvent le point faible des organisations qui ont adopté le no-code de manière organique. Les applications se multiplient dans différents services, sur différentes plateformes, avec des niveaux de maturité et de documentation très variables. Sans inventaire clair, impossible de prendre des décisions éclairées sur la consolidation, la sécurisation ou l'évolution du parc applicatif.

Un catalogue d'applications no-code doit répondre à cinq questions fondamentales pour chaque application référencée. Premièrement, quelle est sa finalité ? Quel problème métier résout-elle, et pour quels utilisateurs ? Deuxièmement, qui en est responsable ? Pas seulement qui l'a construite, mais qui est garant de son fonctionnement, de sa maintenance et de son évolution dans le temps. Troisièmement, sur quelle stack technique repose-t-elle ? Quelles plateformes, quelles intégrations, quelles sources de données ? Quatrièmement, quel est son niveau de criticité ? Si elle tombe en panne pendant 24 heures, quel est l'impact sur l'activité ? Cinquièmement, quel est son cycle de vie prévu ? Est-ce un prototype temporaire, un outil de transition, ou une solution destinée à durer plusieurs années ?

La construction de ce catalogue peut sembler fastidieuse, mais elle devient rapidement un exercice collectif valorisant. Organisez des sessions de recensement par département, où les makers présentent leurs créations à leurs pairs et à la DSI. Ces moments de partage ont une double vertu : ils permettent de cartographier l'existant, mais aussi de créer une communauté de pratique où les bonnes idées circulent. Vous découvrirez souvent que trois équipes différentes ont construit des solutions similaires pour des problèmes voisins, ouvrant la porte à des mutualisations bénéfiques.

© 2026 Oussama Ghorbel

Le catalogue doit être vivant, pas figé. Intégrez-le dans vos processus existants : toute nouvelle application no-code doit y être référencée avant sa mise en production, toute modification significative doit être documentée, toute application abandonnée doit être archivée proprement. Certaines organisations utilisent Notion ou Airtable pour gérer ce catalogue, créant ainsi une méta-application no-code qui supervise toutes les autres. D'autres préfèrent des solutions plus formelles intégrées à leur portail IT. L'important n'est pas l'outil, mais la discipline de maintenance.

Chaque application du catalogue doit avoir un propriétaire clairement identifié. Cette notion de ownership est cruciale. Le propriétaire n'est pas nécessairement le développeur original. C'est la personne ou l'équipe qui assume la responsabilité de l'application face aux utilisateurs et à l'organisation. Quand un bug survient, c'est vers le propriétaire que les utilisateurs se tournent. Quand une évolution est nécessaire, c'est le propriétaire qui priorise et valide. Quand le développeur original quitte l'organisation, le propriétaire garantit la continuité de service. Sans ownership clair, les applications orphelines deviennent des dettes techniques qui s'accumulent silencieusement jusqu'à ce qu'une crise les révèle.

23.3. Classification des données : public, interne, confidentiel, sensible

Les données sont le carburant des applications no-code. Chaque formulaire collecte des informations, chaque base stocke des enregistrements, chaque automatisation transfère des données d'un système à l'autre. Pourtant, toutes les données n'ont pas la même sensibilité, et toutes les applications ne méritent pas le même niveau de protection. Une application de gestion des menus de la cantine n'a pas les mêmes exigences de sécurité qu'un CRM contenant les coordonnées bancaires des clients. La classification des données fournit le langage commun pour calibrer la gouvernance en fonction du risque réel.

Une classification efficace repose sur quatre niveaux distincts. Les données publiques sont celles que l'organisation accepte de partager librement avec l'extérieur : informations générales sur l'entreprise, communiqués de presse, catalogues produits. Elles peuvent être stockées

© 2026 Oussama Ghorbel

et traitées avec des niveaux de sécurité minimaux. Les données internes sont destinées à un usage strictement organisationnel, mais ne présentent pas de risque majeur en cas de divulgation : annuaires d'employés, procédures opérationnelles, planning d'équipe. Elles nécessitent un contrôle d'accès basique, mais ne justifient pas de mesures de protection extrêmes.

Les données confidentielles sont celles dont la divulgation pourrait causer un préjudice significatif à l'organisation ou à ses partenaires : stratégies commerciales, données financières non publiées, résultats de recherche et développement. Elles exigent des contrôles d'accès stricts, du chiffrement, et une traçabilité des consultations. Enfin, les données sensibles sont celles dont la compromission entraînerait des conséquences graves, souvent régulées par la loi : données personnelles identifiantes, informations de santé, données de paiement, secrets industriels critiques. Elles imposent le respect de cadres réglementaires spécifiques comme le RGPD, avec toutes les obligations de consentement, de minimisation et de droit à l'effacement que cela implique.

Cette classification doit être appliquée systématiquement à chaque application no-code. Lors de la conception, le maker identifie les types de données que l'application manipulera. Cette identification détermine automatiquement les exigences de sécurité : authentification renforcée, chiffrement au repos et en transit, logs d'audit, rétention limitée, droits d'accès granulaires. Une application qui traite des données sensibles ne peut pas être hébergée sur une plateforme qui ne garantit pas la conformité réglementaire. Une application qui ne manipule que des données internes peut bénéficier d'un processus d'approbation allégé.

La classification doit également guider les décisions d'intégration. Quand une automatisation connecte deux systèmes, elle crée un flux de données entre eux. Si l'un manipule des données sensibles et l'autre des données publiques, le niveau de protection doit être aligné sur le plus élevé des deux. Cette règle simple évite les fuites accidentelles où des données confidentielles se retrouvent exposées dans un système mal configuré. Elle impose aussi une réflexion en amont sur la nécessité réelle de ces intégrations : a-t-on vraiment besoin de transférer ces données sensibles vers cet outil tiers, ou peut-on atteindre le même

© 2026 Oussama Ghorbel

objectif avec des données anonymisées ou agrégées ?

La sensibilisation à la classification des données est un enjeu de formation continue. Les makers métier n'ont pas toujours conscience des implications juridiques et sécuritaires de leurs choix techniques. Un workshop de deux heures sur le RGPD et la classification des données peut prévenir des mois de problèmes de conformité. L'objectif n'est pas de transformer chaque citoyen développeur en expert juridique, mais de lui donner les réflexes de base : avant de créer un champ dans une base de données, se demander si cette information est vraiment nécessaire ; avant de partager une vue, vérifier qui y aura accès ; avant de connecter une API, comprendre quelles données transitent et où elles sont stockées.

23.4. Process d'approbation light pour nouvelles apps

L'un des paradoxes du no-code est qu'il accélère la création d'applications, mais que les processus d'approbation traditionnels n'ont pas suivi cette accélération. Demander à un maker de soumettre un cahier des charges de cinquante pages et d'attendre trois mois qu'un comité de validation se réunit, c'est tuer dans l'œuf l'agilité que le no-code promet. Mais laisser chaque initiative se d ployer sans aucun contrôle, c'est s'exposer à la prolifération anarchique que nous avons évoquée. La solution réside dans un processus d'approbation proportionné au risque, suffisamment rigoureux pour protéger l'organisation, suffisamment léger pour ne pas décourager l'innovation.

Un processus light repose sur trois principes. Premièrement, l'auto-évaluation avant la soumission. Le maker remplit un questionnaire structuré qui l'oblige à réfléchir aux aspects critiques de son application : quelles données sont manipulées, qui sont les utilisateurs finaux, quel est l'impact en cas d'indisponibilité, quelles intégrations externes sont nécessaires. Ce questionnaire n'est pas une formalité administrative. C'est un outil pédagogique qui structure la pensée et fait émerger les points d'attention avant qu'ils ne deviennent des problèmes. Deuxièmement, la validation par seuil de criticité. Les applications à faible risque, celles qui ne manipulent que des données publiques ou internes et dont l'indisponibilité n'a pas d'impact opérationnel majeur, peuvent être déployées après une simple notification à l'équipe de

© 2026 Oussama Ghorbel

gouvernance. Les applications à risque modéré nécessitent une revue rapide par un référent no-code qui vérifie la conformité aux standards. Les applications à risque élevé, celles qui traitent des données sensibles ou supportent des processus critiques, passent par une validation formelle impliquant la DSI et éventuellement les équipes juridiques et conformité.

Troisièmement, la documentation minimale viable. Chaque application doit être accompagnée d'une fiche descriptive concise qui précise son objectif, son propriétaire, ses dépendances techniques, et ses modalités de maintenance. Cette fiche ne doit pas excéder une page. Elle sert de contrat entre le maker et l'organisation : en échange de l'autorisation de déployer, le maker s'engage à maintenir cette documentation à jour et à alerter en cas de changement significatif. Cette approche pragmatique reconnaît que la documentation exhaustive est souvent un vœu pieux. Mieux vaut une page maintenue que dix pages obsolètes.

Le processus d'approbation doit inclure une dimension temporelle. Toutes les applications ne sont pas destinées à durer. Certaines sont des prototypes pour valider une hypothèse, des outils temporaires pour gérer un événement ponctuel, ou des solutions de transition en attendant un système définitif. Pour ces applications éphémères, le processus peut être encore plus allégé, à condition qu'une date de péremption soit fixée dès le départ. À l'échéance, l'application est soit archivée, soit réévaluée pour une éventuelle pérennisation. Cette discipline évite l'accumulation de solutions temporaires qui finissent par devenir permanentes par inertie.

L'approbation ne se limite pas au déploiement initial. Elle doit aussi couvrir les évolutions significatives. Ajouter un nouveau champ dans un formulaire est anodin. Connecter l'application à une nouvelle API de paiement ou exporter des données vers un partenaire externe est un changement matériel qui nécessite une nouvelle évaluation. Définir ce qui constitue un changement significatif est un exercice de jugement qui dépend du contexte de chaque organisation. L'important est que les makers sachent quand ils peuvent agir de manière autonome et quand ils doivent solliciter une validation.

© 2026 Oussama Ghorbel

23.5. Partenariat IT / métier : CoE, ambassadeurs, formations

La gouvernance no-code ne peut pas être imposée d'en haut par une DSI qui dicterait ses règles à des équipes métier récalcitrantes. Elle ne peut pas non plus émerger spontanément d'initiatives dispersées sans coordination. Elle nécessite un partenariat structuré où chaque partie apporte son expertise : les métiers connaissent leurs besoins et leurs contraintes opérationnelles, l'IT maîtrise les enjeux de sécurité, d'intégration et de maintenabilité à long terme. Ce partenariat s'incarne dans plusieurs mécanismes complémentaires.

Le Center of Excellence, ou CoE, est l'instance qui incarne cette collaboration. Il ne s'agit pas d'un comité bureaucratique qui se réunit une fois par trimestre pour valider des roadmaps. C'est une équipe opérationnelle, composée de représentants de la DSI et de makers métier reconnus pour leur expertise, qui travaille au quotidien pour faciliter l'adoption du no-code dans de bonnes conditions. Le CoE a plusieurs missions. Il définit et fait évoluer les standards techniques : quelles plateformes sont approuvées pour quels usages, quelles bonnes pratiques de développement doivent être respectées, quels patterns d'architecture sont recommandés. Il assure un rôle de support de niveau deux : quand un maker rencontre un problème technique complexe, il peut solliciter l'expertise du CoE plutôt que de rester bloqué. Il organise la veille technologique et évalue les nouvelles plateformes et fonctionnalités pour enrichir continuellement la boîte à outils de l'organisation. Enfin, il capitalise les retours d'expérience et les apprentissages pour éviter que les mêmes erreurs se répètent dans différents services.

Les ambassadeurs no-code sont le prolongement terrain du CoE. Ce sont des makers expérimentés, identifiés dans chaque département, qui servent de relais entre la gouvernance centrale et les équipes opérationnelles. Ils ne sont pas nécessairement des experts techniques de haut niveau, mais ils incarnent les bonnes pratiques et peuvent guider leurs collègues dans leurs premières réalisations. Quand un collaborateur veut créer sa première application, il peut se tourner vers l'ambassadeur de son service pour obtenir des conseils concrets adaptés à son contexte métier. Ce maillage humain est essentiel car la gouvernance ne se décrète pas uniquement par des politiques et des procédures. Elle se

© 2026 Oussama Ghorbel

diffuse par l'exemple, par le mentoring informel, par les échanges entre pairs qui se font confiance.

La formation est le troisième pilier du partenariat IT/métier. Elle doit être conçue comme un parcours progressif plutôt que comme un événement ponctuel. Le premier niveau s'adresse à tous les collaborateurs et couvre les fondamentaux : qu'est-ce que le no-code, quels sont les cas d'usage typiques, comment identifier une opportunité d'automatisation, quelles sont les règles de base de la gouvernance. Ce niveau peut être délivré sous forme de modules e-learning courts, complétés par des ateliers pratiques où les participants construisent leur première application simple. Le deuxième niveau cible les makers qui veulent aller plus loin et approfondir leurs compétences techniques : modélisation de données avancée, conception de workflows complexes, intégrations API, gestion des erreurs. Ce niveau combine formations en présentiel, projets guidés, et certification interne qui reconnaît officiellement les compétences acquises. Le troisième niveau s'adresse aux makers les plus avancés et aux membres du CoE : architecture applicative, sécurité avancée, optimisation des performances, passage à l'échelle. Ce niveau fait souvent appel à des formations externes spécialisées et à des conférences qui permettent de rester à la pointe des évolutions du secteur.

Ce partenariat IT/métier repose sur un changement culturel profond. La DSI doit accepter de passer d'un rôle de gatekeeper qui contrôle et filtre les demandes à un rôle d'enabler qui facilite et accélère les initiatives métier. Cela ne signifie pas abdiquer toute responsabilité en matière de sécurité et de conformité. Cela signifie reconnaître que la centralisation absolue est impossible dans un monde où les outils no-code permettent à chacun de construire ses propres solutions. Les équipes métier, de leur côté, doivent accepter que la liberté créative s'accompagne de responsabilités en matière de documentation, de maintenance et de respect des standards. Ce contrat de confiance mutuelle est la condition sine qua non d'une adoption durable du no-code à l'échelle de l'organisation.

23.6. Indicateurs de maturité no-code de l'organisation

© 2026 Oussama Ghorbel

Comment savoir où en est votre organisation dans son parcours no-code ? Comment identifier les forces sur lesquelles capitaliser et les faiblesses qu'il faut adresser en priorité ? Les indicateurs de maturité fournissent une boussole pour naviguer dans cette transformation complexe. Ils ne s'agit pas de metrics vanity qui flattent l'ego sans informer la décision, mais de mesures actionnables qui reflètent la réalité de l'adoption et de la gouvernance.

Dix dimensions clés permettent d'évaluer la maturité no-code d'une organisation. La première est l'adoption : combien de collaborateurs utilisent activement des outils no-code, et dans quelle proportion des départements sont-ils représentés ? Une adoption concentrée dans un seul service indique un phénomène isolé, tandis qu'une adoption transversale suggère une dynamique organisationnelle. La deuxième dimension est la diversité des usages : les applications se limitent-elles à des automatisations simples, ou couvrent-elles des cas d'usage variés incluant des applications complètes, des portails clients, des tableaux de bord décisionnels ? La troisième dimension est la qualité technique : les applications respectent-elles les standards de conception, sont-elles documentées, font-elles l'objet de tests et de revues de code ?

La quatrième dimension est la gouvernance : existe-t-il un catalogue à jour des applications, un processus d'approbation clair, une classification des données appliquée systématiquement ? La cinquième dimension est la sécurité : les applications respectent-elles les politiques de sécurité, les accès sont-ils gérés correctement, les données sensibles sont-elles protégées ? La sixième dimension est l'intégration : les applications no-code s'insèrent-elles harmonieusement dans l'écosystème IT existant, ou créent-elles des silos isolés ? La septième dimension est la maintenance : qui assure le support des applications, comment sont gérées les évolutions et les corrections de bugs, existe-t-il une stratégie de gestion de la dette technique ?

La huitième dimension est la formation : quel pourcentage des collaborateurs a suivi une formation no-code, existe-t-il un parcours de montée en compétences structuré, les ambassadeurs sont-ils actifs et reconnus ? La neuvième dimension est l'innovation : les équipes utilisent-elles le no-code pour expérimenter de nouvelles idées, existe-t-il un mécanisme pour transformer les prototypes réussis en

© 2026 Oussama Ghorbel

solutions pérennes, l'organisation capitalise-t-elle sur les apprentissages ? La dixième dimension est la culture : le no-code est-il perçu comme un outil légitime ou comme un jouet marginal, les makers sont-ils valorisés pour leurs contributions, existe-t-il une communauté active qui partage et s'entraide ?

Chacune de ces dimensions peut être évaluée sur une échelle de un à cinq. Le niveau un correspond à l'absence totale de pratique ou à des initiatives purement individuelles sans coordination. Le niveau deux indique des initiatives isolées mais sans cadre commun. Le niveau trois marque l'émergence de standards et de processus, mais avec une application inégale selon les services. Le niveau quatre reflète une adoption large avec une gouvernance efficace et des pratiques matures. Le niveau cinq représente l'excellence : le no-code est pleinement intégré dans la stratégie IT, la gouvernance est proactive plutôt que réactive, et l'organisation figure parmi les références du secteur.

Ces indicateurs doivent être mesurés régulièrement, idéalement tous les six mois, pour suivre la progression et identifier les régressions. La mesure ne doit pas être perçue comme un outil de contrôle punitif, mais comme un diagnostic partagé qui alimente la réflexion stratégique. Un score faible dans une dimension n'est pas un échec, c'est une opportunité d'amélioration ciblée. Un score élevé dans toutes les dimensions ne signifie pas que l'organisation peut se reposer sur ses lauriers, car l'écosystème no-code évolue rapidement et les standards d'aujourd'hui seront obsolètes demain.

L'évaluation de maturité est aussi un outil de communication envers la direction générale. Elle permet de traduire en termes mesurables les progrès accomplis et les investissements nécessaires pour franchir les prochaines étapes. Elle fournit des arguments concrets pour défendre le budget formation, justifier l'acquisition de nouvelles plateformes, ou demander le renforcement de l'équipe CoE. Surtout, elle ancre le no-code dans une logique de transformation continue plutôt que dans l'illusion d'un projet avec une date de fin définitive.

Exercices pratiques

© 2026 Oussama Ghorbel

Exercice 1 : Construire le catalogue d'apps no-code de votre organisation

Cet exercice vous invite à réaliser un inventaire complet des applications no-code existantes dans votre organisation, qu'elles soient officielles ou issues du shadow IT. L'objectif est double : cartographier l'existant pour mieux le gouverner, et créer un moment de partage qui renforce la communauté des makers.

Commencez par identifier les sources d'information. Listez toutes les plateformes no-code utilisées dans votre organisation : Make, Zapier, n8n, Airtable, Notion, Bubble, Glide, Power Apps, et autres. Pour chaque plateforme, identifiez les comptes organisationnels et les comptes personnels utilisés à des fins professionnelles. Sollicitez les managers de chaque département pour qu'ils recensent les outils utilisés par leurs équipes, y compris ceux qui n'ont pas été validés officiellement.

Créez un template de fiche applicative dans l'outil de votre choix. Chaque fiche doit contenir les informations suivantes : nom de l'application, description concise de sa finalité, département propriétaire, personne responsable, plateforme technique utilisée, liste des intégrations et des sources de données, classification des données manipulées selon les quatre niveaux définis dans ce chapitre, nombre d'utilisateurs actifs, date de mise en production, statut actuel, et date de prochaine revue prévue.

Organisez une session de recensement collectif où les makers présentent leurs applications. Cette session a une valeur pédagogique : elle permet à chacun de découvrir ce qui existe ailleurs dans l'organisation, d'identifier des synergies potentielles, et de partager des astuces techniques. Elle a aussi une valeur politique : elle rend visible le shadow IT sans le stigmatiser, et crée un espace de dialogue constructif entre les makers et la DSI.

À l'issue de cet exercice, vous devriez disposer d'un catalogue initial qui servira de base à votre gouvernance. Ce catalogue n'est pas figé : il évoluera au rythme des créations, des modifications et des retraits d'applications. L'important est d'avoir posé la première pierre et d'avoir instauré la discipline de le maintenir à jour.

© 2026 Oussama Ghorbel

Exercice 2 : Rédiger la politique d'usage acceptable en 2 pages

Cet exercice vous demande de synthétiser les principes de gouvernance évoqués dans ce chapitre en un document concis et actionnable. L'objectif n'est pas de produire un pavé juridique exhaustif que personne ne lira, mais un guide pratique que chaque maker peut consulter rapidement pour savoir ce qui est autorisé, ce qui nécessite une validation, et ce qui est interdit.

Structurez votre document en quatre sections. La première section, intitulée Principes généraux, expose en quelques paragraphes la philosophie de votre gouvernance : pourquoi le no-code est encouragé, quelles responsabilités il implique, et comment l'organisation accompagne les makers. Cette section donne le ton et montre que la gouvernance est là pour faciliter, pas pour entraver.

La deuxième section, intitulée Règles selon la classification des données, présente sous forme de tableau les exigences applicables à chaque niveau de sensibilité. Pour les données publiques et internes, listez les pratiques recommandées et les plateformes autorisées en self-service. Pour les données confidentielles, précisez les validations requises et les mesures de sécurité obligatoires. Pour les données sensibles, détaillez le processus d'approbation formel et les contraintes réglementaires à respecter.

La troisième section, intitulée Responsabilités du maker, liste les engagements que prend tout collaborateur qui déploie une application no-code : documenter son application dans le catalogue, maintenir une documentation technique minimale, assurer le support de premier niveau pour les utilisateurs, alerter en cas de problème de sécurité ou de dysfonctionnement critique, et archiver proprement les applications qui ne sont plus utilisées.

La quatrième section, intitulée Processus et contacts, fournit les liens vers les formulaires de déclaration d'application, les coordonnées des ambassadeurs no-code par département, et l'email du Center of Excellence pour les questions techniques ou les demandes de validation.

© 2026 Oussama Ghorbel

Relisez votre document en vous mettant à la place d'un maker débutant qui veut créer sa première application. Est-ce que ce document lui donne envie de se lancer tout en étant conscient des règles du jeu ? Est-ce que les informations essentielles sont faciles à trouver ? Est-ce que le ton est encourageant plutôt que répressif ? Si la réponse est oui à ces trois questions, vous avez réussi l'exercice.

Exercice 3 : Calculer le score de maturité no-code sur 10 critères

Cet exercice vous invite à réaliser une auto-évaluation honnête de la maturité no-code de votre organisation selon les dix dimensions présentées dans la section 23.6. L'objectif est d'obtenir un diagnostic factuel qui servira de point de départ pour définir votre feuille de route d'amélioration.

Pour chaque dimension, attribuez une note de un à cinq selon les critères suivants. Pour l'adoption, un point signifie qu'aucune utilisation no-code n'est identifiée, trois points indiquent une adoption dans plusieurs départements mais sans coordination, cinq points reflètent une adoption généralisée et transversale. Pour la diversité des usages, un point correspond à des automatisations basiques uniquement, trois points à un mix d'automatisations et d'applications simples, cinq points à des applications complexes couvrant des cas d'usage stratégiques. Pour la qualité technique, évaluez le respect des standards, la documentation, et les pratiques de test.

Pour la gouvernance, notez l'existence et l'efficacité du catalogue d'apps, du processus d'approbation, et de la classification des données. Pour la sécurité, évaluez la conformité aux politiques, la gestion des accès, et la protection des données sensibles. Pour l'intégration, mesurez la connexion harmonieuse avec l'écosystème IT existant versus la création de silos isolés. Pour la maintenance, évaluez la clarté des responsabilités de support et la gestion de la dette technique.

Pour la formation, notez la couverture des programmes de formation et l'existence d'un parcours de montée en compétences. Pour l'innovation, évaluez la capacité à expérimenter et à transformer les prototypes en solutions pérennes. Pour la culture, mesurez la perception du no-code dans l'organisation et la vitalité de la communauté des

© 2026 Oussama Ghorbel

makers.

Additionnez les dix notes pour obtenir un score global sur cinquante. Un score inférieur à vingt indique une organisation au stade embryonnaire où le no-code existe mais sans cadre ni vision. Un score entre vingt et trente reflète une organisation en phase d'exploration avec des initiatives prometteuses mais une gouvernance encore faible. Un score entre trente et quarante correspond à une organisation en phase de structuration où les fondations sont posées mais l'exécution reste inégale. Un score supérieur à quarante indique une organisation mature où le no-code est pleinement intégré dans la stratégie et les opérations.

Identifiez les trois dimensions où votre score est le plus faible. Ce sont vos priorités d'amélioration pour les six prochains mois. Pour chacune, définissez deux ou trois actions concrètes qui permettront de progresser d'au moins un niveau. Identifiez également les dimensions où votre score est élevé : ce sont vos forces sur lesquelles vous pouvez vous appuyer pour accélérer votre transformation. Partagez ce diagnostic avec votre direction et utilisez-le pour construire un plan d'action réaliste et ambitieux.

© 2026 Oussama Ghorbel

Chapitre 24 – Monétiser, transmettre, évoluer : la vie après le lancement

24. Monétiser, transmettre, évoluer : la vie après le lancement

Lancer une application no-code n'est pas la fin du parcours : c'est le moment où l'objet technique devient un actif. Un actif se monétise, se transmet, se fait grandir — ou se remplace sans regret quand il a rempli son rôle. Ce chapitre s'intéresse à ce qui se passe après le « go-live » : la vente, le métier de freelance, la construction d'une agence, la migration éventuelle vers du code, la veille, et enfin la trajectoire personnelle de celui ou celle qui construit.

24.1. Vendre une app no-code : valorisation, due diligence, transfert

Une application no-code peut se vendre. Cela se fait plus souvent qu'on ne le croit, sur des marketplaces dédiées à la cession de SaaS et de businesses en ligne. La spécificité du no-code n'est pas l'impossibilité de vendre — c'est le déplacement de l'attention de l'acheteur : moins sur le code source, davantage sur les données, les abonnements, la documentation et la reproductibilité des opérations.

Ce que l'acheteur achète vraiment

Un acheteur sérieux n'achète pas une interface jolie. Il achète :

• **Des revenus récurrents vérifiables** : MRR (revenu mensuel récurrent), taux de churn, ancienneté des abonnés, concentration du chiffre d'affaires sur quelques comptes.

• **Un coût d'exploitation maîtrisé** : abonnements aux outils (Bubble, Airtable, Make…), coût des API, temps passé par le propriétaire pour maintenir la machine en marche.

• **Une documentation qui rend le transfert possible** : schéma de données, parcours utilisateur, automatisations cartographiées, mots de passe et procédures d'accès.

© 2026 Oussama Ghorbel

• **Un marché et un avantage défendables** : une niche comprise, un canal d'acquisition qui fonctionne, une base d'utilisateurs qui reste active.

Le fait que l'app soit construite en no-code n'est ni un atout absolu ni un tueur de deal. C'est un arbitrage : un acheteur moins technique peut reprendre la main plus facilement ; un acheteur très technique peut craindre les plafonds de l'outil. Dans les deux cas, la réponse est la même : une documentation irréprochable et des chiffres transparents.

Valoriser sans se mentir

Les méthodes de valorisation appliquées aux SaaS reposent généralement sur des multiples du chiffre d'affaires ou de l'EBITDA (bénéfice avant intérêts, impôts, dépréciations et amortissements). Gardez à l'esprit que toute cession d'actif numérique comporte des enjeux fiscaux et juridiques nécessitant un accompagnement professionnel. Pour un petit SaaS no-code, on observe souvent des fourchettes prudentes, surtout si le produit est jeune, dépendant fortement du propriétaire, ou construit sur une stack dont l'acheteur se méfie. Plutôt que de chercher un multiple « idéal », concentrez-vous sur les leviers que vous contrôlez :

1. **Réduire la dépendance au fondateur** : si tout passe par votre boîte mail personnelle, la valeur chute. Déléguez le support, industrialisez les onboarding, documentez.
2. **Stabiliser les revenus** : un MRR qui monte doucement mais régulièrement vaut mieux que des pics promotionnels.
3. **Baisser le ratio temps/revenu** : un SaaS qui tourne à 3 heures par semaine se vend mieux qu'un SaaS exigeant 30 heures, à revenu égal.
4. **Sécuriser la stack** : lister chaque outil, chaque abonnement, chaque dépendance API, avec les coûts mensuels. Un acheteur veut savoir ce qu'il reprendra.

Je recommande de préparer une **fiche de vente** avant même d'en avoir besoin. Elle force la rigueur et sert aussi de diagnostic : si vous n'arrivez pas à remplir proprement les cases MRR, churn et documentation, c'est que votre actif n'est pas encore prêt à être transmis

© 2026 Oussama Ghorbel

— indépendamment de toute vente.

La due diligence côté vendeur

La due diligence n'est pas seulement ce que l'acheteur fait sur vous. C'est aussi ce que vous devez vérifier avant de signer quoi que ce soit :

- **Propriété et licences** : qui possède le compte Bubble, le compte Stripe, le nom de domaine, les assets graphiques, les contenus ? Les freelances qui ont contribué ont-ils cédé leurs droits par écrit ?
- **Données personnelles** : êtes-vous en règle avec le RGPD dans l'usage que vous faites des données de vos utilisateurs ? La vente transfère-t-elle des bases de données clients, et sur quel fondement juridique ?
- **Conditions des outils** : les CGU de votre outil no-code autorisent-ils la revente du compte, ou faut-il migrer vers un compte nouvellement créé par l'acheteur ? C'est une question pratique, souvent négligie, qui peut rallonger le calendrier.
- **Engagements en cours** : contrats clients, abonnements annuels, dettes éventuelles, litiges ouverts.

Le transfert opérationnel

Un bon transfert se prépare sur plusieurs semaines, pas sur un week-end. Un schéma simple :

1. **Phase 0 (avant annonce)** : documentation à jour, chiffres stabilisés, accès préparés.
2. **Phase 1 (signation et dépôt de garantie)** : souvent un dépôt de garantie et des paiements échelonnés sur 6 à 24 mois, liés à une période de transition.
3. **Phase 2 (transfert d'accès)** : comptes, domaines, outils, procédures. Idéalement avec une période d'accompagnement contractuellement définie (heures, durée, canaux).
4. **Phase 3 (suivi)** : vous restez joignable pour les questions prévues au contrat, sans rester le véritable opérateur de fait.

L'erreur classique est de vendre puis de disparaître en espérant que « ça tourne ». La seconde erreur symétrique est de rester tellement présent

© 2026 Oussama Ghorbel

que l'acheteur n'acquiert jamais l'autonomie — et finit par revenir vers vous pour tout, hors contrat.

24.2. Freelance no-code : se positionner, facturer, livrer

Pour beaucoup de makers, la première monétisation n'est pas un SaaS en vente, mais une expertise vendue en temps. Le freelance no-code consiste à construire des apps, des bases de données et des automatisations pour des clients, contre rémunération. C'est un métier à part entière, avec ses codes.

Se positionner : la niche avant l'outil

« Je fais du Bubble et du Make » n'est pas une positionnement : c'est un CV. Les clients achètent des résultats dans un contexte. Une positionnement claire combine :

- **Un secteur** : cabinets d'avocats, e-commerce Shopify, cabinets de recrutement, promoteurs immobiliers, associations…
- **Un type de problème** : mise en place d'un CRM sur mesure, automatisation du devis->facture, portail client, outil interne de suivi de production…
- **Un niveau d'enjeu** : prototype en 2 semaines pour valider une idée, ou refonte d'un processus critique avec accompagnement au changement.

Plus la position est précise, plus les recommandations circulent dans une même communauté, et plus vous évitez le piège du « client qui veut une app comme [gros acteur] pour 10 % du budget ».

Trois façons de facturer

Le temps passé est un indicateur, rarement le meilleur objet de facturation pour le client. Trois modèles classiques :

1. **Forfait au périmètre** : un chiffrage pour un livrable défini (ex. : MVP avec inscription, CRUD, tableau de bord, 2 automatisations). Avantage : prévisibilité pour le client. Risque : périmètre mal cadré -> dérive. Parade : livrables listés, nombre de révisions, ce qui est hors périmètre.

© 2026 Oussama Ghorbel

2. **Rétainer mensuel** : un volume d'heures ou de demandes par mois pour faire évoluer l'app. Idéal après une première livraison, quand le besoin est continu.

3. **Forfait + succès** (plus rare, à manier avec prudence) : base fixe complétée par des objectifs contractualisés. À rédiger très précisément pour éviter les litiges sur ce qui constitue un « succès ».

Dans tous les cas, la facturation doit couvrir ce que les clients oublient souvent : l'administration des accès, les licences d'outils (parfois refacturées ou incluses), la formation des équipes, la documentation, et le support de stabilisation post-lancement.

Livrer comme un pro

La qualité perçue du freelance no-code ne se joue pas uniquement sur la beauté de l'app. Elle se joue sur la **clarté du processus** :

• Un cadrage initial (parfois payant) qui produit une spec visuelle, un schéma de données, une liste d'automatisations.

• Une démo intermédiaire avant la phase de finition — pour corriger la trajectoire tôt.

• Une recette client formalisée : cas de test, signatures de validation.

• Une documentation d'exploitation : comment créer un utilisateur, modifier un tarif, relancer un envoi d'email raté, changer une clé API.

• Une passation : une session de formation filmée vaut souvent mieux qu'un manuel de 40 pages jamais lu.

La confiance se construit sur la prévisibilité : dates tenues, surprises annoncées tôt, périmètre défendu avec respect. Un freelance no-code qui sait dire « cela n'est pas dans le forfait, voici le chiffrage de l'option » se respecte plus que celui qui accumule les extras dans l'espoir d'être généreux — et finit amer.

24.3. Agence no-code : recruter, former, industrialiser

À un certain stade, la demande dépasse la capacité d'une seule personne. L'agence commence alors, même si le mot fait parfois peur. Il désigne ici une structure capable de livrer plusieurs projets en parallèle,

© 2026 Oussama Ghorbel

avec des standards communs.

Passer de l'artisan à l'organisateur

Le passage soudain est douloureux si on le prend comme une simple multiplication de soi. Il faut accepter de perdre la main sur certains détails pour gagner la capacité de répéter une qualité. Concrètement, cela veut dire :

- **Des composants réutilisables** : bibliothèque de blocs d'interface, scénarios Make ou n8n versionnés, gabarits de base de données Airtable ou Notion, checklists de qualité.
- **Des processus nommés** : cadrage -> prototype -> build -> recette -> formation -> support. Chaque étape a un livrable, un responsable et un critère de sortie.
- **Une documentation vivante** : le standard n'existe que s'il est écrit et utilisé. La documentation interne est votre vrai outil d'industrialisation.

Recruter et former

Les profils utiles ne sont pas toujours déjà « no-code ». On recrute souvent des personnes avec un bon sens métier, une appétence pour les outils numériques, une exigence de clarté — puis on les forme à votre stack. Un parcours d'intégration typique :

1. Semaine 1 : immersion produit, lecture des anciennes specs, petit exercice de modélisation de données.
2. Semaines 2–3 : co-construction d'une petite fonctionnalité avec un senior, sur un projet réel.
3. Semaine 4 : livraison autonome d'un module cadré, revue de code… de configuration, en l'occurrence : nommage, structures de données, scénarios, droits.
4. Ensuite : un projet complet encadré, avant d'ouvrir l'autonomie commerciale.

Le mentorat est plus efficace que les formations génériques. Un binôme senior/junior sur chaque projet pendant les premières semaines évite de réparer plus tard.

© 2026 Oussama Ghorbel

Industrialiser sans standardiser à mort

L'industrialisation a un coût : elle peut rigidifier la créativité et faire passer les clients pour des numéros. Conservez des marges de manœuvre explicites (« template de départ » + module spécifique) et gardez un rituel de retour d'expérience après chaque projet. L'agence apprenante est celle qui met à jour ses gabarits, pas celle qui les considère intangibles.

Un point trop souvent négligé : la **gestion des dépendances et des accès**. Comptes outils, rôles, backups, noms de domaine, facturation des abonnements client — dans une agence, c'est un risque opérationnel et parfois juridique. Un registre simple des accès (qui a accès à quoi, sous quel compte, avec quel mode de réinitialisation) vous sauvera un jour.

24.4. Quand migrer vers du code pro : signes et méthode

Le no-code n'est pas un serment. C'est un outil de vitesse et d'exécution. Il existe des situations où une réécriture en code devient rationnelle — et d'autres où l'idée de migrer n'est qu'un réflexe culturel.

Signaux qui méritent une réflexion

• **Des performances qui plafonnent** : temps de chargement, gros volumes de données, calculs lourds récurrents, quotas d'API dépassés de façon structurelle.

• **Des besoins fonctionnels hors des capacités de l'outil** : logique temps réel complexe, algorithmes propriétaires, intégrations très spécifiques, apps natives performantes.

• **Un modèle économique qui le justifie** : le coût mensuel des outils + le temps de contournement dépasse, sur 12 à 24 mois, le budget raisonnable d'une équipe de développement — sans compter les opportunités manquées.

• **Un enjeu de dépendance stratégique** : forte exposition aux changements de pricing, de conditions ou de capacités d'un éditeur unique, sans alternative.

À l'inverse, ne migrez pas parce qu'un investisseur a dit « le code, c'est plus sérieux », parce qu'un développeur a méprisé votre stack, ou parce que vous vous ennuyez. Le code pro apporte de la flexibilité, et

© 2026 Oussama Ghorbel

aussi du coût permanent de maintenance, des recrutements plus difficiles et des délais parfois plus longs pour les petites évolutions.

La méthode : strangler, pas jeter

Le pire scénario est la réécriture totale « big bang ». Le pattern raisonnables est celui du **monolithe étranglé** (strangler fig pattern), adapté au contexte :

1. **Cartographier l'existant** : fonctionnalités utilisées réellement (analytics à l'appui), automatisations, données, intégrations.
2. **Isoler les frontières** : quels modules sont stables et peu changés ? Quels modules sont au cœur de la valeur différenciante ?
3. **Extraire d'abord les pièces douloureuses** : par exemple, remplacer un scénario fragile par un microservice, ou migrer la base de données vers un vrai backend (Supabase, Xano…) tout en gardant l'interface no-code temporairement.
4. **Basculer progressivement** avec des objectifs de non-régression mesurables : mêmes parcours critiques, mêmes temps de réponse cibles, mêmes droits d'accès.
5. **Décommissionner** une fois la migration stable — en gardant un plan de retour arrière pendant une période définie.

Pendant toute la transition, votre documentation no-code devient la spécification implicite du futur système : elle est souvent plus claire que ce que l'équipe « legacy » aurait produit spontanément.

24.5. Veille technologique : où suivre l'évolution du marché

Le marché no-code évolue vite. Des outils changent de positionnement, des fonctionnalités critiques apparaissent d'un trimestre à l'autre, des tarifs bougent, des acquisitions reconfigurent l'écosystème. Une veille légère mais régulière vaut mieux que des marathons d'essais sans lendemain.

Des sources utiles

• **Les blogs et newsletters des éditeurs** : souvent meilleurs pour comprendre les nouveautés que les agrégateurs. Lisez aussi les notes de version : c'est là que se cachent les capacités qui vont vous faire

© 2026 Oussama Ghorbel

gagner du temps.

• **Les communautés** : forums officiels, groupes Slack ou Discord, serveurs n8n, communautés Bubble, Airtable Universe, etc. La valeur vient des retours d'expérience de production, pas seulement des démos.

• **Les comparatifs honnêtes** : préférez les sources qui documentent les limites, pas uniquement les « top 10 » sponsorisés.

• **Les veilles économiques** : financements, acquisitions, pricing — pour anticiper les risques de dépendance.

• **Les retours d'agences et de freelances** : personne n'est mieux placé pour savoir ce que l'outil donne en conditions réelles.

Un rituel simple

Proposez-vous un rythme réaliste, par exemple :

• **Hebdomadaire (30 minutes)** : scan des newsletters et d'un fil communautaire ; noter ce qui touche directement à vos apps en production.

• **Mensuel (2 heures)** : test ciblé d'une nouveauté sur un bac à sable ; mise à jour du registre de vos dépendances.

• **Trimestriel (demi-journée)** : revue de stack : « devons-nous consolider, remplacer, ou laisser tel outil ? » avec critères écrits (coût, fiabilité, capacité à exporter les données, roadmap).

L'objectif n'est pas de tout essayer. C'est de **décider avec des faits**, et de savoir quand une nouveauté mérite une migration — et quand elle n'est qu'un attrape-nouveau.

24.6. Feuille de route personnelle du maker : 6 mois, 2 ans, 5 ans

Les outils et les marchés bougent ; votre trajectoire, vous en êtes plus responsable qu'il n'y paraît. Une feuille de route personnelle ne prédit pas l'avenir : elle donne des critères de décision quand plusieurs portes s'ouvrent.

À 6 mois : crédibilité et répétabilité

© 2026 Oussama Ghorbel

L'objectif n'est pas de devenir riche, mais de devenir **fiable** :

• Une offre claire (produit ou service) avec un périmètre assumé.

• Deux à trois réalisations documentées, même petites, avec résultats mesurés.

• Un processus de livraison éprouvé une fois du début à la fin (cadrage -> build -> formation).

• Une hygiène opérationnelle : comptes, backups, documentation, facturation propre.

Si vous êtes freelance : un ou deux clients récurrents. Si vous êtes maker produit : des premiers retours utilisateurs et un premier indicateur d'usage (activation, rétention, utilisation hebdomadaire).

À 2 ans : actif et indépendance de jugement

À deux ans, vous devez pouvoir choisir parmi plusieurs voies sans paniquer :

• **Soit un portefeuille d'actifs** : un ou plusieurs micro-SaaS ou outils internes monétisés, documentés, avec une charge maîtrisée.

• **Soit une expertise de marché** : une position de freelance ou de petite agence reconnue sur une niche, avec un taux de satisfaction qui génère le bouche-à-oreille.

• **Soit un hybride équilibré** : un produit qui paye les charges, des missions choisies qui paient la trésorerie et nourrissent le réseau.

Les jalons mesurables typiques : un MRR ou un CA récurrent qui couvre vos besoins de base ; une documentation qui permet à quelqu'un d'autre de faire tourner l'essentiel une semaine sans vous ; une veille qui produit au moins deux décisions de stack par an, écrites et argumentées.

À 5 ans : levier et pérennité

À cinq ans, la question n'est plus seulement « que construire ? » mais **« quel levier je veux sur mon temps et mon capital ? »** :

• Votre savoir-faire est-il encodé (processus, templates, équipe) ou tout repose-t-il sur vos mains ?

• Vos revenus sont-ils liés à votre présence systématique, ou existeraient-ils partiellement sans vous ?

© 2026 Oussama Ghorbel

• Avez-vous un plan pour la transmission ou l'évolution d'un actif — vente, recrutement d'un responsable produit, pivot, arrêtAssumeé assumé ?

• Votre santé, vos finances personnelles et vos apprentissages sont-ils traités comme des actifs à part entière ?

Cinq ans est un horizon assez lointain dans le no-code pour que plusieurs plateformes dominantes aient changé de visage. Votre meilleure protection n'est pas l'outil du moment : c'est la capacité à comprendre les données, les processus et les utilisateurs — compétences transférables d'une stack à l'autre.

Synthèse du chapitre

Penser son app comme un actif change la qualité des décisions. On documente plus tôt, on réduit sa dépendance au fondateur, on choisit des outils exportables, on sépare la valeur métier de l'exécution technique. Vendre devient possible, sans mythe ; freelance et agence deviennent des chemins de monétisation assumés ; la migration vers le code cesse d'être un tabou ou une superstition pour devenir un arbitrage ; la veille et la feuille de route gardent le cap quand le marché ondule.

Le lancement n'est pas le sujet. La durée de vie de ce que vous construisez — et la vôtre à côté de lui — l'est davantage.

Exercices

Exercice 1 — Rédiger une fiche de vente de SaaS no-code

Imaginez que vous mettiez en vente un micro-SaaS no-code construit au cours des 18 derniers mois. Rédigez une fiche d'une à deux pages contenant au minimum :

1. **Un pitch de 3 lignes** : pour qui, quel problème, quelle solution.
2. **Des chiffres sur 12 mois** : MRR moyen, MRR actuel, nombre de clients payants, churn mensuel, CA annuel, marge d'exploitation approximative après coûts d'outils.
3. **La stack** : liste des outils et coûts mensuels associés.

© 2026 Oussama Ghorbel

4. **Le temps du propriétaire** : heures par semaine réellement nécessaires, avec détail (support, marketing, maintenance).

5. **Les accès et actifs inclus** : domaine, comptes, assets, documentation, base clients.

6. **Les risques connus** : concentration client, dépendance API, limite technique identifiée.

7. **Une condition de transfert** : période d'accompagnement proposée et modalités d'accès.

Ne cherchez pas à embellir : l'exercice sert à identifier ce qui, dans votre actif actuel, manque encore pour être transmissible.

Exercice 2 — Construire son offre freelance no-code en 3 packages

À partir d'une niche choisie (à noter en une phrase : « j'aide [type de client] à [résultat] grâce à [approche] »), rédigez trois packages :

• **Package Starter** : périmètre minimal, livrable clair, délai, prix, ce qui est inclus et exclu.

• **Package Croissance** : fonctionnalités supplémentaires, automatisations, formation, prix.

• **Package Accompagnement** : retainer mensuel, volume d'heures ou de demandes, délai de réponse, engagements réciproques.

Pour chaque package, ajoutez : le profil de client idéal, le premier jalon mesurable promis, et un argument honnête pour lequel un client devrait choisir ce niveau plutôt que le précédent.

Exercice 3 — Écrire sa feuille de route maker à 2 ans avec jalons mesurables

Partez de votre situation actuelle (une phrase sur votre stack, votre offre ou produit, votre temps disponible par semaine). Sur un horizon de 24 mois, définissez :

• **Un objectif dominant** (ex. : revenu récurrent, spécialisation freelance, vente d'un actif, migration d'une partie de la stack).

• **Quatre jalons trimestriels**, chacun avec : un résultat observable, une échéance, un indicateur chiffré (ex. : « 20 clients payants », « documentation d'exploitation complète pour 100 % des apps en

© 2026 Oussama Ghorbel

production », « MRR de X € après coûts outils »).

• **Trois risques** (personnels ou de marché) et une action de réduction pour chacun.

• **Un critère d'arrêt ou de pivot** : à quel moment accepterez-vous que la trajectoire actuelle ne mérite plus votre énergie, et quelle alternative avez-vous identifiée à l'avance ?

Relisez cette feuille de route tous les six mois. Ce n'est pas un engagement solennel : c'est un outil de décision, qui doit évoluer avec ce que vous apprenez en construisant.

© 2026 Oussama Ghorbel

Conclusion

Conclusion : Le code n'était qu'un détail, la valeur était dans la pensée

Nous y voilà. Vous tenez entre vos mains, ou vous affichez sur votre écran, les dernières lignes d'un ouvrage qui, je l'espère, a profondément modifié votre rapport à la technologie. Lorsque nous avons entamé ce voyage ensemble, la promesse du no-code semblait presque tenir de la magie : bâtir des applications, automatiser des processus entiers et orchestrer des systèmes complexes sans écrire la moindre ligne de code traditionnel. Mais au fil des chapitres, des modélisations de données et des architectures de workflows, vous avez découvert la véritable nature de cette discipline. Le no-code n'est pas une baguette magique qui dispense de la réflexion. C'est un changement de paradigme. C'est le triomphe de la logique métier sur la syntaxe informatique.

Le père John Culkin, grand théoricien des médias, disait de notre relation à la technologie : « Nous façonnons nos outils, puis nos outils nous façonnent. » Pendant des décennies, les langages de programmation ont façonné la manière dont les entreprises pensaient leurs processus. Il fallait tordre le besoin métier pour qu'il entre dans les cases rigides d'une architecture technique coûteuse et lente à déployer. Aujourd'hui, avec les outils visuels, les bases de données relationnelles modernes et les orchestrateurs d'API, c'est l'inverse qui se produit. La technologie s'efface pour épouser la réalité du terrain. Vous n'êtes plus limité par votre capacité à mémoriser une syntaxe, mais uniquement par votre capacité à comprendre, structurer et résoudre un problème.

Les cinq piliers de votre nouvelle expertise

En refermant ce livre, vous n'emportez pas seulement avec vous une liste d'outils à la mode. Les outils naissent, évoluent, fusionnent ou disparaissent. Ce que vous emportez, c'est une méthode, une grille de lecture en cinq piliers qui fera de vous un architecte de solutions, quel que soit l'éditeur de logiciel qui dominera le marché dans cinq ans.

Le premier pilier est la pensée produit. Avant même d'ouvrir Bubble, WeWeb ou Glide, vous avez appris à cartographier le besoin.

© 2026 Oussama Ghorbel

Vous savez désormais qu'une application n'est qu'une réponse à une friction. Penser comme un produit, c'est accepter de tuer ses propres idées sur l'autel de l'utilité réelle. C'est définir des spécifications visuelles, tracer les parcours utilisateurs et comprendre que la valeur ne réside pas dans la complexité de l'outil, mais dans la simplicité de l'expérience qu'il offre.

Le deuxième pilier est la souveraineté de la donnée. C'est le socle invisible de tout l'édifice. Vous avez compris que modéliser ses données sans SQL ne signifie pas les traiter avec légèreté. Qu'elles soient hébergées dans Airtable, Xano, Supabase ou SmartSuite, vos données sont le véritable actif de votre entreprise. Une base de données bien pensée, normalisée et relationnelle est un moteur de croissance ; une base de données conçue comme un simple tableur plat est une bombe à retardement. Vous avez acquis la pensée relationnelle, cette capacité à voir les liens invisibles entre les entités de votre métier.

Le troisième pilier est l'interface comme contrat. Construire des applications visuelles qui tiennent la route exige de la rigueur. Le no-code ne vous dispense pas des règles du design system, de l'accessibilité ou de la gestion des états de chargement et d'erreur. Que vous utilisiez Power Apps pour l'interne ou Toddle pour le grand public, vous avez appris que l'expérience utilisateur (UX) est le reflet du respect que vous portez à ceux qui utiliseront votre création.

Le quatrième pilier est l'orchestration invisible. Les applications ne sont que des vitrines ; les workflows en sont les coulisses. Grâce à Make, Zapier, n8n ou Power Automate, vous avez appris à connecter les silos. Vous maîtrisez les patterns et les anti-patterns de l'automatisation, la gestion des erreurs, les webhooks et les boucles de rétroaction. Vous savez qu'un workflow robuste est celui qui survit à l'imprévu, qui loggue ses échecs et qui alerte avant de bloquer.

Le cinquième pilier est la maturité opérationnelle. Le passage à l'échelle exige de la gouvernance. Travailler en équipe sur du no-code, gérer les droits d'accès, sécuriser les authentifications et collaborer avec la DSI plutôt que de la contourner : voilà ce qui sépare le bricoleur du dimanche de l'architecte d'entreprise. L'industrialisation n'est pas une option, c'est la condition de survie de vos solutions.

© 2026 Oussama Ghorbel

Le grand malentendu : la dette technique visuelle

Il est temps d'adresser un mythe tenace, souvent brandi par les détracteurs du mouvement no-code et low-code. On entend fréquemment que ces outils génèrent inévitablement de la « mauvaise » technologie, des usines à gaz ingérables et une dette technique colossale. Cette critique n'est pas fausse, mais elle est mal dirigée.

La dette technique n'est pas l'apanage du code traditionnel. Un développeur peut écrire des milliers de lignes de Python ou de JavaScript totalement inextricables, sans documentation, sans tests et sans architecture. De la même manière, un maker no-code peut créer un scénario Make de deux cents modules, rempli de filtres redondants, sans gestion d'erreur, consommant des milliers d'opérations pour une tâche qui pourrait être simplifiée. Le « spaghetti code » a trouvé son équivalent visuel : le « spaghetti workflow ».

Ce livre a été conçu pour vous éviter ce piège. La rigueur n'a rien à voir avec la syntaxe ; elle a tout à voir avec la discipline. Tester ses déploiements, versionner ses bases de données, documenter ses logiques métier, implémenter des environnements de staging avant de pousser en production : ces pratiques, héritées du génie logiciel traditionnel, sont absolument vitales en no-code. L'abstraction visuelle rend la création plus rapide, ce qui signifie que vous pouvez construire des erreurs beaucoup plus vite qu'auparavant. La vitesse d'exécution exige une vitesse de vérification équivalente. Vous êtes désormais conscient que la facilité de construction ne doit jamais servir d'excuse à la paresse architecturale.

L'horizon de l'IA : vers la fin du "builder" ?

Au cours de la rédaction de cet ouvrage, une question est revenue fréquemment lors de conférences et d'ateliers : « L'intelligence artificielle générative va-t-elle tuer le no-code ? » C'est une question fascinante, qui mérite que nous nous y attardions pour clore notre réflexion.

L'histoire de l'informatique est une longue suite d'abstractions successives. Nous sommes passés des cartes perforées au langage assembleur, puis au C, à Python, et enfin aux interfaces visuelles du

© 2026 Oussama Ghorbel

no-code. Chaque saut d'abstraction a été accusé de détruire le "vrai" développement. Aujourd'hui, les grands modèles de langage (LLM) et les agents autonomes promettent une nouvelle abstraction : le langage naturel. Demain, vous ne dessinerez peut-être plus des nœuds sur un canevas infini ; vous décrirez votre intention à une IA qui générera l'architecture, le frontend et les connexions API en une fraction de seconde.

Mais cela ne signe pas la fin de l'expert no-code ; cela en redéfinit le rôle. L'IA est exceptionnelle pour répondre au « comment », mais elle est encore aveugle face au « pourquoi ». Elle peut générer une base de données en quelques secondes, mais elle ne sait pas si cette base de données reflète la réalité complexe et nuancée de votre chaîne logistique. Elle peut coder un bouton d'achat, mais elle ne comprend pas la psychologie de votre parcours de conversion.

L'avenir appartient à ceux qui sauront diriger l'IA. Le no-code vous a appris la logique systémique, la modélisation des données et l'architecture des processus. Ces compétences sont exactement celles qu'il faut pour "prompter" des systèmes complexes, pour auditer ce que l'IA génère, et pour assembler les briques qu'elle vous tend. L'IA générative intégrée à vos applications (via des API comme celles d'OpenAI, Anthropic ou des modèles open-source) transformera vos outils statiques en assistants proactifs. Mais le chef d'orchestre, celui qui définit la partition et veille à la cohérence de l'ensemble, c'est vous. Le no-code n'était qu'une étape vers la démocratisation de la création ; l'IA en est l'accélérateur. Votre expertise visuelle et logique est le socle sur lequel cette IA viendra s'appuyer.

La responsabilité du créateur : éthique et avertissement

En vous donnant les clés pour bâtir, automatiser et industrialiser, ce livre vous a également confié une lourde responsabilité. Lorsque vous connectez des API, lorsque vous manipulez des données clients, lorsque vous automatisez des décisions qui impactent des employés ou des utilisateurs, vous devenez le garant de l'éthique et de la sécurité de ces systèmes. Le shadow IT, cette tendance à contourner les services informatiques pour déployer des outils dans son coin, peut sembler grisant par la liberté qu'il procure. Mais il expose l'entreprise à des

© 2026 Oussama Ghorbel

risques majeurs en matière de conformité (comme le RGPD en Europe), de sécurité des données et de continuité de service. L'architecte no-code mature est celui qui sait intégrer la gouvernance, la cybersécurité et le respect de la vie privée dès la première esquisse de son workflow.

Par ailleurs, un avertissement s'impose avant de vous laisser tourner la dernière page. Si ce livre vous a inspiré pour lancer votre propre SaaS, monétiser une application, optimiser les marges de votre entreprise ou investir du temps et du capital dans une startup no-code, gardez à l'esprit que le marché est imprévisible et que tout investissement comporte des risques financiers à évaluer avec prudence. La technologie, aussi élégante soit-elle, ne garantit pas l'adoption par les utilisateurs ni la viabilité économique d'un modèle d'affaires. Le no-code réduit le coût de l'expérimentation, mais il ne supprime pas le risque inhérent à l'entrepreneuriat et à l'innovation.

Épilogue : la page blanche vous appartient

Il y a une émotion particulière que seul le créateur connaît. C'est ce moment précis où, après des heures de modélisation, de tâtonnements sur des interfaces et de débogage de webhooks obscurs, le système s'éveille. Vous cliquez sur un bouton, et la donnée traverse trois applications différentes, déclenche une notification, met à jour un tableau de bord et envoie un email personnalisé, le tout en moins de deux secondes. C'est le frisson de l'orchestration. C'est la matérialisation de votre pensée pure, transformée en action mécanique et infatigable.

Vous n'avez plus besoin de demander la permission à un département technique surchargé pour tester une idée. Vous n'avez plus besoin d'attendre six mois et un budget colossal pour digitaliser un processus papier qui fait perdre des heures à vos équipes. Les barrières à l'entrée de la création logicielle se sont effondrées. Le goulot d'étranglement n'est plus la rareté des développeurs ; c'est la rareté des visionnaires capables de comprendre les problèmes en profondeur et d'imaginer des solutions élégantes.

Le monde regorge de processus cassés, d'interfaces frustrantes et de données enfermées dans des silos poussiéreux. Partout où vous irez, dans votre entreprise, dans votre secteur d'activité, dans votre communauté, vous verrez désormais la matrice. Vous verrez les entités,

© 2026 Oussama Ghorbel

les relations, les déclencheurs et les actions. Vous saurez comment les aligner.

Alors, que allez-vous construire demain ?

N'attendez pas d'avoir le projet parfait. N'attendez pas que l'outil soit dans sa version 2.0. Commencez petit. Modélisez un problème qui vous irrite au quotidien. Créez une table, liez deux champs, déclenchez une automatisation. Laissez la magie opérer, puis itérez. Le no-code est un art vivant, qui s'apprend en faisant, en cassant, en reconstruisant et en partageant. La communauté des makers et des architectes no-code est vaste, bienveillante et incroyablement inventive ; rejoignez-la, contribuez-y, et n'ayez pas peur de repousser les limites de ce que l'on croyait possible sans code.

Le code n'était qu'un détail. La valeur a toujours été, et sera toujours, dans votre capacité à penser le monde pour le rendre un peu plus fluide, un peu plus logique, et un peu plus humain.

La toile est vierge. Les outils sont entre vos mains. À vous de jouer.

Le manifeste de l'architecte no-code

Pour clore définitivement cet ouvrage, je vous propose non pas un résumé, mais un manifeste. Une boussole morale et pratique à garder sous les yeux lorsque vous serez confronté à la complexité, au doute, ou à la pression de livrer vite. Affichez-le dans votre espace de travail, partagez-le avec vos équipes, et laissez-le guider vos décisions techniques et stratégiques.

1. Le problème dicte la solution, jamais l'inverse.

Ne tombez jamais amoureux d'un outil. Bubble, Make, Xano ou Notion sont des moyens, pas des fins. Si un simple tableur partagé résout le problème de votre client en dix minutes, n'essayez pas de construire une application web complexe juste pour le plaisir de la technique. L'élégance réside dans le minimalisme de la solution par rapport à la taille du problème.

2. La donnée est sacrée, l'interface est éphémère.

Les designs changent, les frameworks frontend évoluent, les tendances UX mutent. Mais la structure de vos données, la vérité de

© 2026 Oussama Ghorbel

votre métier, doit être bâtie sur du roc. Investissez votre temps dans la modélisation relationnelle. Une base de données saine survivra à la refonte totale de votre application ; une base de données bancale fera s'effondrer la plus belle des interfaces.

3. L'automatisation doit servir l'humain, jamais le remplacer aveuglément.

Quand vous concevez un workflow, posez-vous toujours la question de l'impact humain. Une automatisation qui envoie cinquante emails par jour à un manager sans lui offrir de tableau de bord de synthèse n'est pas une aide, c'est une nuisance. Automatisez les tâches à faible valeur ajoutée pour libérer du temps de cerveau disponible, afin que vos équipes puissent se concentrer sur l'empathie, la créativité et la prise de décision complexe.

4. La transparence vaut mieux que la magie.

Documentez vos logiques. Nommez vos variables, vos modules et vos colonnes avec une clarté absolue. Le "vous" du futur, ou le collègue qui reprendra votre projet dans deux ans, vous remerciera. Le no-code ne doit pas être une boîte noire incompréhensible, même pour son créateur. La lisibilité visuelle est votre meilleure assurance-vie contre la dette technique.

5. L'échec rapide est une victoire, l'échec caché est une faute.

Le plus grand avantage du no-code est la vitesse d'itération. Profitez-en pour tester des hypothèses à moindre coût. Mais quand ça casse, et ça cassera, assumez-le, logguez l'erreur, alertez l'utilisateur et corrigez. Ne laissez jamais un workflow échouer en silence dans les limbes d'un serveur distant.

La force du collectif : on ne construit jamais seul

Il serait profondément injuste de conclure ce livre sans évoquer l'écosystème qui rend tout cela possible. Le mouvement no-code n'est pas seulement une suite de logiciels SaaS ; c'est une communauté mondiale, vibrante, généreuse et profondément collaborative.

Contrairement au développement traditionnel où le code source est souvent jalousement gardé, la culture no-code a instauré une norme

© 2026 Oussama Ghorbel

fascinante : le partage des templates. Sur les forums, dans les groupes de discussion, sur les plateformes de vidéo ou lors des meetups locaux, des milliers de créateurs partagent gratuitement leurs architectures, leurs scénarios d'automatisation, leurs formules de bases de données et leurs astuces d'interface.

Cette intelligence collective est votre plus grand atout. Quand vous bloquez sur un problème de pagination, sur une jointure complexe ou sur un timeout dans un webhook, sachez que quelqu'un, quelque part dans le monde, a déjà rencontré cette erreur, l'a documentée, et a pris le temps d'y répondre. N'ayez jamais honte de demander de l'aide, et n'oubliez jamais d'en donner en retour. La véritable expertise ne se mesure pas à ce que vous savez garder pour vous, mais à ce que vous êtes capable d'apporter au collectif.

La rédaction de ces lignes a elle-même puisé dans cette communauté une inspiration inépuisable. Les échanges avec des makers de tous horizons — des fondateurs de startups aux responsables des opérations de multinationales, en passant par des professeurs et des artisans — ont nourri chaque chapitre de ce livre. Le no-code est un grand égalisateur. Il ne vous demande pas votre diplôme, votre âge ou votre réseau. Il ne vous demande que votre curiosité et votre persévérance.

Le mot de la fin

Nous arrivons au terme de ce voyage. Lorsque j'ai commencé à structurer les idées qui ont donné naissance à « No-Code : L'art de bâtir et d'automatiser sans écrire une ligne », mon objectif était simple : démystifier la création logicielle. Je voulais prouver que la technologie, trop longtemps confisquée par une élite technique, pouvait et devait redevenir un bien commun, un outil d'émancipation professionnelle et personnelle.

En parcourant ces pages, vous avez franchi un cap. Vous n'êtes plus un simple consommateur de logiciels ; vous en êtes devenu le créateur. Vous avez appris à penser en systèmes, à modéliser la complexité, à orchestrer des flux d'informations et à concevoir des expériences qui ont du sens. Vous possédez désormais un super-pouvoir discret mais redoutable : la capacité de plier la technologie à votre volonté, et non l'inverse.

© 2026 Oussama Ghorbel

Le monde de demain sera façonné par celles et ceux qui sauront faire le pont entre les besoins humains et les capacités machines. Les entreprises qui survivront aux prochaines décennies ne seront pas nécessairement celles qui auront les plus gros budgets informatiques, mais celles qui auront su donner à leurs collaborateurs les moyens de résoudre leurs propres problèmes, en temps réel, avec agilité et bon sens. Vous êtes l'un de ces ponts. Vous êtes l'un de ces catalyseurs.

Gardez ce livre comme une référence, bien sûr. Retournez aux chapitres sur la modélisation des données quand vous douterez de votre architecture. Relisez les patterns de workflows quand vos automatisations deviendront fragiles. Mais surtout, refermez-le dès que vous sentirez l'appel de la création.

Les outils n'attendent que vous. Les API frémissent. Les bases de données sont prêtes à accueillir vos entités. Les utilisateurs espèrent des solutions qui leur ressemblent et qui les comprennent.

Je vous souhaite des nuits courtes mais exaltantes, des révélations mémorables face à des formules imbriquées, et la fierté immense de voir vos collègues ou vos clients utiliser, chaque jour, une solution que vous avez imaginée, bâtie et déployée de vos propres mains.

Merci de m'avoir lu. Merci d'avoir cru en cette promesse.

Maintenant, éteignez la lumière de ce chapitre, ouvrez un nouvel onglet, et allez construire le futur.

— Oussama Ghorbel

© 2026 Oussama Ghorbel

Bibliographie commentée

Bhaskar Ghosh, Bertrand De Biasi, Karthik Narain, *The Automation Advantage* (2021).

Idée maîtresse : L'automatisation intelligente (RPA + IA) transforme l'entreprise à condition de repenser les processus avant de les outiller.

Apport : Cadre stratégique pour choisir quels processus automatiser en priorité et mesurer le ROI.

Eric von Hippel, *Democratizing Innovation* (2005).

Idée maîtresse : Les utilisateurs finaux sont souvent les meilleurs innovateurs quand on leur donne des outils adaptables.

Apport : Fondement théorique du 'citizen developer' : pourquoi libérer la création du service IT.

Thomas H. Davenport, *The AI Advantage* (2018).

Idée maîtresse : L'IA apporte surtout de la valeur en s'insérant dans des processus métier existants, pas en remplaçant les humains.

Apport : Repères pour intégrer des briques d'IA (OCR, NLP) dans des applications no-code.

Mike Cowley, *Mastering Microsoft Power Apps* (2020).

Idée maîtresse : Power Apps permet de construire des applications métier connectées à l'écosystème Microsoft 365 sans coder.

Apport : Patterns de conception d'apps d'entreprise (formulaires, galeries, collections).

Project Management Institute (PMI), *Citizen Developer Playbook* (2021).

Idée maîtresse : Le citizen development doit être encadré pour éviter le shadow IT tout en libérant la créativité.

Apport : Gouvernance, rôles et garde-fous à transposer dans notre chapitre sur l'industrialisation.

© 2026 Oussama Ghorbel

Jesse James Garrett, *The Elements of User Experience* (2010 (2e éd.)).

Idée maîtresse : Les 5 plans de l'UX (stratégie, portée, structure, squelette, surface) s'appliquent à tout produit numérique.

Apport : Grille de conception UX réutilisable pour les apps no-code.

Alan Cooper, Robert Reimann, David Cronin, *About Face: The Essentials of Interaction Design* (2014 (4e éd.)).

Idée maîtresse : Concevoir autour de personas et de scénarios plutôt que de fonctionnalités techniques.

Apport : Méthode personas + storyboards appliquée à la spec d'une app no-code.

Don Jones, *The DSC Book / Automating PowerShell (collectif)* (2017-2020).

Idée maîtresse : L'automatisation repose sur l'idempotence, la versioning et la reproductibilité.

Apport : Principes d'ingénierie applicables aux workflows Make/Zapier (tests, rollback).

John Allspaw, Jez Humble, *Continuous Delivery (référence DevOps)* (2010).

Idée maîtresse : Livrer petit, souvent, avec des tests automatisés et du versioning.

Apport : Culture DevOps transposée au no-code : environnements dev/staging/prod, revues.

David Chappell, *The Low-Code/No-Code Revolution (rapports & articles)* (2019-2023).

Idée maîtresse : Le low-code redéfinit la frontière entre IT et métier ; le shadow IT devient un actif si gouverné.

Apport : Analyse marché et positionnement stratégique pour décideurs.

Gartner (collectif analystes), *Magic Quadrant for Enterprise Low-Code Application Platforms* (2020-2025).

© 2026 Oussama Ghorbel

Idée maîtresse : Cartographie des plateformes (OutSystems, Mendix, Power Apps, ServiceNow, Salesforce) et critères de choix.

Apport : Grille comparative pour le chapitre 'choisir sa plateforme'.

Zapier Inc. (collectif), *The Ultimate Guide to Automation* (2022).

Idée maîtresse : L'automatisation commence par cartographier les tâches répétitives et leurs déclencheurs.

Apport : Méthode d'audit des workflows et catalogues de recettes types.

Mattias Skoog, Johan Bohman, *Bubble.io - Build Web Apps Without Code (guides communautaires)* (2021-2024).

Idée maîtresse : Bubble permet de construire des SaaS complets via une logique visuelle événementielle.

Apport : Patterns SaaS : users, abonnements, Stripe, emails transactionnels.

Kasra Rahbari (et contributeurs Airtable), *Airtable: The Missing Manual (ressources communautaires)* (2020-2024).

Idée maîtresse : Airtable est une base de données relationnelle déguisée en tableur, avec API et automatisations natives.

Apport : Modélisation de données relationnelles accessibles aux non-développeurs.

Indy Sawhney, *No-Code AI (ouvrages et formations)* (2022-2024).

Idée maîtresse : On peut entraîner et déployer des modèles de ML via des interfaces visuelles (Lobe, AutoML, GPT wrappers).

Apport : Briques IA à intégrer dans les apps no-code (classification, génération, extraction).

© 2026 Oussama Ghorbel